Table of Contents (Jupyter)

LEGEND

These are insights got from analysing the data, in Pink color

''' ''' code sections are notes to self, some quick code snippets, or some notes while doing analysis

GITHUB LINK FOR EASY VIEWING OF .ipynb NOTEBOOK

**Please note, when converting to PDF, the links in TOC stop working. You may like to see the original report's .ipynb file. Please see it on my Github Repository here **

- HTML Page
  - https://github.com/amitguptaforwork/dataanalytics_portfolio_projects/blob/main/CaseStudyOla_ChurnPrediction_UsingBaggingBoosting/Ola_AmitG.html
- JupyterNotebook
  - https://github.com/amitguptaforwork/dataanalytics_portfolio_projects/blob/main/CaseStudyOla_ChurnPrediction_UsingBaggingBoosting/Ola_AmitG.ipynb

1.0 Understanding the Problem and the Data

About Ola

Ola is an Indian ride-hailing company that connects passengers with drivers through its mobile app for cabs, auto-rickshaws, and bikes.

Objective of this Exercise

Recruiting and retaining drivers is seen by industry watchers as a tough battle for Ola. Churn among drivers is high and it's very easy for drivers to stop working for the service on the fly or jump to Uber depending on the rates.

As the companies get bigger, the high churn could become a bigger problem. To find new drivers, Ola is casting a wide net, including people who don't have cars for jobs. But this acquisition is really costly. Losing drivers frequently impacts the morale of the organization and acquiring new drivers is more expensive than retaining existing ones.

This case study will focus on predicting whether a driver will be leaving the company or not based on their attributes.

Company is collecting data for the drivers on monthly basis and thus has not only static data (driver id, driver education, gender) but also dynamic data like income, grade, total business value. These attributes offer additional insight into the "professional journey" of the driver.

Given a set of attributes for an OLA driver, determine if they may potentially leave the company

Dataset

The company collected the data of drivers. The dataset has the following features:

Column Profiling:

- MMMM-YY : Reporting Date (Monthly)
- Driver_ID : Unique id for drivers
- Age : Age of the driver
- Gender : Gender of the driver – Male : 0, Female: 1
- City : City Code of the driver
- Education_Level : Education level – 0 for 10+ ,1 for 12+ ,2 for graduate
- Income : Monthly average Income of the driver
- Date Of Joining : Joining date for the driver
- LastWorkingDate : Last date of working for the driver
- Joining Designation : Designation of the driver at the time of joining

- Grade : Grade of the driver at the time of reporting
- Total Business Value : The total business value acquired by the driver in a month (negative business indicates cancellation/refund or car EMI adjustments)
- Quarterly Rating : Quarterly rating of the driver: 1,2,3,4,5 (higher is better)

2.0 Import and Inspect the Data

In [492…
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import scipy.stats as stats
```

In [493…
```
'''
Notes to Self
During this step, looking into the statistics is critical to gain initial know-how of its structure, variable kinds, and capability issues.

'''
```

Out[493… ' \nNotes to Self\nDuring this step, looking into the statistics is critical to gain initial know-how of its structure, variable kinds, and capability issues.\n\n'

In [494…
```
df =pd.read_csv("ola_driver_scaler.csv")
df.columns
```

Out[494…
```
Index(['Unnamed: 0', 'MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City',
       'Education_Level', 'Income', 'Dateofjoining', 'LastWorkingDate',
       'Joining Designation', 'Grade', 'Total Business Value',
       'Quarterly Rating'],
      dtype='object')
```

In [495…
```
#we dont need 'Unnamed: 0' column, as its just row index.
df.drop(columns=['Unnamed: 0'], inplace=True)
df
```

Out[495…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	2381060	2
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	-665480	2
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	0	2
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	0	1
4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	740280	3
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	448370	3
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	0	2
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	200420	2
19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	NaN	2	2	411480	2

19104 rows × 13 columns

In [ ]:

2.2 Statistical summary for numerical columns and Unique values for categorical data

- Instead of manually creating tables and collecting values lets automate the table creation
- We will generate a table of all columns in Markdown format
- We will paste this table into a markdown cell and then we may add additional analysis

In [496…

```
#!pip install tabulate
from tabulate import tabulate
def generate_markdown_table_with_details(df):
    column_details = []
    for col in df.columns:
        dtype = str(df[col].dtype)

        if dtype == 'object' or dtype=='category':
            cat_or_num = 'Categorical'
            details = df[col].unique().tolist()
            if(len(details)>10):
                details= f'{df[col].nunique()} unique values.Ex.{",".join([str(x) for x in details[:3]])}'

        else:
            cat_or_num = 'Numerical'
            details = f"Min: {df[col].min()}, Max: {df[col].max()}"
        column_details.append([col, dtype, cat_or_num, details, ''])

    # Create a DataFrame for the table
    table_df = pd.DataFrame(column_details, columns=['Column Name', 'Data Type', 'Categorical or Numerical', 'Details', 'Business Description(fill manually)'])

    # Convert DataFrame to Markdown
    markdown_table = tabulate(table_df, headers='keys', tablefmt='pipe', showindex=False)
    return markdown_table
```

```
markdown_table = generate_markdown_table_with_details(df)
print(f'There are a total of {df.shape[0]:,} rows and {df.shape[1]} columns')
print(markdown_table)
```

```
There are a total of 19,104 rows and 13 columns
| Column Name          | Data Type   | Categorical or Numerical   | Details                                          | Business Description(fill manually)   |
|:---------------------|:------------|:---------------------------|:-------------------------------------------------|:--------------------------------------|
| MMM-YY               | object      | Categorical                | 24 unique values.Ex.01/01/19,02/01/19,03/01/19   |                                       |
| Driver_ID            | int64       | Numerical                  | Min: 1, Max: 2788                                |                                       |
| Age                  | float64     | Numerical                  | Min: 21.0, Max: 58.0                             |                                       |
| Gender               | float64     | Numerical                  | Min: 0.0, Max: 1.0                               |                                       |
| City                 | object      | Categorical                | 29 unique values.Ex.C23,C7,C13                   |                                       |
| Education_Level      | int64       | Numerical                  | Min: 0, Max: 2                                   |                                       |
| Income               | int64       | Numerical                  | Min: 10747, Max: 188418                          |                                       |
| Dateofjoining        | object      | Categorical                | 869 unique values.Ex.24/12/18,11/06/20,12/07/19  |                                       |
| LastWorkingDate      | object      | Categorical                | 493 unique values.Ex.nan,03/11/19,27/04/20       |                                       |
| Joining Designation  | int64       | Numerical                  | Min: 1, Max: 5                                   |                                       |
| Grade                | int64       | Numerical                  | Min: 1, Max: 5                                   |                                       |
| Total Business Value | int64       | Numerical                  | Min: -6000000, Max: 33747720                     |                                       |
| Quarterly Rating     | int64       | Numerical                  | Min: 1, Max: 4                                   |                                       |
```

2.3 Adding insights about columns and data size.

- We will copy the markdown table generated above and add our inputs by analysing the columns.
- We identify which columns are numerical, which are categorical
- Within categorical, we identify if it is nominal or ordinal data. This helps us to decide best graph to use
- Also ordinal data can be converted into numeric using codes thereby making it available for some kind of graphs.

2.4 Data Dictionary

There are a total of 19,104 rows and 13 columns

Column Name	Data Type	Categorical or Numerical	Details	Business Description(fill manually)
MMM-YY	object	Categorical	24 unique values.Ex.01/01/19,02/01/19,03/01/19	Reporting Date (Monthly)
Driver_ID	int64	Numerical	Min: 1, Max: 2788	Unique id for drivers
Age	float64	Numerical	Min: 21.0, Max: 58.0	Age of the driver
Gender	float64	Numerical	Min: 0.0, Max: 1.0	Gender of the driver – Male : 0, Female: 1
City	object	Categorical	29 unique values.Ex.C23,C7,C13	City Code of the driver
Education_Level	int64	Numerical	Min: 0, Max: 2	Education level – 0 for 10+ ,1 for 12+ ,2 for graduate
Income	int64	Numerical	Min: 10747, Max: 188418	Monthly average Income of the driver
Dateofjoining	object	Categorical	869 unique values.Ex.24/12/18,11/06/20,12/07/19	Joining date for the driver
LastWorkingDate	object	Categorical	493 unique values.Ex.nan,03/11/19,27/04/20	Last date of working for the driver
Joining Designation	int64	Numerical	Min: 1, Max: 5	Designation of the driver at the time of joining

Grade	int64	Numerical	Min: 1, Max: 5	Grade of the driver at the time of reporting

Total Business Value	int64	Numerical	Min: -6000000, Max: 33747720	The total business value acquired by the driver in a month (negative business indicates cancellation/refund or car EMI adjustments. This isn't just a simple revenue column; it's more like a "Net Value Generated by Driver for the Platform" metric)
				Positive Values (Thousands and Lakhs)
				This represents the gross revenue generated by the driver's activities. It's the core positive contribution.
				• **What it likely includes:** The total fare amount of all completed trips (base fare + distance charge + time charge + surge pricing + tolls).
				• **Interpretation:** A driver with a high positive TBV is an active, high-performing driver. They complete many trips, long-distance trips, or trips during peak hours with surge pricing. They are the engine of the business.
				Zero Values
				This is a significant group that needs to be understood. It almost always indicates inactivity.
				• **Interpretation:**
				  • **Inactive/Dormant Drivers:** The most common reason. These drivers are registered on the platform but took zero trips in that specific month.
				  • **Newly Onboarded Drivers:** They might have signed up late in the month and haven't started driving yet.
				  • **Temporarily Inactive:** Drivers who are on vacation, sick, or taking a break.
				  • **Net Zero (Highly Unlikely):** A theoretical case where a driver's ride values were perfectly cancelled out by refunds or adjustments. This is extremely rare.
				Negative Values
				This is where the metric gets interesting and points to costs associated with the driver for that month.
				• **Interpretation based on your definition:**
				  • **Cancellations/Refunds:** If a driver had trips that were later refunded to the customer due to a complaint, this would be a deduction. A large negative value could indicate a significant service quality issue or a fraudulent trip that was reversed.
				  • **Car EMI Adjustments:** This is a crucial piece of information. It suggests your company has a vehicle leasing or financing program for its drivers. The negative value could represent the monthly installment for the car being deducted. **A driver could have a negative TBV for the month not because they are a "bad" driver, but because their car payment exceeded the value of the rides they gave.**
				  • **Penalties or Other Deductions:** There might be other penalties (e.g., for rule violations) that are deducted here.

Quarterly Rating	int64	Numerical	Min: 1, Max: 4	Quarterly rating of the driver: 1,2,3,4,5 (higher is better)

- 19104 records

In [ ]:

2.5 Handling Missing Values

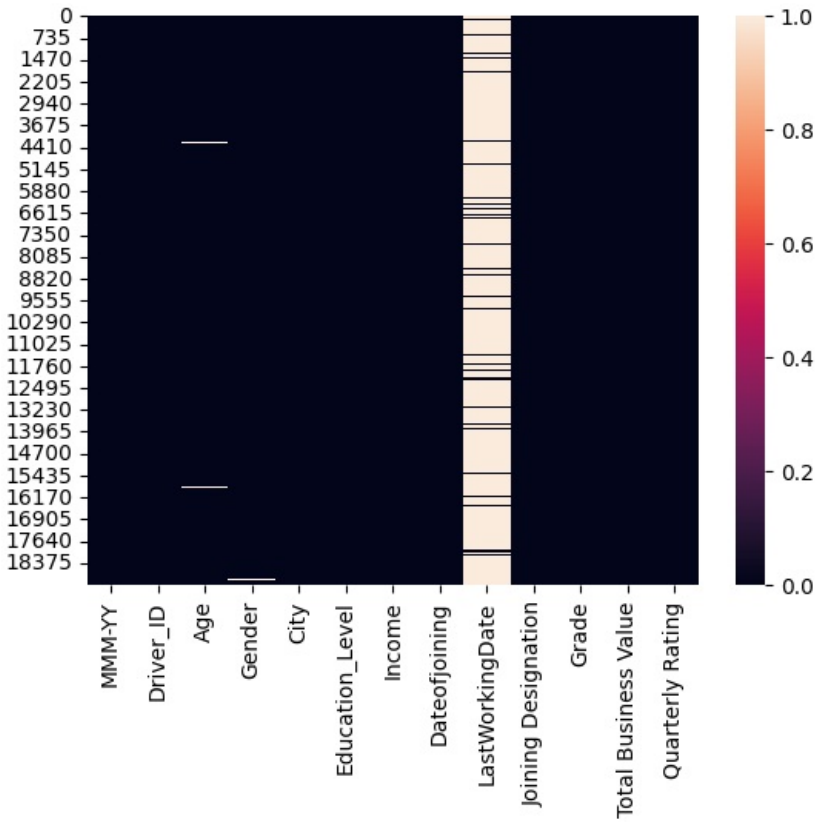
Now let's check if there are any missing values in our dataset or not.

In [497…
```
'''
Notes to Self
Missing Data can also refer to as NA(Not Available) values in pandas. There are several useful functions for detecting, removing, and replacing null values in Pandas D

isnull()
notnull()
dropna()
fillna()
replace()
interpolate()
'''
```

Out[497…
```
' \nNotes to Self\nMissing Data can also refer to as NA(Not Available) values in pandas. There are several useful functions for detecting, removing, and replacing nul
l values in Pandas DataFrame :\n\nisnull()\nnotnull()\ndropna()\nfillna()\nreplace()\ninterpolate()\n'
```

In [498…
```
sns.heatmap(df.isnull())
```

Out[498…
```
<Axes: >
```



- Age and Gender have a few missing values, LastWorkingDay has many, as it indicates drivers who are still there with the company.

In [499…

```
#Create a small dataframe with count of missing values, and their percentage when compared to overall data
df_analysis = pd.DataFrame(df.isnull().sum(), columns=['NumberOfMissingValues'])
df_analysis['Percentage'] = np.round(df_analysis.NumberOfMissingValues*100/len(df),1).astype(int)

#Print only columns that have any missing data
df_analysis[df_analysis.NumberOfMissingValues>0]
```

Out[499…

	NumberOfMissingValues	Percentage
Age	61	0
Gender	52	0
LastWorkingDate	17488	91

- We have 3 columns with some missing data
- LastWorkingDate missing means the driver is still with the company.

2.5.1 Imputation for LastWorkingDate

- We can fill some predefined value here. Later we can compare with that value to identify that this driver is still with the compaany.
- We use SimpleImputer instead of direct Pandas code (fillna)
- SimpleImputer ensures consistent, reusable, and pipeline-friendly imputations across training, testing, and production, unlike direct pandas.fillna()

In [500…
```
from sklearn.impute import SimpleImputer
#Pandas to_datetime interprets two-digit years (%y) using a "pivot year" rule:
# Years 00–68 → 2000–2068
# Years 69–99 → 1969–1999
# So we choose 2067 so that it will get converted to 2067
imp_missing = SimpleImputer(strategy='constant', fill_value="31/12/67")
df[['LastWorkingDate']] = imp_missing.fit_transform(df[['LastWorkingDate']])
```

In [501…
```
df
```

Out[501…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	31/12/67	1	1	2381060	2
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	31/12/67	1	1	-665480	2
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	0	2
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	31/12/67	2	2	0	1
4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	31/12/67	2	2	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	740280	3
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	448370	3
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	0	2
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	200420	2
19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	411480	2

19104 rows × 13 columns

2.5.1 Imputation for Age and Gender

- Let us observe the rows that have missing age. We want to observe if we have missing data for same driver or different drivers.
- We have a hypothesis that if its randomly missing data (MCAR - missing completely at random), then we may have different driver ids.
- The final purpose is, we know we have multiple rows per driver. So if we can establish that driver ids in missing data rows are different, it gives us a chance to copy the missing cells from other rows of the same driver.

In [502…
```
df[df.Age.isna()]
```

Out[502…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
72	02/01/20	20	NaN	1.0	C19	0	40342	25/10/19	31/12/67	3	3	0	1
97	10/01/19	22	NaN	0.0	C10	2	31224	25/05/18	31/12/67	1	1	200000	3
110	07/01/19	24	NaN	0.0	C24	2	76308	25/05/18	31/12/67	1	2	203240	3
212	11/01/19	40	NaN	0.0	C15	0	59182	11/08/19	31/12/67	2	2	0	1
261	05/01/19	49	NaN	0.0	C20	0	53039	25/05/18	31/12/67	1	2	124190	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
18395	05/01/20	2690	NaN	0.0	C11	2	77662	17/07/18	31/12/67	1	2	692600	4
18722	08/01/20	2730	NaN	1.0	C16	2	69924	07/08/19	31/12/67	2	2	161860	2
18780	03/01/19	2738	NaN	0.0	C17	0	23068	09/08/18	31/12/67	1	1	639780	3
18843	01/01/19	2751	NaN	0.0	C17	2	53115	11/05/15	31/12/67	1	1	506550	3
19024	02/01/19	2774	NaN	0.0	C15	1	42313	21/07/18	31/12/67	1	1	1141280	4

61 rows × 13 columns

Indeed, the hypothesis seems to hold - the drivers are different for the missing age rows. Lets "look" at two drivers' rows to further confirm our hypothesis.

In [503…
```
df[(df.Driver_ID==167)|(df.Driver_ID==179)]
```

Out[503…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
1103	01/01/19	167	NaN	0.0	C18	1	35283	03/02/18	31/12/67	1	1	0	2
1104	02/01/19	167	26.0	0.0	C18	1	35283	03/02/18	31/12/67	1	1	100740	2
1105	03/01/19	167	26.0	0.0	C18	1	35283	03/02/18	31/12/67	1	1	1053880	2
1106	04/01/19	167	26.0	0.0	C18	1	35283	03/02/18	31/12/67	1	1	0	1
1107	05/01/19	167	26.0	0.0	C18	1	35283	03/02/18	05/06/19	1	1	0	1
1207	01/01/19	179	26.0	1.0	C11	2	24490	26/05/18	31/12/67	1	1	203420	2
1208	02/01/19	179	NaN	1.0	C11	2	24490	26/05/18	31/12/67	1	1	449310	2
1209	03/01/19	179	26.0	1.0	C11	2	24490	26/05/18	31/12/67	1	1	823030	2
1210	04/01/19	179	26.0	1.0	C11	2	24490	26/05/18	31/12/67	1	1	141020	1
1211	05/01/19	179	26.0	1.0	C11	2	24490	26/05/18	05/04/19	1	1	0	1

Indeed, we can clearly see the Age is missing from one row and can be easily copied from other rows for the same driver. As a final confirmation, lets count how many unique drivers are there across the rows that have Age missing

In [504…
```
df[df.Age.isna()].Driver_ID.nunique()
```

Out[504… 57

- The hypothesis holds. Out of 61 values 59 are unique.
- So there is some drivers who has more than one row of missing value. Lets find those drivers.

In [505…
```
df_analysis = df[df.Age.isna()]
df_analysis2 = df_analysis.groupby('Driver_ID').size()
df_analysis2[df_analysis2>1]
```

Out[505…
```
Driver_ID
49      2
718     2
901     2
2073    2
dtype: int64
```

- So we have these 4 drivers. Let us see how many total rows we have for each of these drivers.
- We group the original dataframe just using driver_id and count the rows, then see that for these drivers, how many rows are there.

In [506…
```
df_analysis3 = df.groupby('Driver_ID',as_index=False).size()
df_analysis3[df_analysis3['Driver_ID'].isin([49,718,901,2073])]
```

Out[506…

	Driver_ID	size
36	49	13
616	718	24
766	901	9
1764	2073	10

- Lets do same analysis for Gender column

In [507…
```
df.Gender.unique()
```

Out[507…
```
array([ 0.,  1., nan])
```

In [508…
```
df[df.Gender.isna()].Driver_ID.nunique()
```

Out[508…
```
51
```

- Out of 61 values 59 are unique. It means that the value is missing from one row only for these drivers and there may be other rows for these drivers where the age value is available. Therefore we can use KNNImputer to find other similar rows and use values from those rows to impute age wherever it is missing. KNNImputer is a class from sklearn.impute that helps you fill (impute) missing values in your dataset using the K-Nearest Neighbors approach. Instead of filling missing values with just the mean or median, it looks at similar rows and uses their values to estimate the missing one.
- Similarly for gender, we had 52 missing values, and 51 are for unique drivers. Again it means that age value may be available for those drivers from other rows. KNNImputer is the right choice for Gender too.

Deciding value of K for KNNImputer

In [509…
```
drivers_missingGender = df[df.Gender.isna()].Driver_ID.values
df[df.Driver_ID.isin(drivers_missingGender)].groupby('Driver_ID').Gender.size()
```

Out[509…
```
Driver_ID
43       2
49      13
68      24
116      9
119      5
225      6
296     24
354      3
365      4
407     24
439     10
446     11
489     13
516     18
541     20
611      4
640      8
709      6
793      2
888     24
951      9
1004     5
1023     8
1028     4
1143    11
1156     5
1205    17
1312     5
1407     9
1452    12
1508    10
1621     9
1804     3
1877    18
1936    24
2057     6
2073    10
2093     5
2215     4
2228    24
2478    14
2517    24
2607     5
2693    13
2716    15
2728    10
2732    14
2738    13
2760     5
2765     2
2774     7
Name: Gender, dtype: int64
```

So we are basically seeing how many rows we have for each driver with any missing gender value. lets find the min value

In [510…
```
df[df.Driver_ID.isin(drivers_missingGender)].groupby('Driver_ID').Gender.size().min()
```

Out[510… np.int64(2)

So there are drivers with two rows and one of them is missing. **So k value we will use should be 1 so that we can cover this driver too**

In [511…
```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=1)

# We cannot pass the entire dataframe to KNN Imputer as we have some categorical data too
# Best is to select he columns on which we want to base imputation on.
cols_for_impute = ['Age', 'Gender', 'Driver_ID']
df_impute = df[cols_for_impute].copy()
df_imputed_values = imputer.fit_transform(df_impute)
```

In [512…
```
df_imputed_values
```

Out[512…
```
array([[2.800e+01, 0.000e+00, 1.000e+00],
       [2.800e+01, 0.000e+00, 1.000e+00],
       [2.800e+01, 0.000e+00, 1.000e+00],
       ...,
       [3.000e+01, 0.000e+00, 2.788e+03],
       [3.000e+01, 0.000e+00, 2.788e+03],
       [3.000e+01, 0.000e+00, 2.788e+03]], shape=(19104, 3))
```

- KNNImputer retured a numpy array.
- df_imputed_values is a numpy array. First value is age, second is gender.
- We will copy these values back to the original dataframe in the right column

In [513…
```
# Replace only the age and gender columns
df['Age'] = df_imputed_values[:, 0]
df['Gender'] = df_imputed_values[:, 1]
```

In [514…
```
df.Gender.unique()
```

Out[514… array([0., 1.])

Verify we dont have any null values left in any columns

In [515…
```
df.isna().sum()
```

Out[515…
```
MMM-YY                 0
Driver_ID              0
Age                    0
Gender                 0
City                   0
Education_Level        0
Income                 0
Dateofjoining          0
LastWorkingDate        0
Joining Designation    0
Grade                  0
Total Business Value   0
Quarterly Rating       0
dtype: int64
```

We have NO missing values

- LastWorkingDate- Uses SimpleImputer to fill constant value of "12/31/2067". That is the max value pandas.to_datetime will convert to an year in 21st century.
- Age - uses KNNImputer
- Gender - uses KNNImputer

In [516…
```
df
```

Out[516…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	31/12/67	1	1	2381060	2
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	31/12/67	1	1	-665480	2
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	0	2
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	31/12/67	2	2	0	1
4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	31/12/67	2	2	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	08/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	740280	3
19100	09/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	448370	3
19101	10/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	0	2
19102	11/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	200420	2
19103	12/01/20	2788	30.0	0.0	C27	2	70254	06/08/20	31/12/67	2	2	411480	2

19104 rows × 13 columns

2.6 Date columns conversion

In [517…
```
# Convert MMM-YY to datetime format
df['MMM-YY'] = pd.to_datetime(df['MMM-YY'], format='%m/%d/%y')

# Convert Dateofjoining and LastWorkingDate to datetime
df['Dateofjoining'] = pd.to_datetime(df['Dateofjoining'], format='%d/%m/%y')
df['LastWorkingDate'] = pd.to_datetime(df['LastWorkingDate'], format='%d/%m/%y')
```

In [518…
```
df
```

Out[518…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	2019-01-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	2381060	2
1	2019-02-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	-665480	2
2	2019-03-01	1	28.0	0.0	C23	2	57387	2018-12-24	2019-11-03	1	1	0	2
3	2020-11-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1
4	2020-12-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	2020-08-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	740280	3
19100	2020-09-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	448370	3
19101	2020-10-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	0	2
19102	2020-11-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	200420	2
19103	2020-12-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	411480	2

19104 rows × 13 columns

2.7 Feature Engineering - Possible candidates for new features after going through existing columns

1. We create a new column called churn that indicates if employee has left. 1= left, 0 = not left
2. Employment length for a driver
3. Flag Creation: Attributes such as income increased, grade increased
4. Date-related features: Extracting day, month, or year can expose temporal patterns.
5. Change in Rating (Example): After aggregating, find the difference between successive quarterly ratings to capture performance trends.
6. Change in income - find if income of employee increased over the months, and by what CAGR

Group at driver id level

- The given data is on monthly basis and thus for each driver, there are many rows
- We expect lot of analysis that can be done at driver level. Thus let us first create the grouped dataframe called drivers.

- During grouping, we need to focus on how the columns that are not part of the group by clause get summarized.
- For most columns, we may have same values. Those are easy to treat. We can just take first or last value
- For some columns, we may need to use mean or median
- This data has a city column which we cannot say remains same unless we check.
  - For ex, a driver may be operating in only one city or may have owrked in multiple cities.
  - We want to check if there is any driver that has that kind of data.

In [519…

```
df_analysis = df.groupby('Driver_ID', as_index=False).City.nunique()
df_analysis.query('City>1')
```

Out[519…

	Driver_ID	City

- We got no rows with city count greater than 1 for a driver. So that confirms there is no driver that has worked in more than one city.
- City can be used in group by or we can just take first or last value while grouping at Driver_ID level

Lets do same for the income column

In [520…
```
df_analysis = df.groupby('Driver_ID', as_index=False).Income.nunique()
df_analysis.query('Income>1')
```

Out[520…

	Driver_ID	Income
18	26	2
40	54	2
46	60	2
80	98	2
230	275	2
256	307	2
267	320	2
312	368	2
368	434	2
460	537	2
488	568	2
500	580	2
502	582	2
550	638	2
615	716	2
672	789	2
756	888	2
877	1031	2
891	1050	2
986	1161	2
990	1165	2
1025	1206	2
1064	1249	2
1083	1274	2
1116	1316	2
1126	1327	2
1507	1770	2
1517	1783	2
1545	1817	2

1564	1840	2
1576	1852	2
1599	1877	2
1634	1918	2
1706	2008	2
1762	2070	2
1773	2087	2
1871	2198	2
1937	2272	2
2034	2390	2
2050	2407	2
2168	2543	2
2189	2567	2
2240	2625	2
2293	2690	2

- So there are drivers who have different incomes in their rows. We need to think what to do with income column while grouping.
  - We can try to find % increase in income which will give idea of how the income increased.
  - This would require custom logic and cant be done as part of group by. We will do it after grouping
- We now go ahead with the grouping of columns based on driver_id.

In [521…

```
create_driver_dict = {'Age':'last',
                      'Gender':'last',
                      'City':'last',
                      'Education_Level':'last',
                      'Dateofjoining':'first',
                      'LastWorkingDate':'last',
                      'Joining Designation':'first',
                      'Grade':'last',
                      'Total Business Value':'sum',
                      'Quarterly Rating':'last'
                      }

df_original = df.copy()
del(df)
drivers = df_original.groupby('Driver_ID', as_index=False).agg(create_driver_dict)
drivers
```

Out[521…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2
...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2

2381 rows × 11 columns

Data as been gropuped. Now lets add additional column related to Income, as we discussed earlier.

2.7.1 Adding IncomeCAGR, IncomeLast

- Now we will try to add a column CAGR that will indicate how much the income increased for the driver over the reporting period
- We create a separate DF to do the calculations and finally merge it with drivers dataframe.
- CAGR Formula

$$\text{CAGR} = \left(\frac{\text{last}}{\text{first}}\right)^{1/\text{years}} - 1$$

In [522…

```
#Sort df dataframe to ensure we have the correct setup for CAGR calculation
df_original = df_original.sort_values(['Driver_ID', 'MMM-YY'])

#Step 1 – Aggregate first & last data per driver
driver_cagr = (
    df_original.groupby('Driver_ID')
      .agg(
          IncomeFirst=('Income', 'first'),
          IncomeLast=('Income', 'last'),
          first_date=('MMM-YY', 'first'),
          last_date=('MMM-YY', 'last')
      )
)
#Step 2 – Compute time difference in years
driver_cagr['years'] = (driver_cagr['last_date'] - driver_cagr['first_date']).dt.days / 365

#Step 3 – Compute CAGR safely
#We'll handle:
#Drivers with only one record (years == 0)
#Missing or invalid data (NaN or zero division)
```

```
#If the driver has only one entry → years == 0, we set CAGR = 0.

driver_cagr['IncomeCAGR'] = np.where(
    driver_cagr['years'] > 0,
    ((driver_cagr['IncomeLast'] / driver_cagr['IncomeFirst']) ** (1 / driver_cagr['years']) - 1) * 100,
    0
)

#Merge back to driver dataframe
drivers = drivers.merge(driver_cagr[['IncomeCAGR','IncomeLast']], on='Driver_ID', how='left')
```

In [523…
```
drivers
```

Out[523…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeCAGR	IncomeLast
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2	0.0	57387
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1	0.0	67016
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1	0.0	65603
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1	0.0	46368
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2	0.0	78728
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4	0.0	82815
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1	0.0	12105
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1	0.0	35370
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1	0.0	69498
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2	0.0	70254

2381 rows × 13 columns

2.7.2 Adding IncomeIncreased

We want to add a flag column for further easier interpretation of CAGR.

In [524…
```
drivers['IncomeIncreased'] = np.where(drivers['IncomeCAGR'] > 0, 1, 0)
```

2.7.3 Adding GradeIncreased

We have two columns 'Joining Designation' and 'Grade'. Its not given if they represent same thing. Lets find out. We assume that a person joined at grade 3 will not go "down" to grade 2...let see if that hypothesis holds. If it does, it means these two field represent same quantity - Grade of a driver at different points in their employment

In [525…
```
df_analysis = df_original[['Joining Designation','Grade']]
df_analysis[df_original['Joining Designation']>df_original.Grade]
```

Out[525…

	Joining Designation	Grade

So there are NO rows where 'Joining Designation' is more than current grade. It proves that these two fields are representing same thing - GRADE - at the time of joining and in reporting month.

So we can calculate a grade increase column that represents if the grade of the driver increased, since they joined

In [526…
```
drivers['GradeIncreased'] = np.where(drivers['Grade'] > drivers['Joining Designation'], 1, 0)
```

In [527…
```
drivers
```

Out[527…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2	0.0	57387	0	0
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1	0.0	67016	0	0
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1	0.0	65603	0	0
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1	0.0	46368	0	0
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2	0.0	78728	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4	0.0	82815	0	1
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1	0.0	12105	0	0
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1	0.0	35370	0	0
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1	0.0	69498	0	0
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2	0.0	70254	0	0

2381 rows × 15 columns

2.7.4 Adding RatingMean, RatingTrend, RatingTrendCode, Ratings (array)

We want to analyse the trend in rating and categorize it.

In [528…
```
# ------------------------------------------------------------------------------
# Function: classify_driver_rating(subdf)
# Purpose:
#   Analyze a single driver's quarterly ratings and classify their performance trend.
#
# Intention:
#   Each driver has multiple "Quarterly Rating" values over time (e.g., 1–5 scale).
#   We want to understand whether the driver is:
#       - improving over time,
#       - degrading over time,
#       - or remaining consistently high/low/stable.
#
# Logic:
#   1. Treat each driver's quarterly ratings as a time series.
#   2. Fit a simple Linear Regression (y = β0 + β1 * t),
#      where:
#         X = sequential time points (0, 1, 2, …)
#         y = quarterly rating values
#      β1 (the slope) shows the direction and strength of the trend.
#
#   3. Compute the mean rating to understand performance level.
```

```
#
#   4. Classify the driver as:
#         - 'improving'        → if slope > +0.5 (ratings increasing significantly)
#         - 'degrading'        → if slope < -0.5 (ratings decreasing significantly)
#         - 'consistent_high'  → if slope ≈ 0 and mean rating ≥ 4
#         - 'consistent_low'   → if slope ≈ 0 and mean rating ≤ 2
#         - 'stable'           → if slope ≈ 0 and mean rating is mid-range (2–4)
#
#   5. Return summary stats for that driver:
#         - average rating (RatingMean)
#         - trend classification (RatingTrend)
#         - original ratings list (Ratings)
#
# Output:
#   A summary DataFrame where each driver_id has one row with these fields:
#       driver_id | RatingMean | RatingTrend | Ratings
#
# Example Interpretation:
#   - Driver with ratings [4,5,4,5] → consistent_high
#   - Driver with ratings [1,1,1,2] → consistent_low
#   - Driver with ratings [1,2,3,4] → improving
#   - Driver with ratings [4,3,2,2] → degrading
# ------------------------------------------------------------------------------

from sklearn.linear_model import LinearRegression
def classify_driver_rating(subdf):
    X = np.arange(len(subdf)).reshape(-1, 1)
    y = subdf['Quarterly Rating'].values

    model = LinearRegression().fit(X, y)
    slope = model.coef_[0]
    intercept = model.intercept_
    mean_rating = y.mean()

    threshold = 0.5

    if abs(slope) < threshold:
        if mean_rating >= 4:
            trend = 'consistent_high'
            trendCode=5
        elif mean_rating <= 2:
            trend = 'consistent_low'
            trendCode=1
        else:
            trend = 'stable'
            trendCode=3
    elif slope > threshold:
        trend = 'improving'
        trendCode=4
    else:
        trend = 'degrading'
        trendCode=2

    y_pred = model.predict(X)

    return pd.Series({
        'RatingMean': mean_rating,
        'RatingTrend': trend,
```

```
        'RatingTrendCode': trendCode,
        'Ratings': [y]
    })

# Apply classification per driver
driver_rating = df_original.groupby('Driver_ID').apply(classify_driver_rating, include_groups=False).reset_index()
```

In [529…

```
driver_rating
```

Out[529…

	Driver_ID	RatingMean	RatingTrend	RatingTrendCode	Ratings
0	1	2.000000	consistent_low	1	[[2, 2, 2]]
1	2	1.000000	consistent_low	1	[[1, 1]]
2	4	1.000000	consistent_low	1	[[1, 1, 1, 1, 1]]
3	5	1.000000	consistent_low	1	[[1, 1, 1]]
4	6	1.600000	consistent_low	1	[[1, 1, 2, 2, 2]]
...	...	...	...	...	...
2376	2784	2.625000	stable	3	[[3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 3, 3, 3, 3,...
2377	2785	1.000000	consistent_low	1	[[1, 1, 1]]
2378	2786	1.666667	consistent_low	1	[[2, 2, 2, 2, 2, 2, 1, 1, 1]]
2379	2787	1.500000	consistent_low	1	[[2, 2, 2, 1, 1, 1]]
2380	2788	2.285714	stable	3	[[1, 3, 3, 3, 2, 2, 2]]

2381 rows × 5 columns

In [530…

```
drivers
```

Out[530…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2	0.0	57387	0	0
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1	0.0	67016	0	0
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1	0.0	65603	0	0
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1	0.0	46368	0	0
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2	0.0	78728	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4	0.0	82815	0	1
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1	0.0	12105	0	0
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1	0.0	35370	0	0
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1	0.0	69498	0	0
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2	0.0	70254	0	0

2381 rows × 15 columns

In [531…

```
# Merge driver-level rating trends into the master 'drivers' table
# This adds summarized performance metrics — mean rating, trend classification,
# and historical rating list — for each driver based on their quarterly ratings.
drivers = drivers.merge(driver_rating, on='Driver_ID', how='left')
drivers
```

Out[531…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	Ratir
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2	0.0	57387	0	0	2.
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1	0.0	67016	0	0	1.
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1	0.0	65603	0	0	1.
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1	0.0	46368	0	0	1.
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2	0.0	78728	0	0	1.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4	0.0	82815	0	1	2.
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1	0.0	12105	0	0	1.
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1	0.0	35370	0	0	1.
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1	0.0	69498	0	0	1.
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2	0.0	70254	0	0	2.

2381 rows × 19 columns

2.7.5 Adding Churn

In [532…

```
# Create churn: 1 = left (has a real LastWorkingDate), 0 = still with company (placeholder 2067-12-31)
dummyDate = pd.Timestamp('2067-12-31')
drivers['Churn'] = (drivers['LastWorkingDate'] != dummyDate).astype(int)
drivers
```

Out[532…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	Ratir
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	2	0.0	57387	0	0	2.
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	1	0.0	67016	0	0	1.
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	1	0.0	65603	0	0	1.
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	1	0.0	46368	0	0	1.
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	2	0.0	78728	0	0	1.
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	4	0.0	82815	0	1	2.
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	1	0.0	12105	0	0	1.
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	1	0.0	35370	0	0	1.
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	1	0.0	69498	0	0	1.
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	2	0.0	70254	0	0	2.

2381 rows × 20 columns

2.7.6 Adding EmploymentAge in months

In [533…

```
# Calculate credit_history_age in months (issue_d - earliest_cr_line)
average_days_per_month = 365.25/12

#Lets use the maximum value in MMM-YY as the date on which we calculate the employment age.
maxDate = max(df_original['MMM-YY']) #Timestamp('2020-12-01 00:00:00')

def calculateEmploymentAge(row):
    if row.Churn==1:
        return np.floor((row.LastWorkingDate - row.Dateofjoining).days/average_days_per_month)
    else:
        return np.floor((maxDate - row.Dateofjoining).days/average_days_per_month)


drivers['EmploymentAge'] = drivers.apply(calculateEmploymentAge, axis=1)
drivers
```

Out[533…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	RatingMean
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	...	0.0	57387	0	0	2.000000
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	...	0.0	67016	0	0	1.000000
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	...	0.0	65603	0	0	1.000000
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	...	0.0	46368	0	0	1.000000
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	...	0.0	78728	0	0	1.600000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	...	0.0	82815	0	1	2.625000
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	...	0.0	12105	0	0	1.000000
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	...	0.0	35370	0	0	1.666667
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	...	0.0	69498	0	0	1.500000
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	...	0.0	70254	0	0	2.285714

2381 rows × 21 columns

In [534…

```
#creating a dataframe with categorical columns of gender and education level replaced with actual values, this df we will use during visualization for easier interpre

driversWithCategories = drivers.replace({
    'Gender': {0: 'Male', 1: 'Female'},
    'Education_Level': {0: '10+', 1: '12+', 2: 'Graduate'},
    'Churn': {0: 'NotChurned', 1: 'Churned'},
}).copy()
driversWithCategories
```

Out[534…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	RatingMean
0	1	28.0	Male	C23	Graduate	2018-12-24	2019-11-03	1	1	1715580	...	0.0	57387	0	0	2.000000
1	2	31.0	Male	C7	Graduate	2020-06-11	2067-12-31	2	2	0	...	0.0	67016	0	0	1.000000
2	4	43.0	Male	C13	Graduate	2019-07-12	2020-04-27	2	2	350000	...	0.0	65603	0	0	1.000000
3	5	29.0	Male	C9	10+	2019-09-01	2019-07-03	1	1	120360	...	0.0	46368	0	0	1.000000
4	6	31.0	Female	C11	12+	2020-07-31	2067-12-31	3	3	1265000	...	0.0	78728	0	0	1.600000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	Male	C24	10+	2015-10-15	2067-12-31	2	3	21748820	...	0.0	82815	0	1	2.625000
2377	2785	34.0	Female	C9	10+	2020-08-28	2020-10-28	1	1	0	...	0.0	12105	0	0	1.000000
2378	2786	45.0	Male	C19	10+	2018-07-31	2019-09-22	2	2	2815090	...	0.0	35370	0	0	1.666667
2379	2787	28.0	Female	C20	Graduate	2018-07-21	2019-06-20	1	1	977830	...	0.0	69498	0	0	1.500000
2380	2788	30.0	Male	C27	Graduate	2020-08-06	2067-12-31	2	2	2298240	...	0.0	70254	0	0	2.285714

2381 rows × 21 columns

3.0 Explore Data Characteristics

In [535…

```
'''
Notes to self
By exploring the characteristics of your information very well, you can gain treasured insights into its structure, pick out capability problems or anomalies, and inf
Documenting any findings or observations from this step is critical, as they may be relevant for destiny reference or communication with stakeholders.
Let's start by exploring the data according to the dataset.

for categorical columns, use
-------------------------------
sns.countplot(data=df, x='Product')
df.Product.value_counts().plot.bar()
df['Product'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90, colors=sns.color_palette('pastel'))



for numerical columns use
```

```
---------------------------
df.hist() # Quickly create histograms for all numeric columns
sns.histplot(data=df["Age"], kde=True)
df.Age.plot.hist() #hist for Specific column
df.Age.plot.kde()
df.Age.plot.box()

df.plot.box()#Print boxplots of all numerical columns in one go

sns.violinplot(data=df , x='Age')




'''
```

Out[535…
```
' \nNotes to self\nBy exploring the characteristics of your information very well, you can gain treasured insights into its structure, pick out capability problems or anomalies, and inform your subsequent evaluation and modeling choices. \nDocumenting any findings or observations from this step is critical, as they may be relevant for destiny reference or communication with stakeholders.\nLet\'s start by exploring the data according to the dataset.\n\nfor categorical columns, use\n----------\nsns.countplot(data=df, x=\'Product\')\ndf.Product.value_counts().plot.bar()\ndf[\'Product\'].value_counts().plot.pie(autopct=\'%1.1f%%\', startangle=90, colors=sns.color_palette(\'pastel\'))\n\n\n\nfor numerical columns use\n----------\ndf.hist() # Quickly create histograms for all numeric columns\nsns.histplot(data=df["Age"], kde=True)\ndf.Age.plot.hist() #hist for Specific column\ndf.Age.plot.kde()\ndf.Age.plot.box()\n\ndf.plot.box()#Print boxplots of all numerical columns in one go\n\nsns.violinplot(data=df , x=\'Age\')\n\n\n\n\n'
```

In [536…
```
df_original.columns
```

Out[536…
```
Index(['MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Income', 'Dateofjoining', 'LastWorkingDate', 'Joining Designation',
       'Grade', 'Total Business Value', 'Quarterly Rating'],
      dtype='object')
```

In [537…
```
drivers.columns
```

Out[537…
```
Index(['Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Dateofjoining', 'LastWorkingDate', 'Joining Designation', 'Grade',
       'Total Business Value', 'Quarterly Rating', 'IncomeCAGR', 'IncomeLast',
       'IncomeIncreased', 'GradeIncreased', 'RatingMean', 'RatingTrend',
       'RatingTrendCode', 'Ratings', 'Churn', 'EmploymentAge'],
      dtype='object')
```

In [538…
```
drivers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2381 entries, 0 to 2380
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Driver_ID             2381 non-null   int64
 1   Age                   2381 non-null   float64
 2   Gender                2381 non-null   float64
 3   City                  2381 non-null   object
 4   Education_Level       2381 non-null   int64
 5   Dateofjoining         2381 non-null   datetime64[ns]
 6   LastWorkingDate       2381 non-null   datetime64[ns]
 7   Joining Designation   2381 non-null   int64
 8   Grade                 2381 non-null   int64
 9   Total Business Value  2381 non-null   int64
 10  Quarterly Rating      2381 non-null   int64
 11  IncomeCAGR            2381 non-null   float64
 12  IncomeLast            2381 non-null   int64
 13  IncomeIncreased       2381 non-null   int64
 14  GradeIncreased        2381 non-null   int64
 15  RatingMean            2381 non-null   float64
 16  RatingTrend           2381 non-null   object
 17  RatingTrendCode       2381 non-null   int64
 18  Ratings               2381 non-null   object
 19  Churn                 2381 non-null   int64
 20  EmploymentAge         2381 non-null   float64
dtypes: datetime64[ns](2), float64(5), int64(11), object(3)
memory usage: 390.8+ KB
```

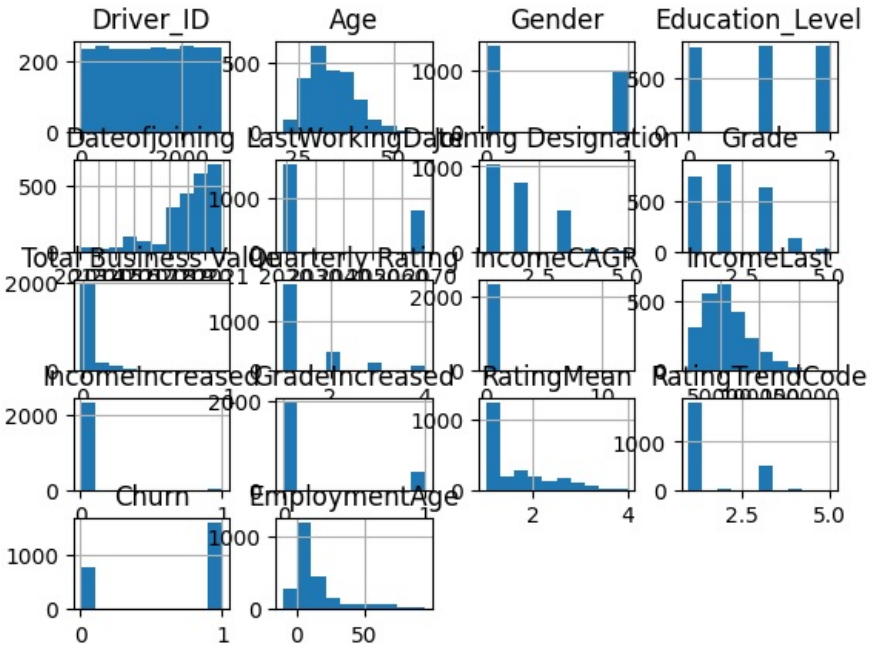
3.1 Univariate Analysis (Analysis of one attribute at a time.)

In [539…

```
#Lets first quickly plot all variables' histograms
drivers.hist()
```

Out[539…

```
array([[<Axes: title={'center': 'Driver_ID'}>,
        <Axes: title={'center': 'Age'}>,
        <Axes: title={'center': 'Gender'}>,
        <Axes: title={'center': 'Education_Level'}>],
       [<Axes: title={'center': 'Dateofjoining'}>,
        <Axes: title={'center': 'LastWorkingDate'}>,
        <Axes: title={'center': 'Joining Designation'}>,
        <Axes: title={'center': 'Grade'}>],
       [<Axes: title={'center': 'Total Business Value'}>,
        <Axes: title={'center': 'Quarterly Rating'}>,
        <Axes: title={'center': 'IncomeCAGR'}>,
        <Axes: title={'center': 'IncomeLast'}>],
       [<Axes: title={'center': 'IncomeIncreased'}>,
        <Axes: title={'center': 'GradeIncreased'}>,
        <Axes: title={'center': 'RatingMean'}>,
        <Axes: title={'center': 'RatingTrendCode'}>],
       [<Axes: title={'center': 'Churn'}>,
        <Axes: title={'center': 'EmploymentAge'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



Helper functions

drawDetailedBoxPlot

Creating a helper function to plot a boxplot that shows detailed quartile information

In [540…

```
def drawDetailedBoxPlot(data,ax):
    boxplot = ax.boxplot(data, vert=False, patch_artist=True)

    # Calculate the five-number summary
    min_val = np.min(data)
    q1 = np.percentile(data, 25)
    median = np.median(data)
    q3 = np.percentile(data, 75)
    max_val = np.max(data)

    iqr = q3-q1
    # Calculate outlier boundaries
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr


    arrowprops0 = dict(facecolor='black', shrink=0.05,mutation_scale=10)
    arrowprops1=dict(
                        arrowstyle='->',
```

```
                    color='black',
                    lw=1.8,           # Line width
                    mutation_scale=20 # Head size
                )

    # Annotate the five-number summary on the boxplot


    ax.annotate(f'Min', xy=(min_val, 1), xytext=(min_val, 1.2),
                arrowprops=arrowprops1 , ha='center'  )

    ax.annotate(f'{min_val:.2f}', xy=(min_val, 1), xytext=(min_val, 0.87),ha='center')


    ax.annotate(f'Q1', xy=(q1, 1.07), xytext=(q1, 1.25),
                arrowprops=arrowprops1 , ha='center'  )
    ax.annotate(f'{q1:.2f}', xy=(q1, 1), xytext=(q1, 0.87),ha='center')


    ax.annotate(f'Median', xy=(median, 1.07), xytext=(median, 1.32),
                arrowprops=arrowprops1,ha='center'  )
    ax.annotate(f'{median:.2f}', xy=(median, 1), xytext=(median, 0.77),ha='center')

    ax.annotate(f'Q3', xy=(q3, 1.07), xytext=(q3, 1.25),
                arrowprops=arrowprops1 , ha='center')
    ax.annotate(f'{q3:.2f}', xy=(q3, 1), xytext=(q3, 0.87),ha='center')

    ax.annotate(f'Max', xy=(max_val, 1), xytext=(max_val, 1.2),
                arrowprops=arrowprops1 , ha='center')
    ax.annotate(f'{max_val:.2f}', xy=(max_val, 1), xytext=(max_val, 0.87),ha='center')

    # Add vertical dotted lines for outlier boundaries
    ax.axvline(lower_bound, color='red', linestyle='--', alpha=0.7, label=f'Lower Bound ({lower_bound:.1f})')
    ax.axvline(upper_bound, color='red', linestyle='--', alpha=0.7, label=f'Upper Bound ({upper_bound:.1f})')
    # Add vertical text labels along the lines
    ax.text(lower_bound, .7, 'Lower Bound', rotation=90, ha='center', va='bottom', color='black', fontsize=10)
    ax.text(upper_bound, .7, 'Upper Bound', rotation=90, ha='center', va='bottom', color='black', fontsize=10)
    return ax
```

annotate_bars

Helper function to add values on top of bars.

In [541…

```
#Helper function to add values on top of bars.
def annotate_bars(barplot,fmt='%.1f'):
    # --- Annotate the bars ---
    for container in barplot.containers:
        barplot.bar_label(
            container,
            fmt=fmt,       # Format as 1 decimal + %
            label_type='edge',  # Label outside bar; use 'center' to put inside
            fontsize=9,
            padding=2
        )
```

univariate_ordinal_analysis

Creating a helper function to plot multiple graphs in one figure for ordinal data

In [542…

```python
import pandas as pd
import matplotlib.pyplot as plt

def univariate_ordinal_analysis(series, column_name, order=None ):
    """
    Print summary statistics and plot distribution for an ordinal categorical Series.

    Parameters:
        series : pd.Series
            The ordinal categorical column.
        order : list (optional)
            Ordered list of categories (if not the Series' CategoricalDtype).
    """

    # Ensure categorical with order
    if not pd.api.types.is_categorical_dtype(series):
        series = series.astype("category")
    if order is not None:
        series = series.cat.set_categories(order, ordered=True)

    # Value counts in order
    counts = series.value_counts().reindex(series.cat.categories)
    perc = counts / counts.sum() * 100



    # Plot
    fig, ax = plt.subplots(1, 2, figsize=(10, 4))

    # Bar plot (counts)
    bars = counts.plot(kind="bar", ax=ax[0], color="skyblue")
    ax[0].set_title(f"{column_name} Counts by Category")
    ax[0].set_ylabel("Count")

    # --- Annotate each bar with count and percentage ---
    total = sum(p.get_height() for p in ax[0].patches)
    for patch in ax[0].patches:
        height = patch.get_height()
        if height > 0:
            pct = height / total * 100
            ax[0].text(
                patch.get_x() + patch.get_width() / 2,  # center of bar
                height + (total * 0.005),               # position slightly above bar
                f"{int(height)}\n({pct:.1f}%)",         # count + % in next line
                ha='center', va='bottom', fontsize=8, color='black'
            )

    # Cumulative percentage
    perc.cumsum().plot(marker="o", ax=ax[1], color="red")
    ax[1].set_title("Cumulative % by Category")
    ax[1].set_ylabel("Cumulative %")
    ax[1].set_ylim(0, 100)
    ax[1].grid(True, linestyle="--", alpha=0.5)

    plt.tight_layout()
    plt.show()
```

```
        # Print summary
    print(f"\n Summary Statistics for {column_name}")
    print("="*50)
    print(f"Mode   : {series.mode().iloc[0]}")
    print(f"Median : {series.sort_values().iloc[len(series)//2]}")
    # print("\nValue Counts (%):")
    # df_analysis=pd.DataFrame({"Count": counts, "Percent": perc.round(2)})
    # print(df_analysis)
    # df_analysis.sort_values(by="Percent", ascending=False, inplace=True)
    # print("Breakup of grades-"+", ".join([f"{idx} ({row['Percent']}%)" for idx, row in df_analysis.iterrows()]))
```

univariate_numerical_analysis

Creating a helper function to plot multiple graphs and other interesting information for a numerical column

In [543…

```
import scipy.stats as stats
import matplotlib.gridspec as gridspec
def univariate_numerical_analysis(data, column_name="Numerical Variable", binsAndLabels=None):
    """
    Perform detailed univariate analysis for a numerical variable, combining descriptive
    statistics, normality testing, and visual insights into a single 4-panel plot.
    It also gives option where we can bin the data.
    Binning adds an extra layer of insight — it transforms raw, continuous data into meaningful
    segments that reveal patterns we might otherwise miss (like which age group dominates or where
    behavior shifts occur).
    Generate 4 key graphs for univariate analysis of numerical data

    Parameters
    ----------
    data : array-like or pandas Series
        Numerical data to analyze.
    column_name : str, optional
        Name of the variable (used for titles and axis labels). Default is "Numerical Variable".
    binsAndLabels : dict, optional
        Dictionary defining custom bins and labels for grouped frequency visualization.
        Example: {"bins": [0, 30, 40, 50, float('inf')], "labels": ["<30", "30-40", "40-50", "50+"]}
        If None, a histogram with KDE is plotted.


    Example usage:
        bins = [float('-inf'),     30,             40,      50,       float('inf')]
        labels = [          'Young(20-30)', 'Established(30-40)', 'Mature(40-50)','Senior(50+)']
        univariate_numerical_analysis(drivers[col], f"{col} Data", {"bins":bins, "labels":labels})

    Description
    -----------
    Generates a 2x3 grid figure with the following components:
      1. **Distribution Plot** — Histogram with KDE (or bar chart if bins provided).
      2. **Q–Q Plot** — Tests normality visually.
      3. **Box Plot** — Displays spread, quartiles, and outliers.
      4. **Summary Panel** — Lists computed statistics and normality results.

    Computed Statistics
    -------------------
    - Mean, Standard Deviation
```

```
    - Min, Max, Quartiles (Q1, Median, Q3), IQR
    - Outlier boundaries and counts (Tukey's 1.5×IQR rule)
    - Skewness, Kurtosis
    - Shapiro–Wilk test statistic and p-value (sampled if n > 300)

    Notes
    -----
    - Automatically adjusts x-axis label rotation for long category names.
    - Prints key insights to console (e.g., central range, outlier summary).
    """
    #Calculations
    mean = np.mean(data)
    q1= np.percentile(data,25)
    q2= np.percentile(data,50)
    q3= np.percentile(data,75)
    iqr = q3-q1
    # Calculate outlier boundaries
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    #Normality test
    #For the Shapiro–Wilk normality test, the recommended maximum sample size is 5,000,
    # but the results become unreliable or overly sensitive beyond roughly 300–500 samples.
    if(len(data)>300):
        test_stat, p_value = stats.shapiro(data.sample(300))

    else:
        test_stat, p_value = stats.shapiro(data)

    # Create subplots
    fig = plt.figure(figsize=(10, 6))
    gs = gridspec.GridSpec(2, 3, figure=fig)
    fig.suptitle(f'Univariate Analysis: {column_name}', fontsize=16, fontweight='bold')

    if(binsAndLabels is None):
    # 1. Histogram with KDE
        ax1 = fig.add_subplot(gs[0, 0:2])
        ax1.hist(data, bins=30, alpha=0.7, color='skyblue', edgecolor='black', density=True)
        # Add KDE curve
        x = np.linspace(data.min(), data.max(), 100)
        kde = stats.gaussian_kde(data)
        ax1.plot(x, kde(x), 'r-', linewidth=2, label='KDE')
        ax1.set_title('Distribution (Histogram + KDE)')
        ax1.set_xlabel(column_name)
        ax1.set_ylabel('Density')
        ax1.legend()
        ax1.grid(True, alpha=0.3)
    else:
        bins = binsAndLabels['bins']
        labels = binsAndLabels['labels']
        df_analysis = pd.cut(data, bins=bins, labels=labels)
        ax1 = fig.add_subplot(gs[0, 0:2])
        df_analysis.value_counts().sort_index().plot.bar(ax1)

        # --- Annotate each bar with count and percentage ---
        total = sum(p.get_height() for p in ax1.patches)
        for patch in ax1.patches:
            height = patch.get_height()
            if height > 0:
```

```
            pct = height / total * 100
            ax1.text(
                patch.get_x() + patch.get_width() / 2,  # center of bar
                height + (total * 0.005),               # position slightly above bar
                f"{int(height)}\n({pct:.1f}%)",         # count + % in next line
                ha='center', va='bottom', fontsize=8, color='black'
            )

    ax1.set_title('Distribution using custom Bins')
    ax1.set_xlabel(column_name)
    labelTextLength = np.array([len(label) for label in labels]).sum()
    if(labelTextLength/len(labels)>10):
        degreeOfXtickRotation = 45
        ha='right'
    else:
        degreeOfXtickRotation = 0
        ha='center'
    plt.setp(ax1.get_xticklabels(), rotation=degreeOfXtickRotation, ha=ha)
    # ax1.tick_params(axis='x', labelrotation=degreeOfXtickRotation, ha=ha)   # we have space to make xticks horizontal
    ax1.set_ylabel('Count')
    ax1.legend()
    ax1.grid(True, alpha=0.3)



# Q-Q Plot (Normality Check)
ax2 = fig.add_subplot(gs[0, 2])
stats.probplot(data, dist="norm", plot=ax2)
ax2.set_title('Q-Q Plot (Normality Test)')
ax2.grid(True, alpha=0.3)

# Box Plot
ax3 = fig.add_subplot(gs[1, 0:2])
drawDetailedBoxPlot(data,ax3)

    # Clear the right subplot and add formatted text
ax4 = fig.add_subplot(gs[1, 2])
ax4.clear()
ax4.set_xlim(0, 1)
ax4.set_ylim(0, 1)
ax4.axis('off')

# Add title
ax4.annotate('Summary Statistics', xy=(0.5, 0.95),
                ha='center', va='top', fontsize=12, fontweight='bold')

#Find outliers
lower_outliers =[]
upper_outliers=[]
for x in data:
    if x < lower_bound:
        lower_outliers.append(x)
    elif x > upper_bound:
        upper_outliers.append(x)
outliers = lower_outliers + upper_outliers
# Add statistics in a formatted way
statistics = [
    ('Mean:', f"{mean:.2f}"),
```

```
        ('Std Dev:',f"{np.std(data):.2f}" ),
        ('Min Max:',f"({np.min(data):.2f}, {np.max(data):.2f})" ),
        ('25, 50, 75 Percentile:', f"{q1}, {q2:.2f}, {q3}" ),
        ('IQR:', iqr ),
        ('Outliers boundary:', f"({lower_bound},{upper_bound})" ),
        ('Outliers count:', f"{len(outliers)} out of {len(data)} ({np.round(len(outliers)*100/len(data),1)})%" ),
        ('Skewness:', f"{stats.skew(data):.2f}"),
        ('Kurtosis:', f"{stats.kurtosis(data):.2f}"),
    ]

    if p_value<0.05:
        statistics.append(("Distribution",f"NOT Normal. Shapiro pval {p_value:.3f}"))
    else:
        statistics.append(("Distribution",f"normal Shapiro pval {p_value}"))

    print(f"Mean value is {mean:.2f} with 50% of data lying between {q1} and {q3}")
    if(len(outliers)>0):
        print(f"There are {len(outliers)} outliers. {len(lower_outliers)} outliers are on lower side and {len(upper_outliers)} outliers are on upper side\n")

    y_pos = 0.85
    for label, value in statistics:
        ax4.annotate(f'{label:.<20} {value}', xy=(0.1, y_pos),
                     ha='left', va='top', fontfamily='monospace', fontsize=8)
        y_pos -= 0.08

    plt.tight_layout()
    plt.show()
```

make_normal

- If data is not normal, we can use this function.
- It automatically detects whether the numerical data can use Box–Cox or needs Yeo–Johnson, applies it, and optionally plots before–after comparisons and returns the transformed data.

In [544…

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.preprocessing import PowerTransformer

def make_normal(data, column_name="Numerical Variable", plot=True):
    """
    Apply a power transformation (Box–Cox or Yeo–Johnson) to make a numerical column
    approximately normal.

    Parameters
    ----------
    data : array-like or pandas Series
        Numerical data to transform.
    column_name : str, optional
        Name of the variable (used in plots).
    plot : bool, optional
        If True, shows before–after histograms and Q–Q plots.

    Returns
    -------
```

```
    transformed_data : numpy.ndarray
        The transformed data as a NumPy array (mean≈0, std≈1).
    transformer : sklearn.preprocessing.PowerTransformer
        The fitted transformer object (can be reused for inverse_transform).

    Notes
    -----
    - Automatically chooses:
        • Box–Cox if all values > 0
        • Yeo–Johnson if data includes zero or negative values
    - Uses Shapiro–Wilk test (n≤300) to assess normality after transformation.
    """

    data = np.asarray(data).reshape(-1, 1)

    # Choose transformation method
    if (data <= 0).any():
        method = 'yeo-johnson'
        print(f"Detected zero/negative values → using Yeo–Johnson transformation for '{column_name}'")
    else:
        method = 'box-cox'
        print(f"All values positive → using Box–Cox transformation for '{column_name}'")

    # Apply transformation
    pt = PowerTransformer(method=method, standardize=True)
    transformed = pt.fit_transform(data).flatten()

    # Shapiro–Wilk test for normality
    sample = transformed if len(transformed) <= 300 else np.random.choice(transformed, 300, replace=False)
    stat, p_value = stats.shapiro(sample)
    print(f"Shapiro–Wilk p-value after transformation: {p_value:.4f}")
    if p_value > 0.05:
        print("✅ Distribution is approximately normal.")
    else:
        print("⚠️  Distribution still deviates slightly from normality.")

    # Optional visualization
    if plot:
        fig, axes = plt.subplots(2, 2, figsize=(10, 6))
        fig.suptitle(f"Normality Transformation: {column_name}", fontsize=14, fontweight='bold')

        # Histogram before
        axes[0, 0].hist(data, bins=30, color='skyblue', edgecolor='black', density=True)
        axes[0, 0].set_title("Original Distribution")

        # Histogram after
        axes[0, 1].hist(transformed, bins=30, color='lightgreen', edgecolor='black', density=True)
        axes[0, 1].set_title("Transformed Distribution")

        # Q-Q before
        stats.probplot(data.flatten(), dist="norm", plot=axes[1, 0])
        axes[1, 0].set_title("Q–Q Plot (Before)")

        # Q-Q after
        stats.probplot(transformed, dist="norm", plot=axes[1, 1])
        axes[1, 1].set_title("Q–Q Plot (After)")

        plt.tight_layout()
```

```
        plt.show()

    return transformed, pt
```

In [545…
```
drivers.columns
```

Out[545…
```
Index(['Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Dateofjoining', 'LastWorkingDate', 'Joining Designation', 'Grade',
       'Total Business Value', 'Quarterly Rating', 'IncomeCAGR', 'IncomeLast',
       'IncomeIncreased', 'GradeIncreased', 'RatingMean', 'RatingTrend',
       'RatingTrendCode', 'Ratings', 'Churn', 'EmploymentAge'],
      dtype='object')
```

Driver_ID

In [546…
```
col = 'Driver_ID'

len(drivers[col])
```

Out[546…
```
2381
```

- We have 2381 drivers' data

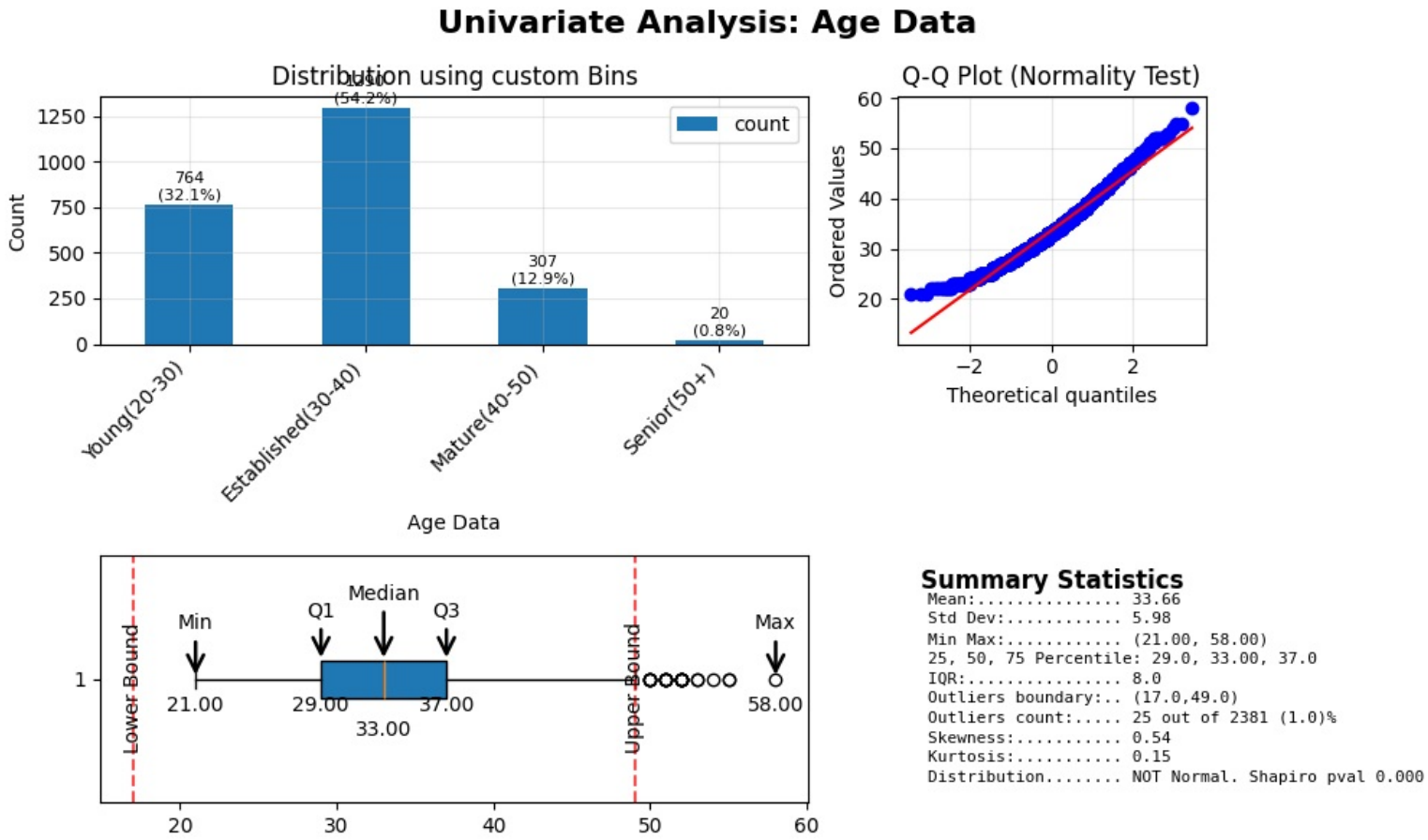
Age

We try to bin the drivers in following categories

Bin	Age Range	Label	Description
1	20–29	Young entrants	Early career, energetic but less experienced
2	30–39	Established	Peak stamina and experience
3	40–49	Mature	Reliable, possibly long-time drivers
4	50–60	Senior	Experience-rich, may prefer shorter shifts

In [547…
```
col = 'Age'

bins = [float('-inf'),     30,             40,      50,       float('inf')]
labels = [          'Young(20-30)', 'Established(30-40)', 'Mature(40-50)','Senior(50+)']

univariate_numerical_analysis(drivers[col], f"{col} Data", {"bins":bins, "labels":labels})
```

```
Mean value is 33.66 with 50% of data lying between 29.0 and 37.0
There are 25 outliers. 0 outliers are on lower side and 25 outliers are on upper side
```



Summary Statistics

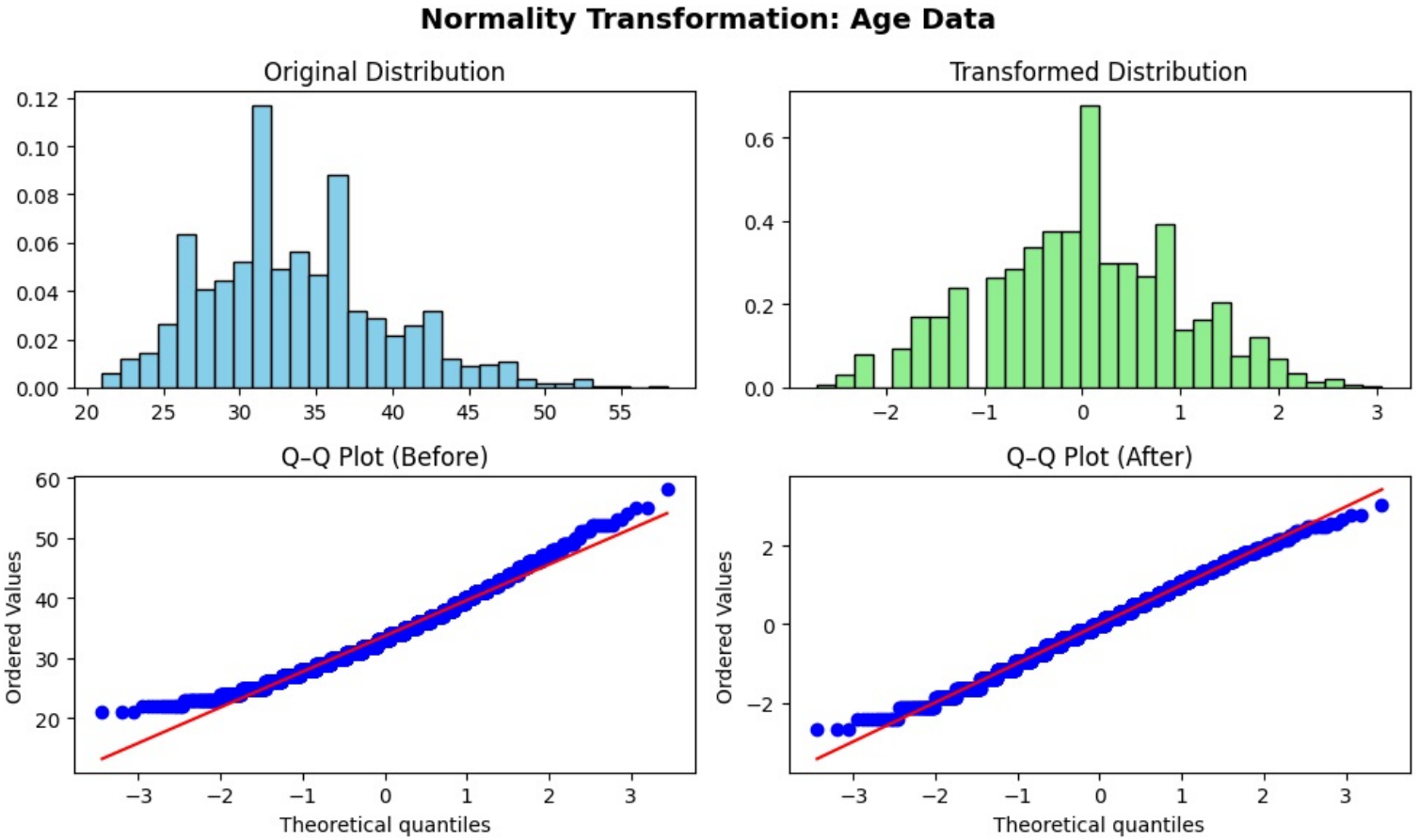
```
Mean:.............. 33.66
Std Dev:........... 5.98
Min Max:........... (21.00, 58.00)
25, 50, 75 Percentile: 29.0, 33.00, 37.0
IQR:............... 8.0
Outliers boundary:.. (17.0,49.0)
Outliers count:..... 25 out of 2381 (1.0)%
Skewness:.......... 0.54
Kurtosis:.......... 0.15
Distribution....... NOT Normal. Shapiro pval 0.000
```

- 50% of drivers are between age of 29 and 37. Mean age is 33
- 32% drivers are between age 20-30, usually energetic but less experienced
- 54% drivers are between age 30-40, can be considered to have peak stamina and experience
- 32% drivers are between age 40-50, generally reliable, possibly long-time drivers
- 32% drivers are between age 50+, Experience-rich, may prefer shorter shifts
- Youngest driver is 21 yrs and oldest is 58 yrs of age

In [548…

```
make_normal(drivers[col],f"{col} Data")
```

```
All values positive → using Box–Cox transformation for 'Age Data'
Shapiro–Wilk p-value after transformation: 0.2133
✅ Distribution is approximately normal.
```

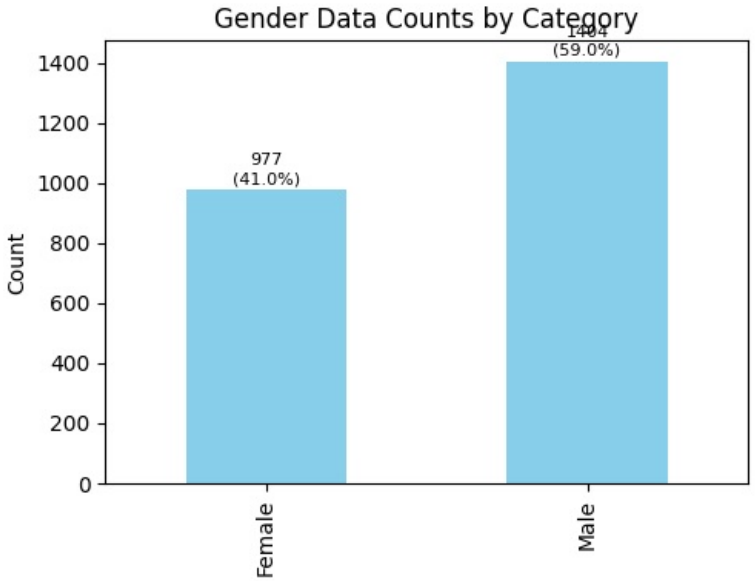


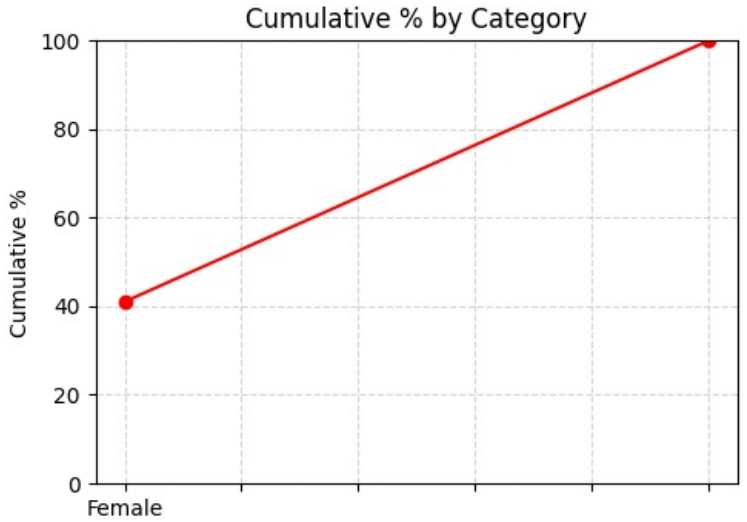
Out[548… (array([-0.96064613, -0.36789145, 1.4650352 , ..., 1.71117629,
        -0.96064613, -0.55765357], shape=(2381,)),
 PowerTransformer(method='box-cox'))

Gender

In [549…
```
col = 'Gender'

univariate_ordinal_analysis(driversWithCategories[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for Gender Data
==================================================
Mode   : Male
Median : Male
```

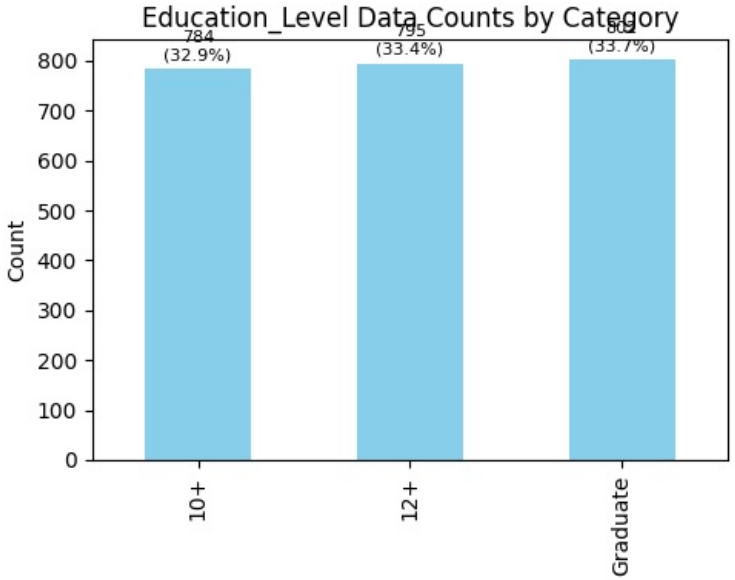
- 59% of drivers are males and 41% are females. That is an awesome ratio

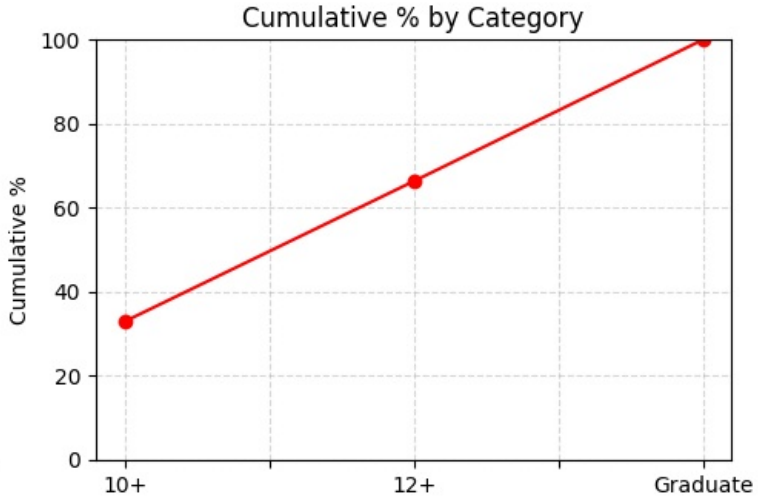
Education_Level

In [550…

```
col = 'Education_Level'

univariate_ordinal_analysis(driversWithCategories[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for Education_Level Data
==================================================
Mode   : Graduate
Median : 12+
```

- The education level stats are evenly distributed across 10+ , 12+ and graduates. Each category is roughly 33%

Dateofjoining

Lets make a separate dataframe and then add year and month columns for further analysis

In [551…

```
df_doj = drivers[['Dateofjoining']].copy()
df_doj['year'] = df_doj.Dateofjoining.dt.year
df_doj['month'] = df_doj.Dateofjoining.dt.month
# Create a 'year_month' column using dt.to_period('M') for 'Month'
df_doj['year_month'] = df_doj['Dateofjoining'].dt.to_period('M')
df_doj
```

Out[551…

	Dateofjoining	year	month	year_month
0	2018-12-24	2018	12	2018-12
1	2020-06-11	2020	6	2020-06
2	2019-07-12	2019	7	2019-07
3	2019-09-01	2019	9	2019-09
4	2020-07-31	2020	7	2020-07
...	...	...	...	...
2376	2015-10-15	2015	10	2015-10
2377	2020-08-28	2020	8	2020-08
2378	2018-07-31	2018	7	2018-07
2379	2018-07-21	2018	7	2018-07
2380	2020-08-06	2020	8	2020-08

2381 rows × 4 columns

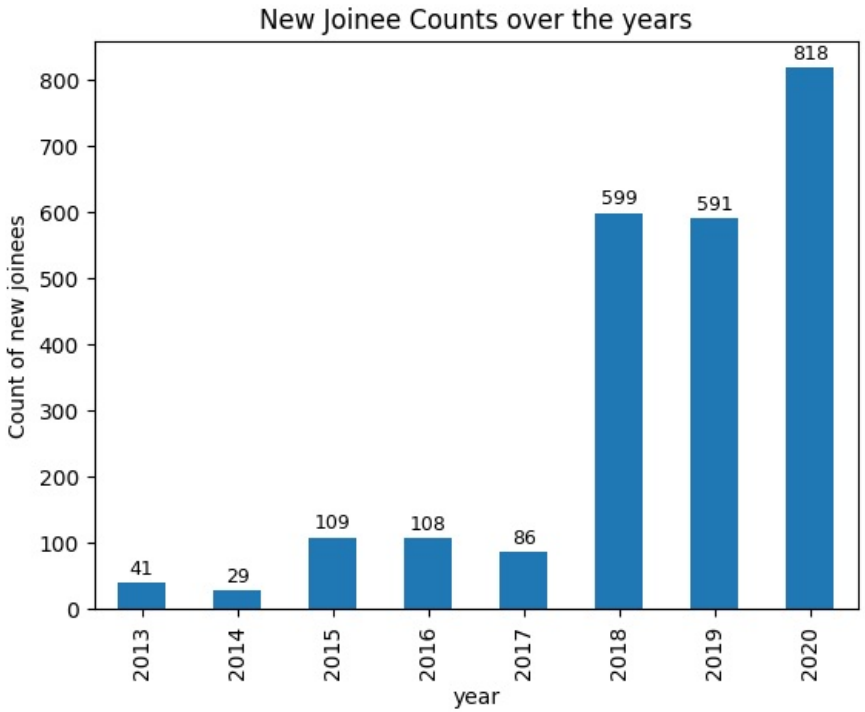
Overall Trend Analysis (Yearly Growth/Decline)

- How is the overall number of new joiners changing year after year?

In [552…

```
bars = df_doj.year.value_counts().sort_index().plot.bar()
annotate_bars(bars,'%.0f')
plt.ylabel("Count of new joinees")
plt.title("New Joinee Counts over the years")
plt.plot()
```

Out[552… []



Overall Trend Analysis (Monthly Growth/Decline)

I want to see the trend on monthly basis over the years.

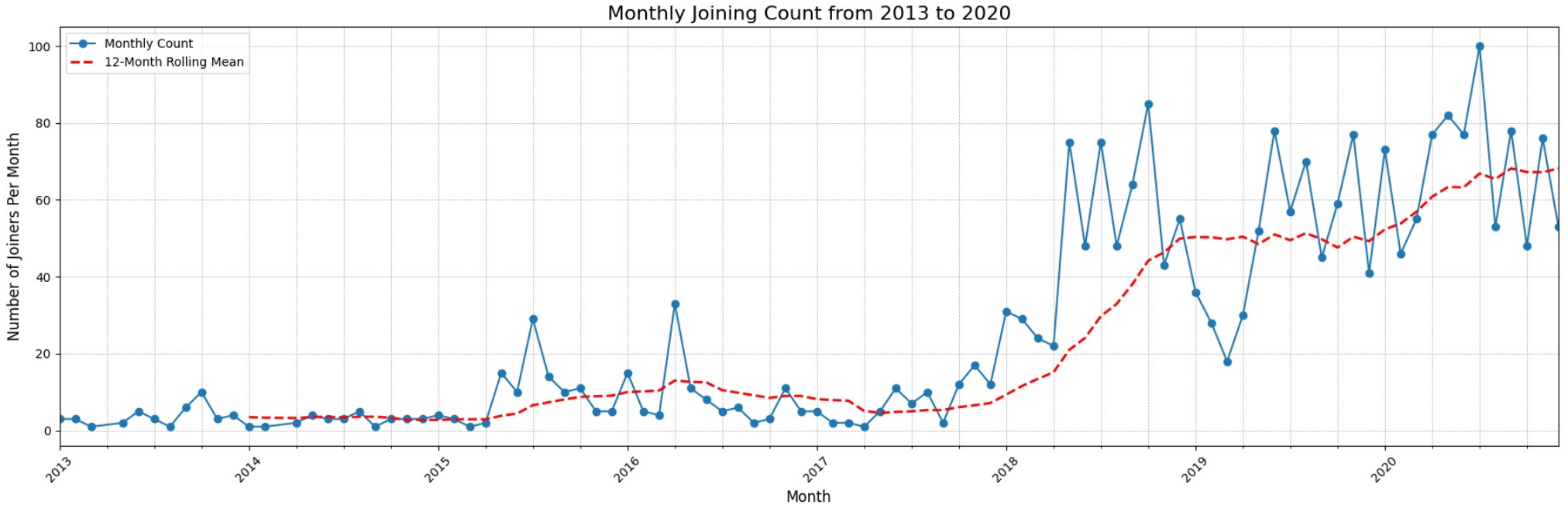
In [553…

```
# Group by this column and get the count of joiners for each month
```

```
monthly_counts_series_joinees = df_doj.groupby('year_month').size()
```

In [554…

```
plt.figure(figsize=(18, 6)) # A wider figure is needed for a long time axis

# Directly plot the series
# Pandas is smart enough to plot the PeriodIndex correctly
# Plot the monthly data
monthly_counts_series_joinees.plot(kind='line', marker='o', linestyle='-', label='Monthly Count')

# Compute and plot the n-month rolling mean
window=12
rolling_mean = monthly_counts_series_joinees.rolling(window=window).mean()
rolling_mean.plot(color='red', linewidth=2, linestyle='--', label=f'{window}-Month Rolling Mean')


# Add informative titles and labels
plt.title('Monthly Joining Count from 2013 to 2020', fontsize=16)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Number of Joiners Per Month', fontsize=12)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

# Improve readability of x-axis labels
plt.xticks(rotation=45) # Rotate labels to prevent overlap
plt.tight_layout() # Adjust plot to ensure everything fits without overlapping
plt.legend()
# Display the plot
plt.show()
```



In [555…

```
df_doj.year.value_counts().mean()
```

Out[555… np.float64(297.625)

- Hiring peaked in 2018 but has been declining since.
- On yearly basis, it is still more than pre-2018 levels though.
- On average 2388 drivers join the company every year
- Observation from the 12-month rolling mean (red dashed line) and the raw monthly counts (blue line):
  - 2013–2017:
    - Very low and irregular joining numbers.
    - Rolling mean is flat to slightly declining → stable but modest hiring.
  - Late 2017–2018:
    - A sharp inflection upward in both monthly counts and rolling mean.
    - Indicates a period of rapid growth or expansion.
  - 2019–2020:

    v
    - Despite some fluctuations, the rolling mean continues to rise, reaching its highest level by late 2020.
    - This suggests sustained and accelerating hiring activity.
  - 2020-Onwards
    - During the pandemic (2020 onward), many people lost traditional employment or faced income uncertainty.
    - The gig platforms absorbed part of that displaced workforce, leading to a visible rise in joiners even when the broader economy was weak.
    - This aligns with global data showing spikes in gig work registrations during and right after lockdown periods.

Overall summary

- Hiring in the mobility platform sector accelerated sharply after 2018 and remained high through 2020, indicating strong supply-side participation as workers sought flexible earning avenues amid pandemic-era job disruptions.

Seasonality Analysis (Which months are busiest overall?)

- We want to find out which months are consistently the busiest, regardless of the year.

In [556…
```
bars=df_doj.month.value_counts().sort_index().plot.bar()
annotate_bars(bars,'%.0f')
plt.title("Hiring trend across months")
plt.ylabel("Count of new joinees")
plt.plot()
```

Out[556… []

- July, June and May are the most prolific hiring months.
- March is lowest in terms of number of hirings

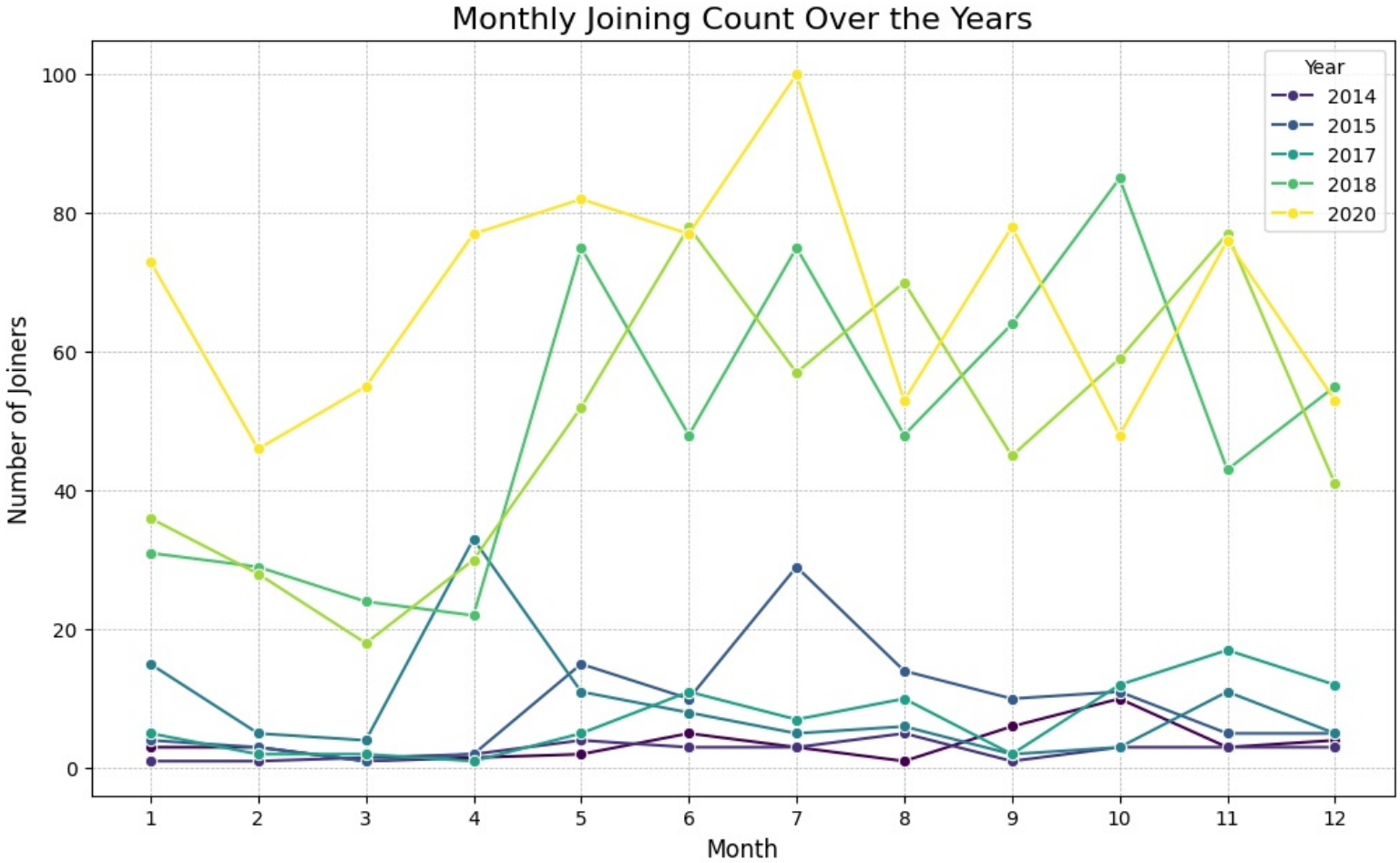
Lets see how the hiring varied across months over the years.

- We use line plot, as it is well-suited for displaying trends and comparing them across different years within the Dateofjoining column.

In [557…

```
monthly_counts = df_doj.groupby(['year', 'month']).size().reset_index(name='count')
# Set the size of the plot
plt.figure(figsize=(12, 7))

# Create the line plot using Seaborn
# x='month': Months on the x-axis
# y='count': The number of people joining on the y-axis
# hue='year': Create a separate colored line for each unique year
sns.lineplot(data=monthly_counts, x='month', y='count', hue='year', marker='o', palette='viridis')
```

```
# Add informative titles and labels
plt.title('Monthly Joining Count Over the Years', fontsize=16)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Number of Joiners', fontsize=12)
plt.xticks(range(1, 13)) # Ensure all 12 months are shown as ticks on the x-axis
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.legend(title='Year')

# Display the plot
plt.show()
```

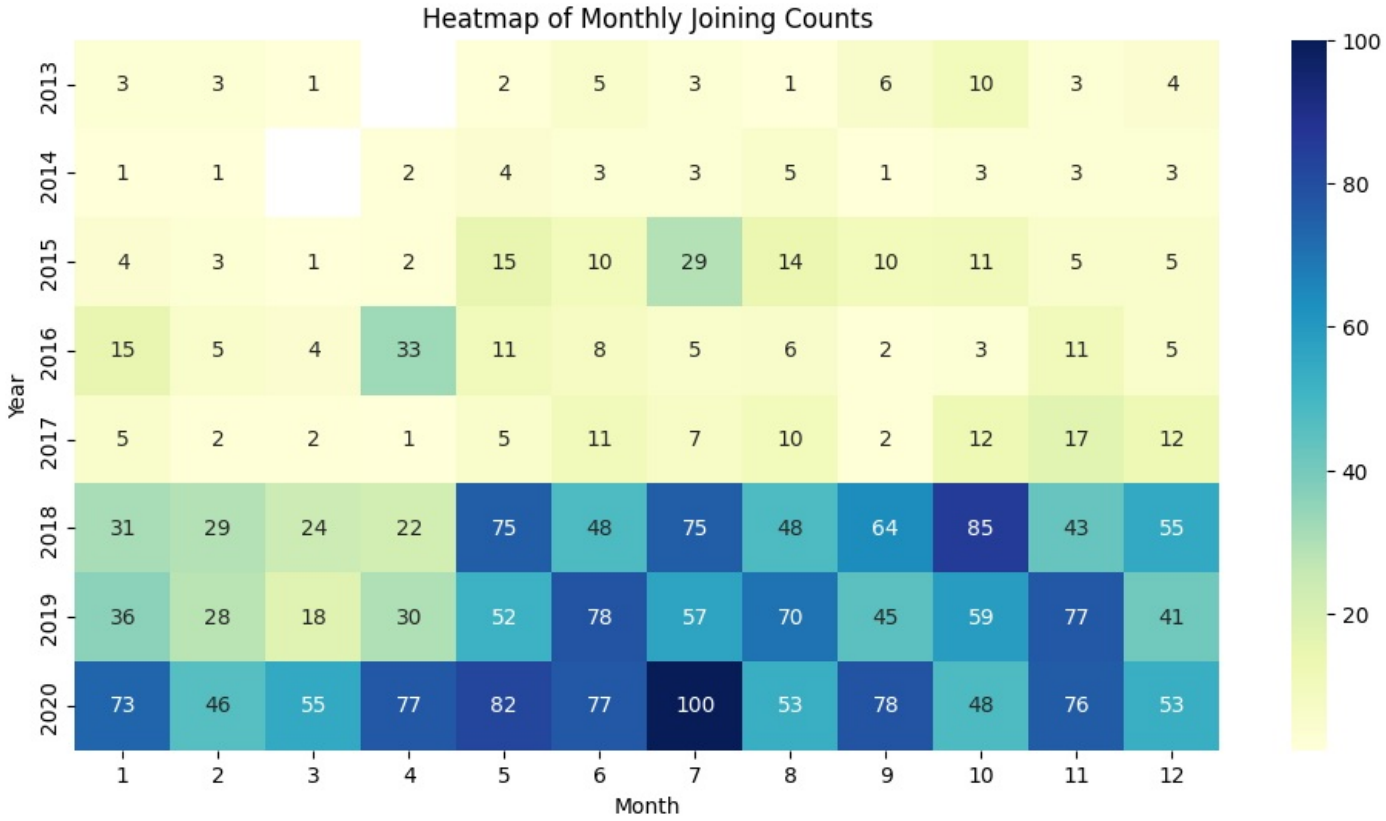


A heatmap is a fantastic way to visualize the same data

In [558…

```
# Pivot the monthly_counts table to get years as rows and months as columns
heatmap_data = monthly_counts.pivot(index='year', columns='month', values='count')

plt.figure(figsize=(12, 6))
sns.heatmap(heatmap_data, annot=True, fmt='g', cmap='YlGnBu') # fmt='g' prevents scientific notation
plt.title('Heatmap of Monthly Joining Counts')
```

```
plt.xlabel('Month')
plt.ylabel('Year')
plt.show()
```



In [559…
```
df_doj.year.value_counts()
```

Out[559…
```
year
2020    818
2018    599
2019    591
2015    109
2016    108
2017     86
2013     41
2014     29
Name: count, dtype: int64
```

- This confirms the trend that last three years have been higher in terms of hiring
- July 2020, the time just after pandemic, when most people were at home, ordering stuff and getting it delivered at doorstep saw the highest hiring for gig workers.

LastWorkingDate

In [560…
```
drivers[['LastWorkingDate','Churn']]
```

Out[560…

	LastWorkingDate	Churn
0	2019-11-03	1
1	2067-12-31	0
2	2020-04-27	1
3	2019-07-03	1
4	2067-12-31	0
...	...	...
2376	2067-12-31	0
2377	2020-10-28	1
2378	2019-09-22	1
2379	2019-06-20	1
2380	2067-12-31	0

2381 rows × 2 columns

- Since we filled the last working day value for current drivers with dummy value, we can filter those rows using the churn field
- Lets create a dataframe df_lwd with only those drivers that left (churned).

In [561…

```
df_lwd = drivers[drivers.Churn==1].LastWorkingDate.to_frame().reset_index()
df_lwd['year'] = df_lwd.LastWorkingDate.dt.year
df_lwd['month'] = df_lwd.LastWorkingDate.dt.month
df_lwd['year_month'] = df_lwd['LastWorkingDate'].dt.to_period('M')
df_lwd
```

Out[561…

	index	LastWorkingDate	year	month	year_month
0	0	2019-11-03	2019	11	2019-11
1	2	2020-04-27	2020	4	2020-04
2	3	2019-07-03	2019	7	2019-07
3	5	2020-11-15	2020	11	2020-11
4	7	2019-12-21	2019	12	2019-12
...	...	...	...	...	...
1611	2373	2020-02-14	2020	2	2020-02
1612	2375	2019-08-16	2019	8	2019-08
1613	2377	2020-10-28	2020	10	2020-10
1614	2378	2019-09-22	2019	9	2019-09
1615	2379	2019-06-20	2019	6	2019-06

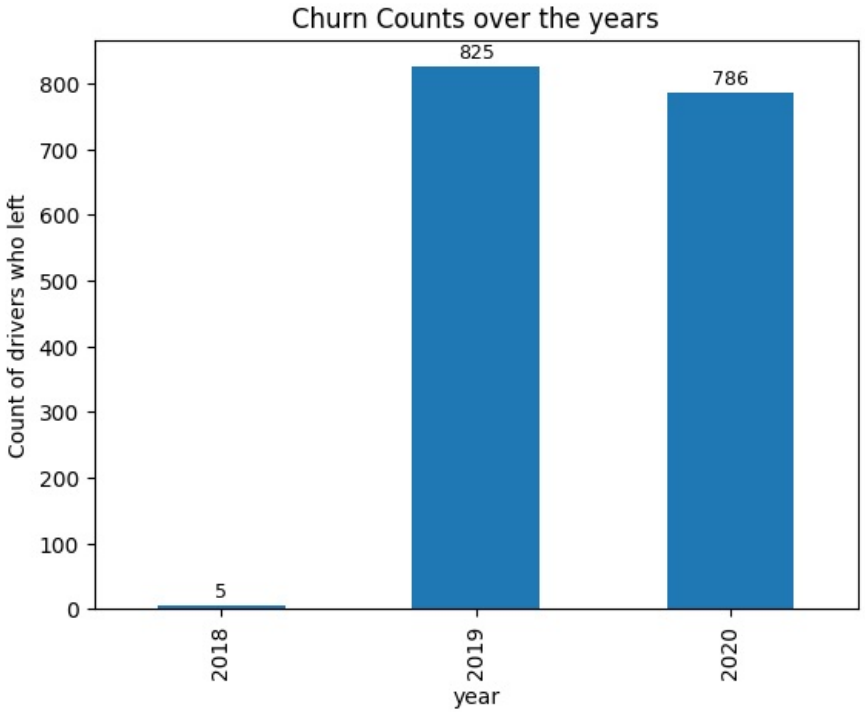
1616 rows × 5 columns

Overall Trend Analysis (Monthly Growth/Decline)

I want to see the trend on monthly basis over the years.

In [562…
```
bars=df_lwd.year.value_counts().sort_index().plot.bar()
annotate_bars(bars,'%.0f')
plt.ylabel("Count of drivers who left")
plt.title("Churn Counts over the years")
plt.plot()
```

Out[562… []



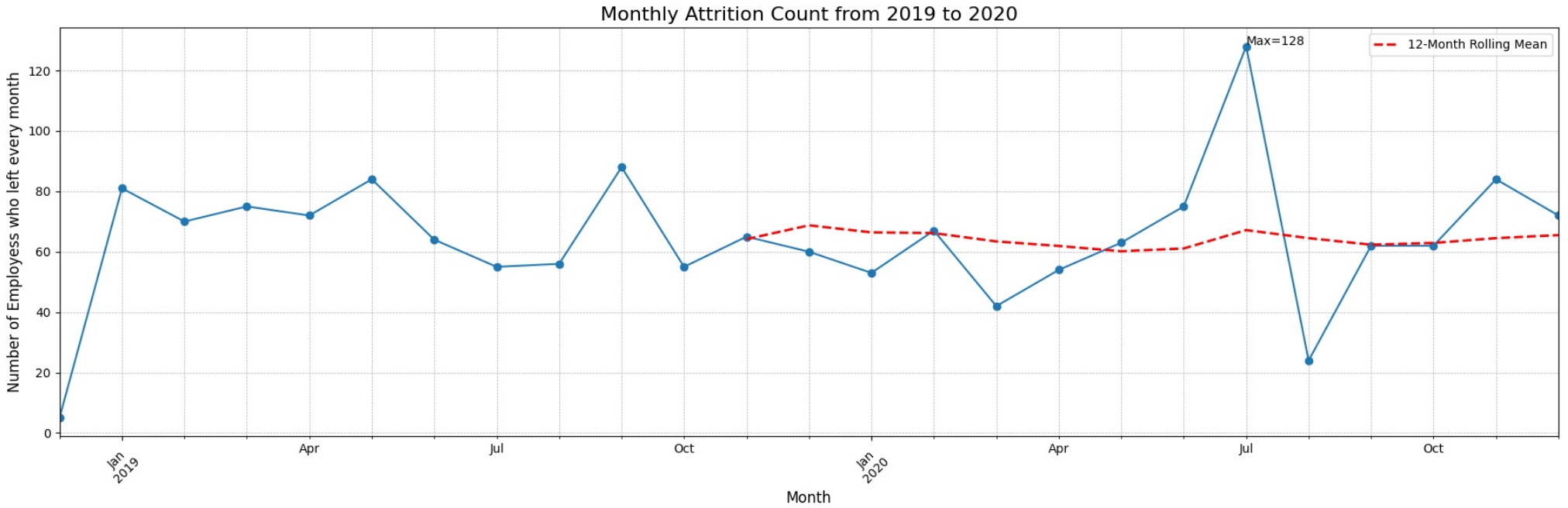
- 780+ people left the organization in each of the past 2 years (2019 and 2020)

Overall Trend Analysis (Monthly Growth/Decline)

I want to see the trend on monthly basis over the years.

In [563…
```
# Group by this column and get the count of joiners for each month
monthly_counts_series_churn = df_lwd.groupby('year_month').size()
plt.figure(figsize=(18, 6)) # A wider figure is needed for a long time axis

# Directly plot the series
# Pandas is smart enough to plot the PeriodIndex correctly
monthly_counts_series_churn.plot(kind='line', marker='o', linestyle='-')

# Compute and plot the n-month rolling mean
window=12
```

```
rolling_mean = monthly_counts_series_churn.rolling(window=window).mean()
rolling_mean.plot(color='red', linewidth=2, linestyle='--', label=f'{window}-Month Rolling Mean')


#Print max value
plt.text(x=monthly_counts_series_churn.index[monthly_counts_series_churn.argmax()],
         y=monthly_counts_series_churn.max()+0.5,

         s=f'Max={monthly_counts_series_churn.max()}')

# Add informative titles and labels
plt.title('Monthly Attrition Count from 2019 to 2020', fontsize=16)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Number of Employess who left every month', fontsize=12)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

# Improve readability of x-axis labels
plt.xticks(rotation=45) # Rotate labels to prevent overlap
plt.tight_layout() # Adjust plot to ensure everything fits without overlapping
plt.legend()
# Display the plot
plt.show()
```



- Mean monthly atttrition is 64 drivers, with maximum at 128 in Jul 2020. So this means both the hiring count (100) and churn count (128) was maximum in July 2020

The joinee count and churn count seems similar. Lets investigate further to see total additions in last 2 yrs to reveal if more number of drivers joined or left the organization. This is done later in Section 3.2.26

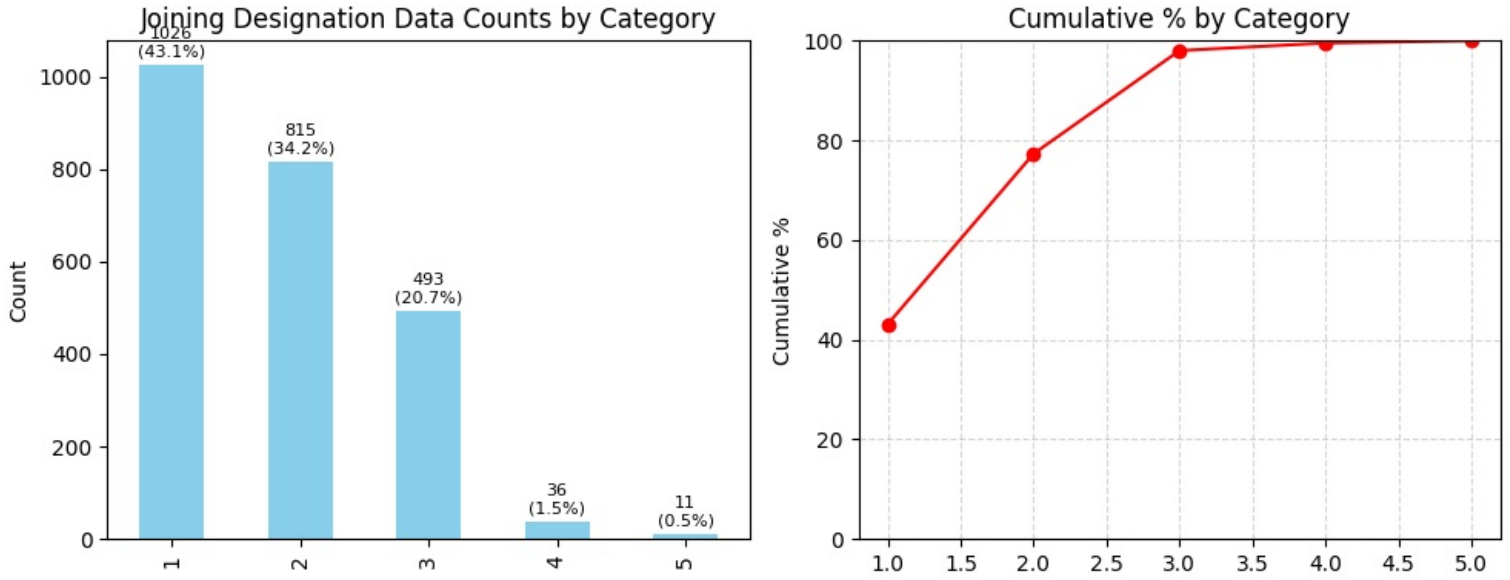
Joining Designation

In [564…

```
col = 'Joining Designation'
```

```
univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```




```
 Summary Statistics for Joining Designation Data
==================================================
Mode   : 1
Median : 2
```

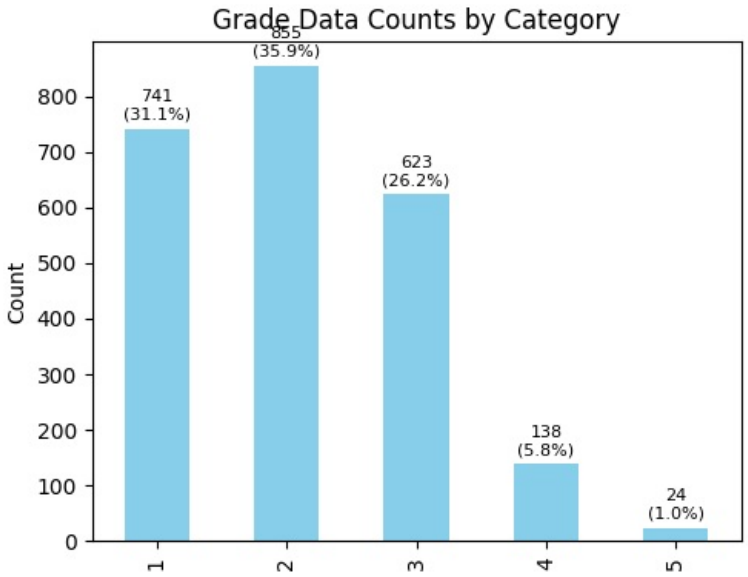
- 43.09% drivers joined at Grade 1, 34.23% at Grade 2, 20.71% at Grade 3 and only 1.51% at Grade 4 and 0.46% at Grade 5

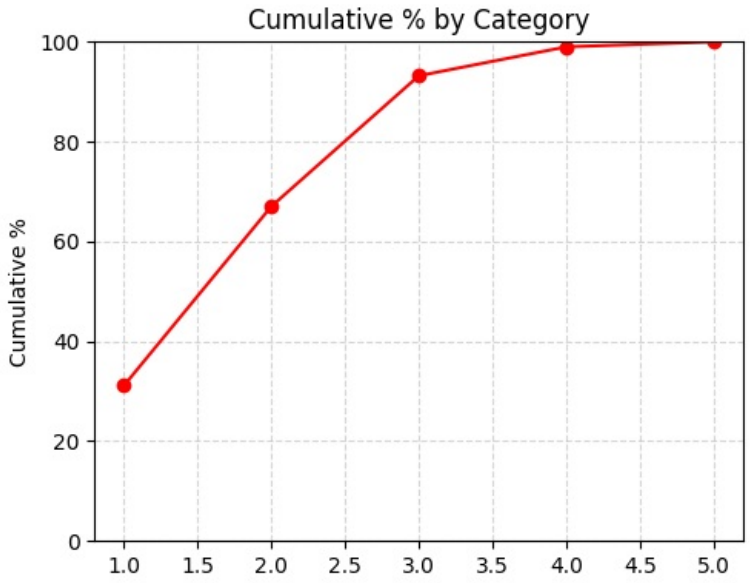
Grade

In [565…

```
col = 'Grade'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for Grade Data
==================================================
Mode   : 2
Median : 2
```

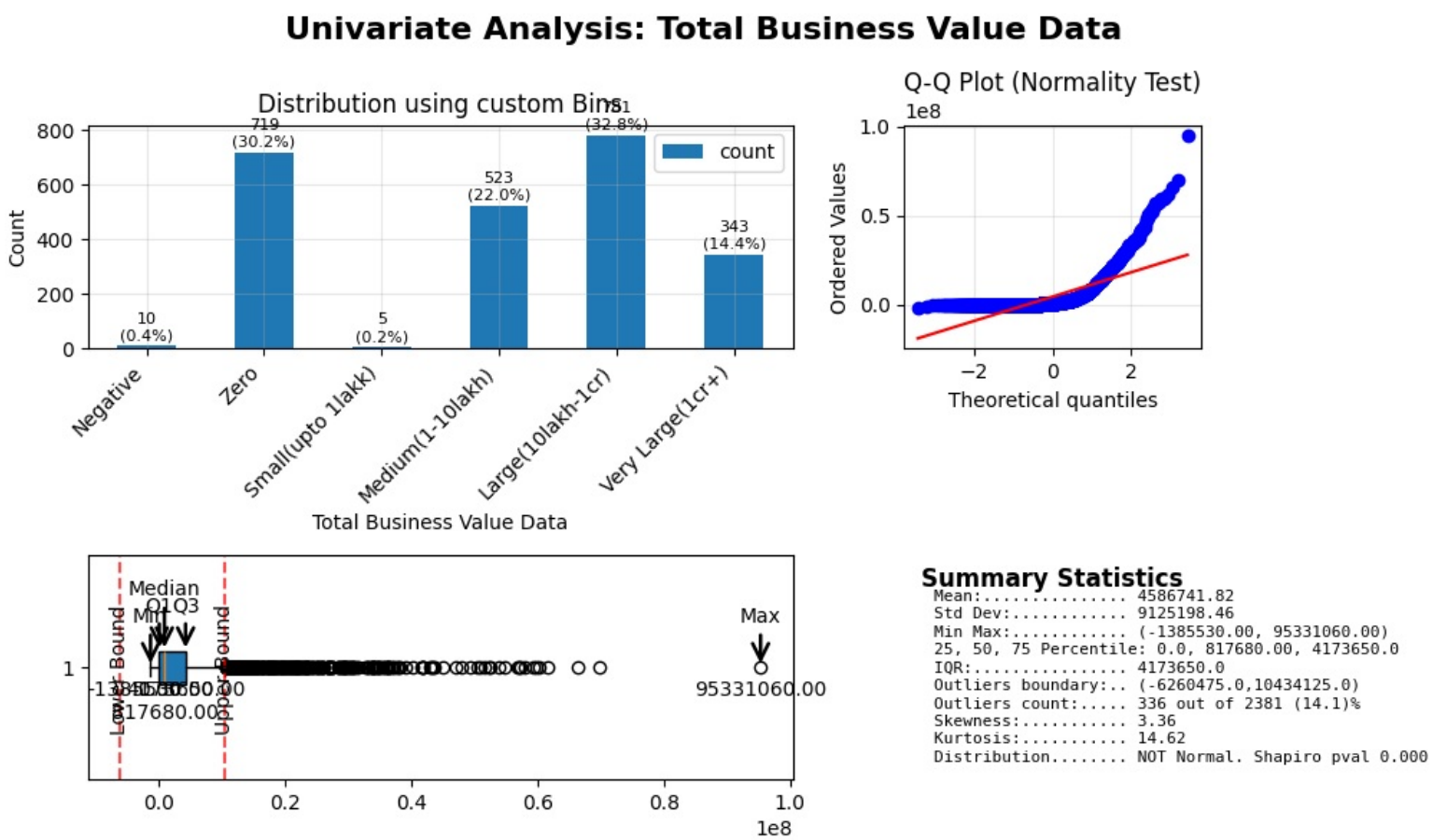
- At present, 31% of drivers are at Grade 1, 36% at Grade 2, 26% at Grade 3, 6% at Grade 4 and 1% at Grade 5

Total Business Value

In [566…

```
col = 'Total Business Value'

bins = [-np.inf,   -0.1,    0.1,   1_00_000,       10_00_000,      1_00_00_000,          np.inf]
labels = ['Negative', 'Zero','Small(upto 1lakk)',   'Medium(1-10lakh)',      'Large(10lakh-1cr)', 'Very Large(1cr+)']
univariate_numerical_analysis(drivers[col], f"{col} Data", {"bins":bins, "labels":labels})
```

```
Mean value is 4586741.82 with 50% of data lying between 0.0 and 4173650.0
There are 336 outliers. 0 outliers are on lower side and 336 outliers are on upper side
```



Summary Statistics

```
Mean:............... 4586741.82
Std Dev:............ 9125198.46
Min Max:............ (-1385530.00, 95331060.00)
25, 50, 75 Percentile: 0.0, 817680.00, 4173650.0
IQR:................ 4173650.0
Outliers boundary:.. (-6260475.0,10434125.0)
Outliers count:..... 336 out of 2381 (14.1)%
Skewness:........... 3.36
Kurtosis:........... 14.62
Distribution........ NOT Normal. Shapiro pval 0.000
```

- Mean value of cumulative total business value per driver is Rs 8.1 lakh
- 14% of drivers contributed betweeen 1 cr and 10 cr
- 54% of drivers contributed betweeen 1lakh and 1 cr
- 30% of drivers contributed zero
- 0.4% of drivers contributed negative

- We have large range of Total business value, including zero and negative values. We use np.log1p() to handle zeros safely.
- This means each value plotted on the x-axis is $\log_e(1 + x)$, where x = original “Total Business Value”.
- In X axis, actually we have log, but we will try to show actual rupee value for better interpretation

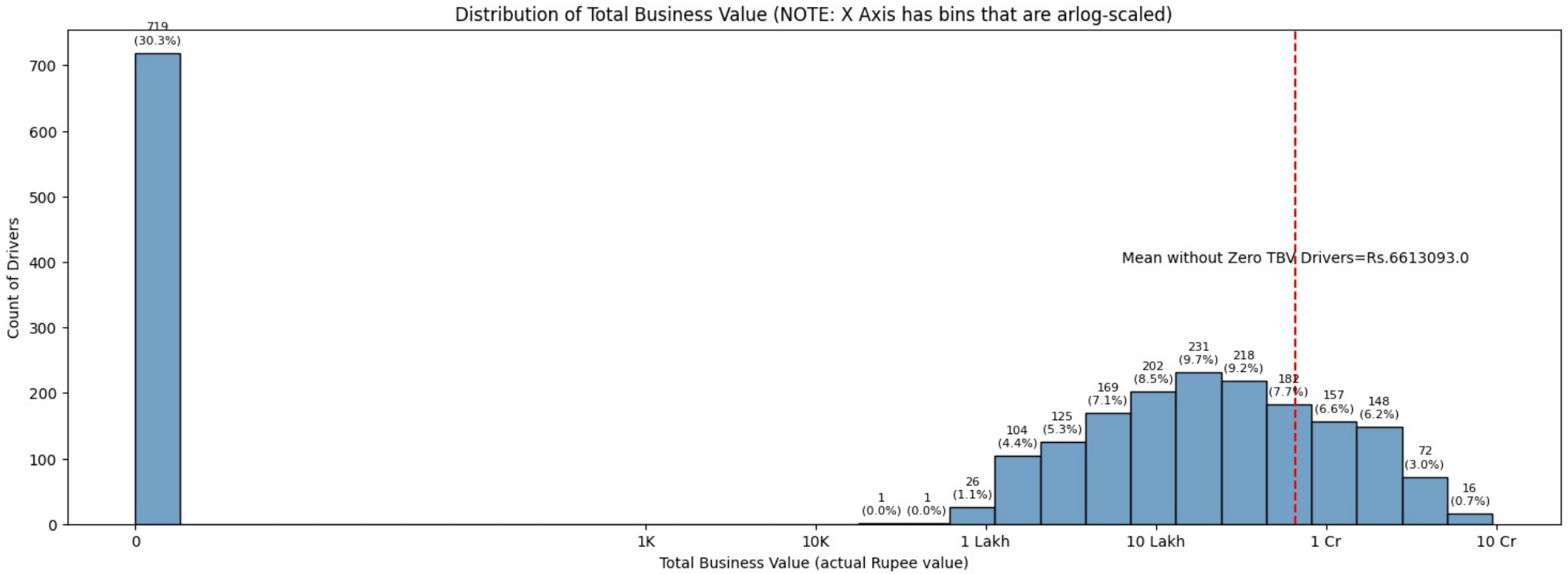
In [567…

```
#sns.histplot(np.log1p(drivers['Total Business Value']))


x = drivers['Total Business Value']
```

```
# Create log-transformed version (log(1+x))
log_vals = np.log1p(x)
plt.figure(figsize=(18, 6)) # A wider figure is needed for a long time axis
# Plot
ax=sns.histplot(log_vals, bins=30, color='steelblue', edgecolor='black')

# --- Custom tick labels ---
# Define the actual numeric values you want to show on x-axis
tick_values = [0, 1_000, 10_000, 1_00_000, 10_00_000, 1_00_00_000, 10_00_00_000]
# Convert those to their log1p equivalents for positioning
tick_positions = np.log1p(tick_values)
# Apply them
#plt.xticks(tick_positions, [f"{v:,}" for v in tick_values])
plt.xticks(tick_positions, ["0", "1K", "10K", "1 Lakh", "10 Lakh", "1 Cr", "10 Cr"])


# --- Annotate each bar with count and percentage ---
total = sum(p.get_height() for p in ax.patches)
heights=[]
for patch in ax.patches:
    height = patch.get_height()
    heights.append(height)
    if height > 0:
        pct = height / total * 100
        ax.text(
            patch.get_x() + patch.get_width() / 2,  # center of bar
            height + (total * 0.005),               # position slightly above bar
            f"{int(height)}\n({pct:.1f}%)",         # count + % in next line
            ha='center', va='bottom', fontsize=8, color='black'
        )
meanWithoutZeroTBV = x[x>0].mean()
plt.axvline(x=np.log1p(meanWithoutZeroTBV), c='r',linestyle='--')

#Print mean value
plt.text(x=np.log1p(meanWithoutZeroTBV),
         y=400,
         s=f'Mean without Zero TBV Drivers=Rs.{np.floor(meanWithoutZeroTBV)}', ha='center')
# Label axes
plt.xlabel("Total Business Value (actual Rupee value)")
plt.ylabel("Count of Drivers")
plt.title("Distribution of Total Business Value (NOTE: X Axis has bins that are arlog-scaled)")
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\pandas\core\arraylike.py:399: RuntimeWarning:

invalid value encountered in log1p
```

Out[567… Text(0.5, 1.0, 'Distribution of Total Business Value (NOTE: X Axis has bins that are arlog-scaled)')



- Log scale makes it easy to observe a very broad range of data.
- Mean TBV value after removing drivers with Zero TBV is Rs 66lakh

Categorize drivers based on TBV

In [568…

```
count= (drivers['Total Business Value'] > 1_00_00_000).sum()
print(f'Driver count above 1 cr TBV-{ count}, who comprise {count*100/len(drivers)} % ')
count= (drivers['Total Business Value'] > 10_00_000).sum()
print(f'Driver count above 10 lakh TBV-{ count}, who comprise {count*100/len(drivers)} % ')
```

```
Driver count above 1 cr TBV-343, who comprise 14.405711885762285 %
Driver count above 10 lakh TBV-1124, who comprise 47.207055858882825 %
```

Analysing positive values

- Let us see max value

- Let us see top values

In [569…
```
drivers[drivers[col] == drivers[col].max()]
```

Out[569…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	RatingMean
2153	2522	36.0	1.0	C13	1	2013-01-04	2067-12-31	2	5	95331060	...	0.0	90057	0	1	2.75

1 rows × 21 columns

- Driver with ID 2522 has a business value of 9,53,31,060 (9 crore 53 lakh)
- There are 343 drivers out 2381 (14%) that have added a total business value of 1crore+
- Interpretation of 14% - It points to a classic business phenomenon - **The Power-Law Distribution (or Pareto Principle)**
  - This is a textbook example of a power-law distribution, often related to the Pareto Principle (the "80/20 rule").
  - **What it means:** A small percentage of drivers (the "Superstars" or "Power Drivers") are generating a massively disproportionate amount of the total business value.
  - **Why it's critical:** These 14% of drivers are the most valuable asset the company has. Losing even a small number of these drivers would have a much more significant negative impact on revenue than losing a large number of the low-value or zero-value drivers.

In [570…
```
df_original[df_original.Driver_ID==2522]
```

Out[570…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
17092	2019-01-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	9830180	4
17093	2019-02-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	165730	4
17094	2019-03-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	20596160	4
17095	2019-04-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	13475510	4
17096	2019-05-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	1710980	4
17097	2019-06-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	0	4
17098	2019-07-01	2522	34.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	496420	3
17099	2019-08-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	2000000	3
17100	2019-09-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	2317970	3
17101	2019-10-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	2822560	2
17102	2019-11-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	240000	2
17103	2019-12-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	242540	2
17104	2020-01-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	151100	1
17105	2020-02-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	772670	1
17106	2020-03-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	1381500	1
17107	2020-04-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	0	2
17108	2020-05-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	1400000	2
17109	2020-06-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	2193640	2
17110	2020-07-01	2522	35.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	502320	2
17111	2020-08-01	2522	36.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	431000	2
17112	2020-09-01	2522	36.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	2157900	2
17113	2020-10-01	2522	36.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	0	4
17114	2020-11-01	2522	36.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	-1304840	4
17115	2020-12-01	2522	36.0	1.0	C13	1	90057	2013-01-04	2067-12-31	2	5	33747720	4

Analyzing Zero Values

In [571…

```
zeroTBVDrivers = drivers[drivers[col] == 0]
print (f'{len(zeroTBVDrivers)}/{len(drivers)} = {len(zeroTBVDrivers)/len(drivers)}')
```

```
719/2381 = 0.30197396052078956
```

Lets observe these rows to see if we can find out more about them

In [572…

```
zeroTBVDrivers
```

Out[572…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	RatingMean
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	...	0.0	67016	0	0	1.0
5	8	34.0	0.0	C2	0	2020-09-19	2020-11-15	3	3	0	...	0.0	70656	0	0	1.0
6	11	28.0	1.0	C19	2	2020-07-12	2067-12-31	1	1	0	...	0.0	42172	0	0	1.0
9	14	39.0	1.0	C26	0	2020-10-16	2067-12-31	3	3	0	...	0.0	19734	0	0	1.0
12	18	27.0	1.0	C17	1	2019-09-01	2019-04-30	1	1	0	...	0.0	31631	0	0	1.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2371	2776	27.0	1.0	C27	1	2019-04-01	2019-04-04	2	2	0	...	0.0	54397	0	0	1.0
2372	2778	35.0	0.0	C13	2	2020-11-29	2067-12-31	2	2	0	...	0.0	50180	0	0	1.0
2373	2779	28.0	0.0	C26	0	2020-01-26	2020-02-14	3	3	0	...	0.0	95133	0	0	1.0
2375	2782	26.0	0.0	C19	1	2019-05-16	2019-08-16	1	1	0	...	0.0	29582	0	0	1.0
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	...	0.0	12105	0	0	1.0

719 rows × 21 columns

Lets see their Gender

In [573…
```
zeroTBVDrivers.Gender.value_counts(normalize=True)
```

Out[573…
```
Gender
0.0    0.605007
1.0    0.394993
Name: proportion, dtype: float64
```

- 60% are men

Lets see their city

In [574…
```
zeroTBVDrivers.City.value_counts(normalize=True, sort=True)
```

Out[574…
```
City
C20    0.072323
C27    0.047288
C8     0.045897
C15    0.043115
C5     0.041725
C29    0.041725
C10    0.040334
C17    0.038943
C4     0.036161
C12    0.036161
C21    0.036161
C2     0.034771
C9     0.033380
C3     0.033380
C1     0.031989
C28    0.031989
C25    0.030598
C13    0.030598
C19    0.029207
C24    0.029207
C14    0.029207
C11    0.029207
C16    0.029207
C22    0.027816
C23    0.026426
C6     0.025035
C7     0.023644
C18    0.023644
C26    0.020862
Name: proportion, dtype: float64
```

In [575…
```
zeroTBVDrivers.Dateofjoining.dt.year.value_counts()
```

Out[575…
```
Dateofjoining
2020    381
2019    202
2018    124
2017      4
2016      4
2013      3
2015      1
Name: count, dtype: int64
```

In [576…
```
zeroTBVDrivers.Dateofjoining.dt.year.value_counts(normalize=True)
```

Out[576…
```
Dateofjoining
2020    0.529903
2019    0.280946
2018    0.172462
2017    0.005563
2016    0.005563
2013    0.004172
2015    0.001391
Name: proportion, dtype: float64
```

- 52% joined in 2020 (less than 12 months ago), 28% joined in 2019(less than 24 months ago), 18% joined in 2018

In [577…
```
zeroTBVDrivers.Churn.value_counts(normalize=True)
```

Out[577…
```
Churn
1    0.767733
0    0.232267
Name: proportion, dtype: float64
```

- 76% of these drivers have LEFT the organization. That explains to a large extent the zero value.

In [578…
```
pd.crosstab(zeroTBVDrivers.Dateofjoining.dt.year,zeroTBVDrivers.Churn)
```

Out[578…

Churn	0	1
Dateofjoining		
2013	0	3
2015	0	1
2016	0	4
2017	0	4
2018	0	124
2019	6	196
2020	161	220

- The crosstab data clearly confirms that these drivers have basically left the company, which explain the zero TBV value. There are 161 drivers that have recently joined but may not have started taking active rides.
- Let us analyze these 161 drivers who joined in last year and have zero TBV

In [579…
```
zeroTBVDriversLastYear = zeroTBVDrivers[(zeroTBVDrivers.Dateofjoining.dt.year==2020) & (zeroTBVDrivers.Churn==0)]
zeroTBVDriversLastYear
```

Out[579…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeCAGR	IncomeLast	IncomeIncreased	GradeIncreased	RatingMean
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	...	0.0	67016	0	0	1.0
6	11	28.0	1.0	C19	2	2020-07-12	2067-12-31	1	1	0	...	0.0	42172	0	0	1.0
9	14	39.0	1.0	C26	0	2020-10-16	2067-12-31	3	3	0	...	0.0	19734	0	0	1.0
48	62	27.0	0.0	C9	2	2020-11-28	2067-12-31	1	1	0	...	0.0	62376	0	0	1.0
52	66	27.0	1.0	C4	2	2020-01-21	2067-12-31	3	3	0	...	0.0	104286	0	0	1.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2250	2635	35.0	0.0	C20	0	2020-11-17	2067-12-31	2	2	0	...	0.0	66890	0	0	1.0
2253	2640	32.0	1.0	C29	1	2020-09-28	2067-12-31	2	2	0	...	0.0	80788	0	0	1.0
2365	2770	32.0	0.0	C26	0	2020-11-29	2067-12-31	3	3	0	...	0.0	49295	0	0	1.0
2370	2775	27.0	0.0	C9	0	2020-02-10	2067-12-31	3	3	0	...	0.0	85112	0	0	1.0
2372	2778	35.0	0.0	C13	2	2020-11-29	2067-12-31	2	2	0	...	0.0	50180	0	0	1.0

161 rows × 21 columns

In [580…
```
zeroTBVDriversLastYear.IncomeLast.mean()
```

Out[580… np.float64(58908.27329192546)

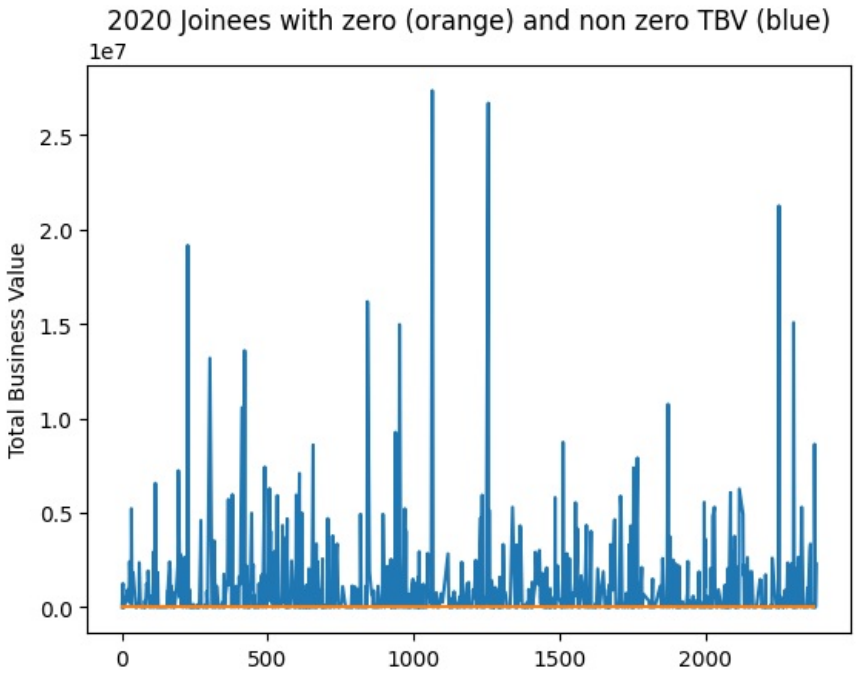
- These 161 drivers have consistently poorest rating of 1, but surprisingly, they have a positive income, averaging Rs 58900 monthly!

Let us see if there are other drivers that are also joined in 2020 but have non zero total business value.

In [581…
```
plt.plot(drivers[drivers.Dateofjoining.dt.year==2020]['Total Business Value'])
plt.plot(zeroTBVDriversLastYear['Total Business Value'])
print(len(drivers[drivers.Dateofjoining.dt.year==2020])-len(zeroTBVDriversLastYear))
plt.ylabel("Total Business Value")
plt.title("2020 Joinees with zero (orange) and non zero TBV (blue)")
plt.plot()
```

657

Out[581… []



- Indeed, there are a 657 more drivers that too joined in 2020 but have non zero total business value.
- Company can investigate if this is a simple case of missing data or what is the reason for zero business value of these 161 recenly joined drivers.

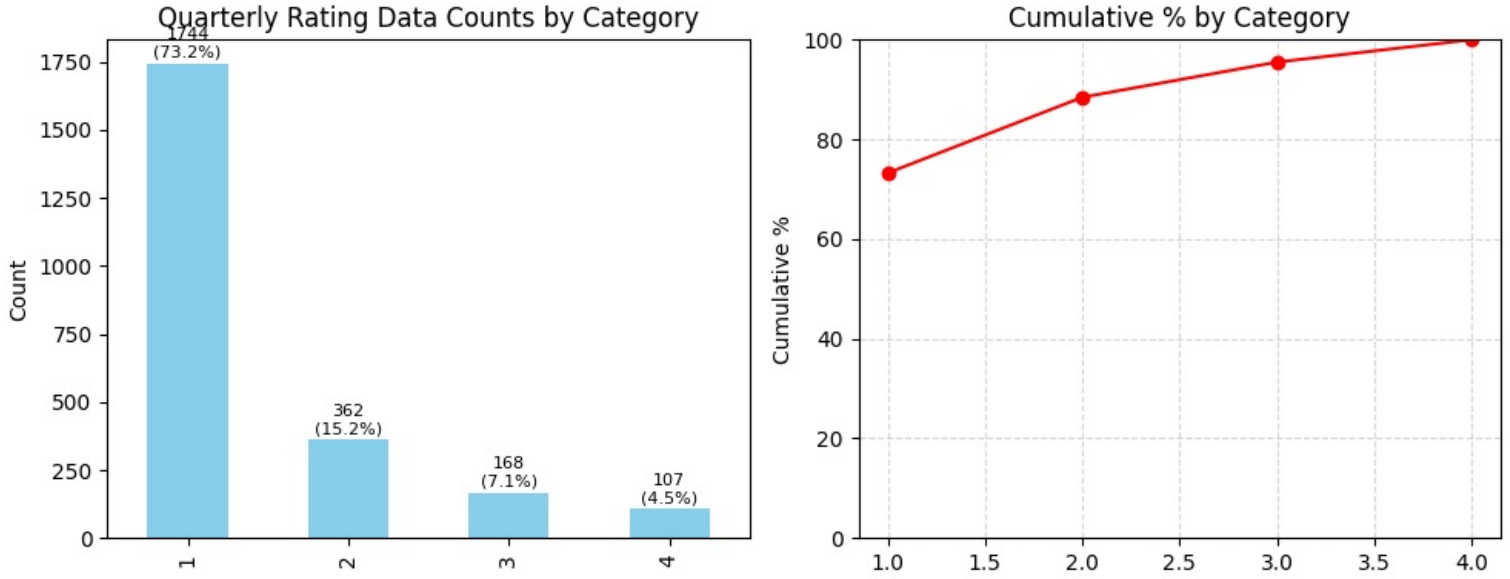
- 30% (719 of 2381) of drivers have not contributed anything to the total business value (value is zero)
- 60% of these non contributing drivers are men
- City C20 has 7% of these drivers.
- 52% (381 of 719) joined in 2020 (less than 12 months ago), 28% joined in 2019(less than 24 months ago), 18% joined in 2018
- 76% (552 of 719) of these drivers have LEFT the organization. That explains to a large extent the zero value.
- Out of the remaining 167 drivers that are still with the company, 161 drivers have recently joined (2020) but have zero total business value. We find that these drivers have non zero income, infact their avg income is Rs 58908. So surely they are serving the customers. There were a 657 more drivers that too joined in 2020 but have non zero total business value. Company can investigate if this is a simple case of missing data or what is the reason for zero business value of these 161 recenly joined drivers.

Quarterly Rating

We are looking at latest values of quarterly ratings for each driver

In [582…
```
col = 'Quarterly Rating'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```




```
 Summary Statistics for Quarterly Rating Data
==================================================
Mode   : 1
Median : 1
```

- 73% of drivers have a rating 1, 15% have rating 2, 7% have rating 3 and only 4.4% have top rating of 4

RatingMean

Lets bin the ratingmean value as it will help in the analysis.

In [583…
```
bins = [float('-inf'), 1, 2, 3, float('inf')]
labels = ['1', '2', '3', '4']

df_analysis = pd.cut(drivers['RatingMean'], bins=bins, labels=labels)
df_analysis.value_counts()
```

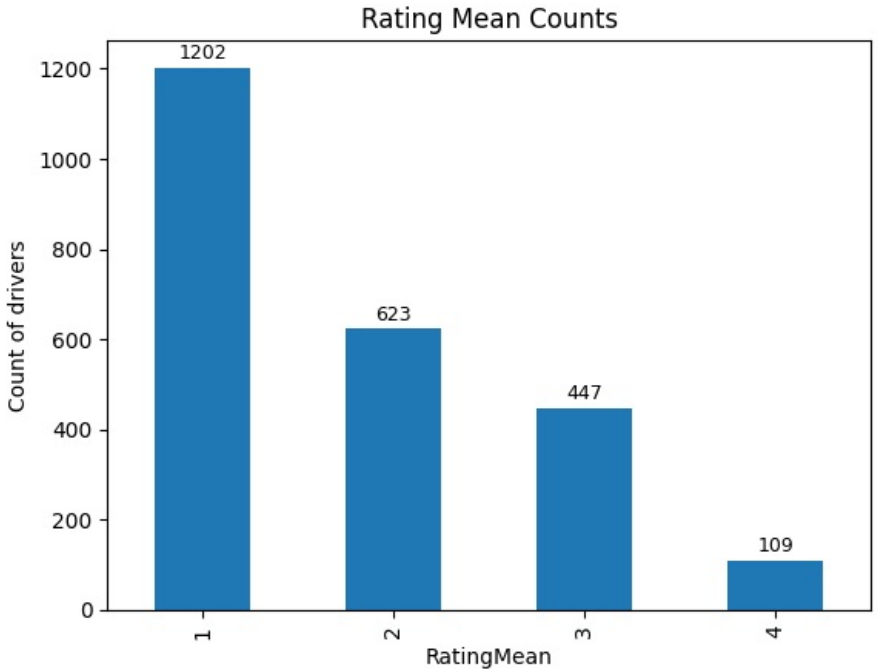
Out[583…
```
RatingMean
1    1202
2     623
3     447
4     109
Name: count, dtype: int64
```

In [584…
```
df_analysis.value_counts(normalize=True)
```

Out[584…
```
RatingMean
1    0.504830
2    0.261655
3    0.187736
4    0.045779
Name: proportion, dtype: float64
```

In [585…
```
bars=df_analysis.value_counts().plot.bar()
annotate_bars(bars,'%.0f')
plt.title("Rating Mean Counts")
plt.ylabel("Count of drivers")
plt.plot()
```

Out[585…
```
[]
```



- 50% of drivers have rating 1 or less, 26% have mean rating of 1 to 2, 18% have between 2 and 3, and 4.5% have between 3 and 4

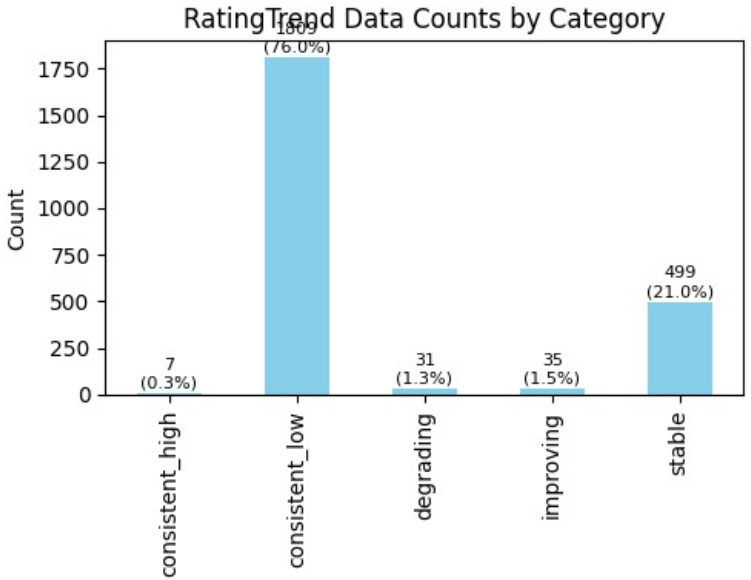
RatingTrend

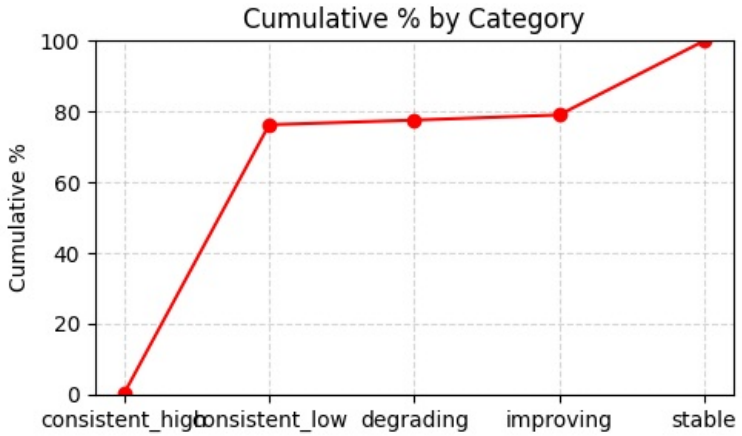
- We created a RatingTrend column during creation of Drivers dataframe and used logic to assign these values to each driver based on the data of their ratings over the months.
- This is how we can interpret the values in RatingTrend
- 'improving' → if slope > +0.5 (ratings increasing significantly)
- 'degrading' → if slope < -0.5 (ratings decreasing significantly)
- 'consistent_high' → if slope ≈ 0 and mean rating ≥ 4

- 'consistent_low' → if slope ≈ 0 and mean rating ≤ 2
- 'stable' → if slope ≈ 0 and mean rating is mid-range (2–4)

In [586…
```
col = 'RatingTrend'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```





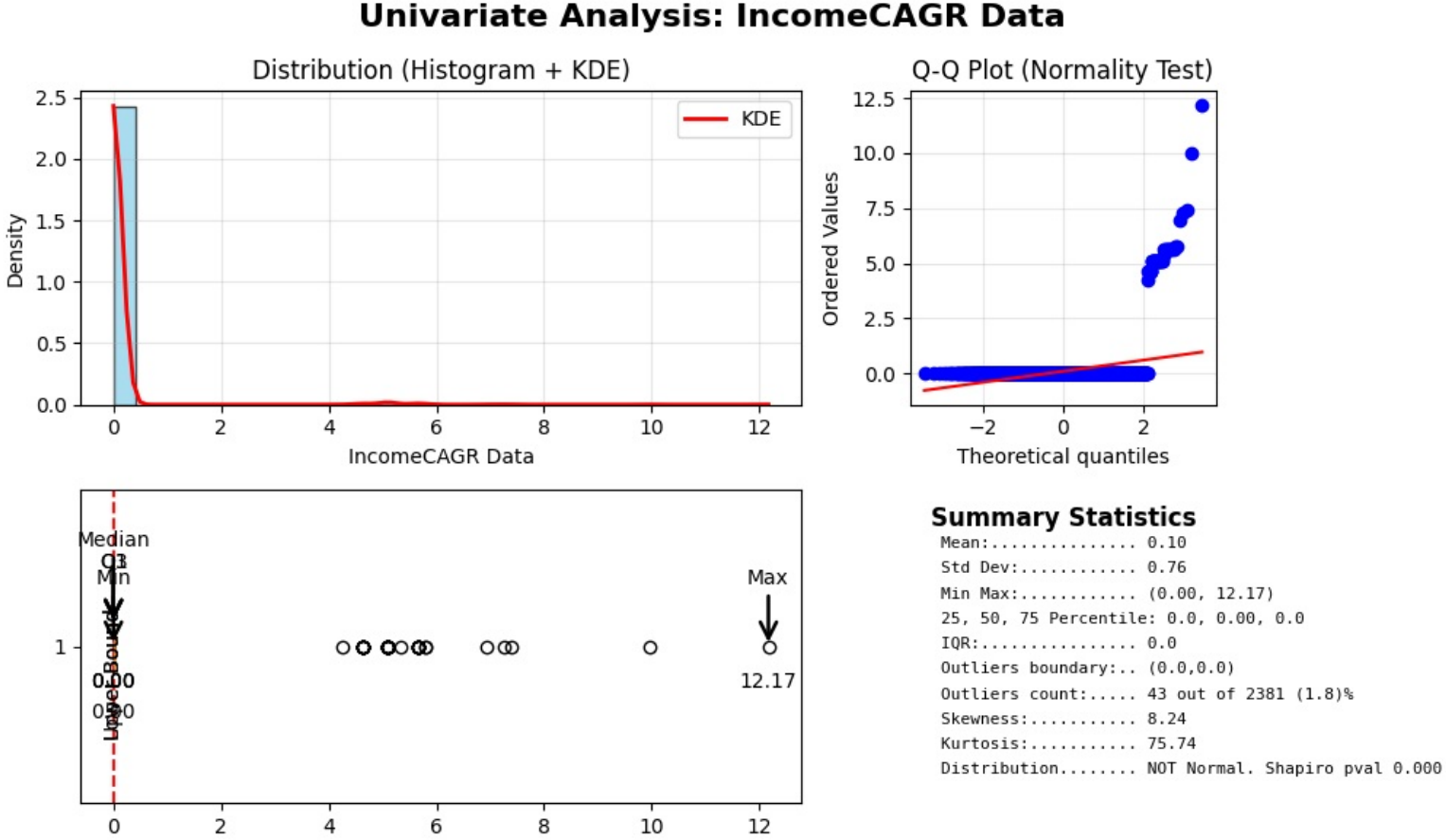

```
 Summary Statistics for RatingTrend Data
==================================================
Mode   : consistent_low
Median : consistent_low
```

- 76% of drivers have consistently low ratings over the months
- 21% have a rating that has remained around 2-3
- Only 1.47% have improved their rating over the months consistently
- Only 1.3% have degraded their rating over the months

IncomeCAGR

In [587…
```
col = 'IncomeCAGR'

univariate_numerical_analysis(drivers[col], f"{col} Data")
```

```
Mean value is 0.10 with 50% of data lying between 0.0 and 0.0
There are 43 outliers. 0 outliers are on lower side and 43 outliers are on upper side
```



The data is too skewed,most values are zero. Lets bin the data to see it more clearly

In [588…
```
bins = [float('-inf'), 0.1, 5, 10, float('inf')]
labels = ['Zero', '0-5', '5-10', '10+']

df_analysis = pd.cut(drivers['IncomeCAGR'], bins=bins, labels=labels)
df_analysis.value_counts()
```

Out[588…
```
IncomeCAGR
Zero    2338
5-10      33
0-5        9
10+        1
Name: count, dtype: int64
```

In [589…
```
df_analysis.value_counts(normalize=True)
```

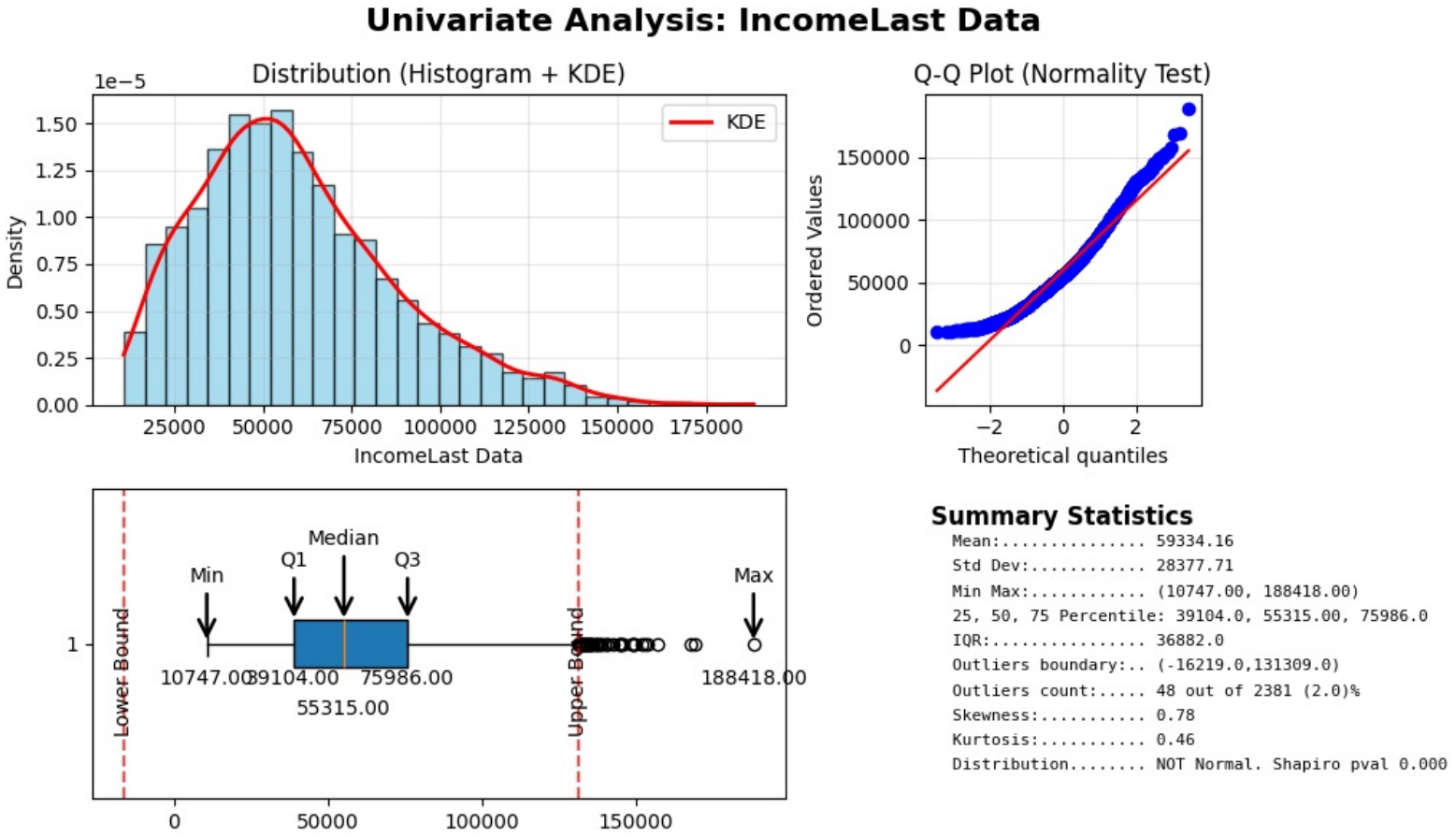
Out[589…
```
IncomeCAGR
Zero    0.98194
5-10    0.01386
0-5     0.00378
10+     0.00042
Name: proportion, dtype: float64
```

- 98% of drivers have not had an increase in income
- 0.3% (9) drivers had an income increase of 0-5%
- 1.3% (33) drivers had an income increase of 5-10%
- Only 1 driver had an income increase of 10+%

IncomeLast

In [590…
```
col = 'IncomeLast'

univariate_numerical_analysis(drivers[col], f"{col} Data")
```

```
Mean value is 59334.16 with 50% of data lying between 39104.0 and 75986.0
There are 48 outliers. 0 outliers are on lower side and 48 outliers are on upper side
```

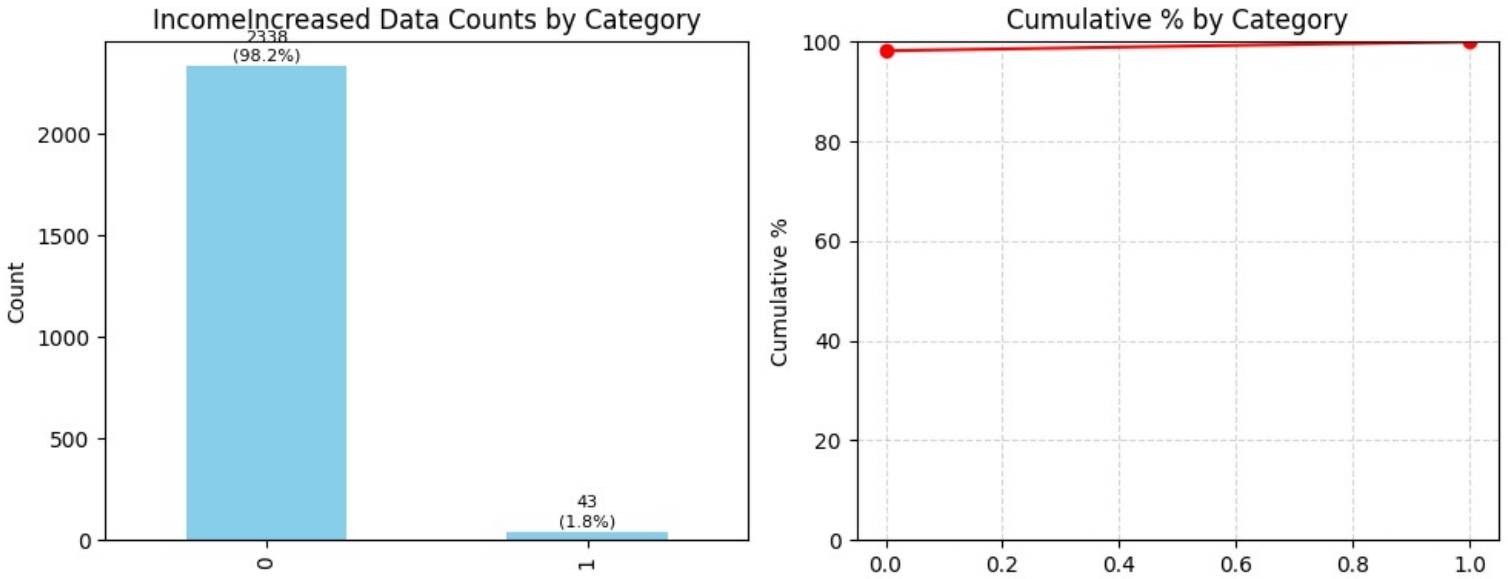


- Average income per driver is Rs 59334
- Maximum income is 1,88,418; Minimum income is 10,747
- 50% of drivers earn between Rs 39104 to 75986
- 48 outliers

IncomeIncreased

In [591…
```
col = 'IncomeIncreased'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```



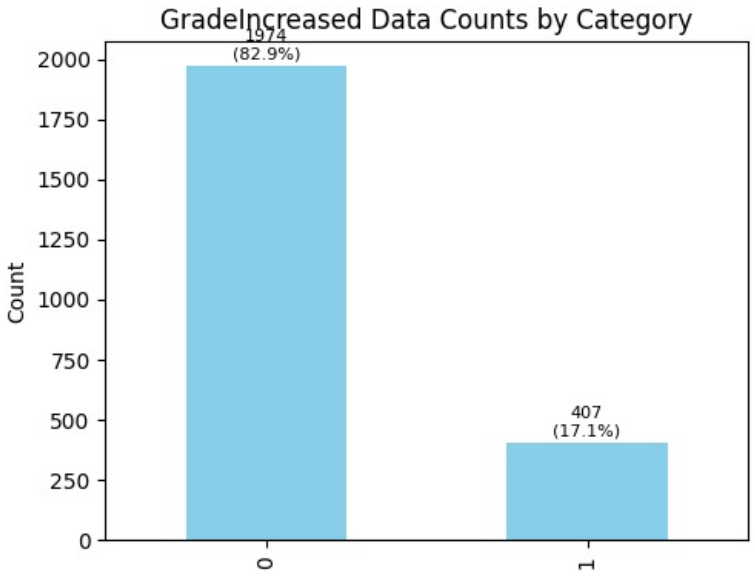

```
 Summary Statistics for IncomeIncreased Data
==================================================
Mode   : 0
Median : 0
```

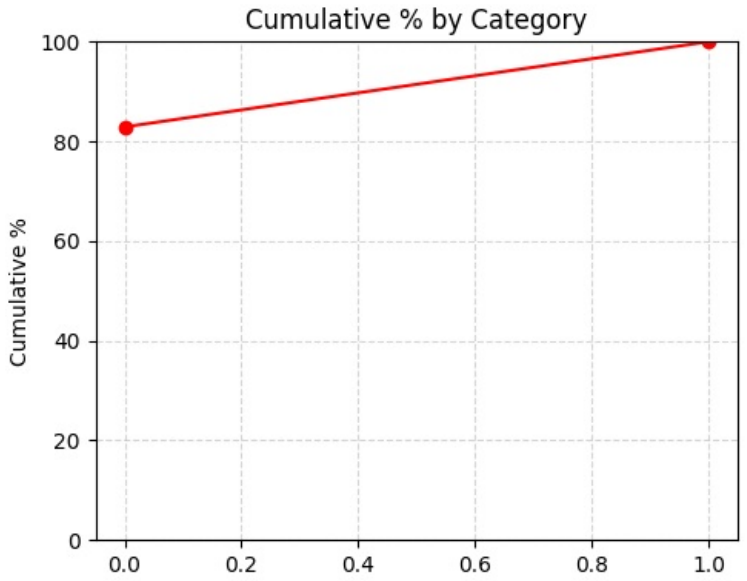
- This confirms what we found in IncomeCAGR. 98% of drivers have not seen a salary increase in over 2 yrs.

GradeIncreased

In [592…
```
col = 'GradeIncreased'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for GradeIncreased Data
==================================================
Mode   : 0
Median : 0
```

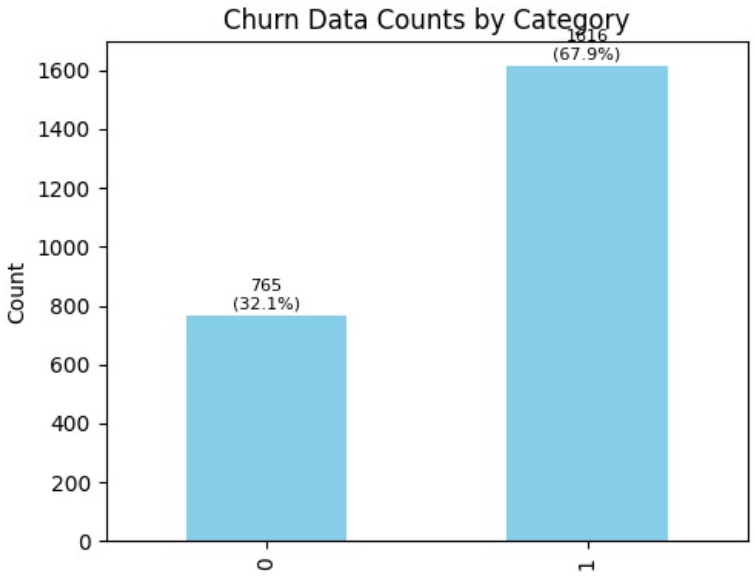
- 83% of drivers have had no increase in their grade for last 2 yrs.

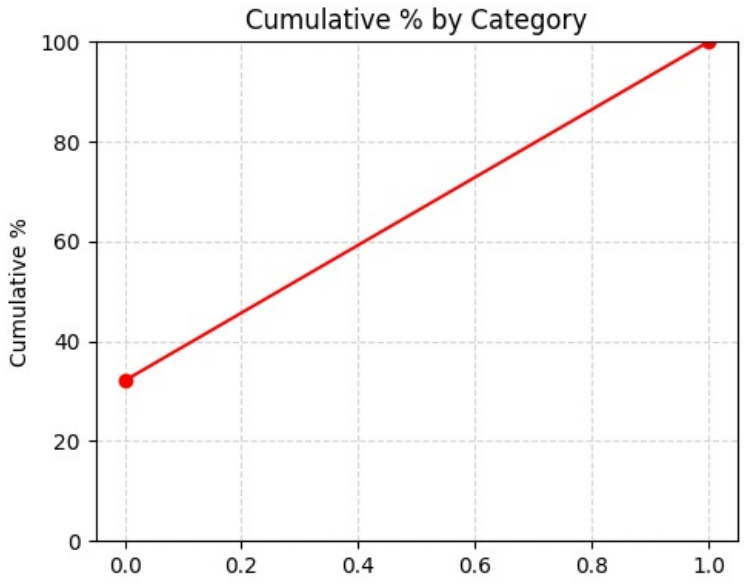
Churn

In [593…

```
col = 'Churn'

univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for Churn Data
==================================================
Mode   : 1
Median : 1
```

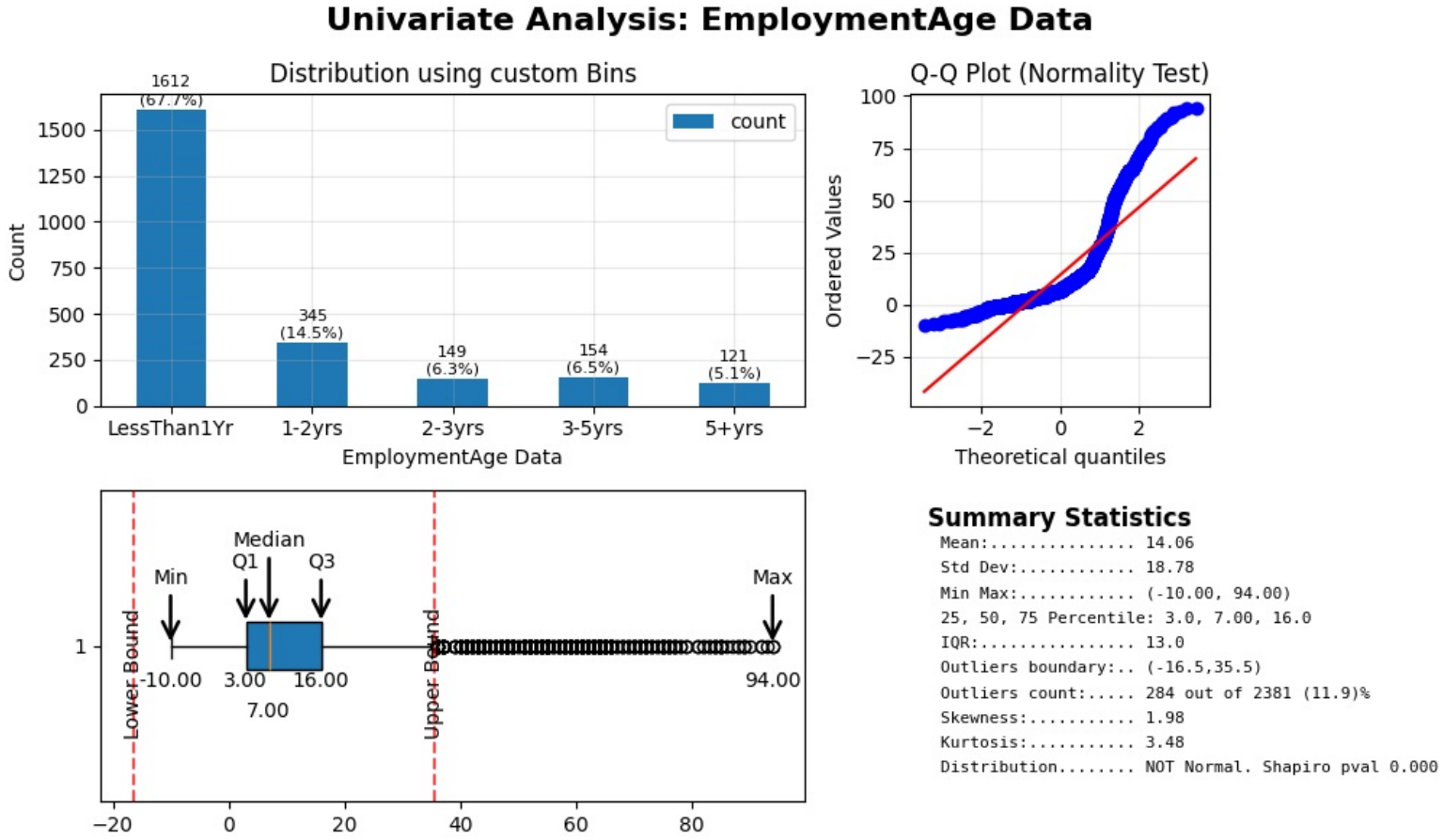
- 68% of drivers have churned over the past years.
- We have imbalanced data.

EmploymentAge

In [594…

```
col = 'EmploymentAge'

bins = [float('-inf'), 12, 24, 36, 60, float('inf')]
labels = ['LessThan1Yr', '1-2yrs', '2-3yrs','3-5yrs', '5+yrs']
univariate_numerical_analysis(drivers[col], f"{col} Data", {"bins":bins, "labels":labels})
```

```
Mean value is 14.06 with 50% of data lying between 3.0 and 16.0
There are 284 outliers. 0 outliers are on lower side and 284 outliers are on upper side
```



- 67% of drivers have joined in last year
- 82% of drivers joined less than 2 yrs back
- 95% of drivers joined less than 5 yrs back and only 5% of drivers are those that have stayed with the company for more than 5 yrs. This points to a huge attrition rate. Surely attrition is a problem in the company.

City

- We established earlier that no driver has changed their city.
- Hence we can use driver level dataframe to analyse the cities.

In [595…

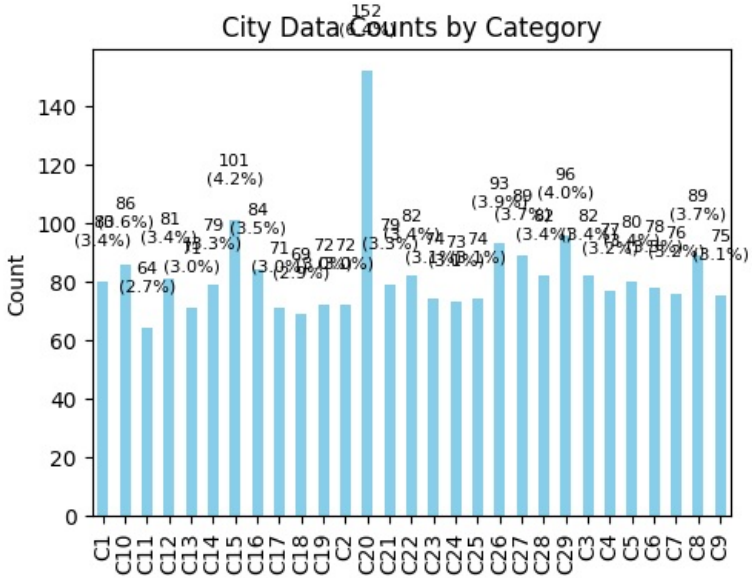
```
col = 'City'
drivers_by_city = drivers[col].value_counts()
print(f'We have {len(drivers_by_city)} cities')
print(f'On average, each city has {np.floor(drivers_by_city.mean())} drivers')
top_city = drivers_by_city.idxmax()
top_count = drivers_by_city.max()
total_count = drivers_by_city.sum()
```

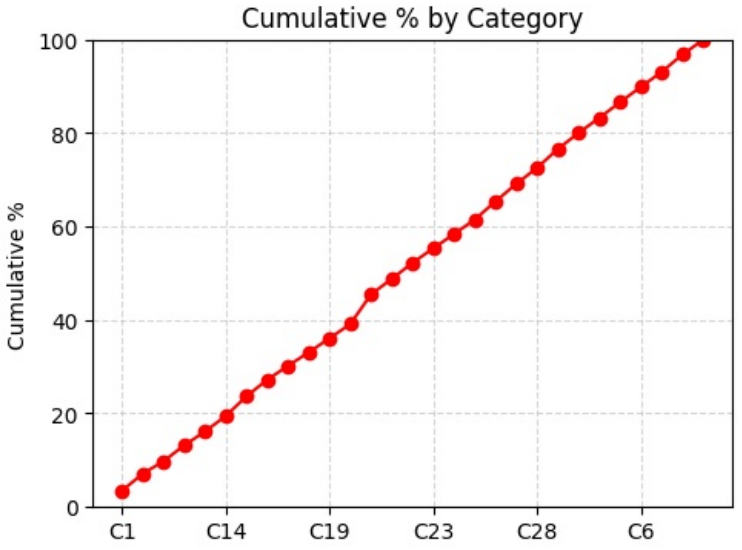
```
percentage = (top_count / total_count) * 100

print(f"City with maximum drivers: {top_city} ({top_count} drivers, {percentage:.2f}% of total)")
```

```
We have 29 cities
On average, each city has 82.0 drivers
City with maximum drivers: C20 (152 drivers, 6.38% of total)
```

In [596…
```
univariate_ordinal_analysis(drivers[col], f"{col} Data")
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\3549804371.py:16: DeprecationWarning:

is_categorical_dtype is deprecated and will be removed in a future version. Use isinstance(dtype, pd.CategoricalDtype) instead
```






```
 Summary Statistics for City Data
==================================================
Mode   : C20
Median : C22
```

- We have 29 cities
- On average, each city has 82.0 drivers
- City with maximum drivers: C20 (152 drivers, 6.38% of total)

3.2 Multivariate Analysis

Reusable helper function for ANOVA/KRUSKAL to compare categorical and numerical columns

In [597…
```
#Reusable function
#It will perform Anova or Kruksal based on input
#It will generate all groups and then perform the specific test.
# log = compact, detailed, compactNoDiagram, detailedNoDiagram
# We can pass a third column to act as hue. This makes it a trivariate analysis.
```

```python
import pingouin as pg


def perform_anova_kruskal(data,numerical_column,categorical_column, test, log="detailed", hueColumn=None):
    #η² for kruskal, ε² for anova
    def interpret_effect(size,test="anova"):
        if(test=='anova'):
            metric="ε²"
        else:
            metric="η²"
        effectMetric = f'{metric}={size:.2f}'
        if(p_value<0.05):
            if size < 0.01:
                effect = f'Even though {categorical_column} values vary across {numerical_column} groups (enough to be statistically significant {test} p_value={p_valu
            elif size < 0.06:
                effect = f'{categorical_column} values vary across {numerical_column} groups (enough to be statistically significant {test} p_value={p_value}), {numer
            elif size < 0.14:
                effect = f'{categorical_column} values vary across {numerical_column} groups (enough to be statistically significant {test} p_value={p_value}), {numer
            else:
                effect = f'{categorical_column} values vary across {numerical_column} groups (enough to be statistically significant {test} p_value={p_value}), {numer
        else:
            if size < 0.01:
                effect = f'{categorical_column} values do not vary across {numerical_column} groups (statistically insignificant {test} p_value={p_value}), which is a
            elif size < 0.06:
                effect = f'{categorical_column} values do not vary across {numerical_column} groups (statistically insignificant {test} p_value={p_value}), which is a
            elif size < 0.14:
                effect = f'{categorical_column} values do not vary across {numerical_column} groups (statistically insignificant{test} p_value={p_value}), {numerical_
            else:
                effect = f'{categorical_column} values vary slightly across {numerical_column} groups (statistically insignificant {test} p_value={p_value}), {numerica
        return effect, effectMetric



    # Group data by category and extract numerical values
    groups = [group[numerical_column].values for _, group in data.groupby(categorical_column, observed=True)]
    p_value=0
    # li="<li>"
    li=""
    Ho=Ha=HaDetail=HaDetail=effectDesc=""
    #Null Hypothesis=
    Ho=f"{li}{categorical_column} and {numerical_column} are independent. Medians of groups of {numerical_column} based on {categorical_column} are same"
    HoDetail=f"{li} In simple words, there's no substantial {numerical_column} difference across {categorical_column} values"
    #Alternate Hypothesis
    Ha=f"{li}{categorical_column} and {numerical_column} are DEPENDENT. Medians of groups of {numerical_column} based on {categorical_column} are different"
    HaDetail=f"{li} In simple words, there's substantial {numerical_column} difference across {categorical_column} values"

    if(test=='kruskal'):


        # Perform Kruskal-Wallis Test
        H, p_value  = stats.kruskal(*groups)

        k = len(groups)
        n = len(data)

        # Effect sizes
        epsilon_squared = (H - k + 1) / (n - k)
        effectDesc,effectMetric = interpret_effect(epsilon_squared,"kruskal")
```

```
    if(log=="detailed"):
        print(f'Ho = {Ho}')
        print(f'Ha = {Ha}')
        print(f'Kruskal-Wallis Result:{H},{p_value}')
        print(f'Effect Size: {effectMetric}')


if(test=='anova'):
    # Perform Anova Test
    #anova_result = stats.f_oneway(*groups)
    # p_value= anova_result.pvalue

    #Using pingouin as this gives eta square value too.
    aov = pg.anova(data=data, dv=numerical_column, between=categorical_column)
    # Extract p-value and eta-squared
    p_value = aov['p-unc'].values[0]
    eta_squared = aov['np2'].values[0]  # np2 is partial eta-squared (same as eta-squared for one-way ANOVA)
    effectDesc,effectMetric = interpret_effect(eta_squared)


    if(log=="detailed"):
        print(f'Ho = {Ho}')
        print(f'Ha = {Ha}')
        print("ANOVA Result:", p_value)
        print(f'Effect Size: {effectMetric}')

if(p_value<0.05):
    if(log=="detailed"):
        print(f"Reject Null Hypothesis. Alternate Hypothesis '{Ha}{HaDetail}' is True, p-value:{p_value:.2f}.\n{effectDesc}")

    else:
        print(f'{Ha}(p:{p_value:.2f}), {effectDesc}')



else:
    if(log=="detailed"):
        print(f"Fail to Reject Null Hypothesis. Null Hypothesis '{Ho}{HoDetail}' is True, p-value:{p_value:.2f}")
    else:
        print(f'{Ho}(p:{p_value:.2f})')

if(log=="detailed"):
    print()

if("NoDiagram" not in log):
    # We can also do visualization
    fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(15,5))
    sns.boxplot(data=data, y=numerical_column, x=categorical_column, ax=ax[0],hue=hueColumn)
    ax[0].set_ylabel(numerical_column)

    if(hueColumn is not None):
        df_analysis = data.groupby([categorical_column, hueColumn], as_index=False)[numerical_column].mean()
        sns.barplot(data=df_analysis, x=categorical_column, y=numerical_column, ax=ax[1], hue=hueColumn)
    else:
        df_analysis = data.groupby([categorical_column], as_index=False)[numerical_column].mean()
        sns.barplot(data=df_analysis, x=categorical_column, y=numerical_column, ax=ax[1])
    ax[1].set_xlabel(categorical_column)
```

```
        ax[1].set_ylabel(numerical_column)
        ax[1].set_title(f'Average {numerical_column} by {categorical_column}')

        sns.kdeplot(data=data, x=numerical_column, hue=categorical_column, ax=ax[2])
        ax[2].set_title(f"Kdeplot for {numerical_column} by {categorical_column}")
        plt.show()
        if(log=='detailed'):
            print(df_analysis)
```

Reusable helper function for chi square independence test for comparing categorical columns

In [598…

```
from scipy.stats import chi2_contingency



def perform_chi_square_test(df, col1, col2, alpha=0.05,log="detailed"):
    """
    Perform Chi-square test on two categorical columns from a DataFrame
    Also checks the Cramer's V value to establish if the association is of practical significance or only statistical.

    Parameters:
    df: pandas DataFrame
    col1: string, name of first categorical column
    col2: string, name of second categorical column
    alpha: float, significance level (default 0.05)

    Returns:
    dict with test results
    """
    li="<li>"
    # Create contingency table
    contingency_table = pd.crosstab(df[col1], df[col2])


    # Perform chi-square test
    Ho=f"{li}{col1} and {col2} are independent."
    Ha=f"{li}{col1} and {col2} are DEPENDENT"
    chi2_stat, p_value, dof, expected = chi2_contingency(contingency_table)

    # Compute effect size (Cramér's V)
    n = contingency_table.sum().sum()
    phi2 = chi2_stat / n
    r, k = contingency_table.shape
    cramers_v = (phi2 / min(k-1, r-1)) ** 0.5

    # p-value Interpretation
    if p_value < 0.05:
        relation = "Dependent (Significant)"
    else:
        relation = "Independent (Not significant)"


    # Cramer V Effect size Interpretation
    if cramers_v < 0.1:
        effectDesc = f"association is Negligible(CramersV={cramers_v:.2f})"
    elif cramers_v < 0.3:
        effectDesc = f"association is Weak(CramersV={cramers_v:.2f})"
```

```
    elif cramers_v < 0.5:
        effectDesc = f"association is Moderate(CramersV={cramers_v:.2f})"
    else:
        effectDesc = f"association is Strong(CramersV={cramers_v:.2f})"

    # Results
    results = {
        'chi2_statistic': chi2_stat,
        'p_value': p_value,
        'degrees_of_freedom': dof,
        'expected_frequencies': expected,
        'contingency_table': contingency_table,
        'significant': (p_value < alpha)
    }

    # Print results
    if("detailed" in log):
        print("Contingency Table:")
        print(contingency_table)
        print("\n")
        print(f"Chi-square Test Results:")
        print(f"Chi-square statistic: {chi2_stat:.4f}")
        print(f"p-value: {p_value:.4f}")
        print(f"Degrees of freedom: {dof}")
        print(f"Significant at α={alpha}: {'Yes' if results['significant'] else 'No'}")

    if results['significant']:
        if("detailed" in log):
            print(f"Reject Null Hypothesis. Alternate Hypothesis '{Ha}' is True, p-value:{results['p_value']}. However, {effectDesc}")
        else:
            print(f'{Ha}(p:{results['p_value']:.2f}), however {effectDesc}')
    else:
        if("detailed" in log):
            print(f"Fail to Reject Null Hypothesis. Null Hypothesis '{Ho}' is True, p-value:{results['p_value']}")
        else:
            print(f'{Ho}(p:{results['p_value']:.2f})')

    if("noplot" not in log):
        # We create two crosstabs
        crossNormalized12 = pd.crosstab(df[col1], df[col2], normalize='index') * 100
        crossNormalized21 = pd.crosstab(df[col2], df[col1], normalize='index') * 100

        # Create 1x2 subplot figure
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))

        colors = [
            "salmon", "skyblue",
            "lightgreen", "gold", "orchid", "coral", "khaki",
            "plum", "turquoise", "lightsteelblue", "tan", "lightpink",
            "mediumseagreen", "wheat", "lightsalmon", "aquamarine",
            "thistle", "palegoldenrod", "powderblue", "peru", "lightcyan",
            "violet", "darkseagreen", "peachpuff", "mistyrose"
        ]
        # determine how many categories you have
        num_categories = len(crossNormalized12.columns)

        # now slice the colors list accordingly
        color_list = colors[:num_categories]
```

```
        # --- First Plot: Col2 by Col1 ---
        bars = crossNormalized12.plot(kind='bar', stacked=False, ax=axes[0], color=color_list)
        axes[0].set_title(f"{col2} by {col1}")
        axes[0].set_ylabel(f"Percentage of {col2}")
        axes[0].set_xlabel(col1)
        axes[0].legend(title=col2)
        axes[0].tick_params(axis='x', rotation=0)
        annotate_bars(bars,'%.1f%%')

        # --- Second Plot: Col1 by Col2 ---
        bars = crossNormalized21.plot(kind='bar', stacked=False, ax=axes[1], color=color_list)
        axes[1].set_title(f"{col1} by {col2}")
        axes[1].set_ylabel(f"Percentage of {col1}")
        axes[1].set_xlabel(col2)
        axes[1].legend(title=col1)
        axes[1].tick_params(axis='x', rotation=0)
        annotate_bars(bars,'%.1f%%')
        plt.show()

    if("detailed" in log):
        print("="*50)
```

Reusable helper function for checking relation between two numerical columns and an optional categorical column.

In [599…

```
def numerical_numerical_analysis(df, x, y, hue=None, bins=5, figsize=(14, 5)):
    """
    Create side-by-side scatter and binned mean plots for two numerical columns.
    Also prints Pearson and Spearman correlation coefficients.

    Parameters:
        df (pd.DataFrame): Input dataframe
        x (str): X-axis numerical column
        y (str): Y-axis numerical column
        hue (str, optional): Optional hue column (categorical/numeric)
        bins (int): Number of quantile bins for x-axis
        figsize (tuple): Overall figure size (width, height)
    """

    # --- Compute Correlations ---
    pearson_corr = df[x].corr(df[y], method='pearson')
    spearman_corr = df[x].corr(df[y], method='spearman')

    print(f" Pearson correlation between {x} and {y}: {pearson_corr:.3f}")
    print(f" Spearman correlation between {x} and {y}: {spearman_corr:.3f}\n")

    # --- Prepare binned data ---
    df = df.copy()
    df[f'{x} bin'] = pd.qcut(df[x], q=bins, duplicates='drop')
    binned_means = df.groupby(f'{x} bin')[y].mean().reset_index()

    # --- Set up 1x2 layout ---
    fig, axes = plt.subplots(1, 2, figsize=figsize)

    # --- Scatter Plot ---
    sns.scatterplot(data=df, x=x, y=y,hue=hue, alpha=0.7, ax=axes[0])
    sns.regplot(data=df, x=x, y=y, scatter=False, color='red', ci=None, ax=axes[0])
    axes[0].set_title(f"Scatter: {y} vs {x}\n(Pearson: {pearson_corr:.2f}, Spearman: {spearman_corr:.2f})")
```

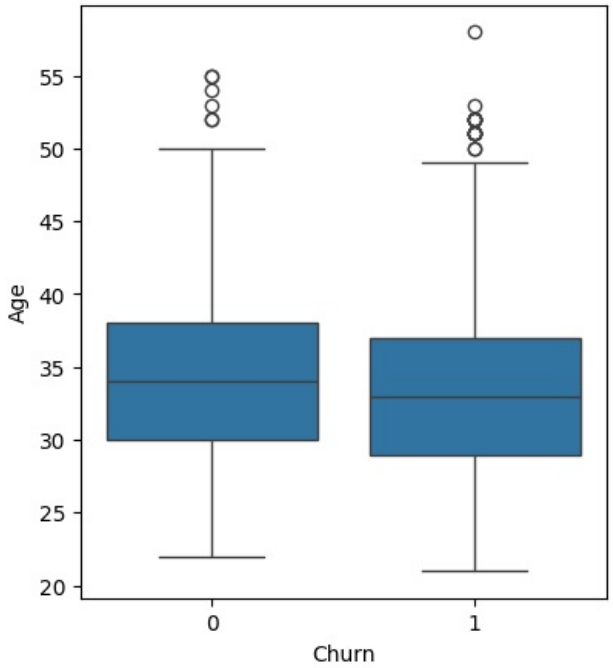
```
    # --- Binned Mean Plot ---
    sns.barplot(data=binned_means, x=f'{x} bin', y=y, color='skyblue', ax=axes[1])
    axes[1].set_title(f"Mean {y} per {x}-bin ({bins} quantiles)")
    axes[1].tick_params(axis='x', rotation=45)

    # --- Layout adjustments ---
    plt.tight_layout()
    plt.show()
```

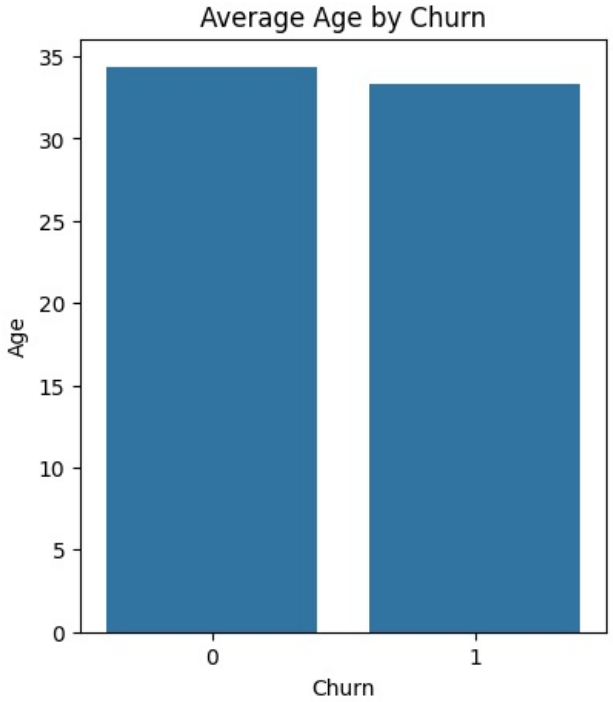
In [600…
```
numerical_columns=['Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Dateofjoining', 'LastWorkingDate', 'Joining Designation', 'Grade',
       'Total Business Value', 'Quarterly Rating', 'IncomeCAGR', 'IncomeLast',
       'IncomeIncreased', 'GradeIncreased', 'RatingMean', 'RatingTrend',
       'RatingTrendCode', 'Ratings', 'EmploymentAge', 'TBVCategory']

# We established during univariate analysis that none of the numerical columns are normal.
# So ANOVA cannot be used
# We shall use Kruksal Wallis test for all the comparisons, therefore.
test= 'kruskal'

categorical_column = 'Churn'
```

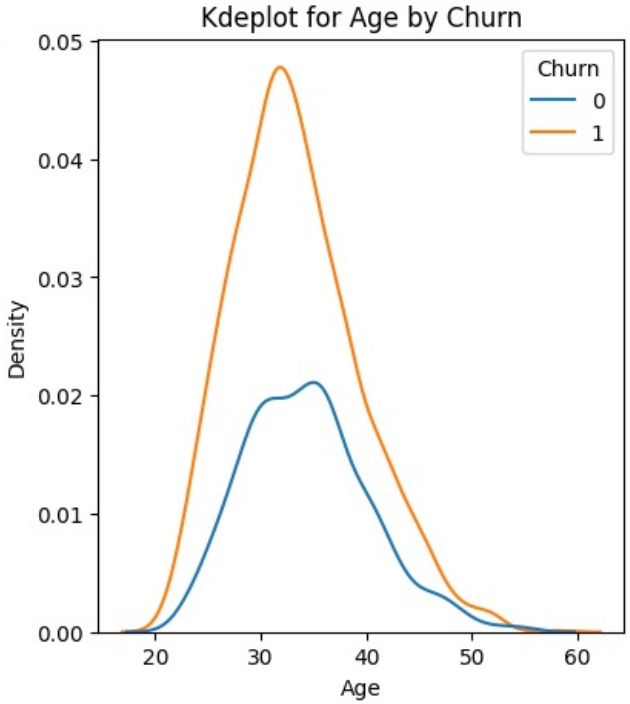
3.2.1 Age and Churn

In [601…
```
perform_anova_kruskal(drivers,numerical_column='Age', categorical_column='Churn', test=test,log="detailed")
```

```
Ho = Churn and Age are independent. Medians of groups of Age based on Churn are same
Ha = Churn and Age are DEPENDENT. Medians of groups of Age based on Churn are different
Kruskal-Wallis Result:16.68680718104635,4.408652546642178e-05
Effect Size: η²=0.01
Reject Null Hypothesis. Alternate Hypothesis 'Churn and Age are DEPENDENT. Medians of groups of Age based on Churn are different In simple words, there's substantial A
ge difference across Churn values' is True, p-value:0.00.
Even though Churn values vary across Age groups (enough to be statistically significant kruskal p_value=4.408652546642178e-05), Age itself doesn't meaningfully influen
ce Churn — the effect is too small (η²=0.01) to matter in practical terms.
```







	Churn	Age
0	0	34.349020
1	1	33.337871

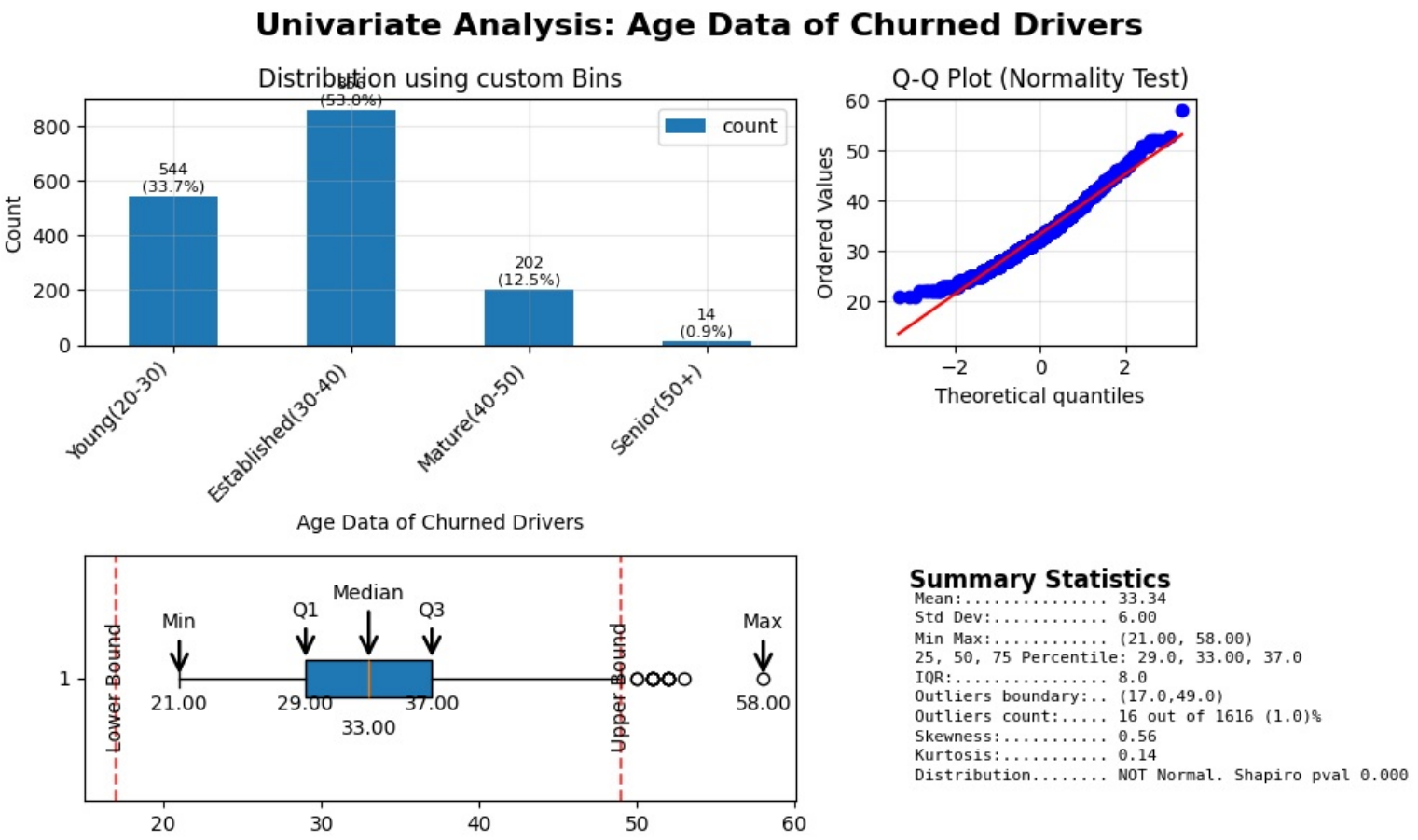
- There's substantial Age difference across Churn values
- Even though Churn rates vary across Age groups (enough to be statistically significant kruskal p_value=4.408652546642178e-05), Age itself doesn't meaningfully influence Churn — the effect is too small (η^2=0.01) to matter in practical terms.

In [602…
```
drivers.columns
```

Out[602…
```
Index(['Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Dateofjoining', 'LastWorkingDate', 'Joining Designation', 'Grade',
       'Total Business Value', 'Quarterly Rating', 'IncomeCAGR', 'IncomeLast',
       'IncomeIncreased', 'GradeIncreased', 'RatingMean', 'RatingTrend',
       'RatingTrendCode', 'Ratings', 'Churn', 'EmploymentAge'],
      dtype='object')
```

In [603…
```
driversChurned = drivers.query('Churn==1')
driversCurrent = drivers.query('Churn==0')
col = 'Age'

bins = [float('-inf'),     30,             40,      50,       float('inf')]
labels = [          'Young(20-30)', 'Established(30-40)', 'Mature(40-50)','Senior(50+)']

univariate_numerical_analysis(driversChurned[col], f"{col} Data of Churned Drivers", {"bins":bins, "labels":labels})
univariate_numerical_analysis(driversCurrent[col], f"{col} Data of Current Drivers", {"bins":bins, "labels":labels})
```

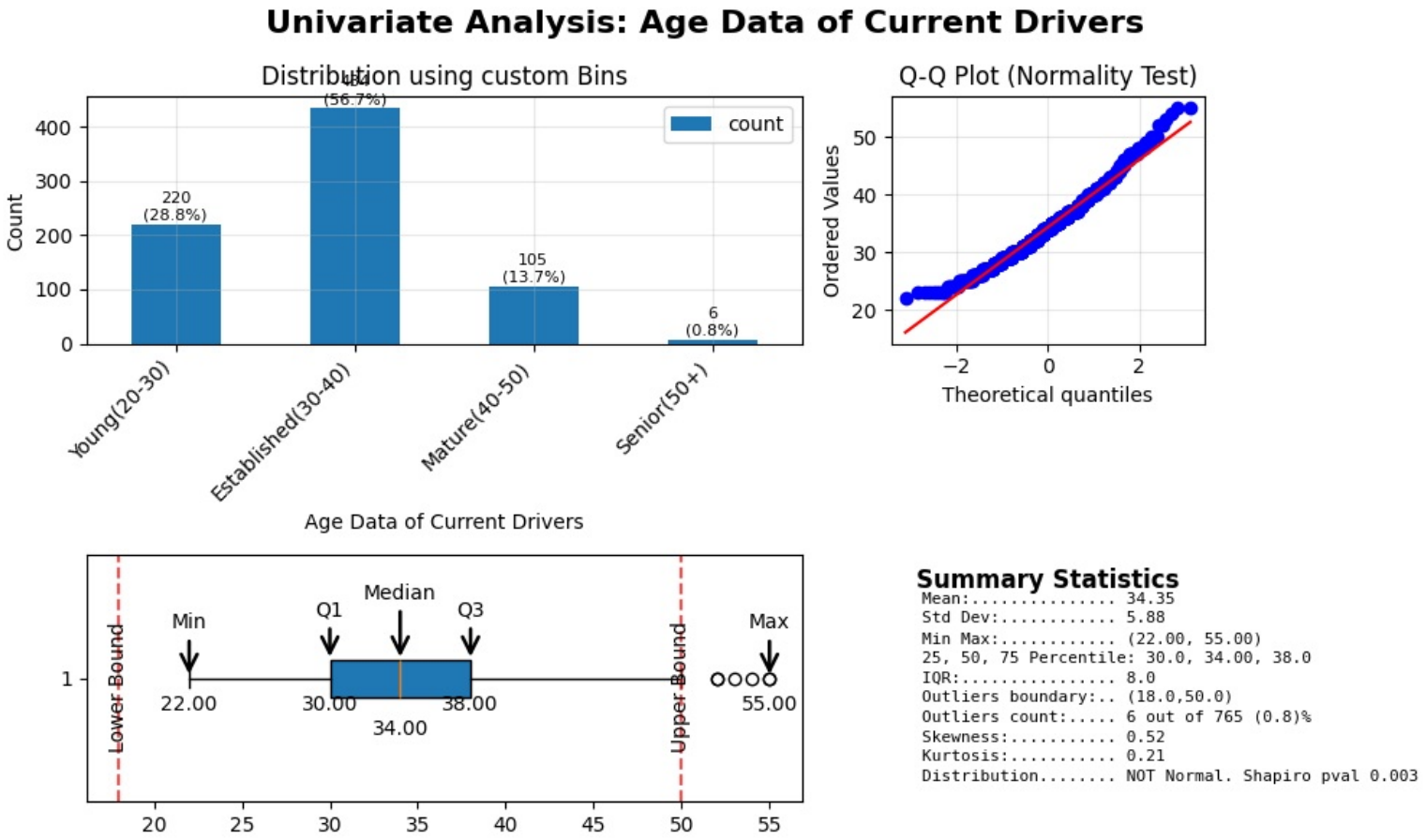
```
Mean value is 33.34 with 50% of data lying between 29.0 and 37.0
There are 16 outliers. 0 outliers are on lower side and 16 outliers are on upper side
```



Summary Statistics

```
Mean:.............. 33.34
Std Dev:........... 6.00
Min Max:........... (21.00, 58.00)
25, 50, 75 Percentile: 29.0, 33.00, 37.0
IQR:............... 8.0
Outliers boundary:.. (17.0,49.0)
Outliers count:..... 16 out of 1616 (1.0)%
Skewness:.......... 0.56
Kurtosis:.......... 0.14
Distribution....... NOT Normal. Shapiro pval 0.000
```

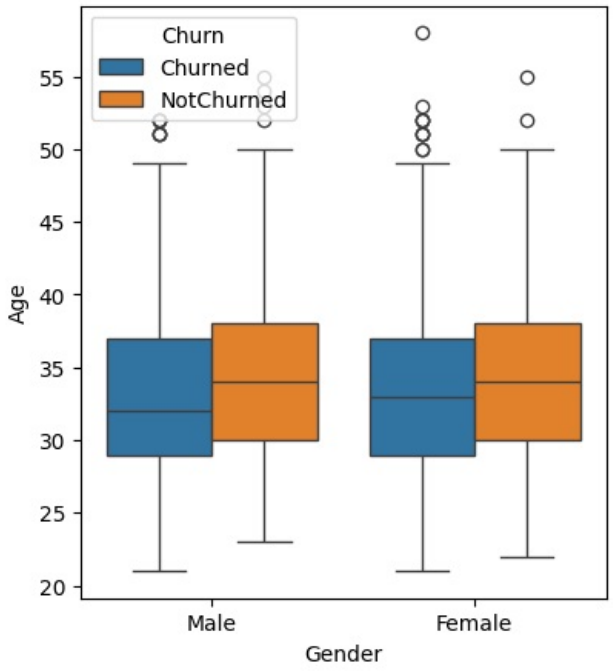
```
Mean value is 34.35 with 50% of data lying between 30.0 and 38.0

There are 6 outliers. 0 outliers are on lower side and 6 outliers are on upper side
```

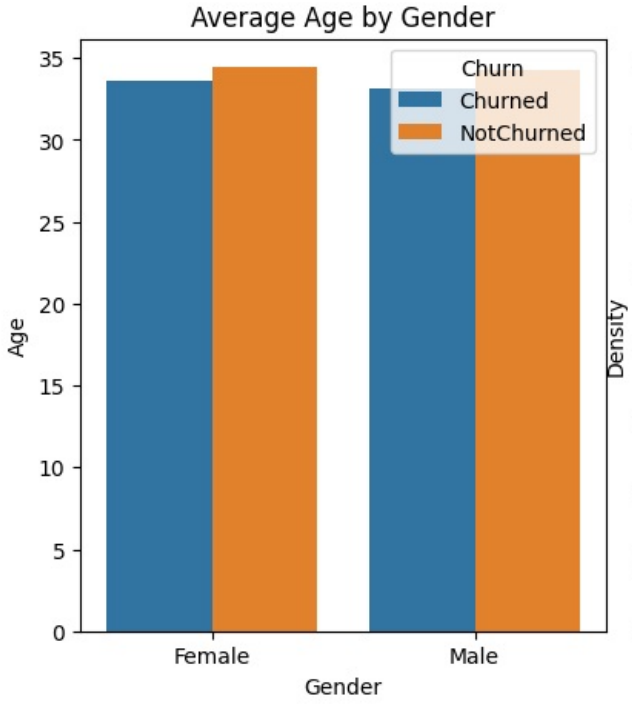


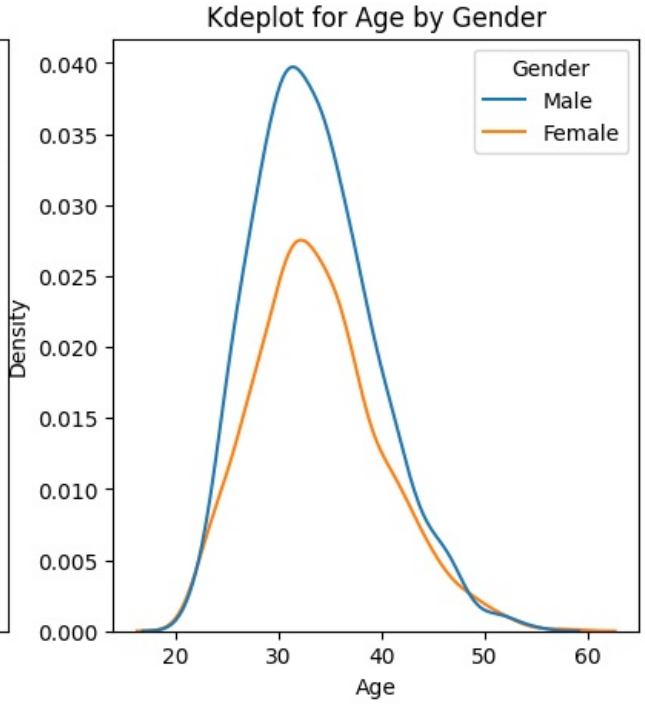
3.2.2 Age and Gender

In [604…
```
perform_anova_kruskal(driversWithCategories,numerical_column='Age', categorical_column='Gender', test=test,log="detailed", hueColumn='Churn')
```

```
Ho = Gender and Age are independent. Medians of groups of Age based on Gender are same
Ha = Gender and Age are DEPENDENT. Medians of groups of Age based on Gender are different
Kruskal-Wallis Result:1.9785838886858578,0.15953967066906793
Effect Size: η²=0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Gender and Age are independent. Medians of groups of Age based on Gender are same In simple words, there’s no substant
ial Age difference across Gender values' is True, p-value:0.16
```







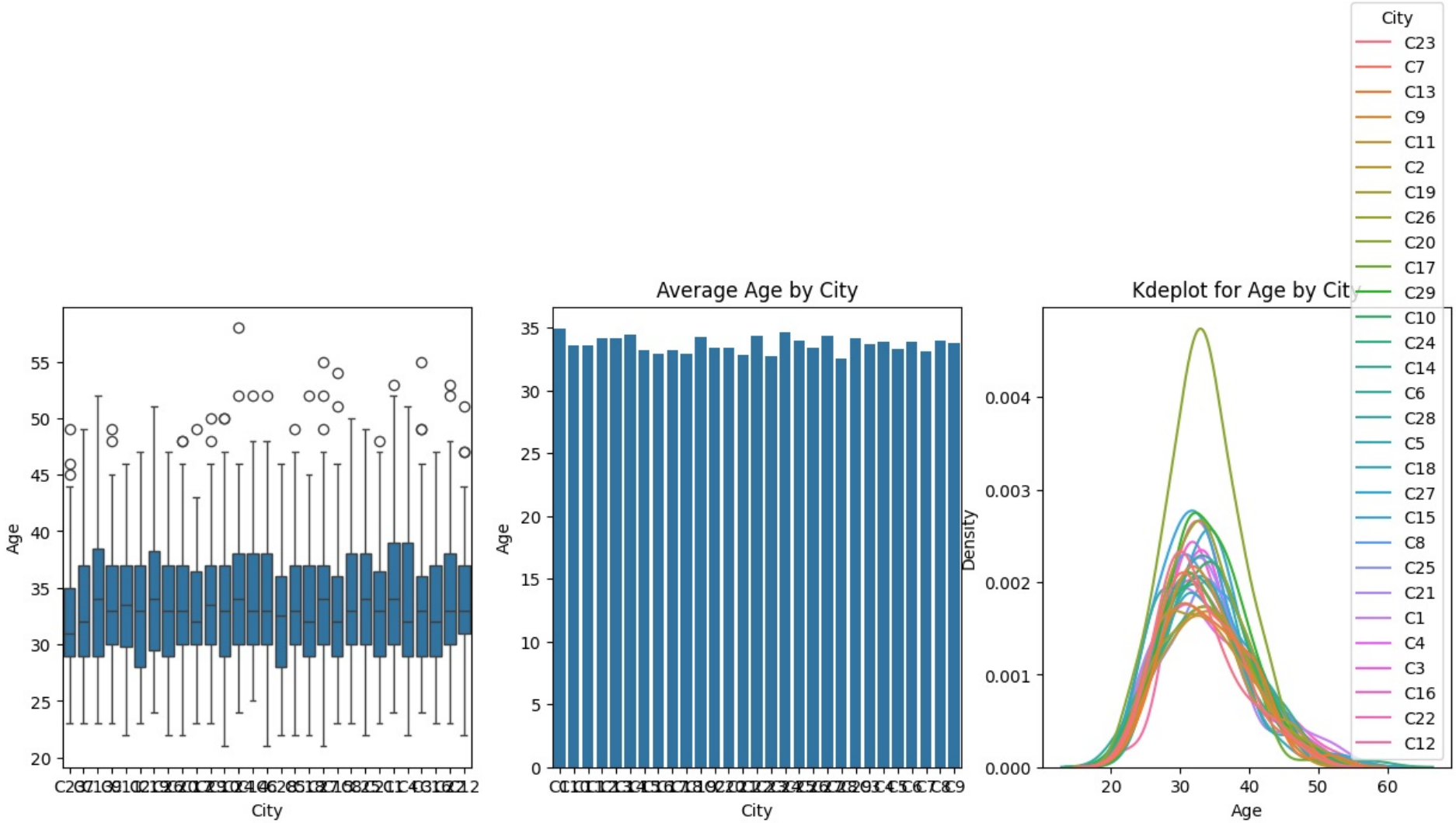
	Gender	Churn	Age
0	Female	Churned	33.624251
1	Female	NotChurned	34.449838
2	Male	Churned	33.136076
3	Male	NotChurned	34.280702

- Gender and Age are independent. Medians of groups of Age based on Gender are same
- In simple words, there's no substantial Age difference across Gender values

3.2.3 Age and City

```
In [605… perform_anova_kruskal(drivers,numerical_column='Age', categorical_column='City', test=test,log="detailed")
```

```
Ho = City and Age are independent. Medians of groups of Age based on City are same
Ha = City and Age are DEPENDENT. Medians of groups of Age based on City are different
Kruskal-Wallis Result:19.116945857154757,0.8944889418402363
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'City and Age are independent. Medians of groups of Age based on City are same In simple words, there's no substantial
Age difference across City values' is True, p-value:0.89
```


Average Age by City
Kdeplot for Age by City
Age
City
Density
City
C23
C7
C13
C9
C11
C2
C19
C26
C20
C17
C29
C10
C24
C14
C6
C28
C5
C18
C27
C15
C8
C25
C21
C1
C4
C3
C16
C22
C12

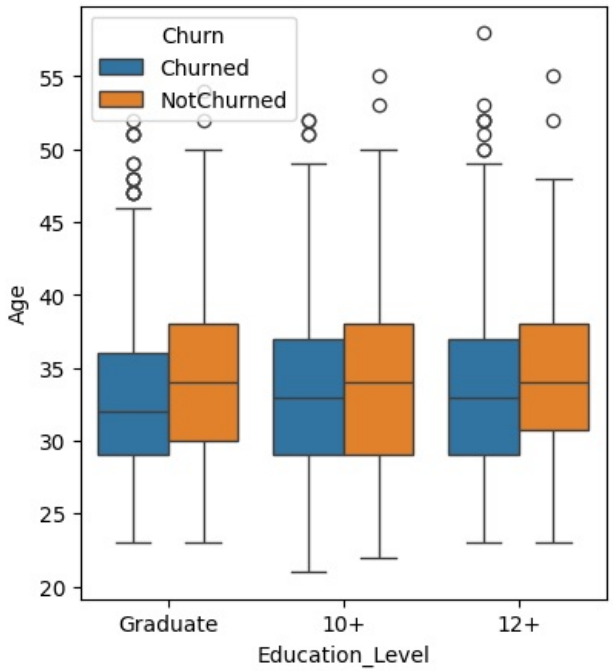
	City	Age
0	C1	34.912500
1	C10	33.616279
2	C11	33.578125
3	C12	34.148148
4	C13	34.154930
5	C14	34.405063
6	C15	33.158416
7	C16	32.928571
8	C17	33.197183
9	C18	32.956522
10	C19	34.208333
11	C2	33.347222
12	C20	33.421053
13	C21	32.822785
14	C22	34.304878
15	C23	32.729730
16	C24	34.589041
17	C25	33.959459
18	C26	33.419355
19	C27	34.303371
20	C28	32.536585
21	C29	34.166667
22	C3	33.634146
23	C4	33.883117
24	C5	33.275000
25	C6	33.858974
26	C7	33.118421
27	C8	33.988764
28	C9	33.760000

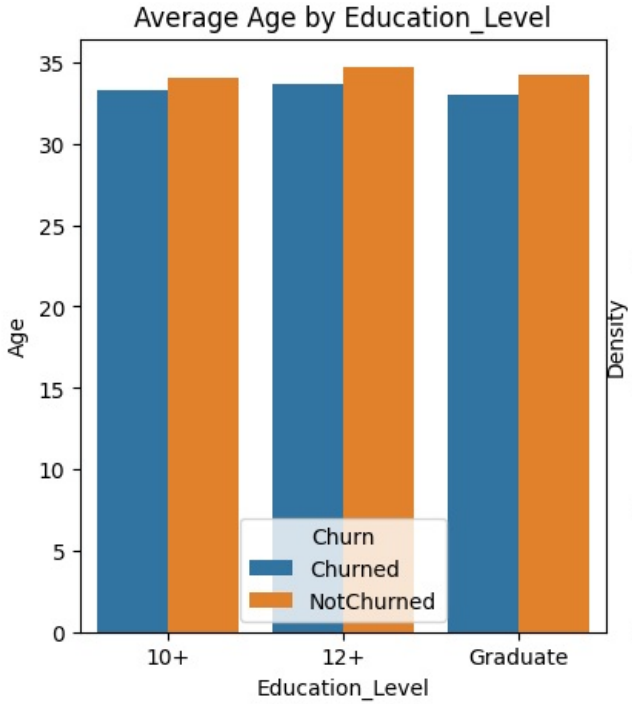
- City and Age are independent. Medians of groups of Age based on City are same

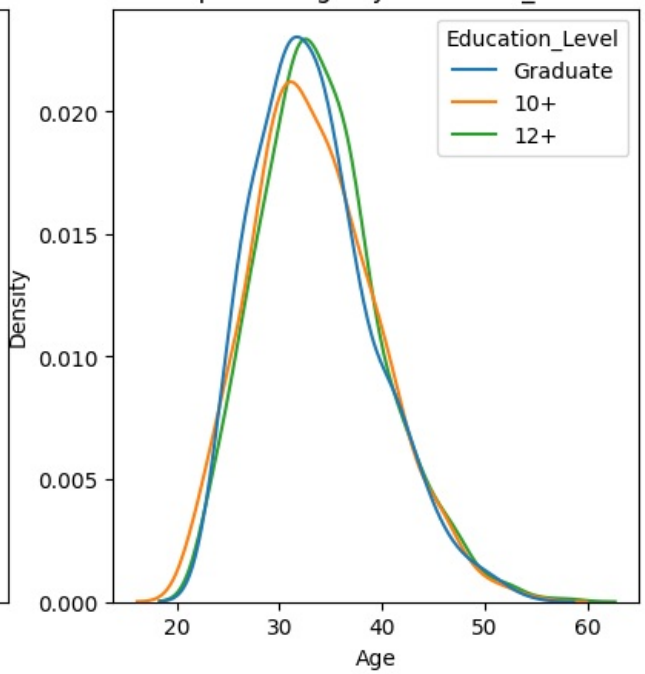
3.2.4 Age and Education Level

In [606…
```
perform_anova_kruskal(driversWithCategories,numerical_column='Age', categorical_column='Education_Level', test=test,log="detailed", hueColumn='Churn')
```

```
Ho = Education_Level and Age are independent. Medians of groups of Age based on Education_Level are same
Ha = Education_Level and Age are DEPENDENT. Medians of groups of Age based on Education_Level are different
Kruskal-Wallis Result:5.715767909743988,0.05739007185174372
Effect Size: η²=0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Education_Level and Age are independent. Medians of groups of Age based on Education_Level are same In simple words, t
here’s no substantial Age difference across Education_Level values' is True, p-value:0.06
```







	Education_Level	Churn	Age
0	10+	Churned	33.309963
1	10+	NotChurned	34.024793
2	12+	Churned	33.681214
3	12+	NotChurned	34.738806
4	Graduate	Churned	33.034735
5	Graduate	NotChurned	34.247059

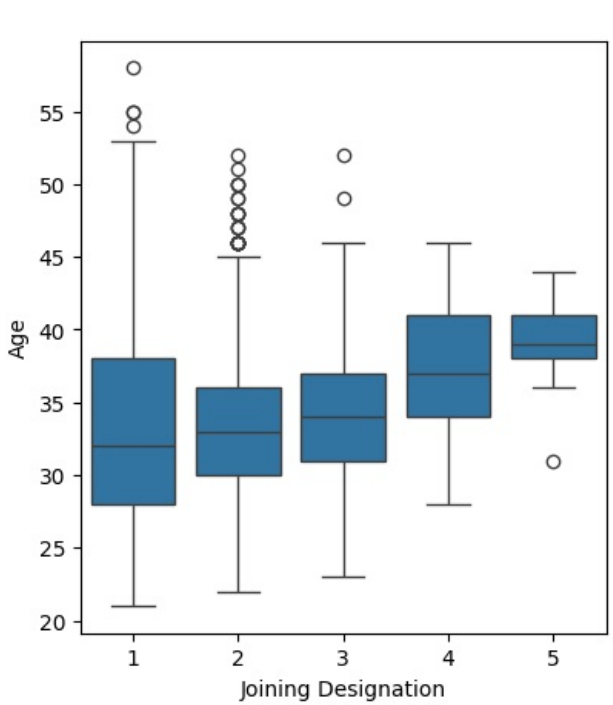
- Education_Level and Age are independent
- There's no substantial age difference across education levels, and churn does not strongly depend on either.

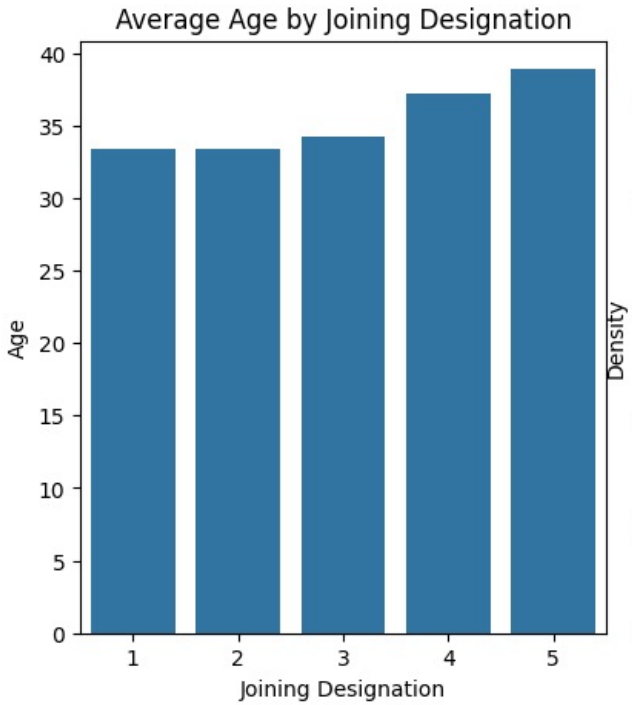
3.2.5 Age and Joining Designation

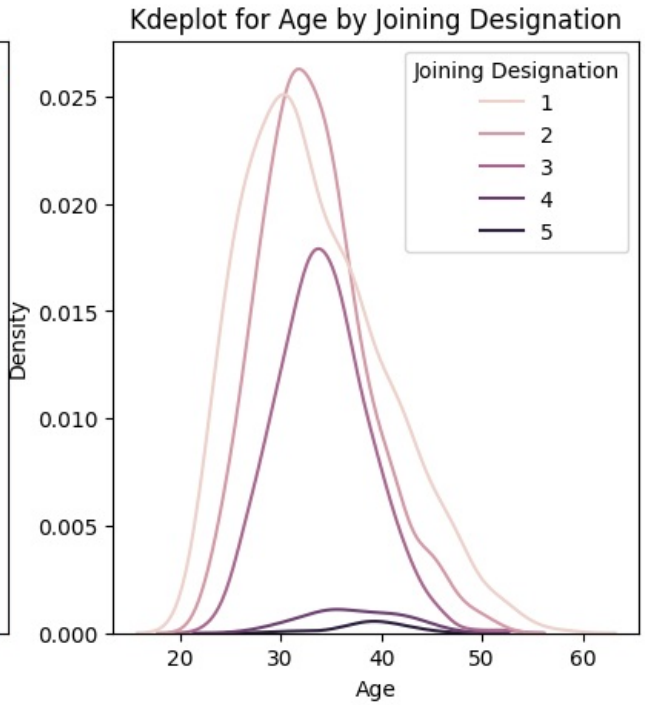
In [607…

```
perform_anova_kruskal(drivers,numerical_column='Age', categorical_column='Joining Designation', test=test,log="detailed")
```

```
Ho = Joining Designation and Age are independent. Medians of groups of Age based on Joining Designation are same
Ha = Joining Designation and Age are DEPENDENT. Medians of groups of Age based on Joining Designation are different
Kruskal-Wallis Result:47.958995786477715,9.625428906028363e-10
Effect Size: η²=0.02
Reject Null Hypothesis. Alternate Hypothesis 'Joining Designation and Age are DEPENDENT. Medians of groups of Age based on Joining Designation are different In simple words, there's substantial Age difference across Joining Designation values' is True, p-value:0.00.
Joining Designation values vary across Age groups (enough to be statistically significant kruskal p_value=9.625428906028363e-10), Age influence on Joining Designation is considered small (η²=0.02)
```







	Joining Designation	Age
0	1	33.411306
1	2	33.397546
2	3	34.247465
3	4	37.222222
4	5	38.909091

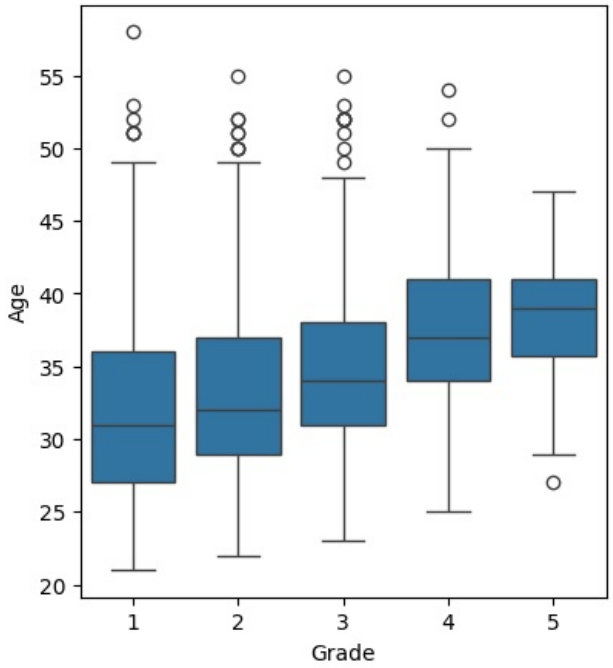
- Joining Designation rates vary slightly across Age groups (enough to be statistically significant kruskal p_value=9.625428906028363e-10), Age influence on Joining Designation is considered small (η²=0.02)
- Avg age of drivers joining in designation 1,2 is 33 while those at grade 4,5 is 37 to 40

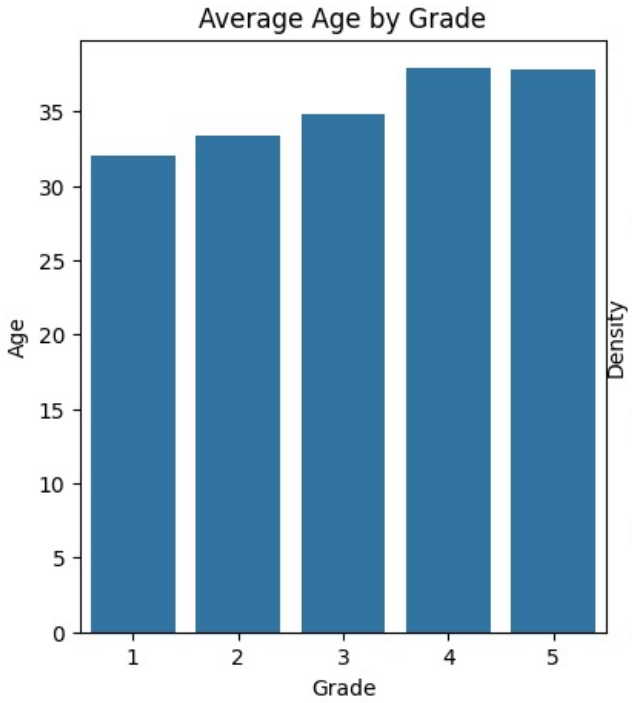
3.2.6 Age and Current Grade

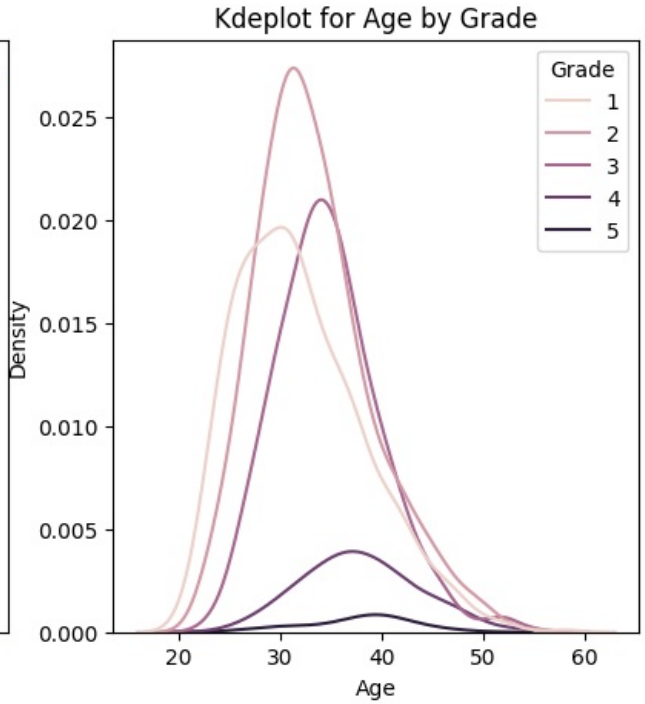
In [608…

```
perform_anova_kruskal(drivers,numerical_column='Age', categorical_column='Grade', test=test,log="detailed")
```

```
Ho = Grade and Age are independent. Medians of groups of Age based on Grade are same
Ha = Grade and Age are DEPENDENT. Medians of groups of Age based on Grade are different
Kruskal-Wallis Result:180.1942034759169,6.773769898624466e-38
Effect Size: η²=0.07
Reject Null Hypothesis. Alternate Hypothesis 'Grade and Age are DEPENDENT. Medians of groups of Age based on Grade are different In simple words, there's substantial A
ge difference across Grade values' is True, p-value:0.00.
Grade values vary across Age groups (enough to be statistically significant kruskal p_value=6.773769898624466e-38), Age does have a medium influence on Grade (η²=0.07)
```







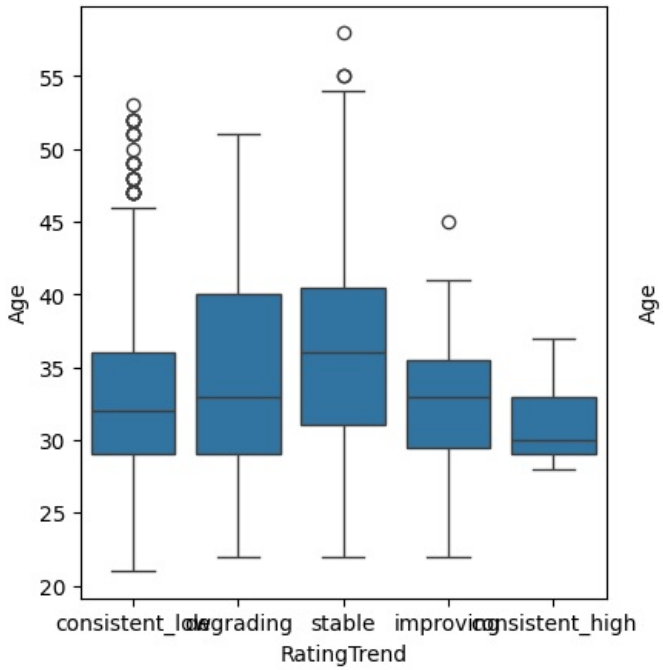
	Grade	Age
0	1	32.056680
1	2	33.409357
2	3	34.815409
3	4	37.927536
4	5	37.833333

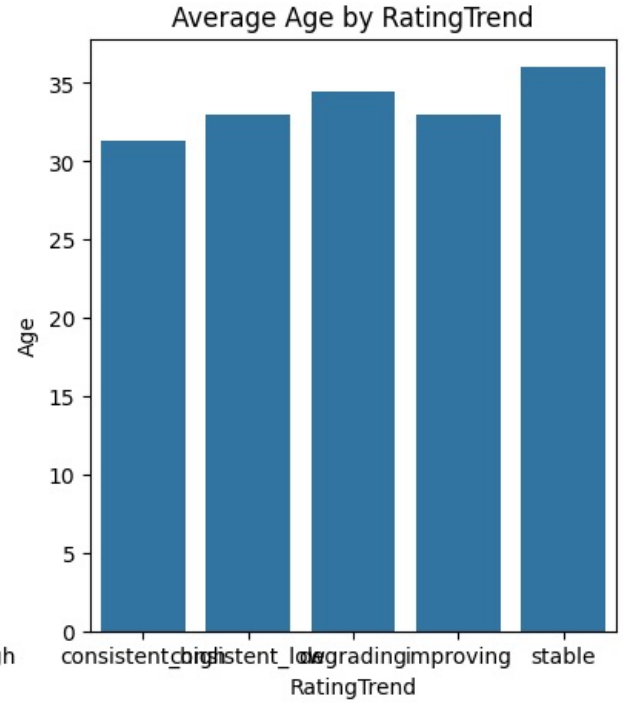
- Grade rates vary across Age groups (enough to be statistically significant kruskal p_value=6.773769898624466e-38), Age does have a medium influence on Grade (η²=0.07)
- Avg age of drivers in grade 1,2,3 is around 33-34 while those at grade 4,5 is 38 to 39

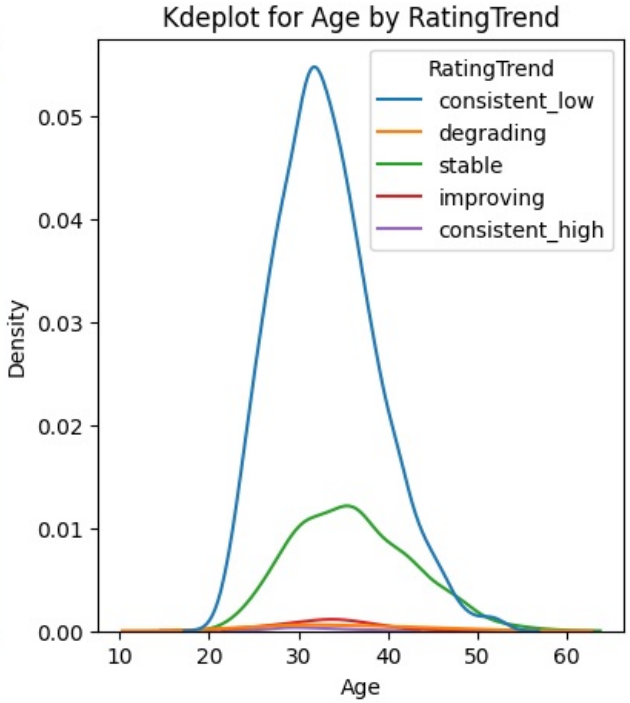
3.2.7 Age and Rating Trend

In [609…
```
perform_anova_kruskal(drivers,numerical_column='Age', categorical_column='RatingTrend', test=test,log="detailed")
```

```
Ho = RatingTrend and Age are independent. Medians of groups of Age based on RatingTrend are same
Ha = RatingTrend and Age are DEPENDENT. Medians of groups of Age based on RatingTrend are different
Kruskal-Wallis Result:83.79395463190693,2.7340133727364485e-17
Effect Size: η²=0.03
Reject Null Hypothesis. Alternate Hypothesis 'RatingTrend and Age are DEPENDENT. Medians of groups of Age based on RatingTrend are different In simple words, there's s
ubstantial Age difference across RatingTrend values' is True, p-value:0.00.
RatingTrend values vary across Age groups (enough to be statistically significant kruskal p_value=2.7340133727364485e-17), Age influence on RatingTrend is considered s
mall (η²=0.03)
```







	RatingTrend	Age
0	consistent_high	31.285714
1	consistent_low	33.017689
2	degrading	34.483871
3	improving	32.971429
4	stable	36.032064

- RatingTrend values vary across Age groups (enough to be statistically significant kruskal p_value=2.7340133727364485e-17), Age influence on RatingTrend is considered small (η²=0.03)
- Youngest drivers (age 31) have consistently high ratings(4,5)
- Oldest drivers (age 36) have consistently stable midrange ratings(2,3,4)
- Drivers around age 33 have consistently low range ratings(1)

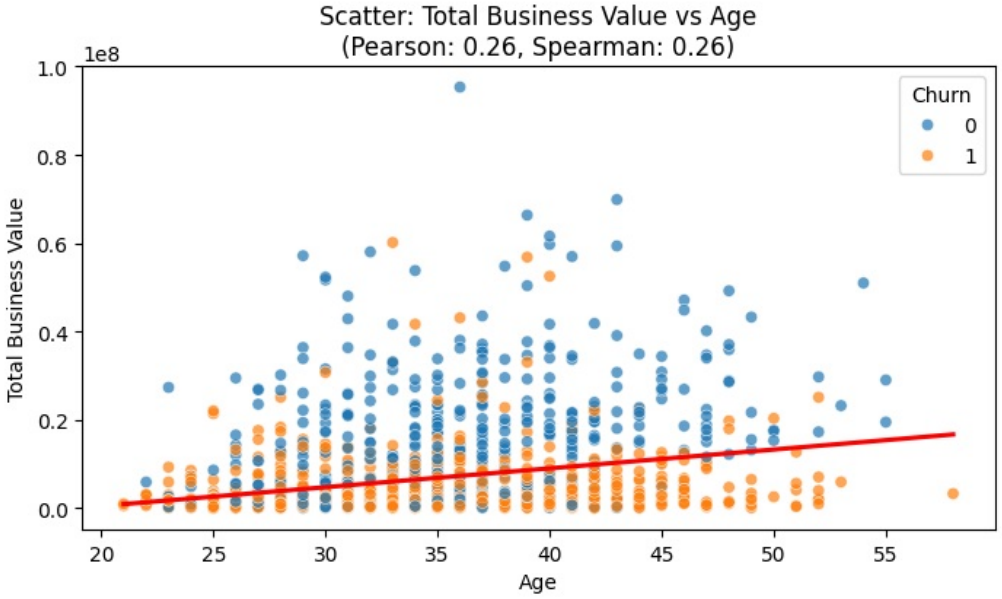
3.2.8 Age and Total Business Value

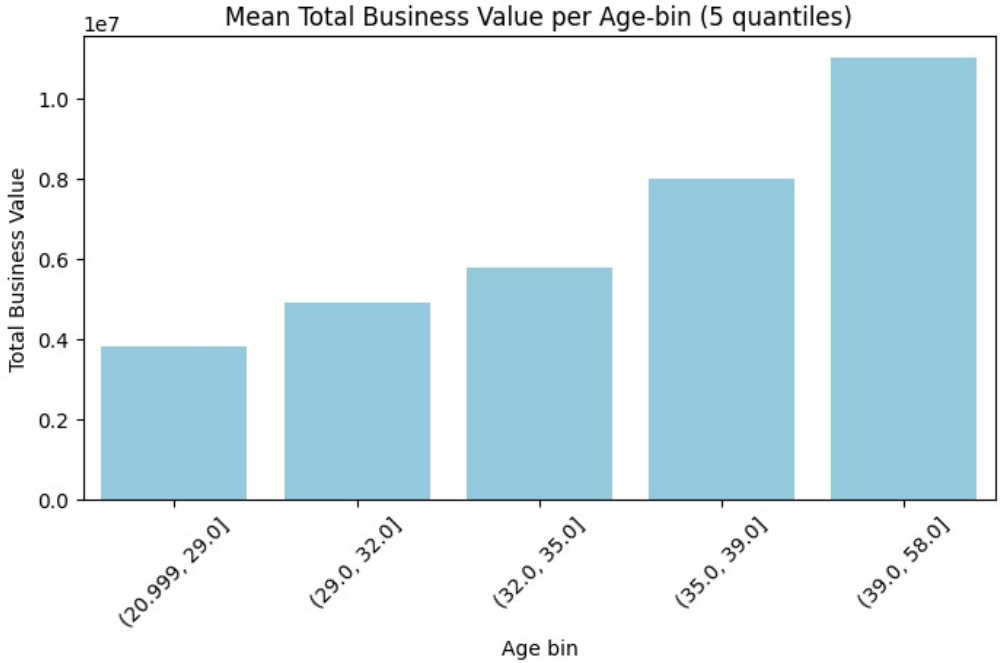
In [610…

```
nonZeroTBVDrivers = drivers[drivers['Total Business Value']>0].copy()
numerical_numerical_analysis(nonZeroTBVDrivers,'Age','Total Business Value', hue='Churn')
```

```
Pearson correlation between Age and Total Business Value: 0.255
Spearman correlation between Age and Total Business Value: 0.263
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\1352136180.py:25: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
to adopt the future default and silence this warning.
```





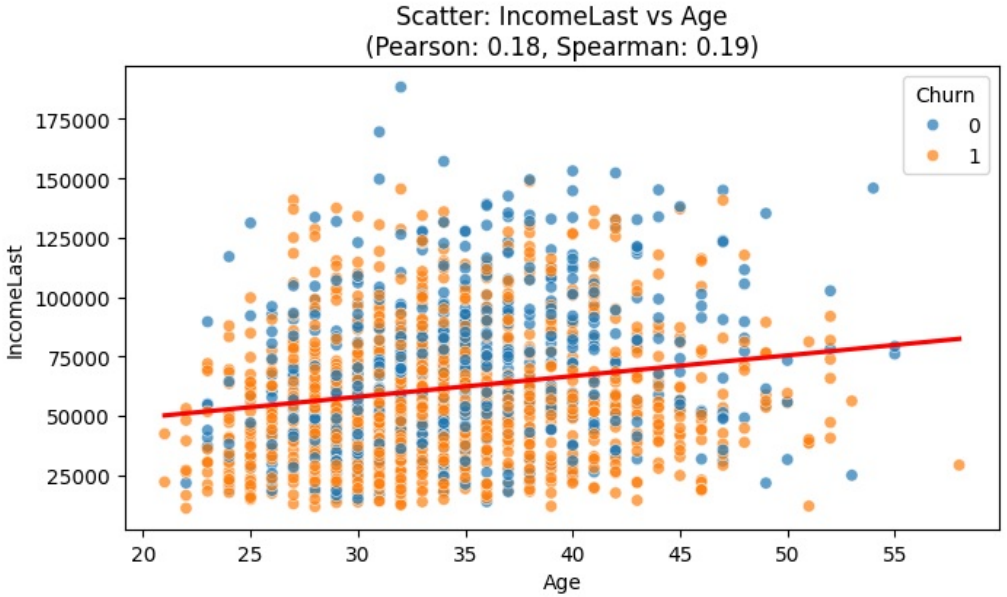
- There is a weak positive correlation between Age and Total business value
- Contribution to total business value for age group 39+ is triple that of age group 20-29
- Contribution to total business value for age group 35-39 is double that of age group 20-29
- Drivers who churned generally had less TBV

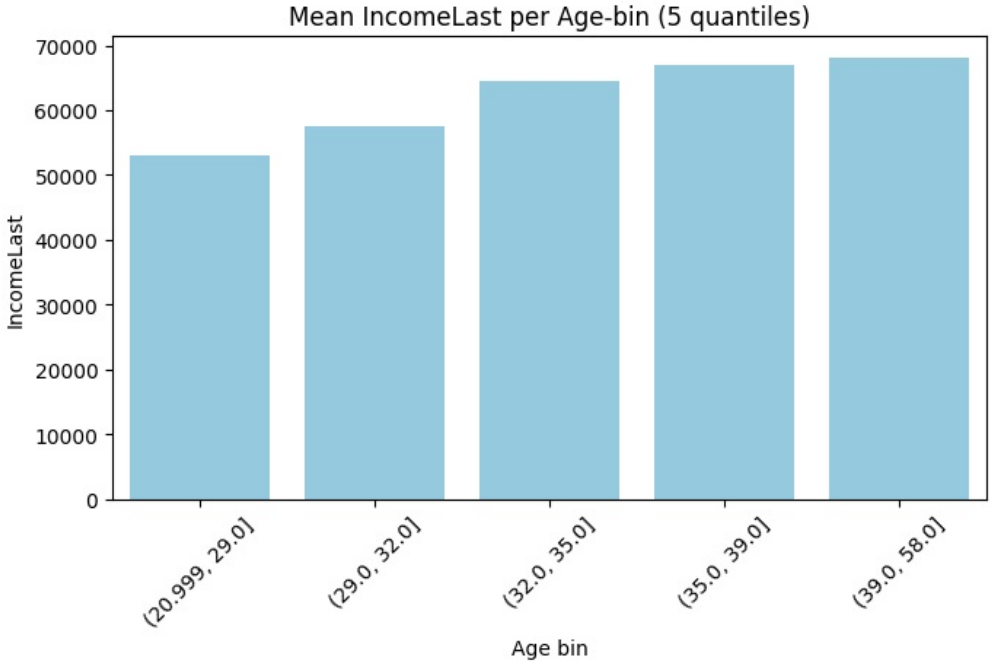
3.2.9 Age and IncomeLast

In [611…
```
numerical_numerical_analysis(nonZeroTBVDrivers,'Age','IncomeLast', hue='Churn')
```

```
Pearson correlation between Age and IncomeLast: 0.184
Spearman correlation between Age and IncomeLast: 0.193
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\1352136180.py:25: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
to adopt the future default and silence this warning.
```





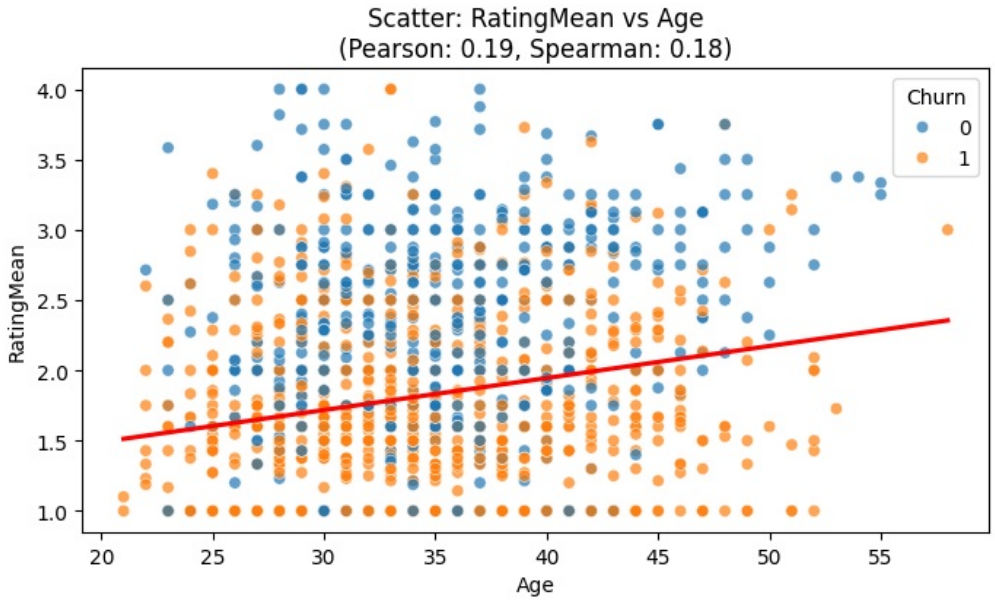
- There is a weak positive correlation between Age and Income
- Income for age group 39+ is 40% more than that of age group 20-29
- There is a steeper increase in income in lower age groups. Income increase beyond 32 is less in percentage terms

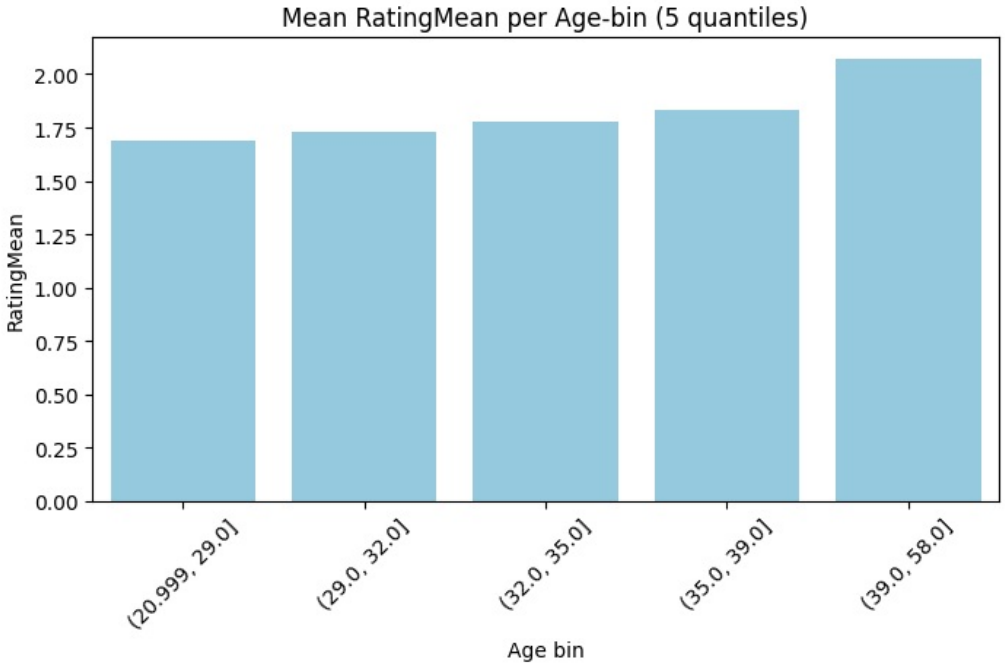
3.2.10 Age and RatingMean

In [612…
```
numerical_numerical_analysis(nonZeroTBVDrivers,'Age','RatingMean', hue='Churn')
```

```
Pearson correlation between Age and RatingMean: 0.191
Spearman correlation between Age and RatingMean: 0.177
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\1352136180.py:25: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
to adopt the future default and silence this warning.
```





- Olders drivers have slightly more mean rating than younger ones, but the correlation of age and rating is very weak

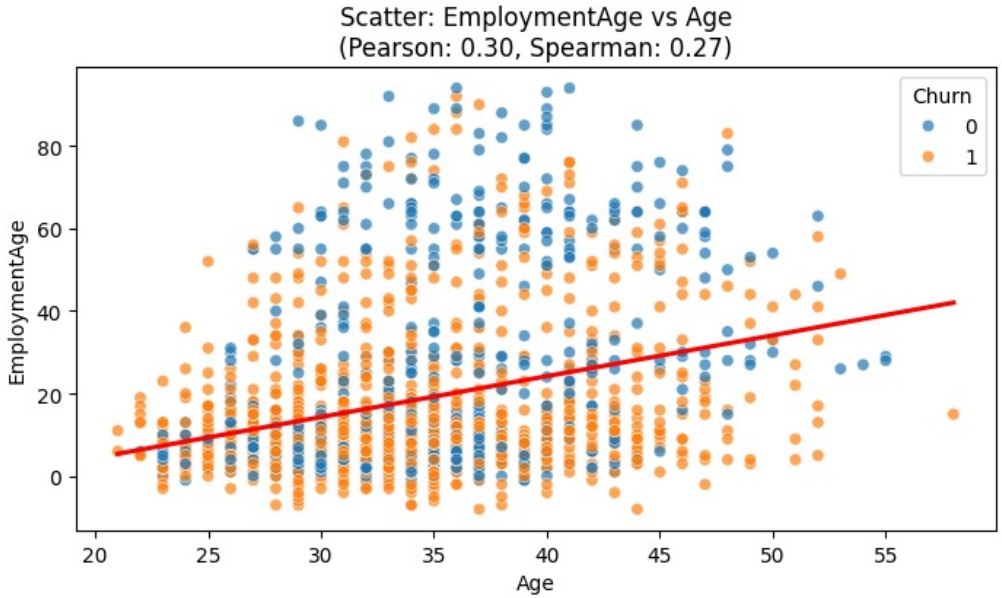
3.2.11 Driver Age and EmploymentAge

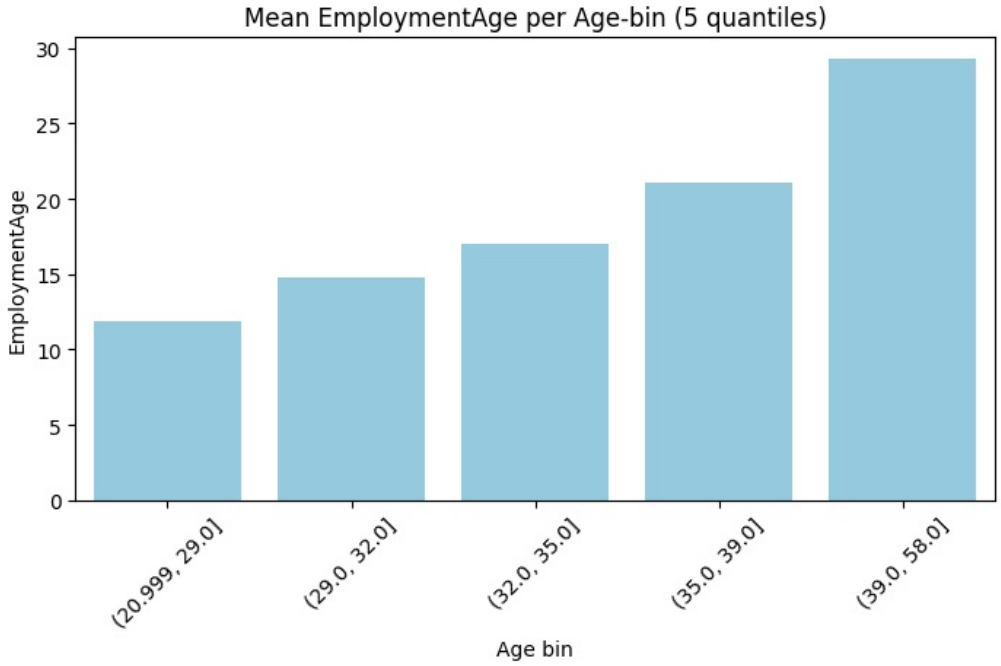
In [613…

```
numerical_numerical_analysis(nonZeroTBVDrivers,'Age','EmploymentAge', hue='Churn')
```

```
Pearson correlation between Age and EmploymentAge: 0.299
Spearman correlation between Age and EmploymentAge: 0.273
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\1352136180.py:25: FutureWarning:

The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True
to adopt the future default and silence this warning.
```





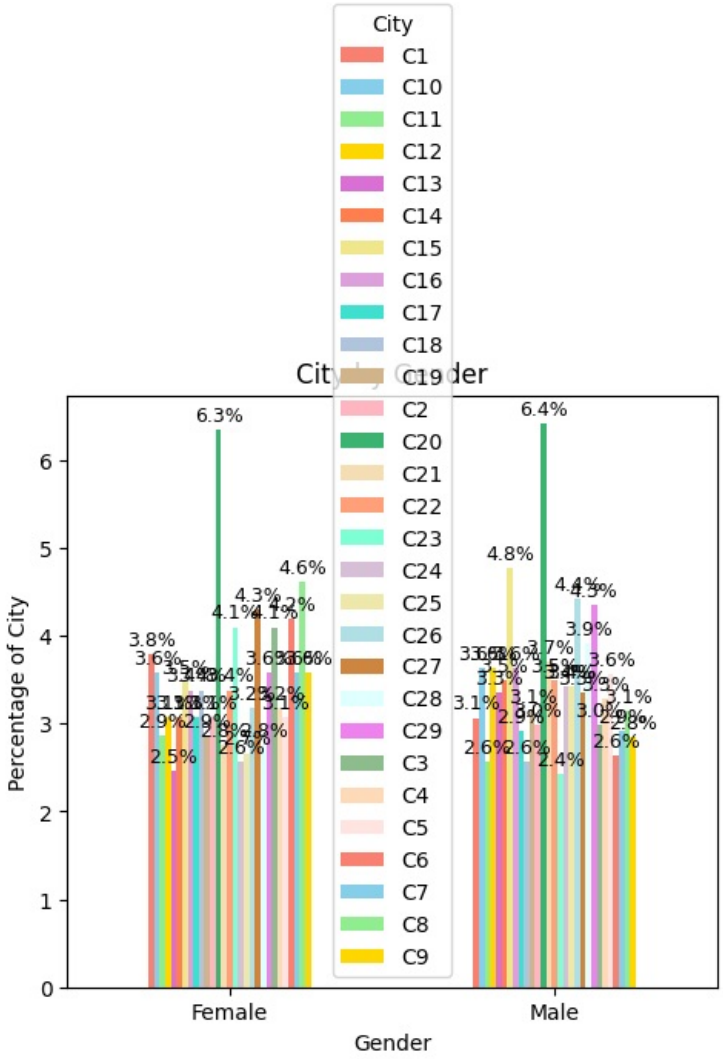
- Age and Employment age are obviously correlated. An older employee will naturally tend to have a longer employment history. But it does show that employees have stayed with the company for past 7 yrs.

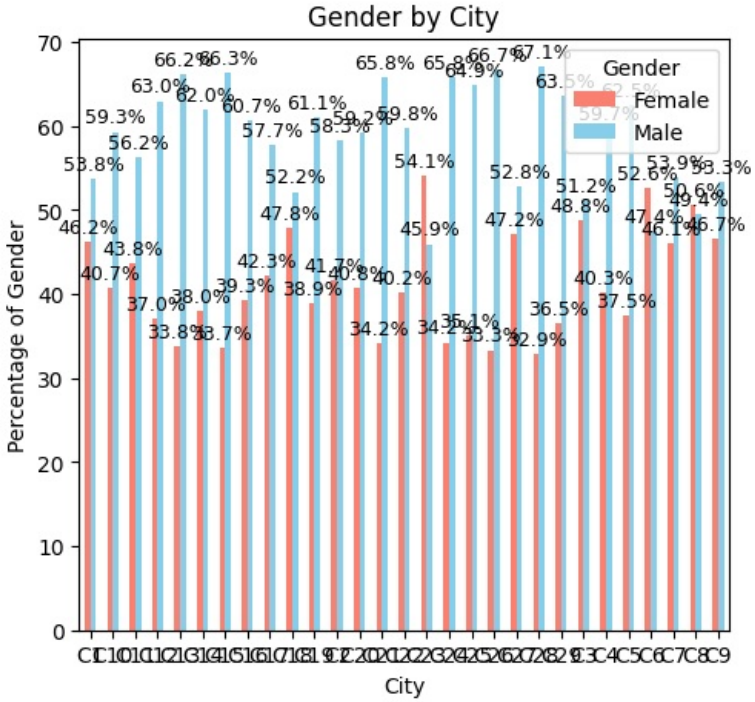
3.2.12 Gender and City

In [614…

```
perform_chi_square_test(driversWithCategories,'Gender','City')
```

```
Contingency Table:
City    C1  C10  C11  C12  C13  C14  C15  C16  C17  C18  ...  C27  C28  C29  \
Gender                                                    ...
Female  37   35   28   30   24   30   34   33   30   33  ...   42   27   35
Male    43   51   36   51   47   49   67   51   41   36  ...   47   55   61

City    C3  C4  C5  C6  C7  C8  C9
Gender
Female  40  31  30  41  35  45  35
Male    42  46  50  37  41  44  40

[2 rows x 29 columns]


Chi-square Test Results:
Chi-square statistic: 35.1509
p-value: 0.1655
Degrees of freedom: 28
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and City are independent.' is True, p-value:0.16551831811805257
```





==========

In [615…
```
crosstab = pd.crosstab(drivers.Gender,drivers.City)
cities = crosstab.columns[crosstab.loc[1] > crosstab.loc[0]].tolist()
print(cities)
```

```
['C23', 'C6', 'C8']
```

- There is no correlation between gender and city
- There are 3 cities 'C23', 'C6', 'C8' out of 29 where females are more than males

In [616…
```
percentageMoreMales = np.round(100*(crosstab.loc[0]-crosstab.loc[1] )/(crosstab.loc[0]),2)
percentageMoreMales
```

Out[616…
```
City
C1     13.95
C10    31.37
C11    22.22
C12    41.18
C13    48.94
C14    38.78
C15    49.25
C16    35.29
C17    26.83
C18     8.33
C19    36.36
C2     28.57
C20    31.11
C21    48.08
C22    32.65
C23   -17.65
C24    47.92
C25    45.83
C26    50.00
C27    10.64
C28    50.91
C29    42.62
C3      4.76
C4     32.61
C5     40.00
C6    -10.81
C7     14.63
C8     -2.27
C9     12.50
dtype: float64
```

In [617…
```
print(percentageMoreMales[percentageMoreMales>49])
print(percentageMoreMales.mean())
```

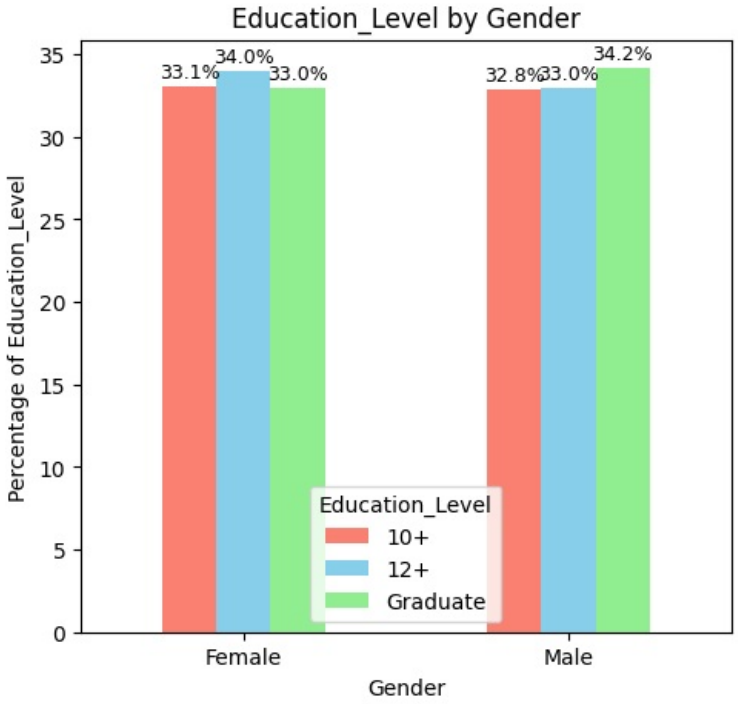
```
City
C15    49.25
C26    50.00
C28    50.91
dtype: float64
28.089655172413792
```

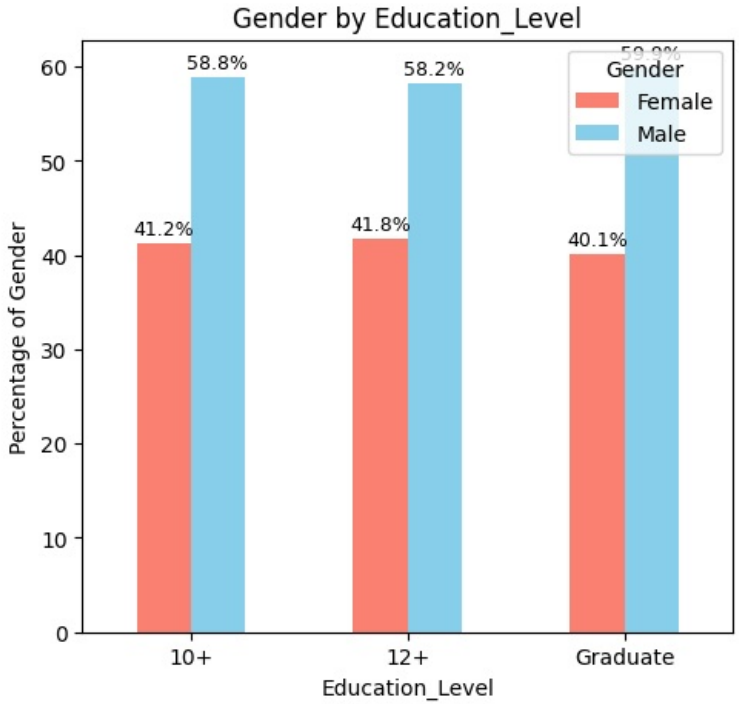
- There are three cities C15, C26 and C28 where percentage males is around 50% more than female drivers.
- In general there are 28% more males in workforce than females

3.2.13 Gender and Education_Level

In [618…
```
perform_chi_square_test(driversWithCategories,'Gender','Education_Level')
```

```
Contingency Table:
Education_Level  10+  12+  Graduate
Gender
Female           323  332       322
Male             461  463       480


Chi-square Test Results:
Chi-square statistic: 0.4417
p-value: 0.8018
Degrees of freedom: 2
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and Education_Level are independent.' is True, p-value:0.8018290006096183
```





==========

- Education levels are similar across genders

3.2.14 Gender and Date of Joining

In [619…

```
df_analysis = driversWithCategories[['Dateofjoining','Gender']].copy()
df_analysis['year'] = df_analysis.Dateofjoining.dt.year
ct = pd.crosstab(df_analysis.year, df_analysis.Gender)
ct['pctMoreMales'] = np.round(100*(ct.Male-ct.Female)/ct.Female,2)
ct['ratioFemaleMale'] = np.round(ct.Female/ct.Male,2)
print(f'In general, company hires {ct['pctMoreMales'].mean()}% more males than female drivers')
print(f'The company hires {np.floor(100*ct['ratioFemaleMale'].mean())} females for every 100 males drivers')
ct
```

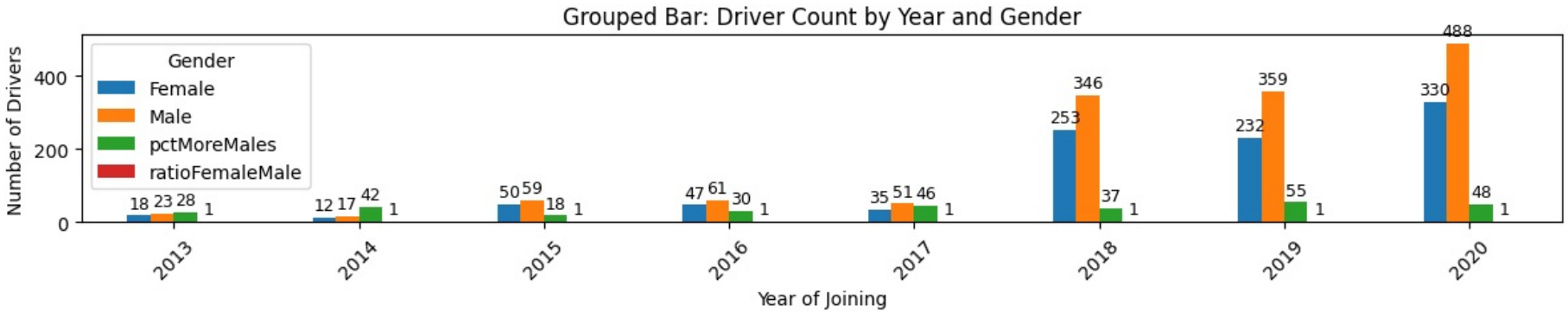
```
In general, company hires 37.791250000000005% more males than female drivers
The company hires 73.0 females for every 100 males drivers
```

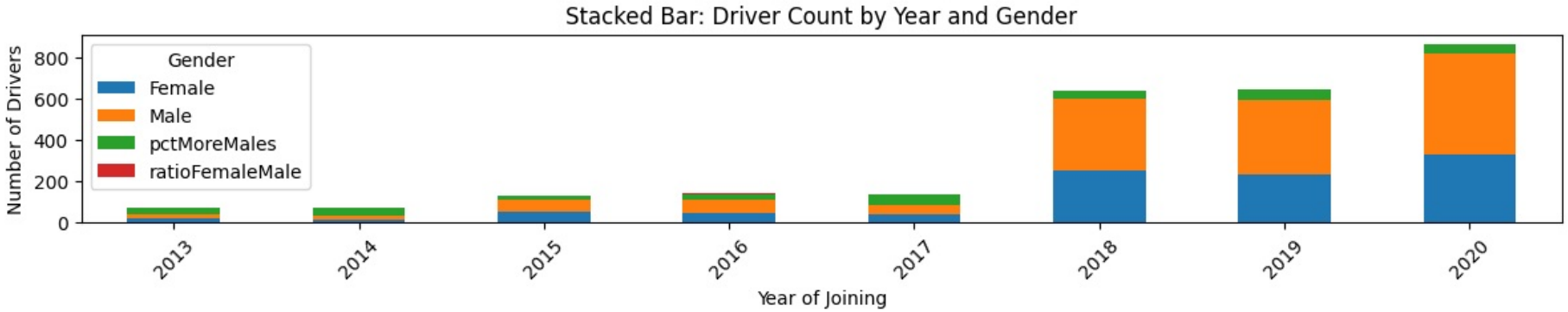
Out[619…

Gender	Female	Male	pctMoreMales	ratioFemaleMale
year				
2013	18	23	27.78	0.78
2014	12	17	41.67	0.71
2015	50	59	18.00	0.85
2016	47	61	29.79	0.77
2017	35	51	45.71	0.69
2018	253	346	36.76	0.73
2019	232	359	54.74	0.65
2020	330	488	47.88	0.68

In [620…

```
# --- Normal (grouped) bar plot ---
fig, axes = plt.subplots(2, 1, figsize=(12, 5))
bars = ct.plot(kind='bar', ax=axes[0])
axes[0].set_title('Grouped Bar: Driver Count by Year and Gender')
axes[0].set_xlabel('Year of Joining')
axes[0].set_ylabel('Number of Drivers')
axes[0].tick_params(axis='x', rotation=45)
axes[0].legend(title='Gender')
annotate_bars(bars,'%.0f')

# --- Stacked bar plot ---
bars=ct.plot(kind='bar', stacked=True, ax=axes[1])
axes[1].set_title('Stacked Bar: Driver Count by Year and Gender')
axes[1].set_xlabel('Year of Joining')
axes[1].set_ylabel('Number of Drivers')
axes[1].tick_params(axis='x', rotation=45)
axes[1].legend(title='Gender')

plt.tight_layout()
plt.show()
```





- Every year, the company hires 37.8% more males than female drivers
- In 2019, the company hired larger ratio of female drivers

3.2.15 Gender and LastWorkingDate

In [621…
```
df_analysis = driversWithCategories.loc[driversWithCategories.Churn=='Churned'][['LastWorkingDate','Gender']].copy()
df_analysis['year'] = df_analysis.LastWorkingDate.dt.year
print(pd.crosstab(df_analysis.year, df_analysis.Gender, normalize='index'))
```

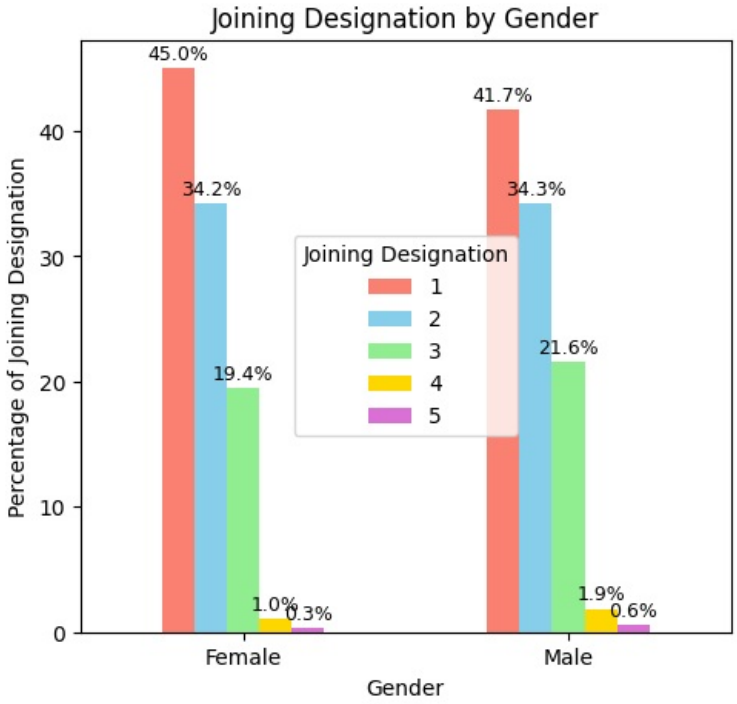
```
Gender    Female      Male
year
2018    0.400000  0.600000
2019    0.416970  0.583030
2020    0.409669  0.590331
```

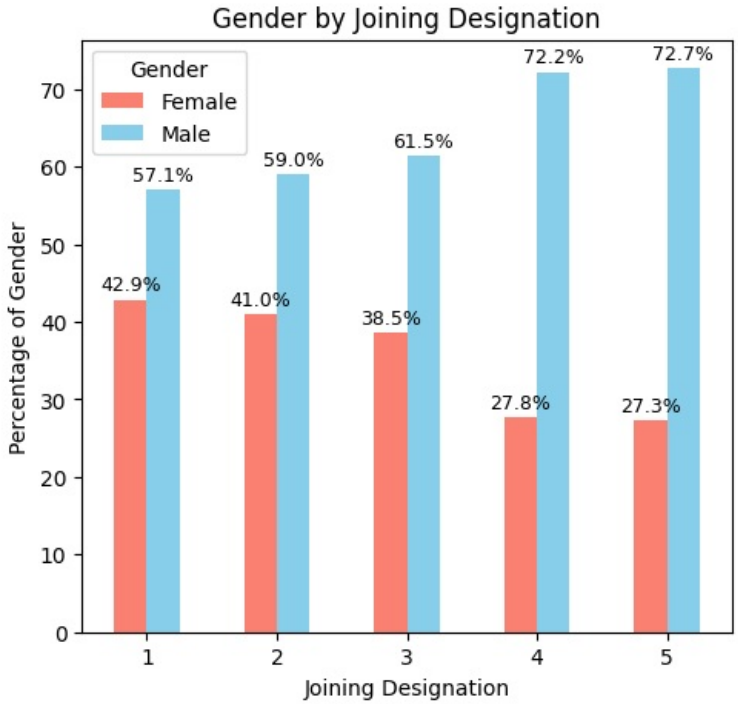
- The ratio of churn across genders is same as the overall ratio of genders. We can say churn is unaffected by gender.

3.2.16 Gender and Joining Designation

In [622…
```
perform_chi_square_test(driversWithCategories,'Gender','Joining Designation')
```

```
Contingency Table:
Joining Designation    1    2    3   4  5
Gender
Female               440  334  190  10  3
Male                 586  481  303  26  8


Chi-square Test Results:
Chi-square statistic: 6.1970
p-value: 0.1849
Degrees of freedom: 4
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and Joining Designation are independent.' is True, p-value:0.18490824967273095
```





==========

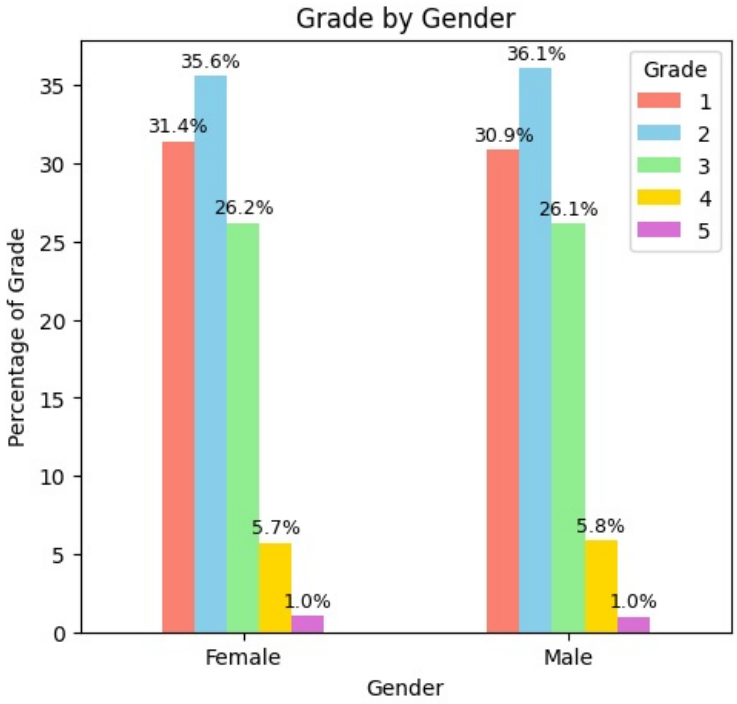
- There are lesser females hired at higher joining designations

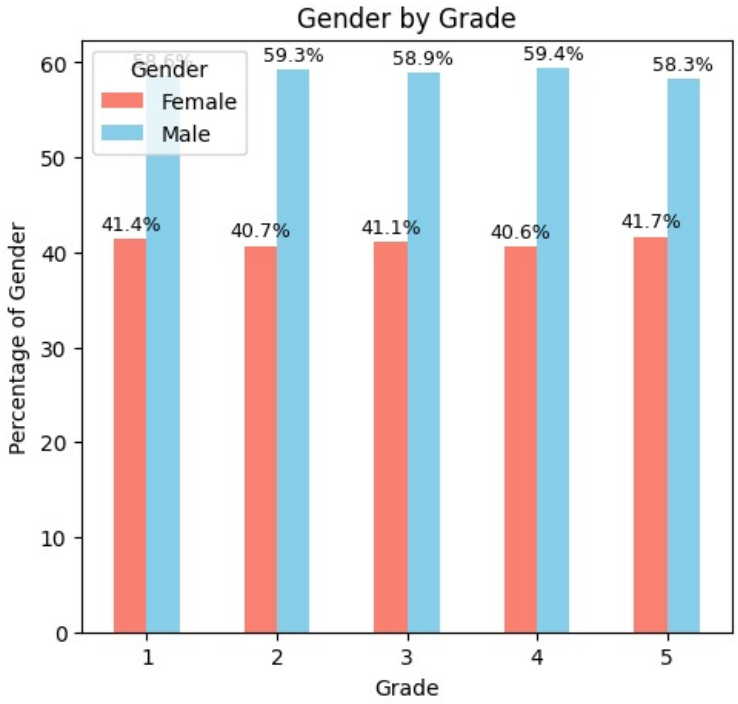
3.2.17 Gender and Joining Grade

In [623…

```
perform_chi_square_test(driversWithCategories,'Gender','Grade')
```

```
Contingency Table:
Grade     1    2    3   4   5
Gender
Female  307  348  256  56  10
Male    434  507  367  82  14


Chi-square Test Results:
Chi-square statistic: 0.1037
p-value: 0.9987
Degrees of freedom: 4
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and Grade are independent.' is True, p-value:0.9987002718014714
```





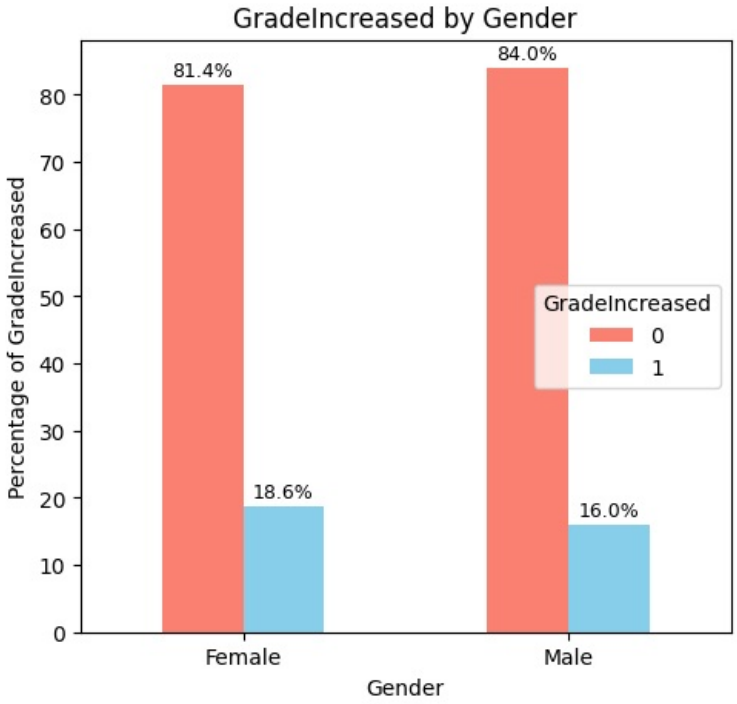
==========

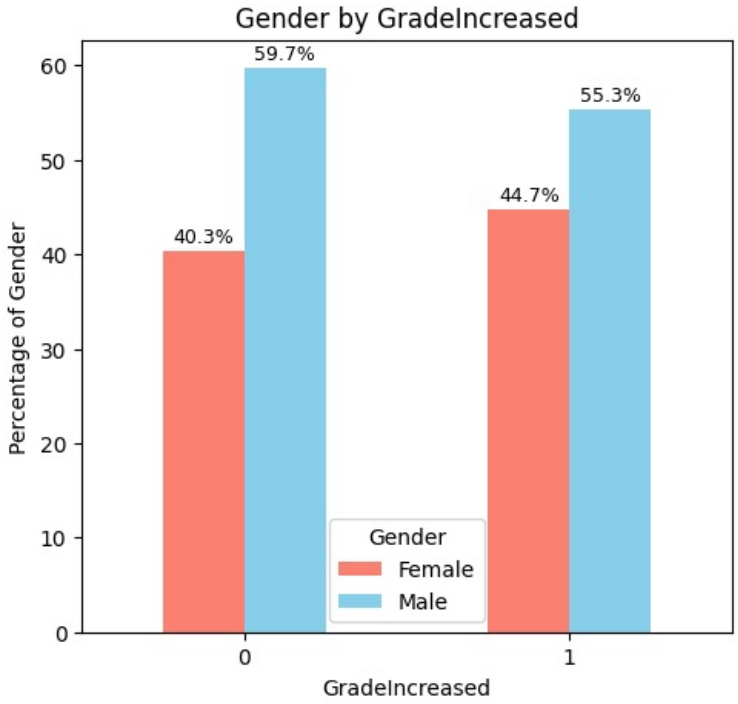
- Gender does not have impact on joining grade

3.2.18 Gender and GradeIncreased

In [624…
```
perform_chi_square_test(driversWithCategories,'Gender','GradeIncreased')
```

```
Contingency Table:
GradeIncreased     0    1
Gender
Female           795  182
Male            1179  225


Chi-square Test Results:
Chi-square statistic: 2.5734
p-value: 0.1087
Degrees of freedom: 1
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and GradeIncreased are independent.' is True, p-value:0.10867323802580135
```





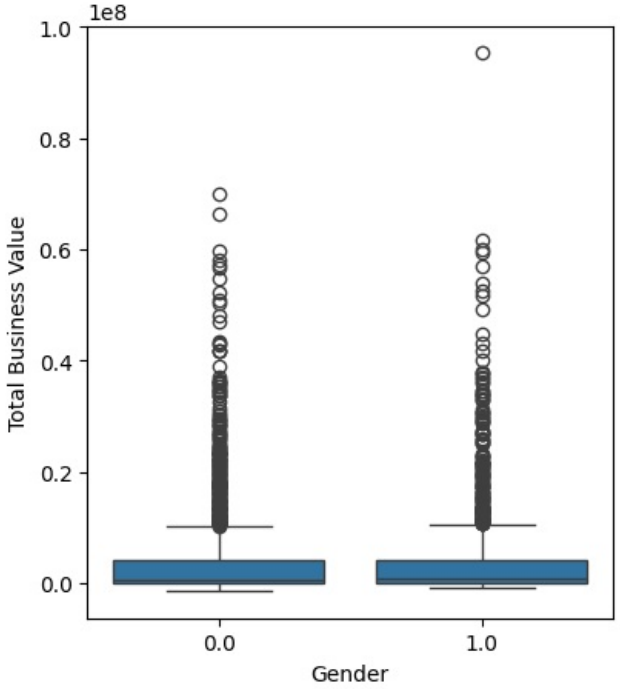
==========

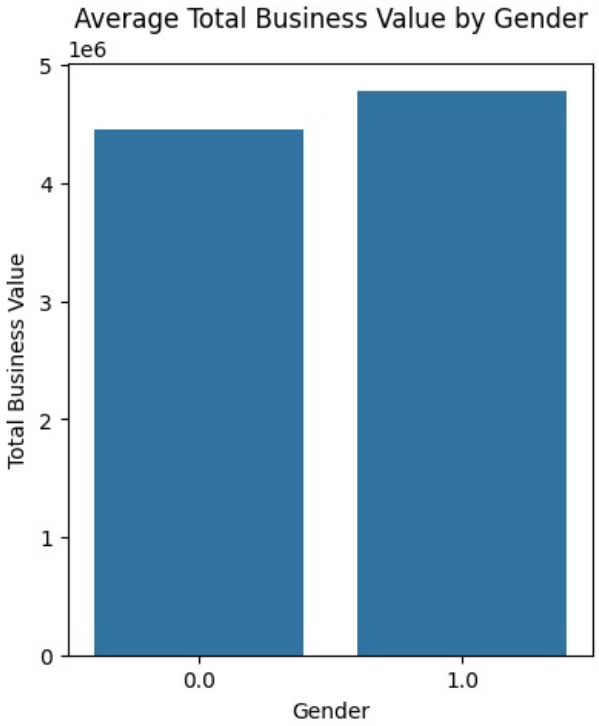
- Gender and GradeIncreased are independent

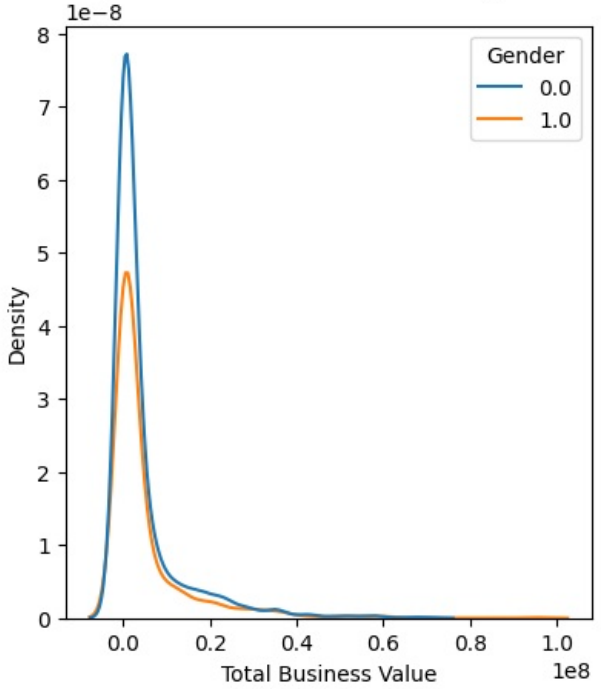
3.2.19 Gender and Total Business Value

In [625…
```
perform_anova_kruskal(drivers,numerical_column='Total Business Value', categorical_column='Gender', test=test,log="detailed")
```

```
Ho = Gender and Total Business Value are independent. Medians of groups of Total Business Value based on Gender are same
Ha = Gender and Total Business Value are DEPENDENT. Medians of groups of Total Business Value based on Gender are different
Kruskal-Wallis Result:0.933393696679572,0.33398262504199605
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Gender and Total Business Value are independent. Medians of groups of Total Business Value based on Gender are same In
simple words, there’s no substantial Total Business Value difference across Gender values' is True, p-value:0.33
```







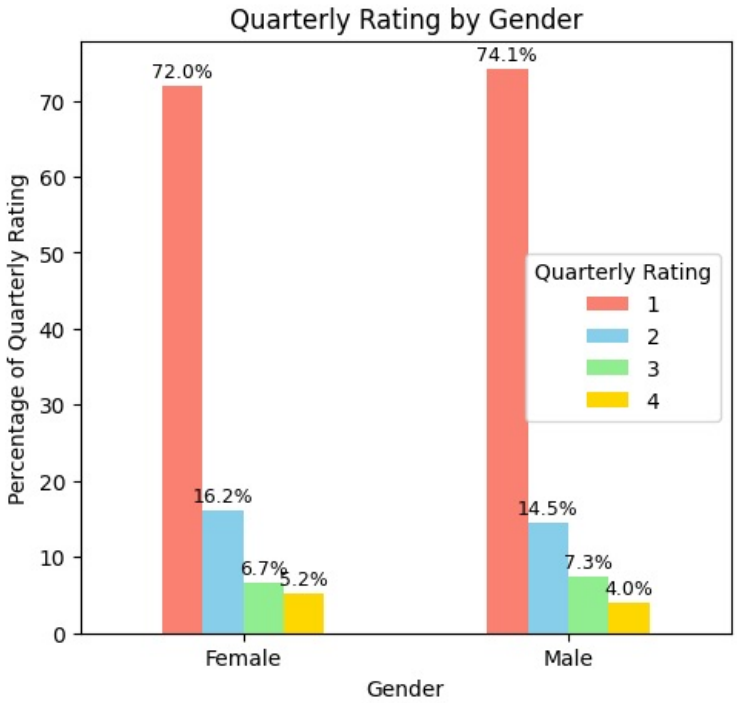
	Gender	Total Business Value
0	0.0	4.453073e+06
1	1.0	4.778831e+06

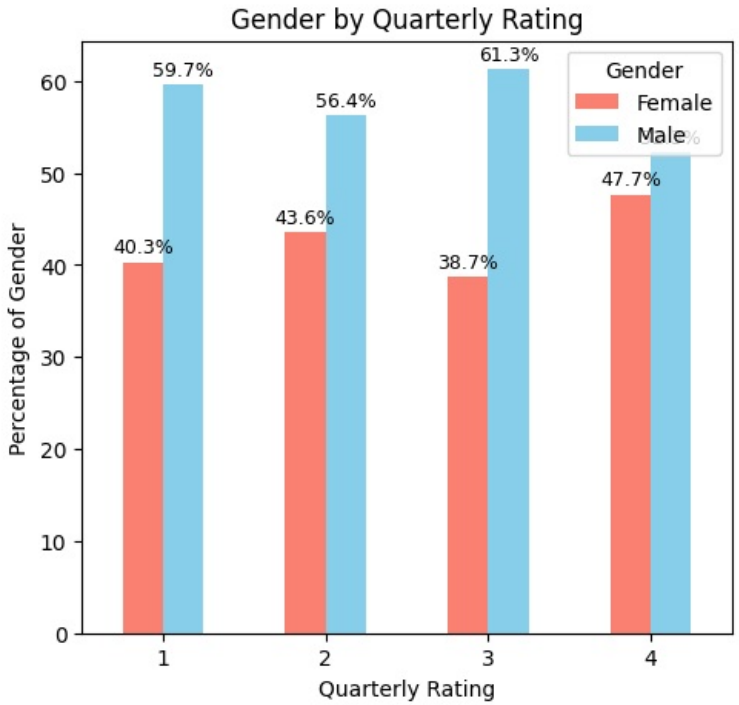
- Gender does not affect Total Business Value

3.2.20 Gender and Quarterly Rating

In [626…
```
perform_chi_square_test(driversWithCategories,'Gender','Quarterly Rating')
```

```
Contingency Table:
Quarterly Rating     1    2    3   4
Gender
Female             703  158   65  51
Male              1041  204  103  56


Chi-square Test Results:
Chi-square statistic: 3.7242
p-value: 0.2928
Degrees of freedom: 3
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and Quarterly Rating are independent.' is True, p-value:0.2928275069846611
```





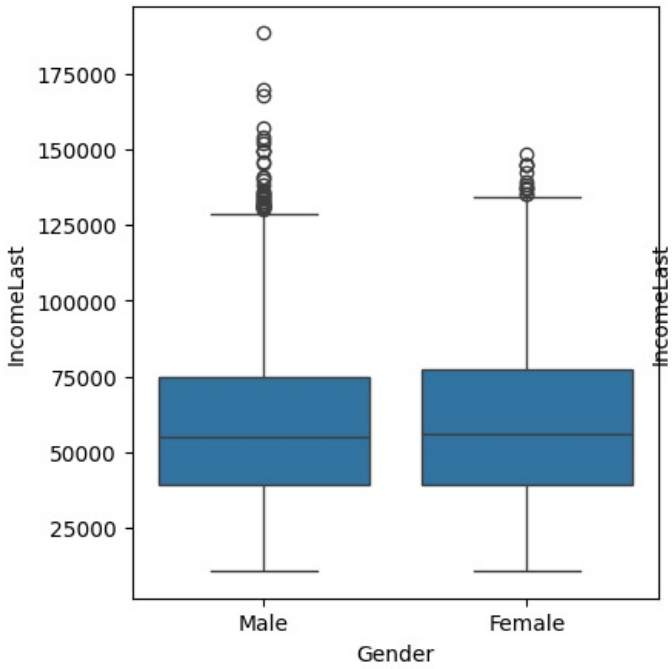
==========

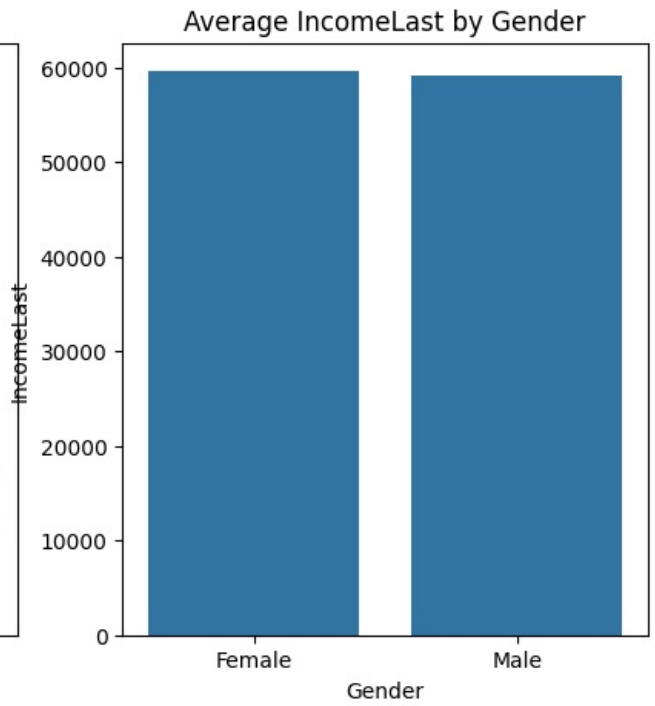
- Quarterly Rating are similar across genders

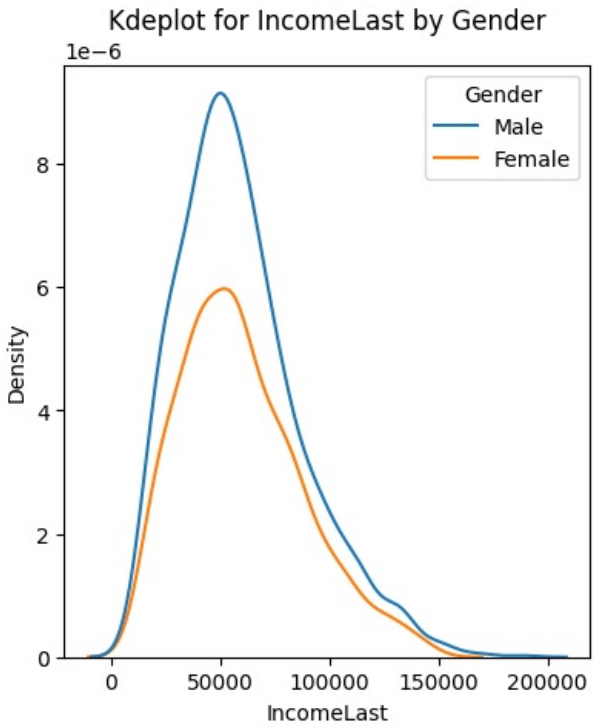
3.2.21 Gender and Income

In [627…
```
perform_anova_kruskal(driversWithCategories,numerical_column='IncomeLast', categorical_column='Gender', test=test,log="detailed")
```

```
Ho = Gender and IncomeLast are independent. Medians of groups of IncomeLast based on Gender are same
Ha = Gender and IncomeLast are DEPENDENT. Medians of groups of IncomeLast based on Gender are different
Kruskal-Wallis Result:0.3846149964109669,0.5351436584318934
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Gender and IncomeLast are independent. Medians of groups of IncomeLast based on Gender are same In simple words, there
's no substantial IncomeLast difference across Gender values' is True, p-value:0.54
```







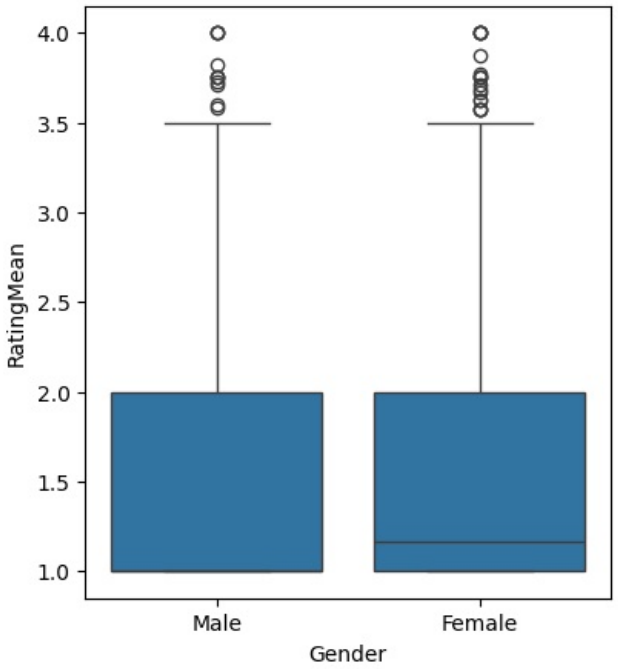
	Gender	IncomeLast
0	Female	59587.636643
1	Male	59157.768519

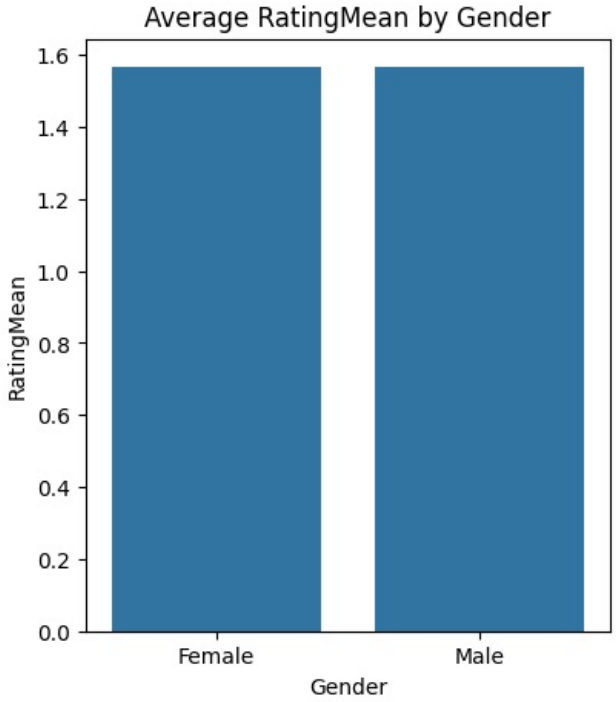
- Gender does not affect Income

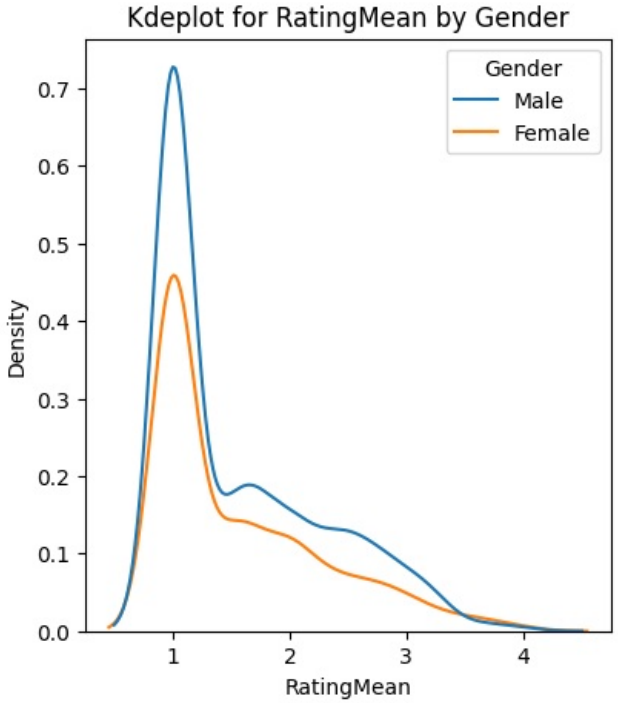
3.2.22 Gender and Rating

In [628…
```
perform_anova_kruskal(driversWithCategories,numerical_column='RatingMean', categorical_column='Gender', test=test,log="detailed")
```

```
Ho = Gender and RatingMean are independent. Medians of groups of RatingMean based on Gender are same
Ha = Gender and RatingMean are DEPENDENT. Medians of groups of RatingMean based on Gender are different
Kruskal-Wallis Result:0.003339625424636433,0.9539163080918562
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Gender and RatingMean are independent. Medians of groups of RatingMean based on Gender are same In simple words, there
’s no substantial RatingMean difference across Gender values' is True, p-value:0.95
```







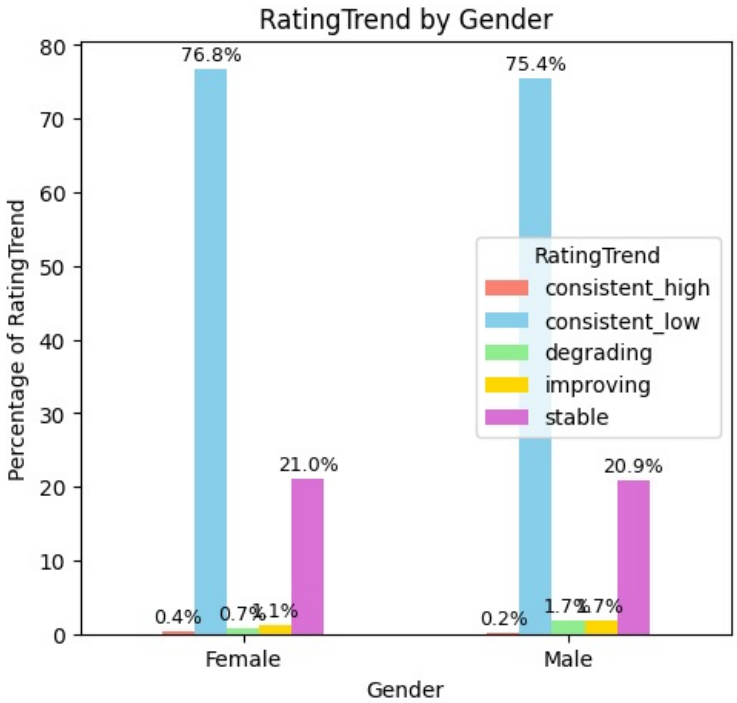

```
   Gender  RatingMean
0  Female    1.566544
1    Male    1.566137
```

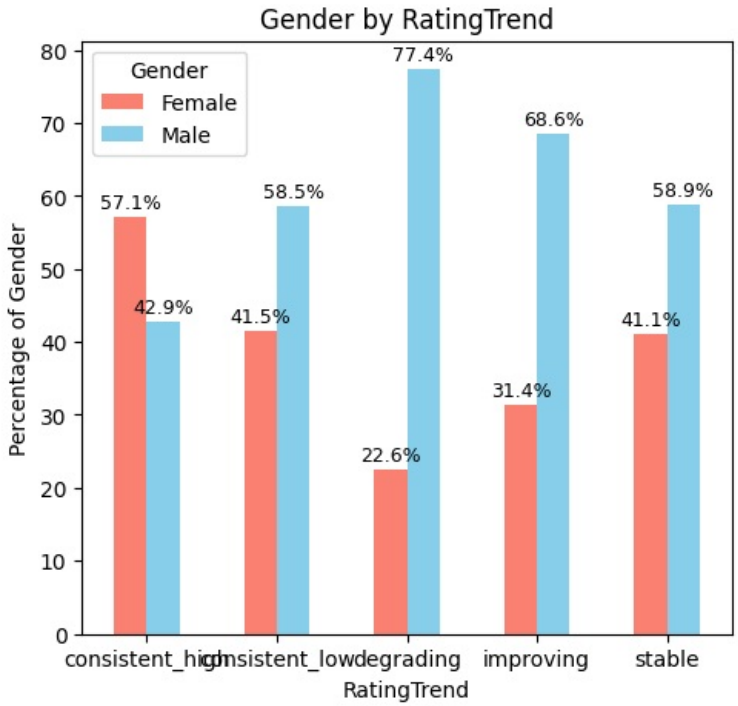
- Ratings are similar across genders

3.2.23 Gender and RatingTrend

In [629…
```
perform_chi_square_test(driversWithCategories,'Gender','RatingTrend')
```

```
Contingency Table:
RatingTrend  consistent_high  consistent_low  degrading  improving  stable
Gender
Female                     4             750          7         11     205
Male                       3            1059         24         24     294


Chi-square Test Results:
Chi-square statistic: 6.5840
p-value: 0.1596
Degrees of freedom: 4
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and RatingTrend are independent.' is True, p-value:0.1595769367752142
```





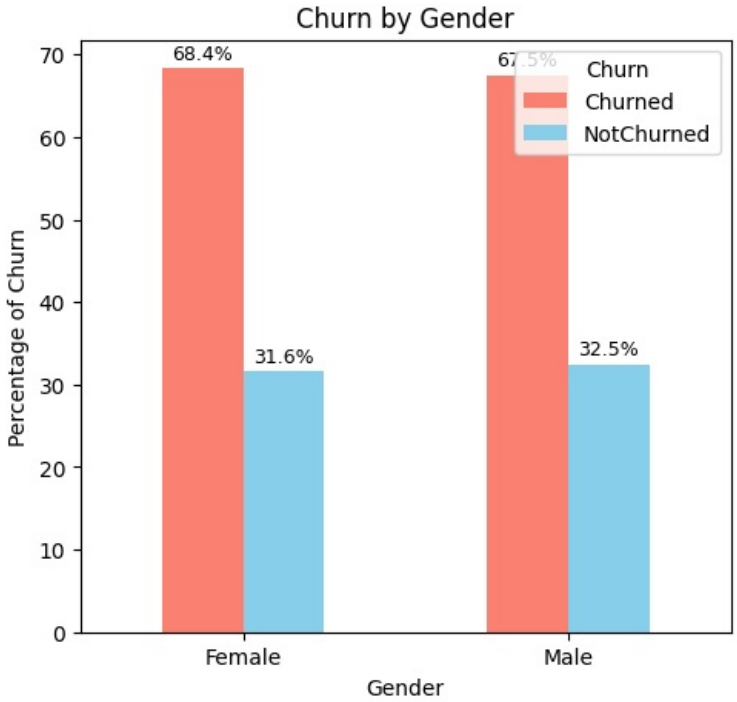
==========

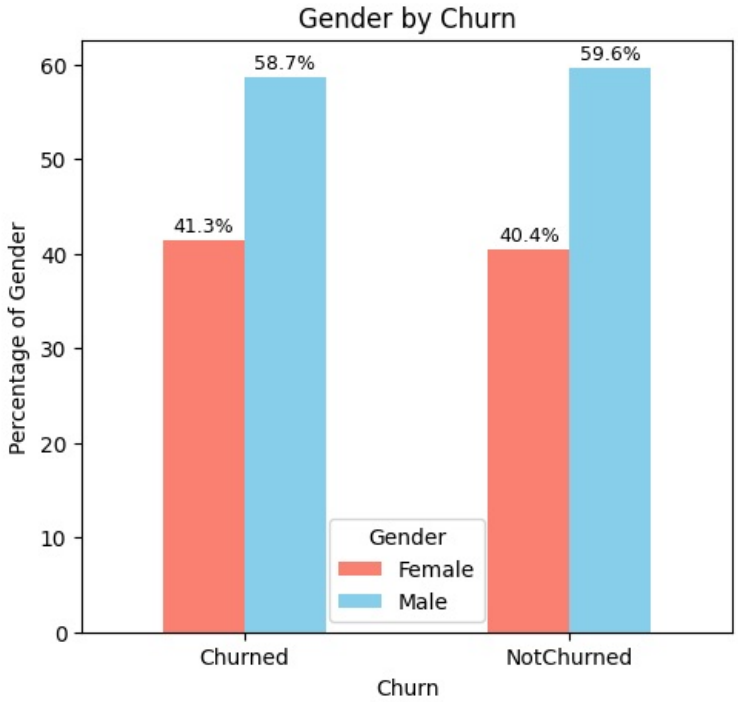
- There are more women than men that have consistently high ratings
- For drivers whose rating is degrading, the ratio of men is more than the general ratio of men to women. It means a larger percentage of men have degrading performance rating.

3.2.24 Gender and Churn

In [630…
```
perform_chi_square_test(driversWithCategories,'Gender','Churn')
```

```
Contingency Table:
Churn   Churned  NotChurned
Gender
Female      668         309
Male        948         456


Chi-square Test Results:
Chi-square statistic: 0.1544
p-value: 0.6944
Degrees of freedom: 1
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Gender and Churn are independent.' is True, p-value:0.6943902798506425
```





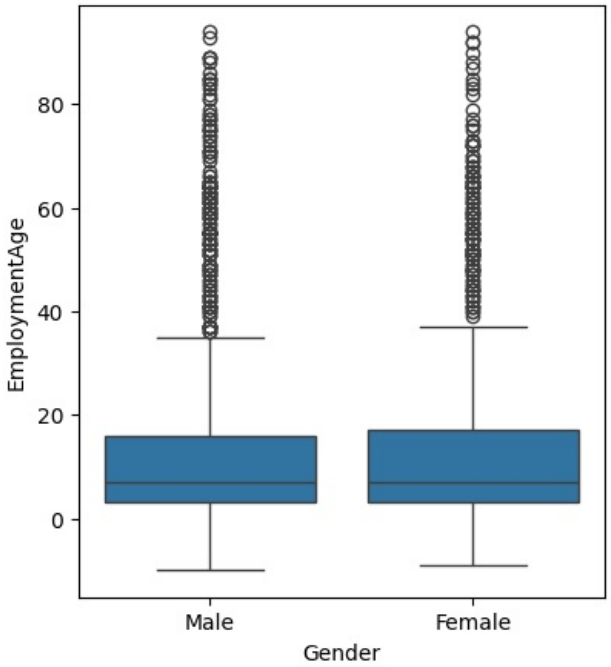
==========

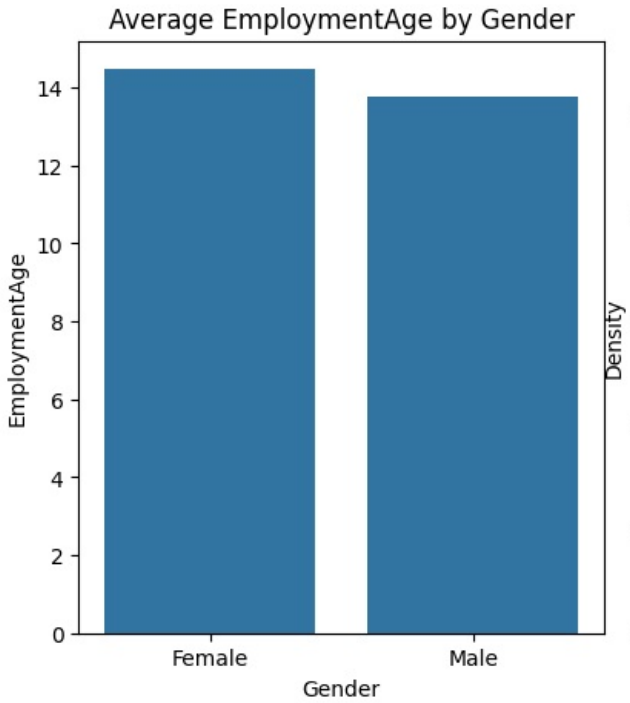
- Churn is similar across genders

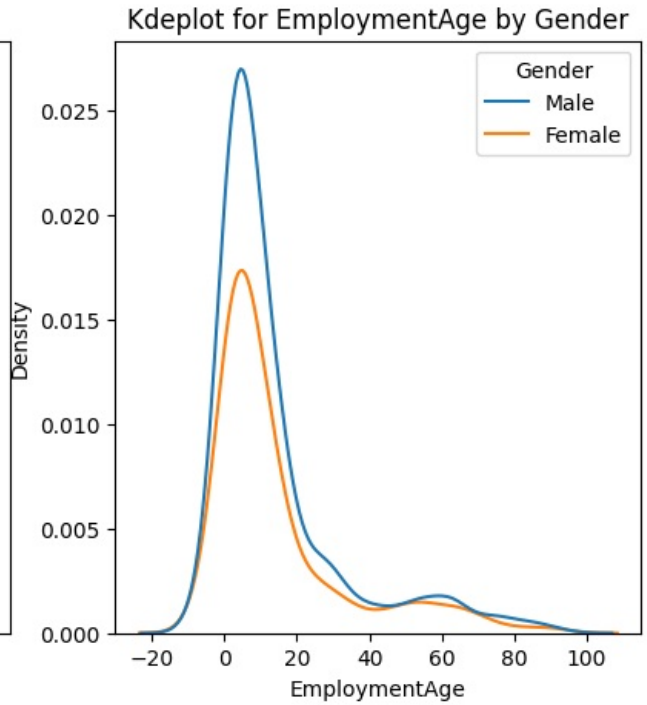
3.2.25 Gender and EmploymentAge

In [631…
```
perform_anova_kruskal(driversWithCategories,numerical_column='EmploymentAge', categorical_column='Gender', test=test,log="detailed")
```

```
Ho = Gender and EmploymentAge are independent. Medians of groups of EmploymentAge based on Gender are same
Ha = Gender and EmploymentAge are DEPENDENT. Medians of groups of EmploymentAge based on Gender are different
Kruskal-Wallis Result:0.21097233242125069,0.6460063650316499
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'Gender and EmploymentAge are independent. Medians of groups of EmploymentAge based on Gender are same In simple words,
there’s no substantial EmploymentAge difference across Gender values' is True, p-value:0.65
```








```
   Gender  EmploymentAge
0  Female      14.479017
1    Male      13.763533
```

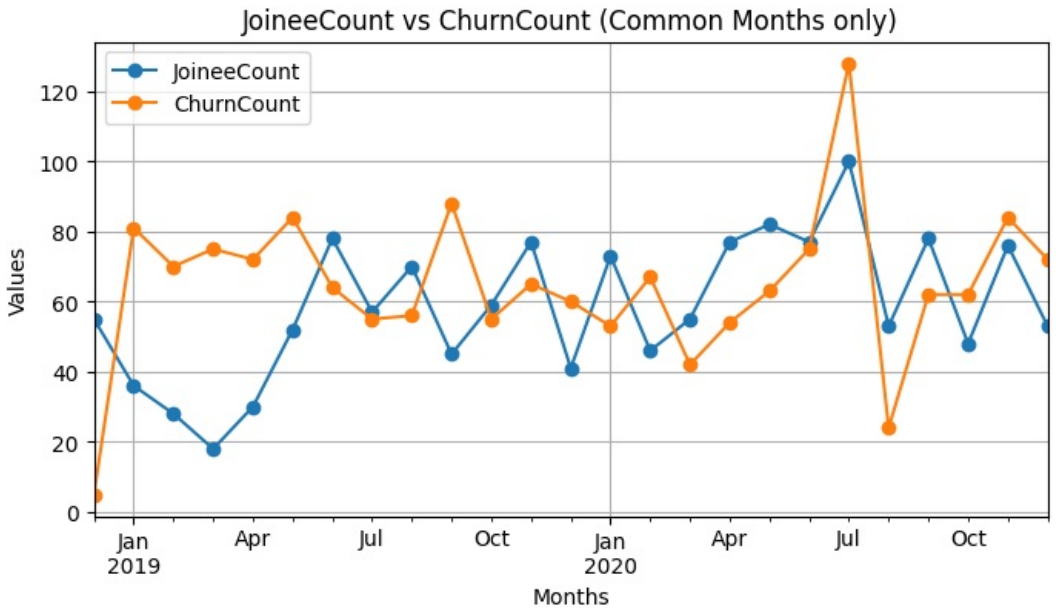
- There's no substantial EmploymentAge difference across Gender values

3.2.26 JoiningDate and LastWorkingDate

- We created two series earlier -monthly_counts_series_joinees, monthly_counts_series_churn
- Lets analyze them together to compare hiring and churning trends

In [632…
```
df_analysis = pd.concat([monthly_counts_series_joinees, monthly_counts_series_churn], axis=1, join='inner')
df_analysis.columns = ['JoineeCount', 'ChurnCount']
# Plot both columns
df_analysis.plot(kind='line', marker='o', figsize=(8,4))
```

```
plt.title('JoineeCount vs ChurnCount (Common Months only)')
plt.xlabel('Months')
plt.ylabel('Values')
plt.grid(True)
plt.show()
```



In [633…
```
df_analysis.sum(axis=0)
```

Out[633…
```
JoineeCount    1464
ChurnCount     1616
dtype: int64
```

- This comparison reveals a stark reality of the hiring numbers and state of affairs in the company.
- While we were under impresssion earlier that hiring has increased, these numbers show that churn has increased even further.
- So while 1464 drivers joined, 1616 drivers left the company !!

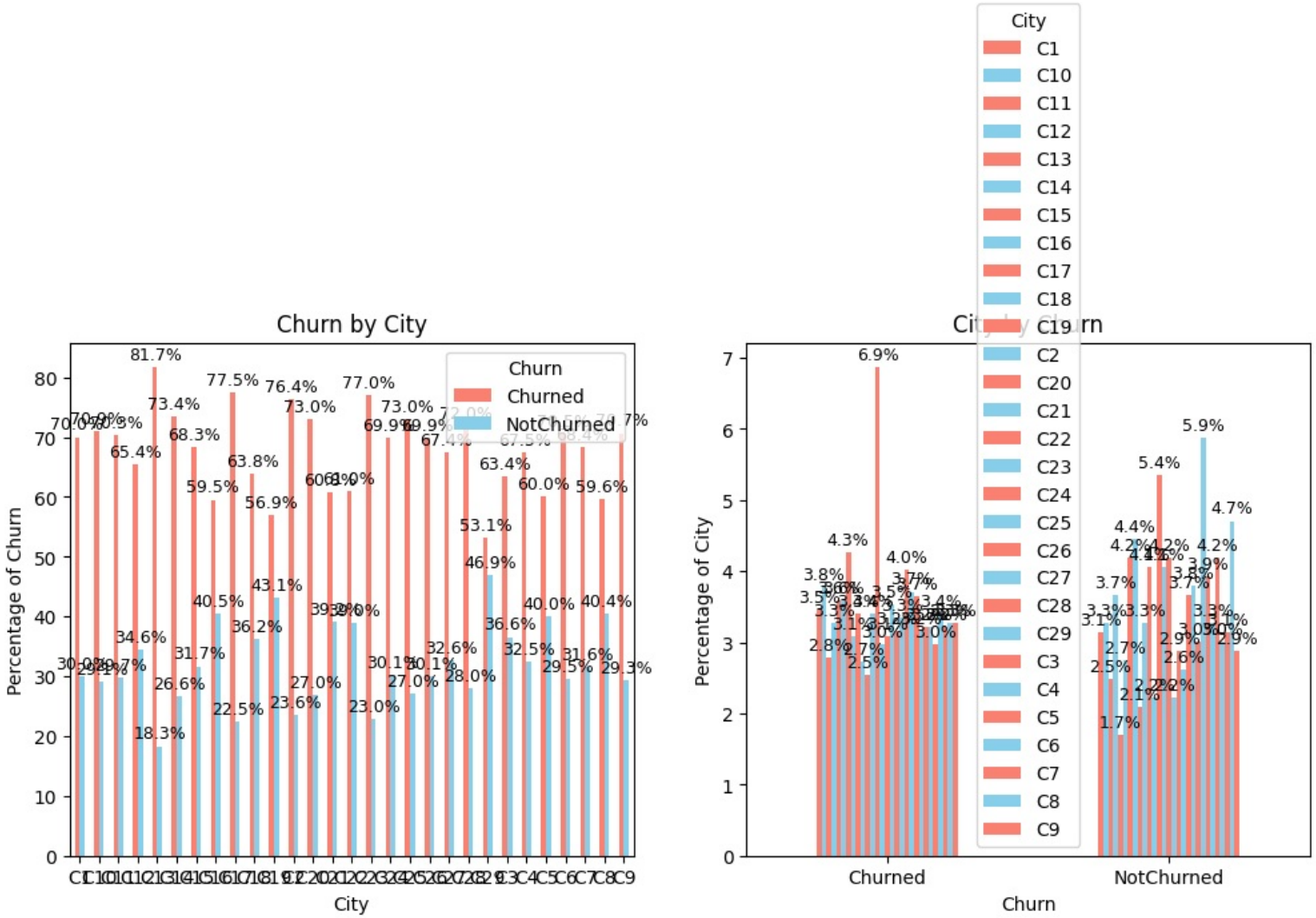
3.2.27 Churn and City

In [634…
```
perform_chi_square_test(driversWithCategories,'City','Churn')
```

Contingency Table:

City	Churned	NotChurned
C1	56	24
C10	61	25
C11	45	19
C12	53	28
C13	58	13
C14	58	21
C15	69	32
C16	50	34
C17	55	16
C18	44	25
C19	41	31
C2	55	17
C20	111	41
C21	48	31
C22	50	32
C23	57	17
C24	51	22
C25	54	20
C26	65	28
C27	60	29
C28	59	23
C29	51	45
C3	52	30
C4	52	25
C5	48	32
C6	55	23
C7	52	24
C8	53	36
C9	53	22

Chi-square Test Results:
Chi-square statistic: 46.9172
p-value: 0.0140
Degrees of freedom: 28
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>City and Churn are DEPENDENT' is True, p-value:0.013977549937173567. However, association is Weak(CramersV=0.14)



==========

- City and Churn are DEPENDENT. It means some cities have more churn compared to others. p-value:0.013977549937173567. However, association is Weak(CramersV=0.14)

Analyse join and churn trend across cities

We want to analyse the joining and churning trend in the cities over the months so as to find if there is any city that has abnormally high data.

In [635…

```
df_original
```

Out[635…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	2019-01-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	2381060	2
1	2019-02-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	-665480	2
2	2019-03-01	1	28.0	0.0	C23	2	57387	2018-12-24	2019-11-03	1	1	0	2
3	2020-11-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1
4	2020-12-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	2020-08-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	740280	3
19100	2020-09-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	448370	3
19101	2020-10-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	0	2
19102	2020-11-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	200420	2
19103	2020-12-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	411480	2

19104 rows × 13 columns

In [636…
```
df_original.columns
```

Out[636…
```
Index(['MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',
       'Income', 'Dateofjoining', 'LastWorkingDate', 'Joining Designation',
       'Grade', 'Total Business Value', 'Quarterly Rating'],
      dtype='object')
```

In [637…
```
df_original['EmploymentStatus'] = np.where(df_original.LastWorkingDate.dt.year==2067,1,0)
df_original
```

Out[637…

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating	EmploymentStatus
0	2019-01-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	2381060	2	1
1	2019-02-01	1	28.0	0.0	C23	2	57387	2018-12-24	2067-12-31	1	1	-665480	2	1
2	2019-03-01	1	28.0	0.0	C23	2	57387	2018-12-24	2019-11-03	1	1	0	2	0
3	2020-11-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1	1
4	2020-12-01	2	31.0	0.0	C7	2	67016	2020-06-11	2067-12-31	2	2	0	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19099	2020-08-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	740280	3	1
19100	2020-09-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	448370	3	1
19101	2020-10-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	0	2	1
19102	2020-11-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	200420	2	1
19103	2020-12-01	2788	30.0	0.0	C27	2	70254	2020-08-06	2067-12-31	2	2	411480	2	1

19104 rows × 14 columns

In [638…

```
city_monthly_data = df_original.groupby(["City","MMM-YY"],as_index=False).EmploymentStatus.sum()
city_monthly_data
```

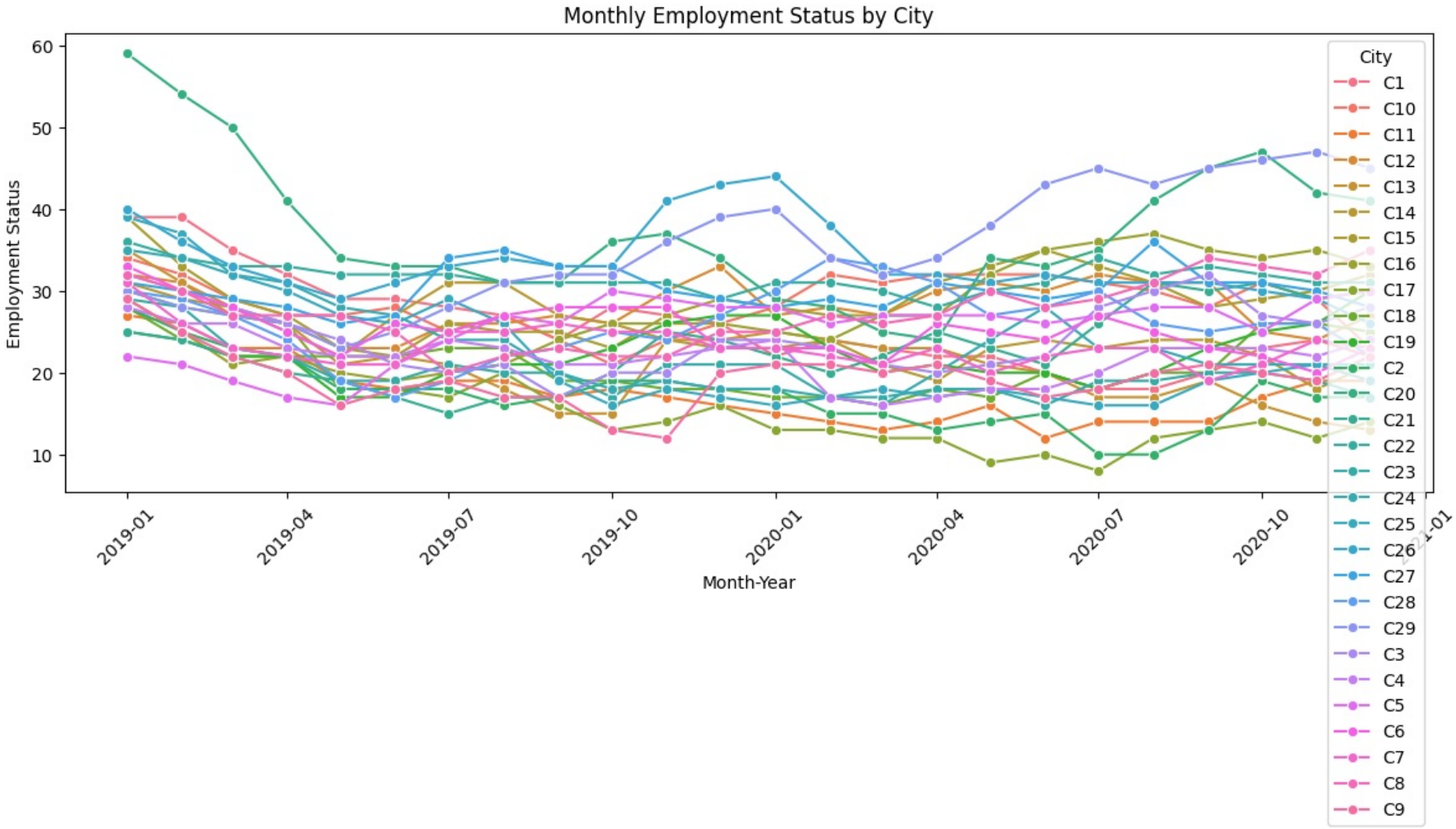
Out[638…

	City	MMM-YY	EmploymentStatus
0	C1	2019-01-01	39
1	C1	2019-02-01	39
2	C1	2019-03-01	35
3	C1	2019-04-01	32
4	C1	2019-05-01	29
...	...	...	...
691	C9	2020-08-01	20
692	C9	2020-09-01	21
693	C9	2020-10-01	20
694	C9	2020-11-01	19
695	C9	2020-12-01	22

696 rows × 3 columns

In [639…

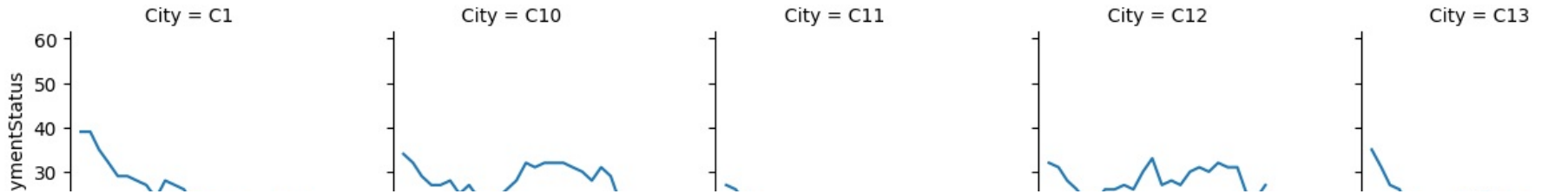
```
# Create line plot
plt.figure(figsize=(12, 6))
sns.lineplot(
    data=city_monthly_data,
    x='MMM-YY',
    y='EmploymentStatus',
    hue='City',
    marker='o'
)

plt.title('Monthly Employment Status by City')
plt.xlabel('Month-Year')
plt.ylabel('Employment Status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

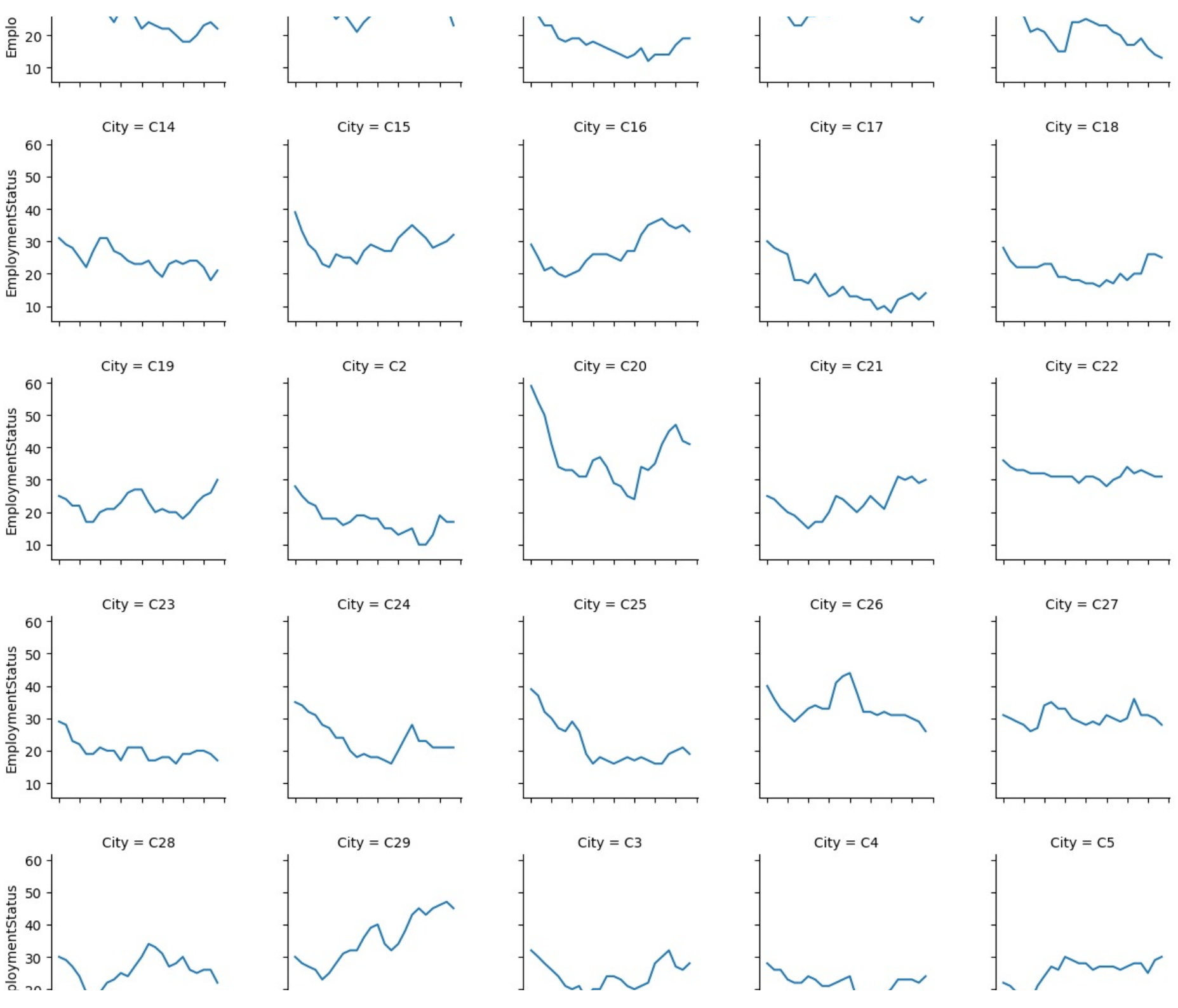


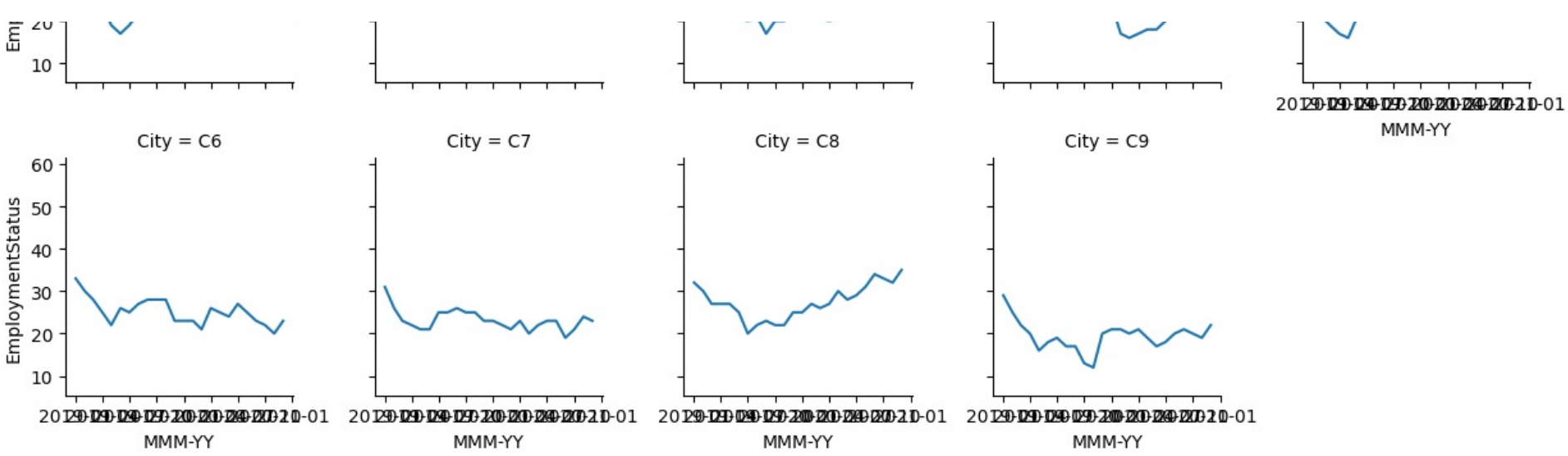
- With 29 cities, it's visually overloaded graph.
- We can use a facet grid to show one small multiple per city

In [640…
```
g = sns.FacetGrid(city_monthly_data, col='City', col_wrap=5, height=2.5)
g.map(sns.lineplot, 'MMM-YY', 'EmploymentStatus')
```

Out[640…
```
<seaborn.axisgrid.FacetGrid at 0x25a5527db50>
```




City = C14
City = C15
City = C16
City = C17
City = C18
City = C19
City = C2
City = C20
City = C21
City = C22
City = C23
City = C24
City = C25
City = C26
City = C27
City = C28
City = C29
City = C3
City = C4
City = C5
EmploymentStatus
10
20
30
40
50
60



Focus on top cities

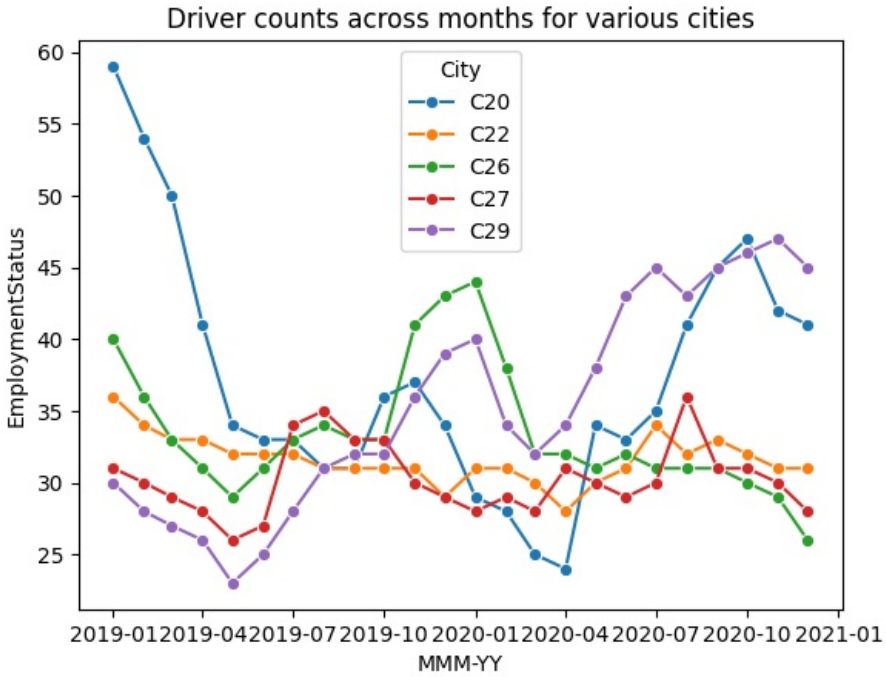
- As cities are too many, let us start by focusing on key cities
- Top 5 cities by average employment

In [641…

```
top_cities = (
    city_monthly_data.groupby('City')['EmploymentStatus']
    .mean()
    .sort_values(ascending=False)
    .head(5)
    .index
)

filtered = city_monthly_data[city_monthly_data['City'].isin(top_cities)]

sns.lineplot(data=filtered, x='MMM-YY', y='EmploymentStatus', hue='City', marker='o')
plt.title("Driver counts across months for various cities")
```

Out[641…

```
Text(0.5, 1.0, 'Driver counts across months for various cities')
```



In 4 of the top 5 employing cities (c20,C22,C26,C27), the employee count DECREASED over the last two years Only in one of the top 5 cities, C29 has the count increased.

- Let us try to analyse City stability / volatility
- We will do this by measuring the standard deviation of driver counts per city

In [642…

```
employeeStabilityAtCityLevel = city_monthly_data.groupby('City',as_index=False)['EmploymentStatus'].std().sort_values(by='EmploymentStatus')
employeeStabilityAtCityLevel.rename(columns={'EmploymentStatus':'DriverCount'}, inplace=True)
employeeStabilityAtCityLevel
```

Out[642…

	City	DriverCount
14	C22	1.689160
26	C7	2.484371
19	C27	2.489107
3	C12	2.947610
23	C4	3.020462
25	C6	3.050006
15	C23	3.141298
9	C18	3.331884
10	C19	3.335144
1	C10	3.400501
28	C9	3.476370
5	C14	3.574264
2	C11	3.943211
24	C5	3.963603
6	C15	4.071819
27	C8	4.096437
11	C2	4.230454
22	C3	4.250959
20	C28	4.333891
18	C26	4.568322
13	C21	4.693312
4	C13	5.368824
16	C24	5.453193
7	C16	5.785884
0	C1	5.810878
8	C17	6.068301
17	C25	6.940221
21	C29	7.511223
12	C20	8.811122

- Cities C22, C7, C27 and C12 have lower attrition rates and stable number of drivers.

In [643…

```
employeeStabilityAtCityLevel[-5:]
```

Out[643…

	City	DriverCount
0	C1	5.810878
8	C17	6.068301
17	C25	6.940221
21	C29	7.511223
12	C20	8.811122

In [644…
```
employeeStabilityAtCityLevel[-5:].index
```

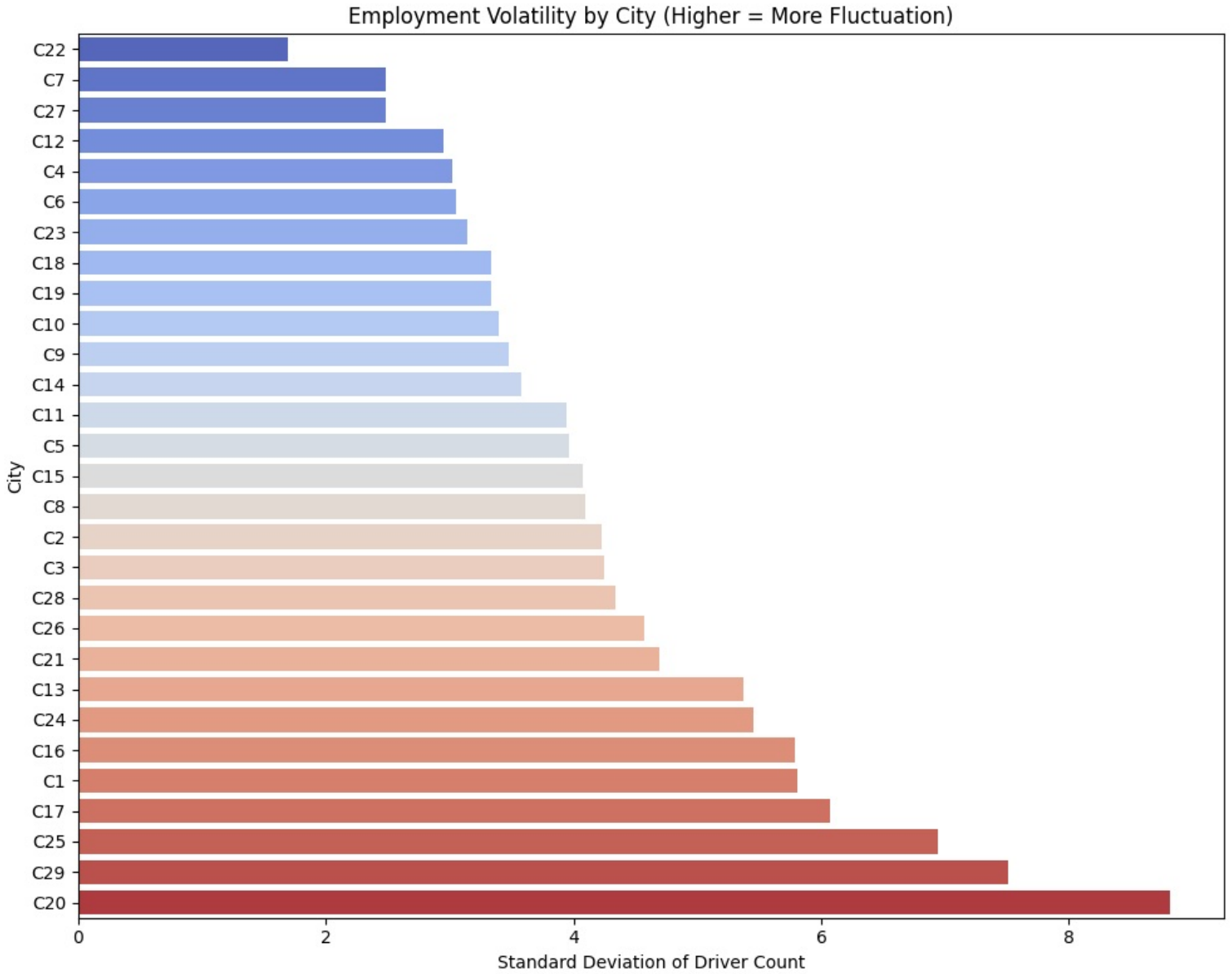
Out[644…
```
Index([0, 8, 17, 21, 12], dtype='int64')
```

- Cities C20, C29, C25, C17, C1 have very high volatility in driver count.
- C20 and C29 are amonng the top 5 employer cities
- For C20 the driver count has decreased the maximum
- For C29 the driver count has incresed the maximum

Lets plot this data for further understanding.

In [645…
```
plt.figure(figsize=(10, 8))
sns.barplot(
    data=employeeStabilityAtCityLevel,
    y='City',
    x='DriverCount',
    palette='coolwarm'
)
plt.title('Employment Volatility by City (Higher = More Fluctuation)')
plt.xlabel('Standard Deviation of Driver Count')
plt.ylabel('City')
plt.tight_layout()
plt.show()
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_10256\1061183991.py:2: FutureWarning:


Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.
```



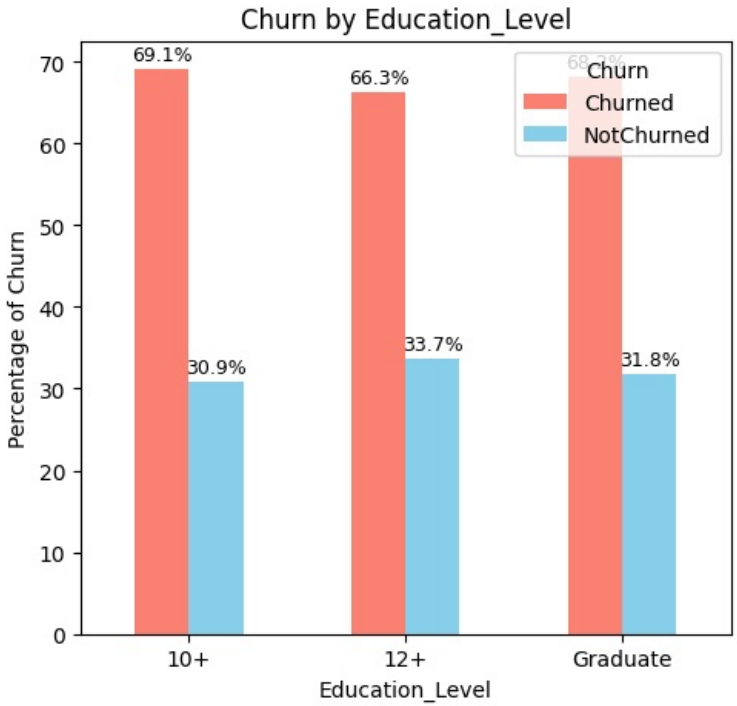
This plot helps easily identify cities with huge volatility in number of drivers versus those that have relatively stable driver count.

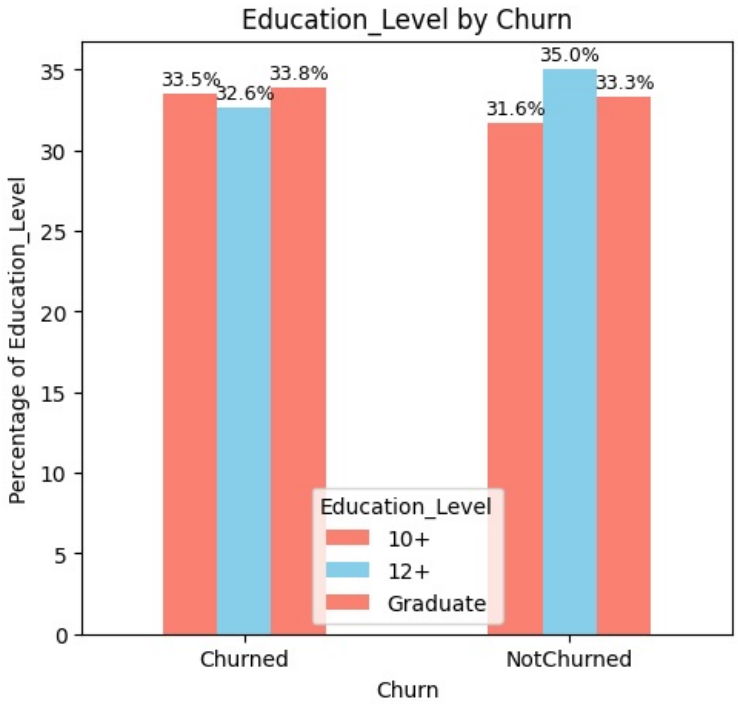
3.2.28 Churn and Education Level

In [646…

```
perform_chi_square_test(driversWithCategories,'Education_Level','Churn')
```

```
Contingency Table:
Churn            Churned  NotChurned
Education_Level
10+                  542         242
12+                  527         268
Graduate             547         255


Chi-square Test Results:
Chi-square statistic: 1.5253
p-value: 0.4664
Degrees of freedom: 2
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Education_Level and Churn are independent.' is True, p-value:0.46643939521309963
```





==========

- Churn is similar across education levels

3.2.29 Churn and Date of Joining

At month level EmploymentStatus column is roughly our churn

In [647…

```
employmentStatus_monthly_data = df_original.groupby(['MMM-YY'], as_index=False).EmploymentStatus.agg({'sum','count'})
employmentStatus_monthly_data['ChurnedEmployees']=employmentStatus_monthly_data.eval('count-sum')
employmentStatus_monthly_data.rename(columns={'count':'TotalEmployees','sum':'ActiveEmployees'},inplace=True)
employmentStatus_monthly_data
```

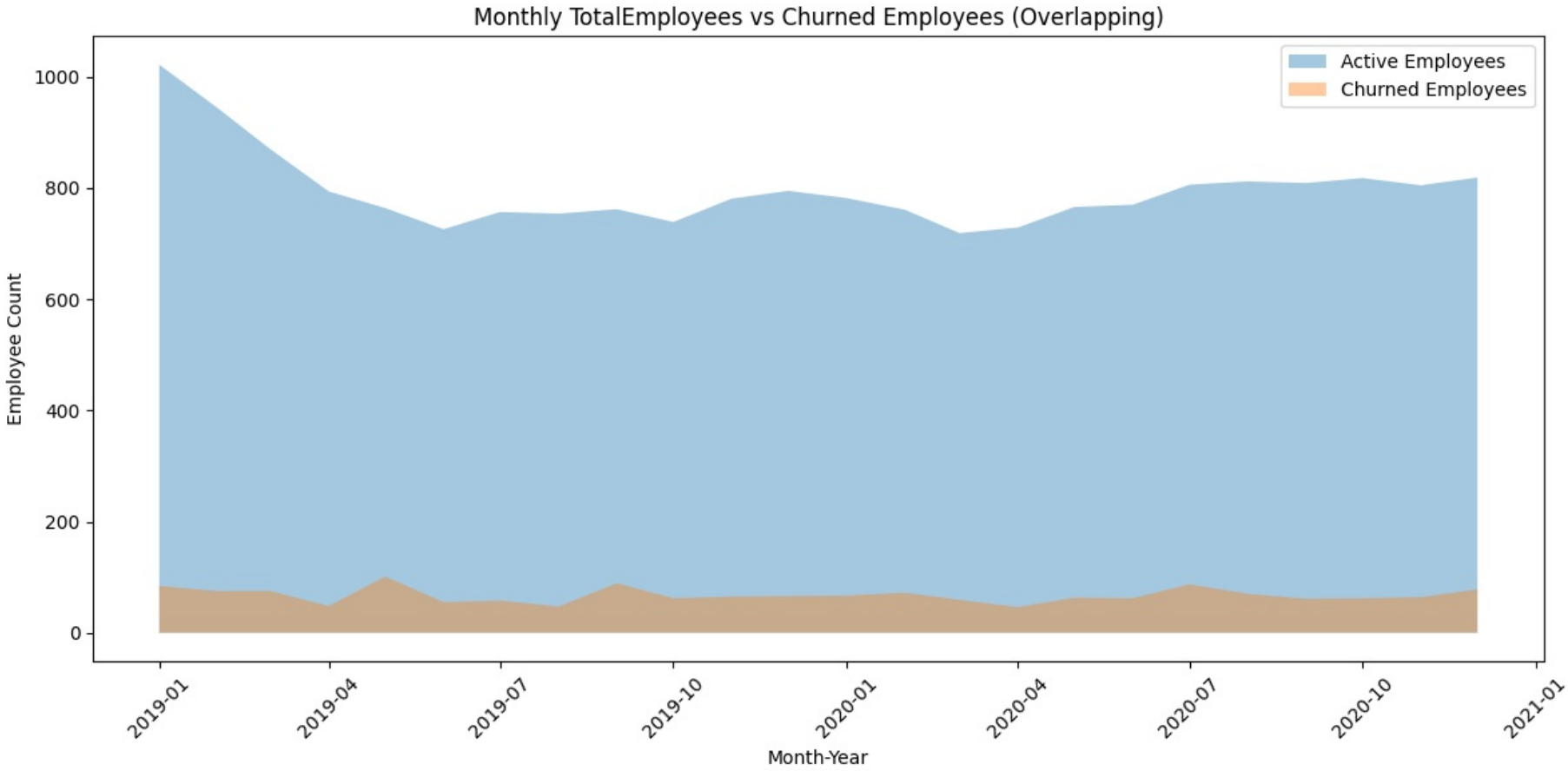
Out[647…

	MMM-YY	ActiveEmployees	TotalEmployees	ChurnedEmployees
0	2019-01-01	938	1022	84
1	2019-02-01	869	944	75
2	2019-03-01	795	870	75
3	2019-04-01	746	794	48
4	2019-05-01	663	764	101
5	2019-06-01	671	726	55
6	2019-07-01	699	757	58
7	2019-08-01	707	754	47
8	2019-09-01	673	762	89
9	2019-10-01	677	739	62
10	2019-11-01	716	781	65
11	2019-12-01	729	795	66
12	2020-01-01	715	782	67
13	2020-02-01	689	761	72
14	2020-03-01	660	719	59
15	2020-04-01	683	729	46
16	2020-05-01	703	766	63
17	2020-06-01	708	770	62
18	2020-07-01	719	806	87
19	2020-08-01	742	812	70
20	2020-09-01	748	809	61
21	2020-10-01	756	818	62
22	2020-11-01	741	805	64
23	2020-12-01	741	819	78

In [648…

```
plt.figure(figsize=(12, 6))
plt.fill_between(
    employmentStatus_monthly_data['MMM-YY'],
    employmentStatus_monthly_data['TotalEmployees'],
    alpha=0.4, label='Active Employees'
)
plt.fill_between(
    employmentStatus_monthly_data['MMM-YY'],
    employmentStatus_monthly_data['ChurnedEmployees'],
    alpha=0.4, label='Churned Employees'
)

plt.title('Monthly TotalEmployees vs Churned Employees (Overlapping)')
plt.xlabel('Month-Year')
plt.ylabel('Employee Count')
```

```
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The plot shows that driver count in the company has remained lower that what it was in start of 2019

In [649…

```
employmentStatus_monthly_data['AttritionRate'] = np.round(employmentStatus_monthly_data.eval('ChurnedEmployees*100/(ActiveEmployees + ChurnedEmployees)'),1)
employmentStatus_monthly_data
```

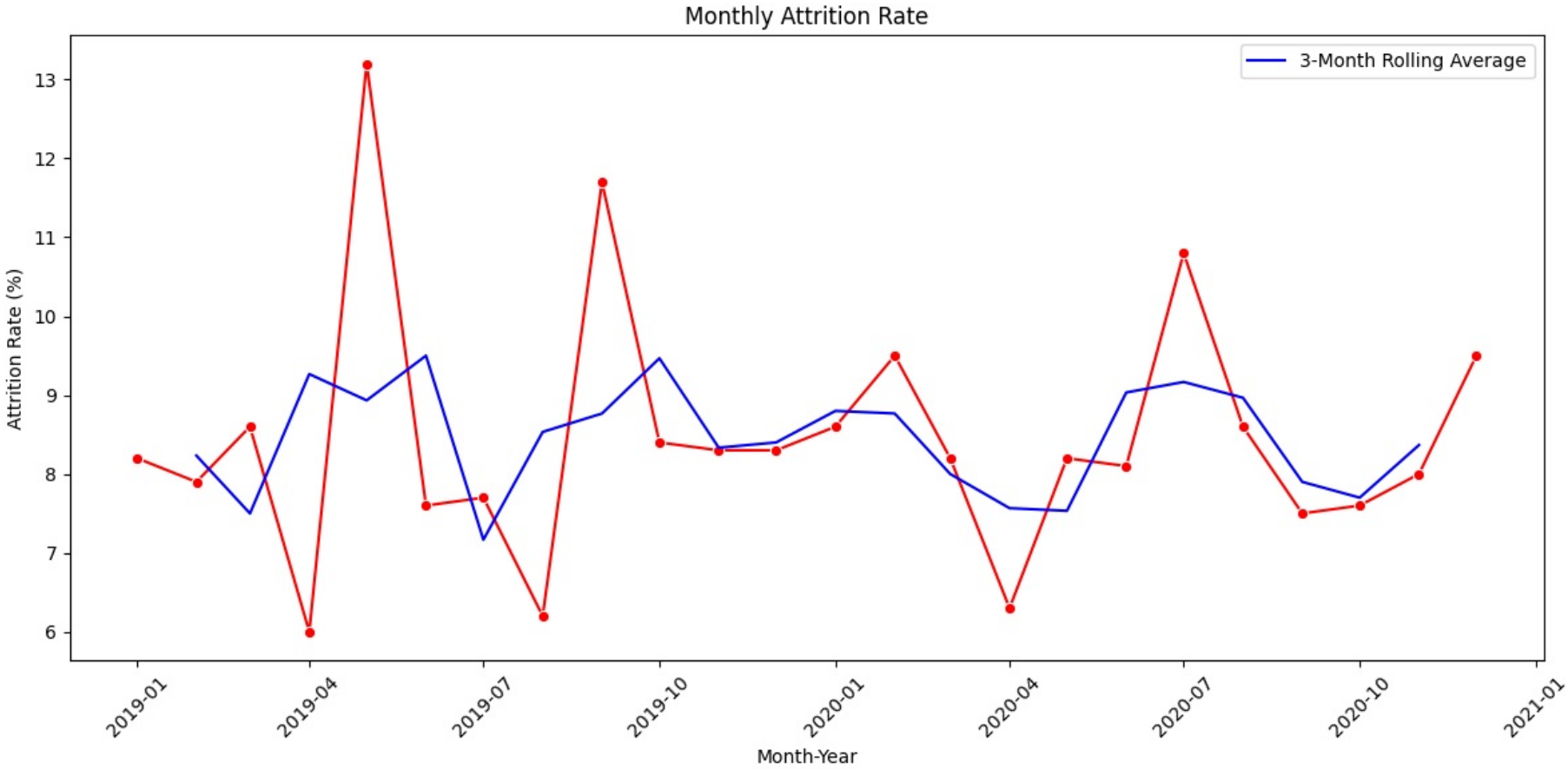
Out[649…

	MMM-YY	ActiveEmployees	TotalEmployees	ChurnedEmployees	AttritionRate
0	2019-01-01	938	1022	84	8.2
1	2019-02-01	869	944	75	7.9
2	2019-03-01	795	870	75	8.6
3	2019-04-01	746	794	48	6.0
4	2019-05-01	663	764	101	13.2
5	2019-06-01	671	726	55	7.6
6	2019-07-01	699	757	58	7.7
7	2019-08-01	707	754	47	6.2
8	2019-09-01	673	762	89	11.7
9	2019-10-01	677	739	62	8.4
10	2019-11-01	716	781	65	8.3
11	2019-12-01	729	795	66	8.3
12	2020-01-01	715	782	67	8.6
13	2020-02-01	689	761	72	9.5
14	2020-03-01	660	719	59	8.2
15	2020-04-01	683	729	46	6.3
16	2020-05-01	703	766	63	8.2
17	2020-06-01	708	770	62	8.1
18	2020-07-01	719	806	87	10.8
19	2020-08-01	742	812	70	8.6
20	2020-09-01	748	809	61	7.5
21	2020-10-01	756	818	62	7.6
22	2020-11-01	741	805	64	8.0
23	2020-12-01	741	819	78	9.5

In [650…

```
plt.figure(figsize=(12, 6))
sns.lineplot(
    data=employmentStatus_monthly_data,
    x='MMM-YY',
    y='AttritionRate',
    marker='o',
    color='red'
)


employmentStatus_monthly_data['AttritionRate_Rolling'] = (
    employmentStatus_monthly_data['AttritionRate']
    .rolling(window=3, center=True)
    .mean()
)
```

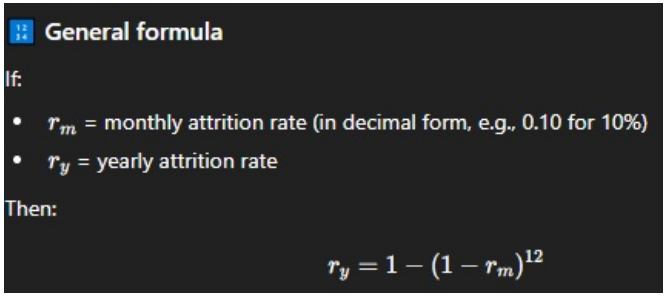
```
sns.lineplot(
    data=employmentStatus_monthly_data,
    x='MMM-YY',
    y='AttritionRate_Rolling',
    color='blue',
    label='3-Month Rolling Average'
)

plt.title('Monthly Attrition Rate')
plt.xlabel('Month-Year')
plt.ylabel('Attrition Rate (%)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

meanMonthlyAttritionRate = employmentStatus_monthly_data.AttritionRate.mean()
meanMonthlyAttritionRate
```



Out[650… np.float64(8.458333333333334)

- Mean monthly attrition is 8.46%. However this does not mean that yearly attrition = 8.46%.

- Attrition rates compound over time — so the yearly attrition will actually be much higher.
- If 8.45% of employees leave every month, only 91.55% remain after each month.



In [651…
```
yearly_rate = 1 - (1 - meanMonthlyAttritionRate/100) ** 12
print(f"Yearly attrition ≈ {yearly_rate * 100:.1f}%")
```

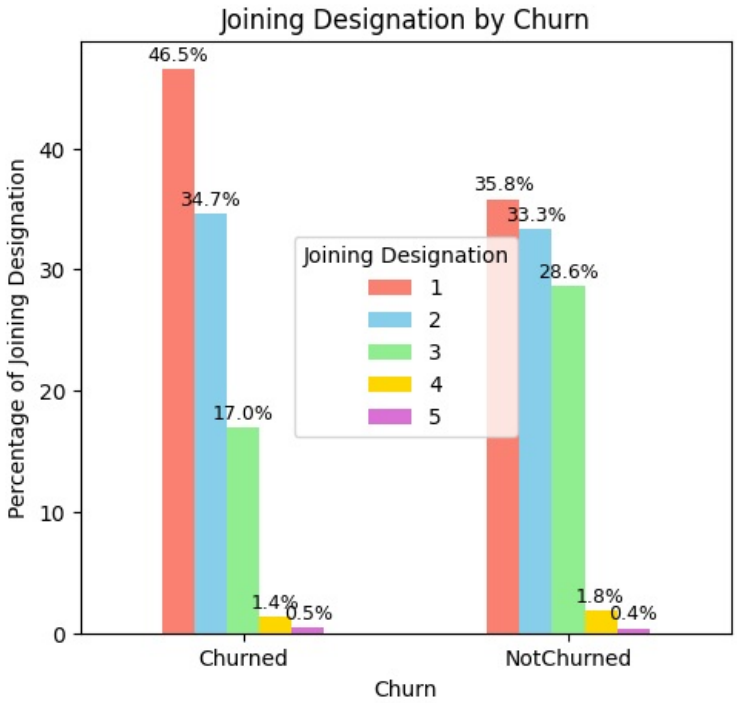
```
Yearly attrition ≈ 65.4%
```

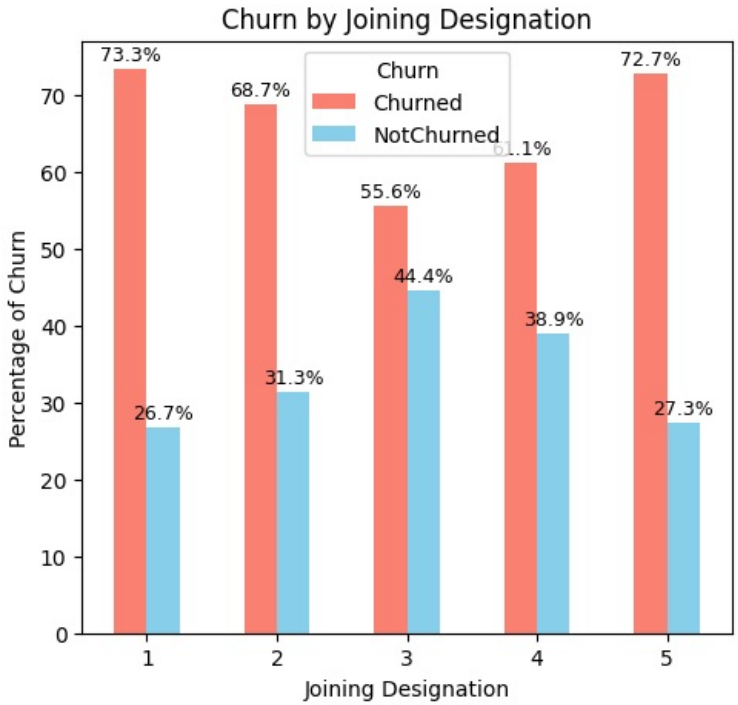
- Yearly attrition rate is 65.4%

3.2.30 Churn and Joining Designation

In [652…
```
perform_chi_square_test(driversWithCategories,'Churn','Joining Designation')
```

```
Contingency Table:
Joining Designation    1    2    3   4  5
Churn
Churned              752  560  274  22  8
NotChurned           274  255  219  14  3


Chi-square Test Results:
Chi-square statistic: 49.1405
p-value: 0.0000
Degrees of freedom: 4
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and Joining Designation are DEPENDENT' is True, p-value:5.457615375535053e-10. However, association is Weak(Cra
mersV=0.14)
```





==========

In [653…
```
crossNormalized12 = pd.crosstab(driversWithCategories['Churn'], driversWithCategories['Joining Designation'], normalize='columns')
crossNormalized12
```

Out[653…

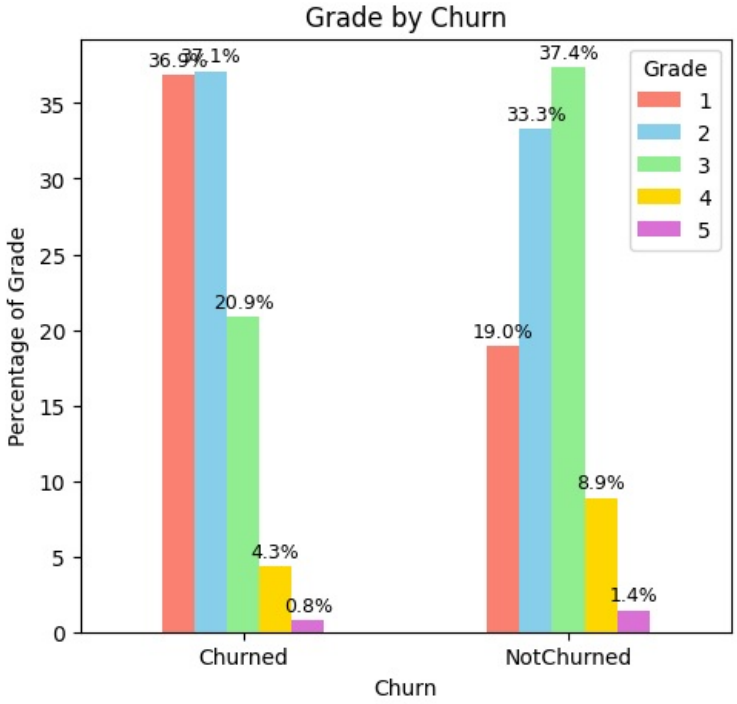
Joining Designation	1	2	3	4	5
Churn					
Churned	0.732943	0.687117	0.555781	0.611111	0.727273
NotChurned	0.267057	0.312883	0.444219	0.388889	0.272727

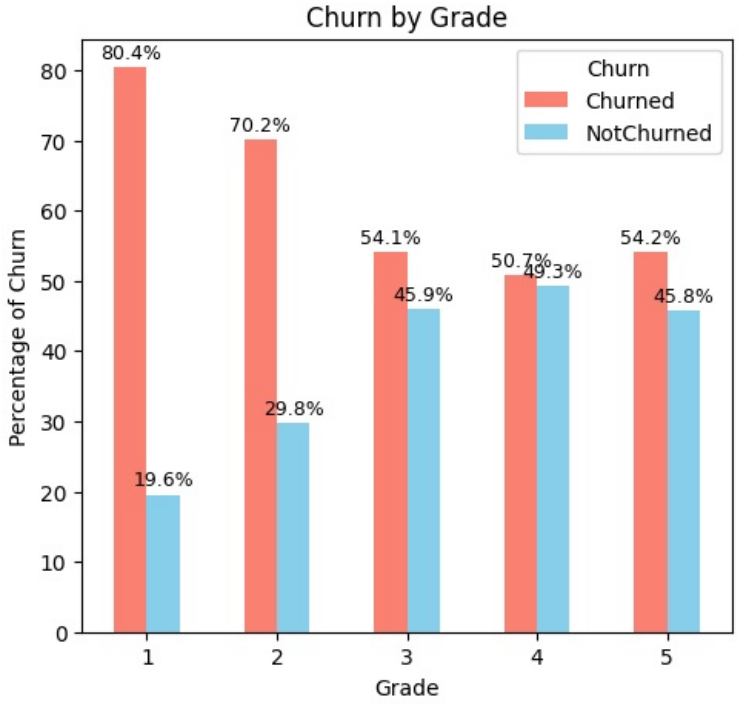
- Drivers joining at Grade 1 and Grade 5 have highest chance of churning, around 73%
- Drivers joining at Grade 3 have lowest chance of churning, around 55%

3.2.31 Churn and Grade

In [654…
```
perform_chi_square_test(driversWithCategories,'Churn','Grade')
```

```
Contingency Table:
Grade         1    2    3   4   5
Churn
Churned     596  600  337  70  13
NotChurned  145  255  286  68  11


Chi-square Test Results:
Chi-square statistic: 130.6016
p-value: 0.0000
Degrees of freedom: 4
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and Grade are DEPENDENT' is True, p-value:2.8955519930847994e-27. However, association is Weak(CramersV=0.23)
```





==========

In [655…
```
crossNormalized12 = pd.crosstab(driversWithCategories['Churn'], driversWithCategories['Grade'], normalize='columns')
crossNormalized12
```

Out[655…

Grade	1	2	3	4	5
Churn					
Churned	0.804318	0.701754	0.540931	0.507246	0.541667
NotChurned	0.195682	0.298246	0.459069	0.492754	0.458333

- Drivers at Grade 1 have a extremely high chance of churn, 80%
- Drivers Grade 4 have lowest chance of churning, around 50%, Grade 3.4,5 have overall lower churning chance, around 50-54%

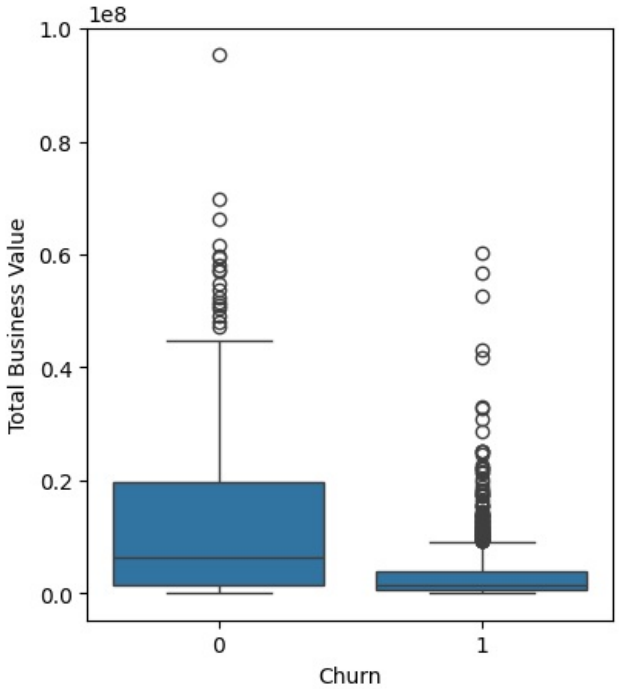
3.2.32 Churn and Total Business value

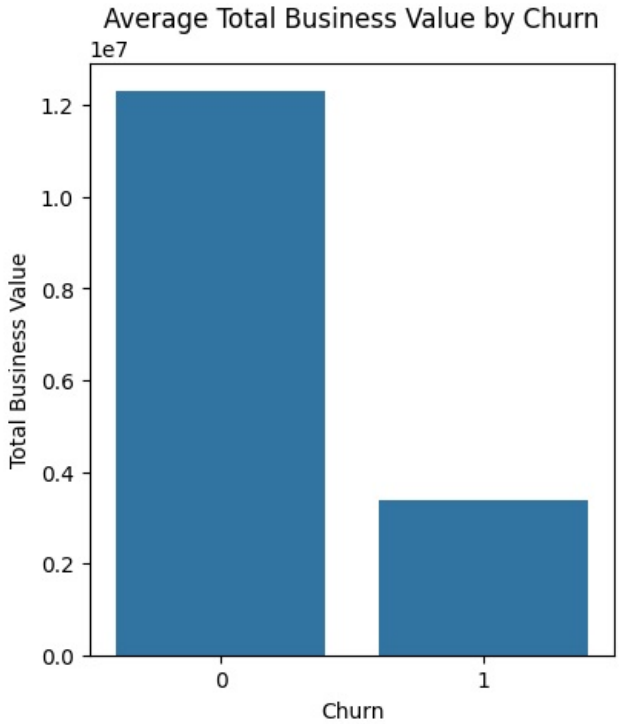
- We established earlier that 161 drivers who have 0 TBV value have substantial income and hence there is chance of missing data there.
- We will ignore those rows and focus on drivers that have non zero total business value and check churn in them
- We already created a dataframe nonZeroTBVDrivers earlier, that we can reuse here for analysis.

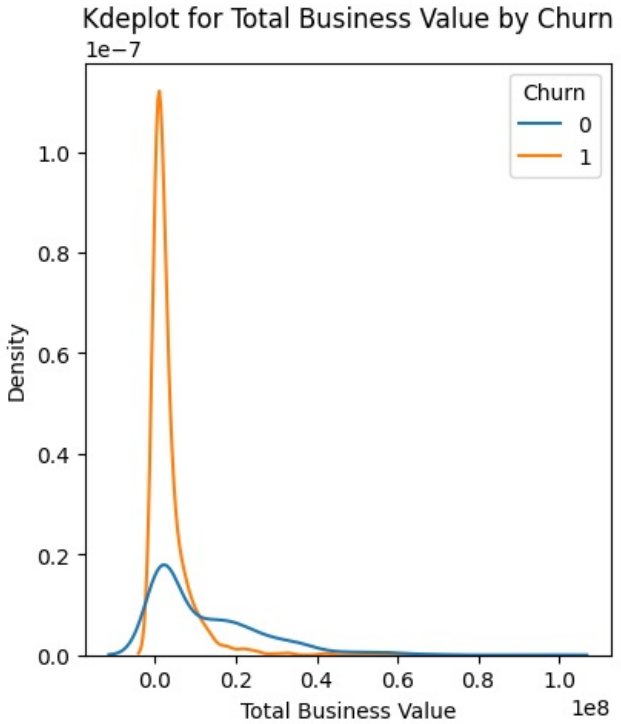
In [656…

```
perform_anova_kruskal(nonZeroTBVDrivers,numerical_column='Total Business Value', categorical_column='Churn', test=test,log="detailed")
```

```
Ho = Churn and Total Business Value are independent. Medians of groups of Total Business Value based on Churn are same
Ha = Churn and Total Business Value are DEPENDENT. Medians of groups of Total Business Value based on Churn are different
Kruskal-Wallis Result:225.24783639406974,6.482707330521164e-51
Effect Size: η²=0.14
Reject Null Hypothesis. Alternate Hypothesis 'Churn and Total Business Value are DEPENDENT. Medians of groups of Total Business Value based on Churn are different In s
imple words, there’s substantial Total Business Value difference across Churn values' is True, p-value:0.00.
Churn values vary across Total Business Value groups (enough to be statistically significant kruskal p_value=6.482707330521164e-51), Total Business Value does have a m
edium influence on Churn (η²=0.14)
```


1e8
1.0
0.8
0.6
0.4
0.2
0.0
Total Business Value
0
1
Churn


Average Total Business Value by Churn
1e7
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Total Business Value
0
1
Churn


Kdeplot for Total Business Value by Churn
1e−7
Churn
0
1
1.0
0.8
0.6
0.4
0.2
0.0
Density
0.0
0.2
0.4
0.6
0.8
1.0
1e8
Total Business Value

```
   Churn  Total Business Value
0      0          1.230732e+07
1      1          3.382402e+06
```

- The figures are little difficult to read.
- Lets recalculate the mean values with some formatting

In [657…
```
nonZeroTBVDrivers.groupby('Churn')['Total Business Value'].mean().apply(lambda x: f"{x:,.0f}")
```

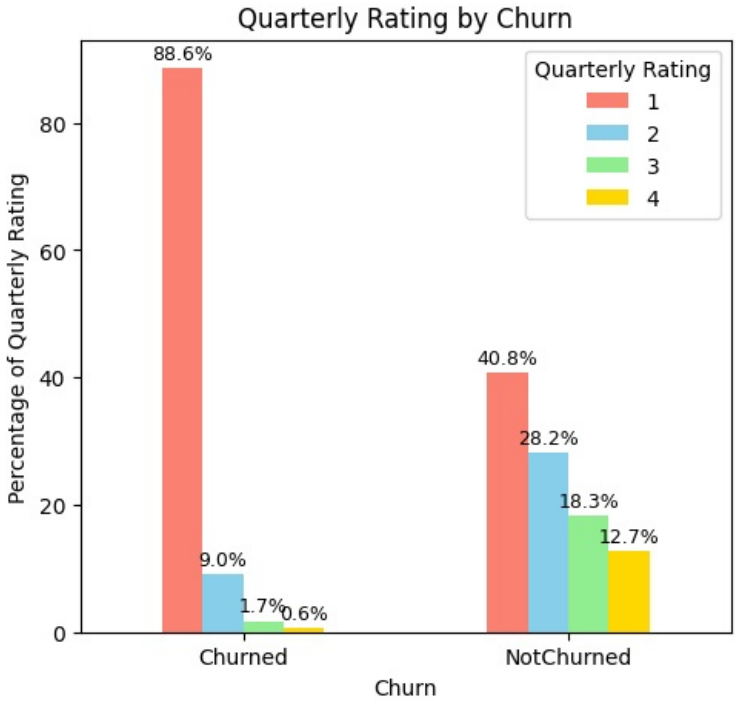
Out[657…
```
Churn
0    12,307,323
1     3,382,402
Name: Total Business Value, dtype: object
```

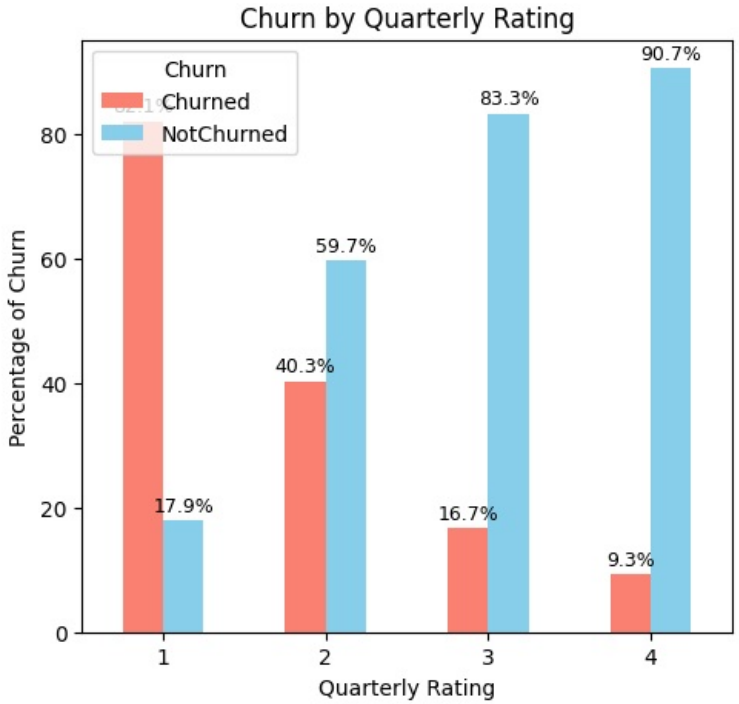
- Drivers who churned have a substantially lower range of total business value
- The average total business value of drivers who churned is one-third (33lakh) of those who did not churn (123 lakh)

3.2.33 Churn and Quarterly Rating

In [658…
```
col = 'Quarterly Rating'
perform_chi_square_test(driversWithCategories,'Churn',col)
```

```
Contingency Table:
Quarterly Rating     1    2    3   4
Churn
Churned           1432  146   28  10
NotChurned         312  216  140  97


Chi-square Test Results:
Chi-square statistic: 658.1194
p-value: 0.0000
Degrees of freedom: 3
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and Quarterly Rating are DEPENDENT' is True, p-value:2.528965659451095e-142. However, association is Strong(Cra
mersV=0.53)
```





==========

- Quarterly rating and Churn are dependent
- 88% of those who churned had the lowest quarterly rating of 1. It is possible company terminated their services due to the poor rating
- Those who have rating greater than 1 have a greater chance of staying

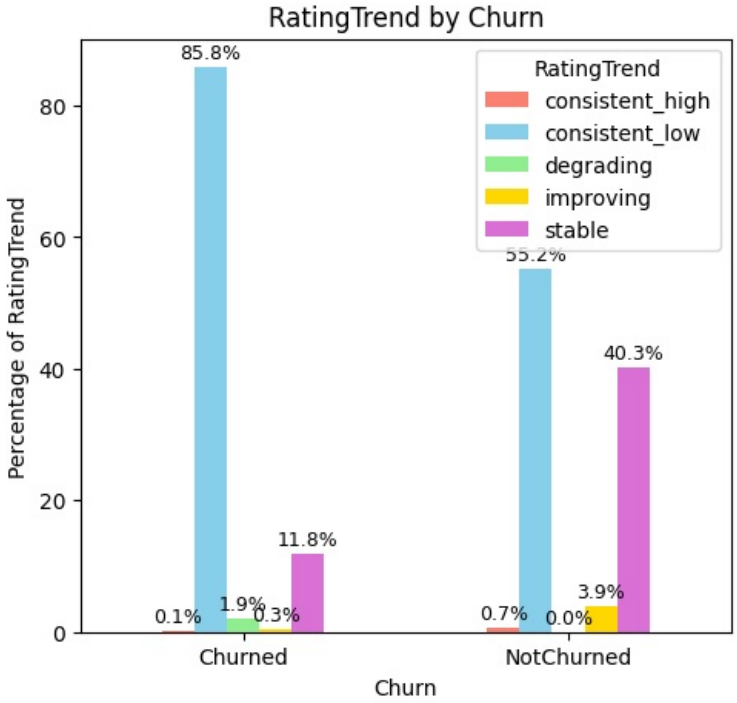
3.2.34 Churn and RatingTrend

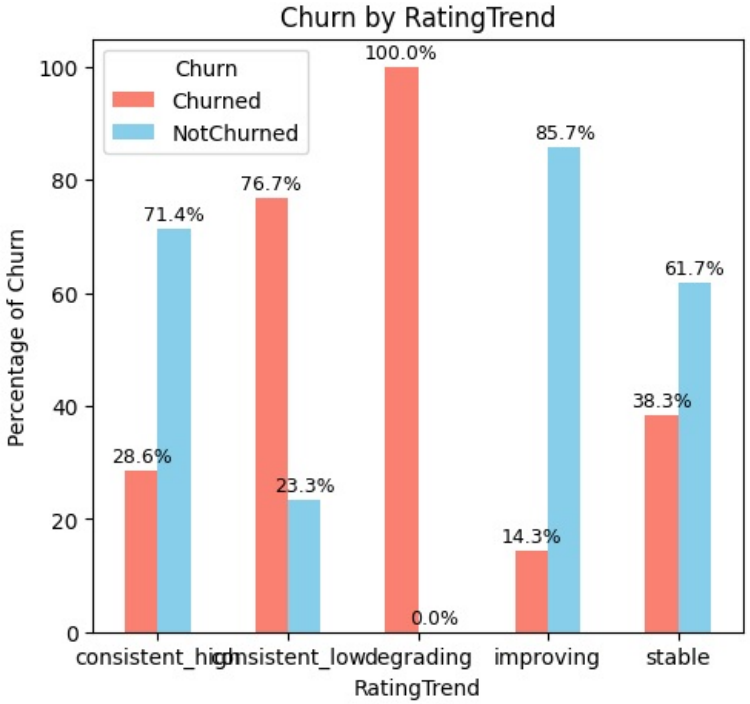
In [659…

```
col = 'RatingTrend'
```

```
perform_chi_square_test(driversWithCategories,'Churn',col)
```

```
Contingency Table:
RatingTrend  consistent_high  consistent_low  degrading  improving  stable
Churn
Churned                    2            1387         31          5     191
NotChurned                 5             422          0         30     308


Chi-square Test Results:
Chi-square statistic: 330.3970
p-value: 0.0000
Degrees of freedom: 4
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and RatingTrend are DEPENDENT' is True, p-value:2.991165703691902e-70. However, association is Moderate(Cramers
V=0.37)
```





==========

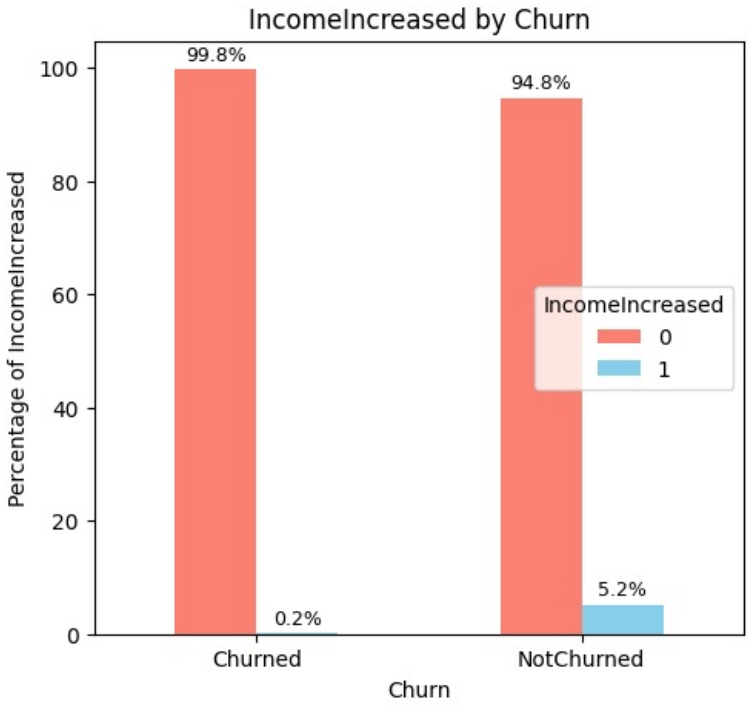
- Rating Trend can help understand churn
- Drivers who have consistently low ratings have 86% of chance of churn
- Drivers who have consistently high ratings have only 29% of chance of churn
- Drivers whose rating has been seen as improving over the months stay with the company 86% of times
- ALL drivers whose rating declined were found to have churned.

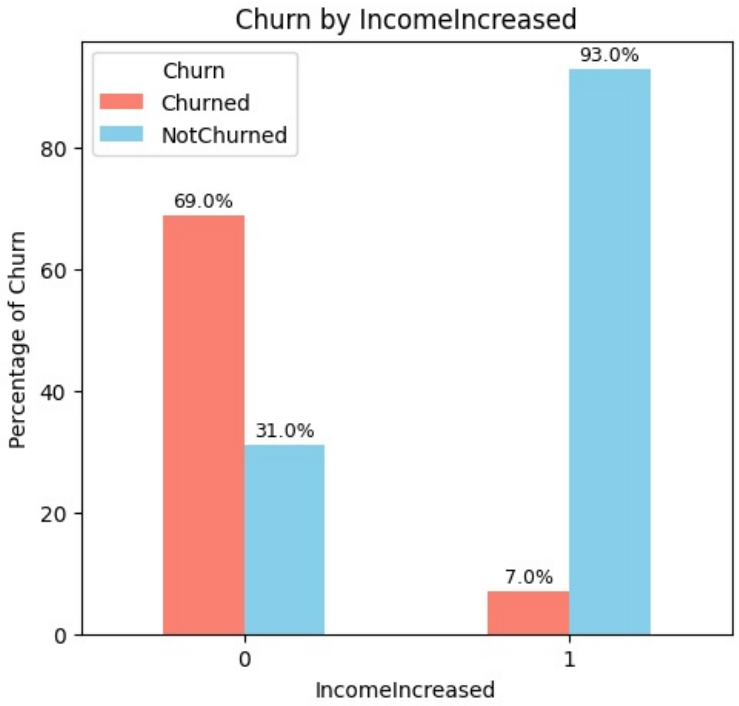
3.2.35 Churn and IncomeIncreased

In [660…

```
col = 'IncomeIncreased'
perform_chi_square_test(driversWithCategories,'Churn',col)
```

```
Contingency Table:
IncomeIncreased     0   1
Churn
Churned          1613   3
NotChurned        725  40


Chi-square Test Results:
Chi-square statistic: 71.6474
p-value: 0.0000
Degrees of freedom: 1
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and IncomeIncreased are DEPENDENT' is True, p-value:2.5729990685015372e-17. However, association is Weak(Cramer
sV=0.17)
```





==========

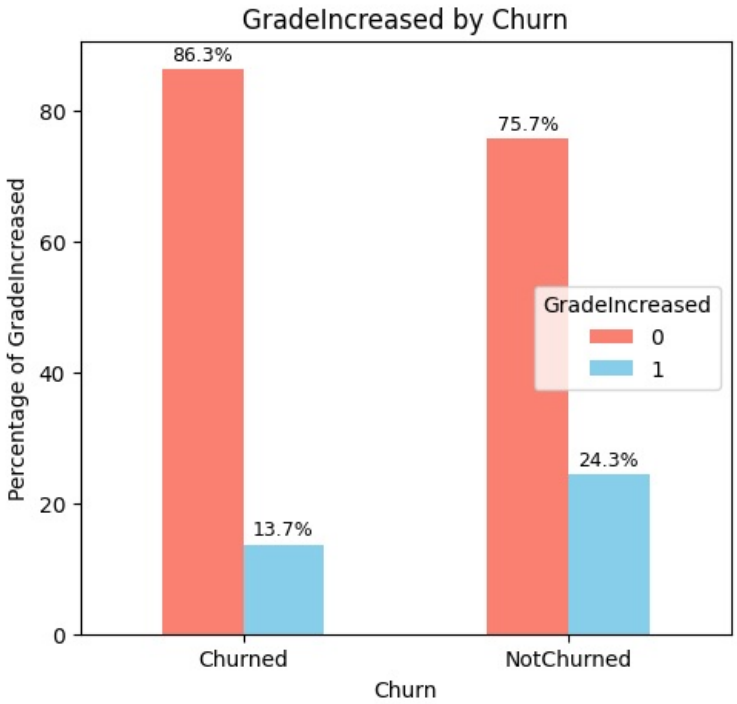
- Almost all the drivers who churned did not have a increase in income over the months.
- This does not mean that all those who did not see increase in income got churned. That figure is 69%
- 93% of drivers who saw an increase in income stayed.

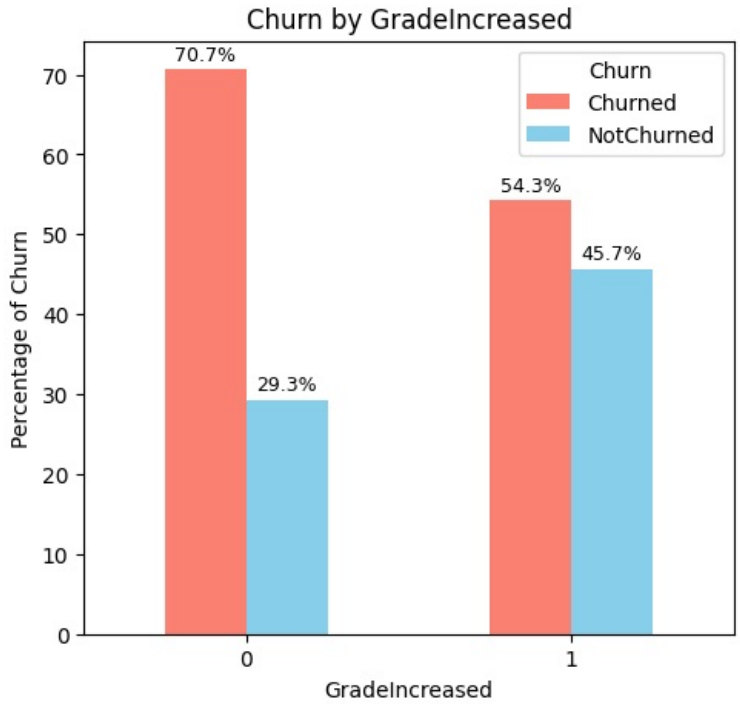
3.2.36 Churn and GradeIncreased

In [661…

```
col = 'GradeIncreased'
perform_chi_square_test(driversWithCategories,'Churn',col)
```

```
Contingency Table:
GradeIncreased     0    1
Churn
Churned         1395  221
NotChurned       579  186


Chi-square Test Results:
Chi-square statistic: 40.7137
p-value: 0.0000
Degrees of freedom: 1
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Churn and GradeIncreased are DEPENDENT' is True, p-value:1.7624761870029749e-10. However, association is Weak(Cramers
V=0.13)
```





==========

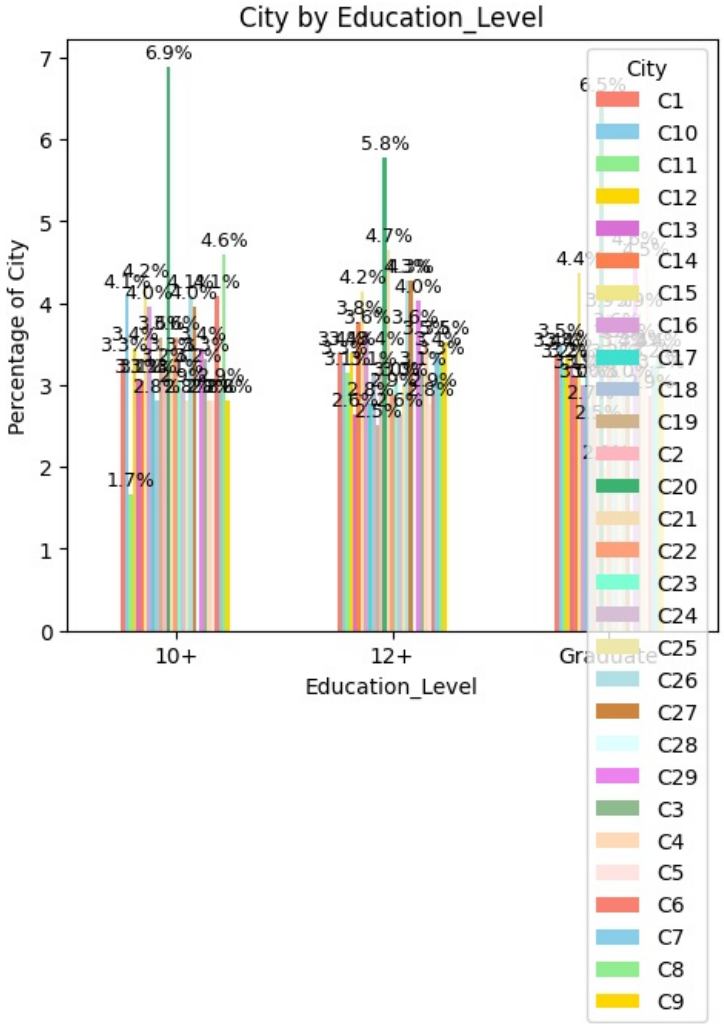
- Among those whose grade did not increase since joining, 71% left the company
- Among those whose grade increased since joining, only 54% left the company
- 86% of drivers who left the company were those that didnt see increase in their grade since joining

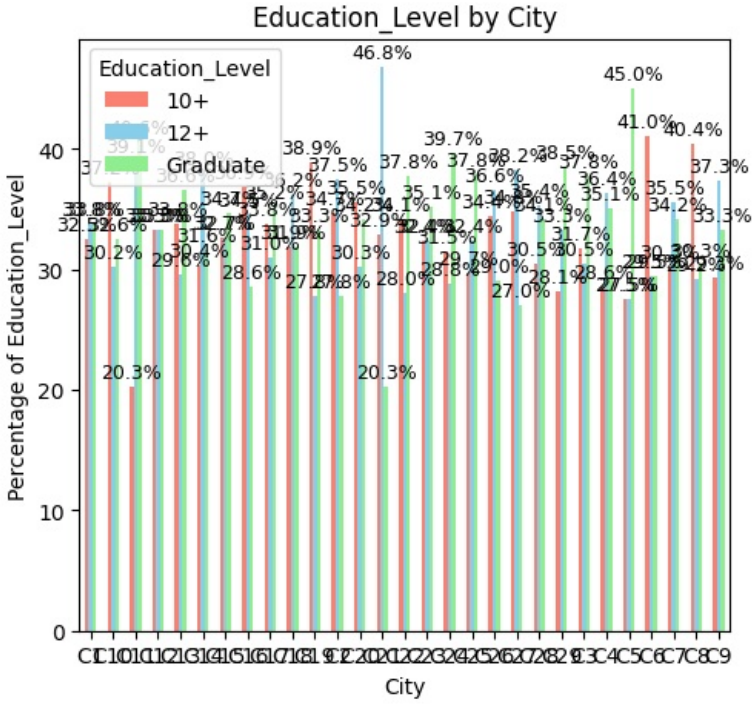
3.2.37 City and Education_Level

In [662…

```
perform_chi_square_test(driversWithCategories,'Education_Level','City')
```

```
Contingency Table:
City             C1  C10  C11  C12  C13  C14  C15  C16  C17  C18  ...  C27  \
Education_Level                                                    ...
10+              26   32   13   27   24   24   33   31   24   22  ...   31
12+              27   26   25   27   21   30   33   29   22   25  ...   34
Graduate         27   28   26   27   26   25   35   24   25   22  ...   24

City             C28  C29  C3  C4  C5  C6  C7  C8  C9
Education_Level
10+               25   27  26  22  22  32  23  36  22
12+               29   32  25  28  22  23  27  26  28
Graduate          28   37  31  27  36  23  26  27  25

[3 rows x 29 columns]


Chi-square Test Results:
Chi-square statistic: 39.2217
p-value: 0.9567
Degrees of freedom: 56
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Education_Level and City are independent.' is True, p-value:0.9567468072754713
```





==========

- Education Level is similar across cities

3.2.38 City and IncomeIncreased

In [663…

```
perform_chi_square_test(driversWithCategories,'IncomeIncreased','City')
```

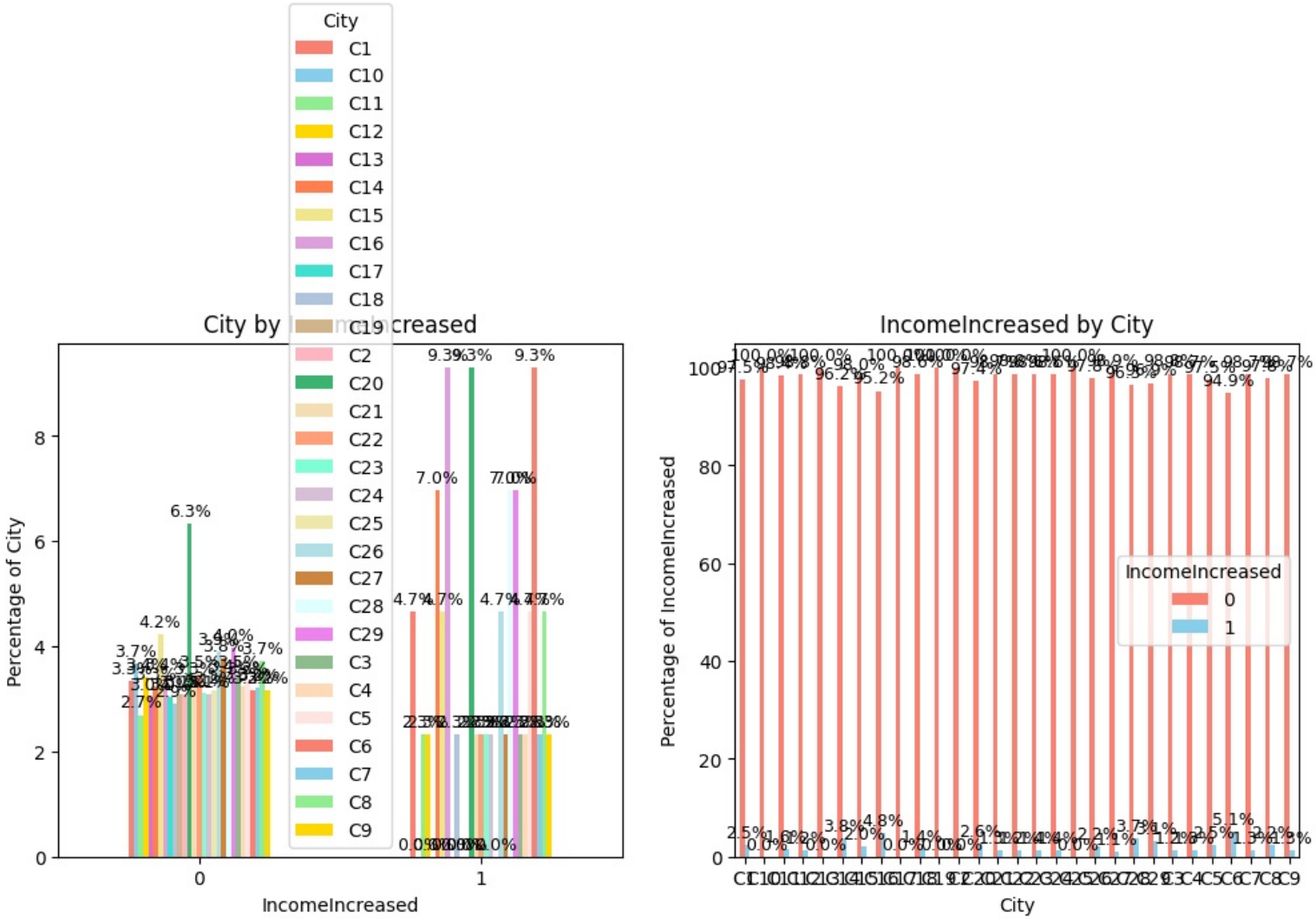
Contingency Table:

City	C1	C10	C11	C12	C13	C14	C15	C16	C17	C18	...	C27	\
IncomeIncreased											...		
0	78	86	63	80	71	76	99	80	71	68	...	88	
1	2	0	1	1	0	3	2	4	0	1	...	1	

City	C28	C29	C3	C4	C5	C6	C7	C8	C9
IncomeIncreased									
0	79	93	81	76	78	74	75	87	74
1	3	3	1	1	2	4	1	2	1

[2 rows x 29 columns]

Chi-square Test Results:
Chi-square statistic: 24.0635
p-value: 0.6782
Degrees of freedom: 28
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>IncomeIncreased and City are independent.' is True, p-value:0.6781787409841961



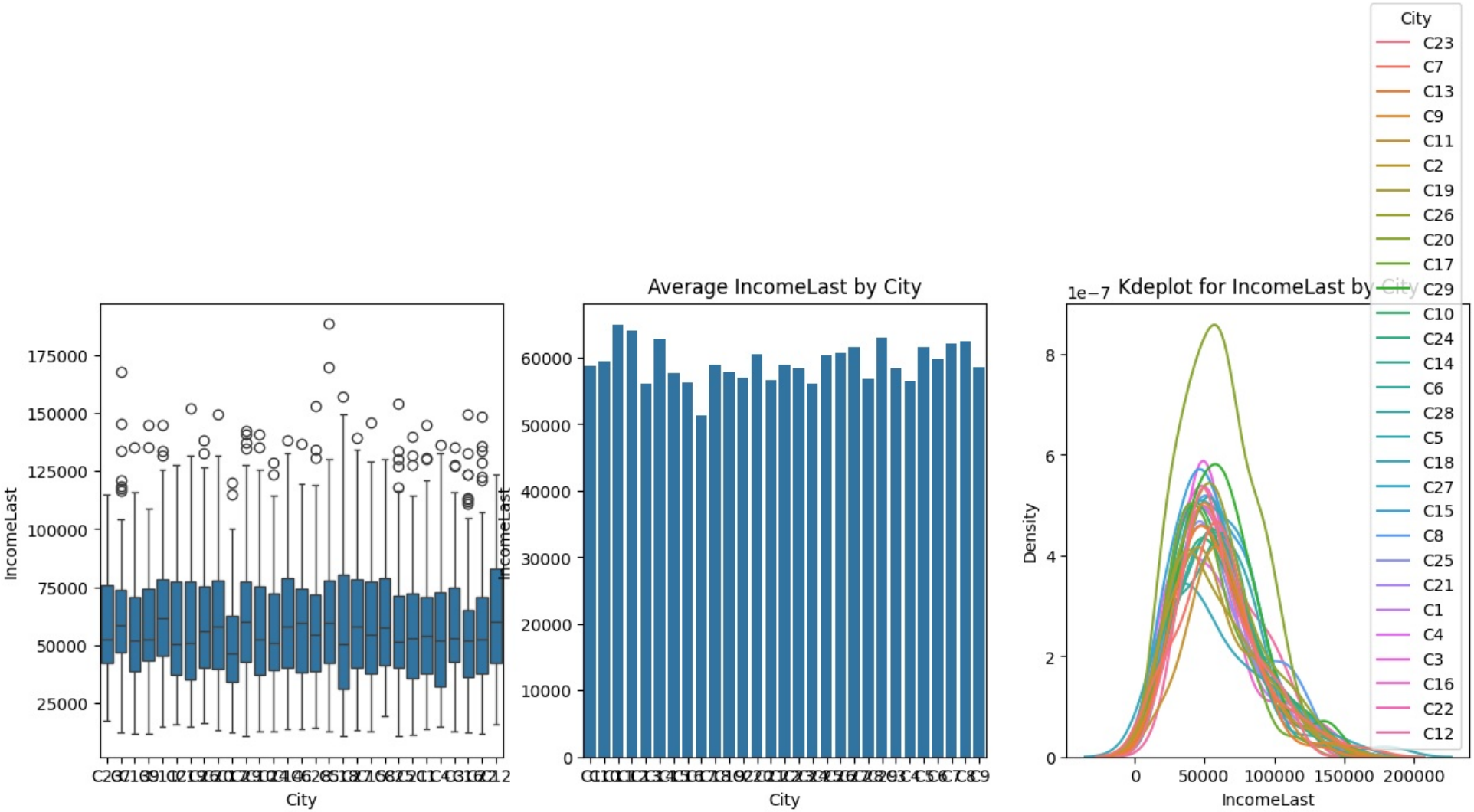
==========

- Tendency of income increased is similar across cities

3.2.39 City and IncomeLast

In [664…
```
perform_anova_kruskal(drivers,numerical_column='IncomeLast', categorical_column='City', test=test,log="detailed")
```

```
Ho = City and IncomeLast are independent. Medians of groups of IncomeLast based on City are same
Ha = City and IncomeLast are DEPENDENT. Medians of groups of IncomeLast based on City are different
Kruskal-Wallis Result:28.232537736924705,0.4521771793142992
Effect Size: η²=0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'City and IncomeLast are independent. Medians of groups of IncomeLast based on City are same In simple words, there’s n
o substantial IncomeLast difference across City values' is True, p-value:0.45
```


IncomeLast
City
Average IncomeLast by City
City
1e−7
Kdeplot for IncomeLast by City
Density
IncomeLast
City
C23
C7
C13
C9
C11
C2
C19
C26
C20
C17
C29
C10
C24
C14
C6
C28
C5
C18
C27
C15
C8
C25
C21
C1
C4
C3
C16
C22
C12

	City	IncomeLast
0	C1	58782.762500
1	C10	59421.290698
2	C11	64937.359375
3	C12	64040.209877
4	C13	56022.830986
5	C14	62901.658228
6	C15	57627.504950
7	C16	56318.821429
8	C17	51240.929577
9	C18	59008.521739
10	C19	57823.013889
11	C2	57057.972222
12	C20	60566.164474
13	C21	56597.291139
14	C22	58863.012195
15	C23	58462.797297
16	C24	56165.506849
17	C25	60382.270270
18	C26	60764.107527
19	C27	61669.966292
20	C28	56849.682927
21	C29	62917.093750
22	C3	58352.426829
23	C4	56520.545455
24	C5	61509.975000
25	C6	59886.525641
26	C7	62104.407895
27	C8	62400.573034
28	C9	58624.626667

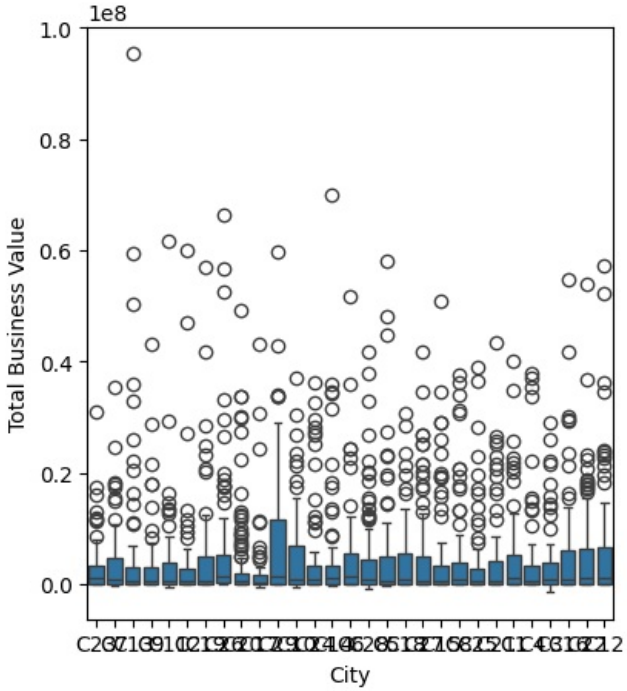
- Income levels are similar across cities

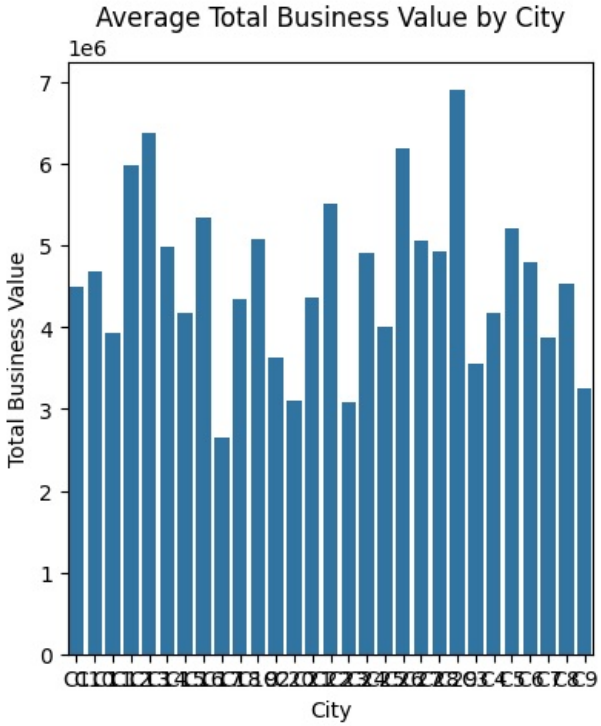
3.2.40 City and Total Business Value

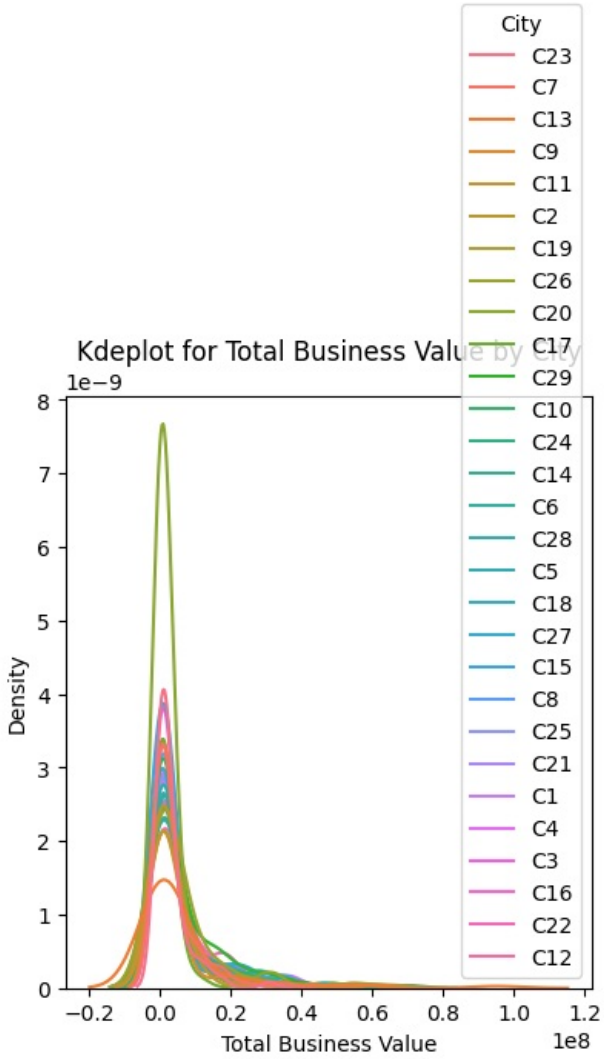
We first use the dataframe that contains drivers with zero TBV value

In [665…
```
perform_anova_kruskal(drivers,numerical_column='Total Business Value', categorical_column='City', test=test,log="detailed")
```

```
Ho = City and Total Business Value are independent. Medians of groups of Total Business Value based on City are same
Ha = City and Total Business Value are DEPENDENT. Medians of groups of Total Business Value based on City are different
Kruskal-Wallis Result:31.575927942560106,0.29210923895603225
Effect Size: η²=0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'City and Total Business Value are independent. Medians of groups of Total Business Value based on City are same In sim
ple words, there’s no substantial Total Business Value difference across City values' is True, p-value:0.29
```


1e8
1.0
0.8
0.6
0.4
0.2
0.0
Total Business Value
City


Average Total Business Value by City
1e6
7
6
5
4
3
2
1
0
Total Business Value
City

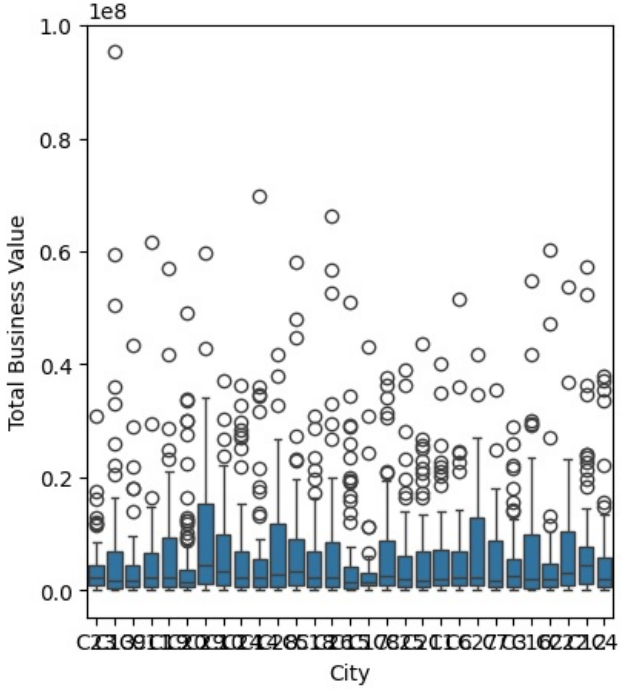

Kdeplot for Total Business Value by City
1e−9
8
7
6
5
4
3
2
1
0
Density
−0.2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1e8
Total Business Value
City
C23
C7
C13
C9
C11
C2
C19
C26
C20
C17
C29
C10
C24
C14
C6
C28
C5
C18
C27
C15
C8
C25
C21
C1
C4
C3
C16
C22
C12

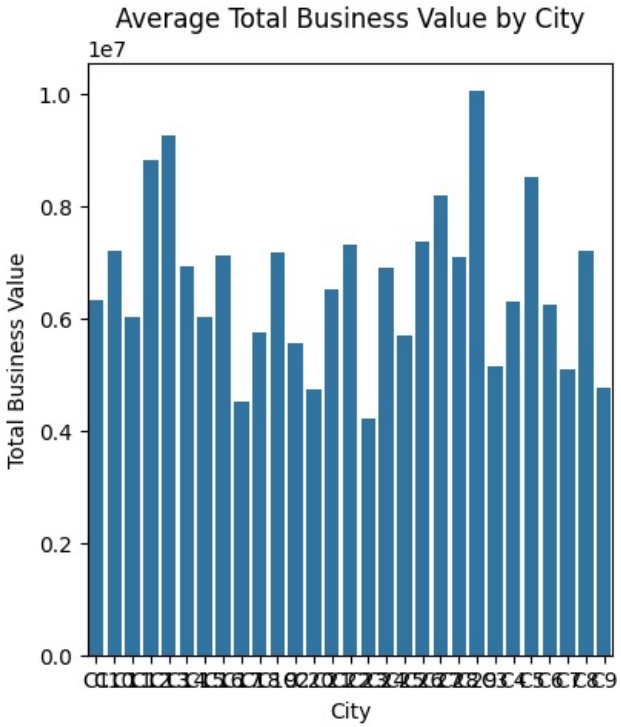
	City	Total Business Value
0	C1	4.498329e+06
1	C10	4.678149e+06
2	C11	3.938141e+06
3	C12	5.989065e+06
4	C13	6.381320e+06
5	C14	4.986578e+06
6	C15	4.168672e+06
7	C16	5.339325e+06
8	C17	2.659584e+06
9	C18	4.337070e+06
10	C19	5.074116e+06
11	C2	3.627616e+06
12	C20	3.107131e+06
13	C21	4.371252e+06
14	C22	5.522406e+06
15	C23	3.082497e+06
16	C24	4.917992e+06
17	C25	4.005728e+06
18	C26	6.184266e+06
19	C27	5.051943e+06
20	C28	4.925986e+06
21	C29	6.905977e+06
22	C3	3.557909e+06
23	C4	4.174303e+06
24	C5	5.205819e+06
25	C6	4.789594e+06
26	C7	3.882930e+06
27	C8	4.530628e+06
28	C9	3.244210e+06

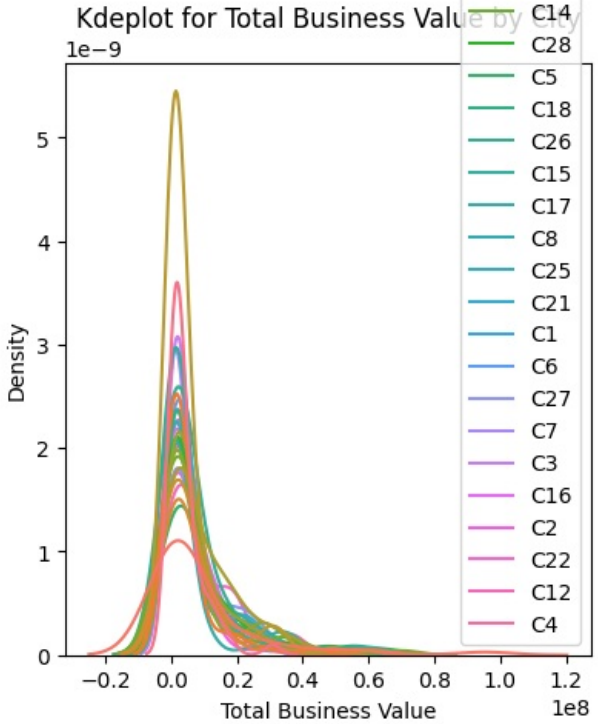
- As per this graph, we can conclude that total business value is similar across cities.
- Let us do one more exercise though
- We established earlier that 161 drivers who have 0 TBV value have substantial income and hence there is chance of missing data there.
- We will ignore those rows and focus on drivers that have non zero total business value and check churn in them
- We already created a dataframe nonZeroTBVDrivers earlier, that we can reuse here for analysis.

In [666…
```
perform_anova_kruskal(nonZeroTBVDrivers,numerical_column='Total Business Value', categorical_column='City', test=test,log="detailed")
```

```
Ho = City and Total Business Value are independent. Medians of groups of Total Business Value based on City are same
Ha = City and Total Business Value are DEPENDENT. Medians of groups of Total Business Value based on City are different
Kruskal-Wallis Result:34.03371501834049,0.199765427507146
Effect Size: η²=0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'City and Total Business Value are independent. Medians of groups of Total Business Value based on City are same In sim
ple words, there’s no substantial Total Business Value difference across City values' is True, p-value:0.20
```


1e8
1.0
0.8
0.6
0.4
0.2
0.0
Total Business Value
City


Average Total Business Value by City
1e7
1.0
0.8
0.6
0.4
0.2
0.0
Total Business Value
City


Kdeplot for Total Business Value by City
1e−9
5
4
3
2
1
0
Density
−0.2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1e8
Total Business Value
City
C23
C13
C9
C11
C19
C20
C29
C10
C24
C14
C28
C5
C18
C26
C15
C17
C8
C25
C21
C1
C6
C27
C7
C3
C16
C2
C22
C12
C4

	City	Total Business Value
0	C1	6.313444e+06
1	C10	7.193228e+06
2	C11	6.011436e+06
3	C12	8.820259e+06
4	C13	9.246402e+06
5	C14	6.913589e+06
6	C15	6.014799e+06
7	C16	7.119100e+06
8	C17	4.505756e+06
9	C18	5.754958e+06
10	C19	7.163458e+06
11	C2	5.557198e+06
12	C20	4.722839e+06
13	C21	6.515640e+06
14	C22	7.303827e+06
15	C23	4.224427e+06
16	C24	6.904104e+06
17	C25	5.700459e+06
18	C26	7.373548e+06
19	C27	8.174962e+06
20	C28	7.099052e+06
21	C29	1.004506e+07
22	C3	5.142702e+06
23	C4	6.302379e+06
24	C5	8.501275e+06
25	C6	6.226472e+06
26	C7	5.089721e+06
27	C8	7.200463e+06
28	C9	4.770897e+06

- As per this graph, we can conclude that total business value is similar across cities.

3.2.40 City and Grade

In [667…
```
perform_chi_square_test(driversWithCategories,'City','Grade',log="detailed_noplot")
```

```
Contingency Table:
Grade   1   2   3   4  5
City
C1     24  32  16   7  1
C10    30  29  20   7  0
C11    16  28  18   1  1
C12    17  27  29   7  1
C13    23  23  18   5  2
C14    23  29  20   6  1
C15    33  33  28   6  1
C16    32  25  21   4  2
C17    26  28  15   2  0
C18    28  16  21   4  0
C19    22  23  21   5  1
C2     30  17  22   3  0
C20    40  60  42   9  1
C21    25  31  20   2  1
C22    27  29  21   4  1
C23    26  30  14   4  0
C24    25  22  25   1  0
C25    26  27  16   4  1
C26    23  39  23   6  2
C27    21  30  31   7  0
C28    23  34  19   5  1
C29    24  28  32  10  2
C3     32  31  17   2  0
C4     28  22  21   5  1
C5     23  34  20   1  2
C6     27  25  21   4  1
C7     23  30  17   5  1
C8     21  39  20   9  0
C9     23  34  15   3  0


Chi-square Test Results:
Chi-square statistic: 101.4975
p-value: 0.7518
Degrees of freedom: 112
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>City and Grade are independent.' is True, p-value:0.7517562728143586
==================================================
```

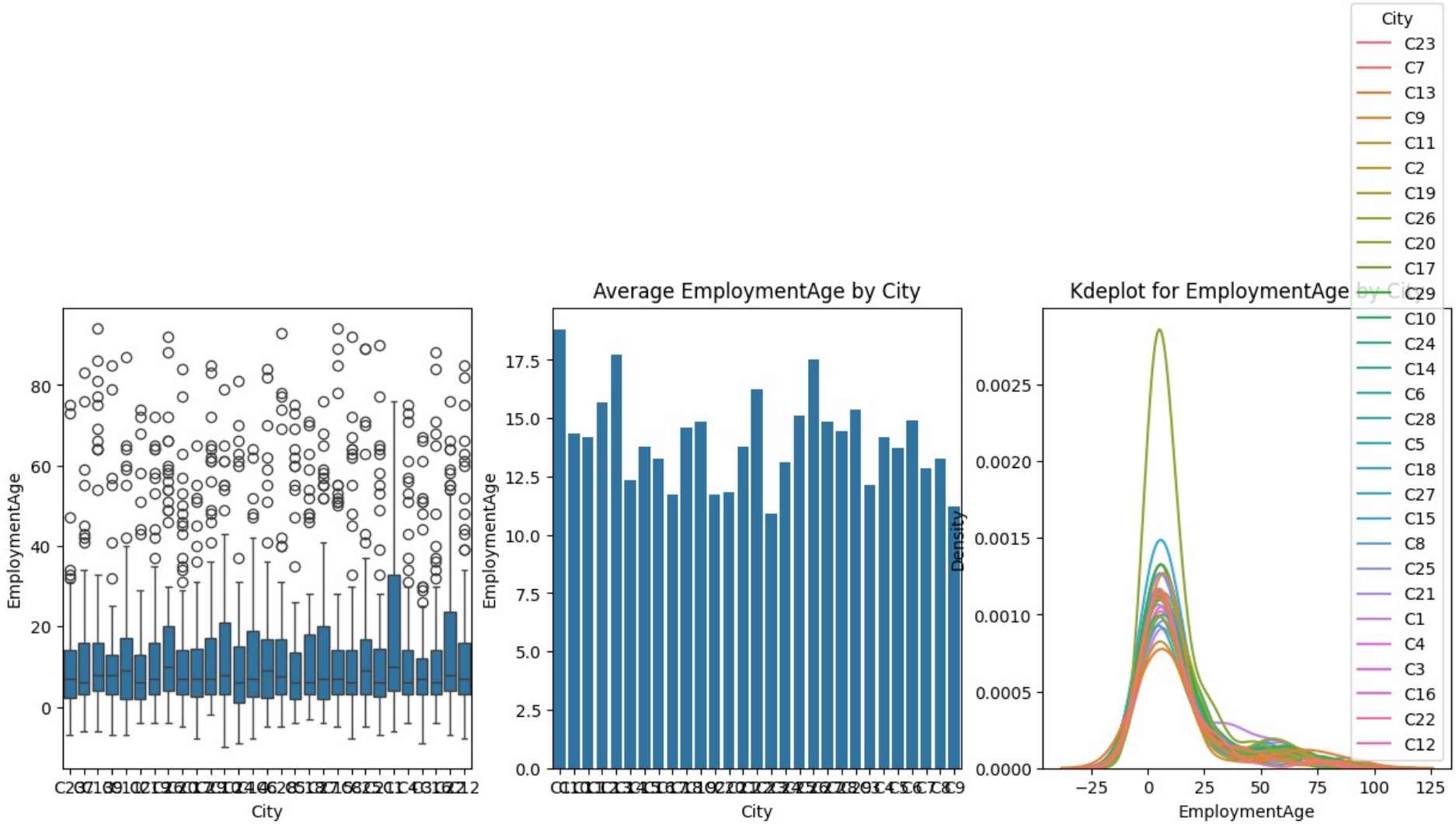
- City and Grade are independent.

3.2.41 City and EmploymentAge

In [668…

```
perform_anova_kruskal(driversWithCategories,numerical_column='EmploymentAge', categorical_column='City', test=test,log="detailed")
```

```
Ho = City and EmploymentAge are independent. Medians of groups of EmploymentAge based on City are same
Ha = City and EmploymentAge are DEPENDENT. Medians of groups of EmploymentAge based on City are different
Kruskal-Wallis Result:21.750559249733087,0.792706461908808
Effect Size: η²=-0.00
Fail to Reject Null Hypothesis. Null Hypothesis 'City and EmploymentAge are independent. Medians of groups of EmploymentAge based on City are same In simple words, the
re’s no substantial EmploymentAge difference across City values' is True, p-value:0.79
```


EmploymentAge
City
Average EmploymentAge by City
EmploymentAge
City
Kdeplot for EmploymentAge by City
Density
EmploymentAge
City
C23
C7
C13
C9
C11
C2
C19
C26
C20
C17
C29
C10
C24
C14
C6
C28
C5
C18
C27
C15
C8
C25
C21
C1
C4
C3
C16
C22
C12

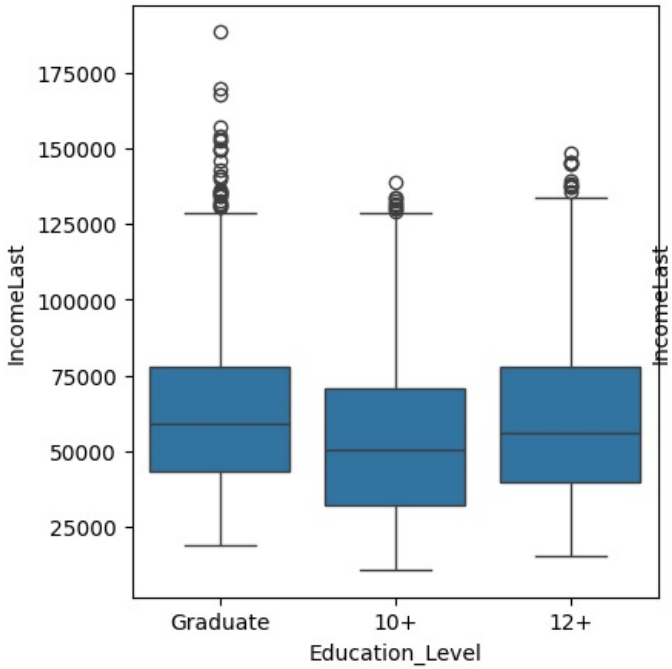
	City	EmploymentAge
0	C1	18.800000
1	C10	14.337209
2	C11	14.187500
3	C12	15.654321
4	C13	17.704225
5	C14	12.329114
6	C15	13.792079
7	C16	13.273810
8	C17	11.746479
9	C18	14.608696
10	C19	14.847222
11	C2	11.750000
12	C20	11.809211
13	C21	13.759494
14	C22	16.231707
15	C23	10.891892
16	C24	13.095890
17	C25	15.135135
18	C26	17.526882
19	C27	14.876404
20	C28	14.439024
21	C29	15.385417
22	C3	12.158537
23	C4	14.181818
24	C5	13.737500
25	C6	14.884615
26	C7	12.855263
27	C8	13.280899
28	C9	11.240000

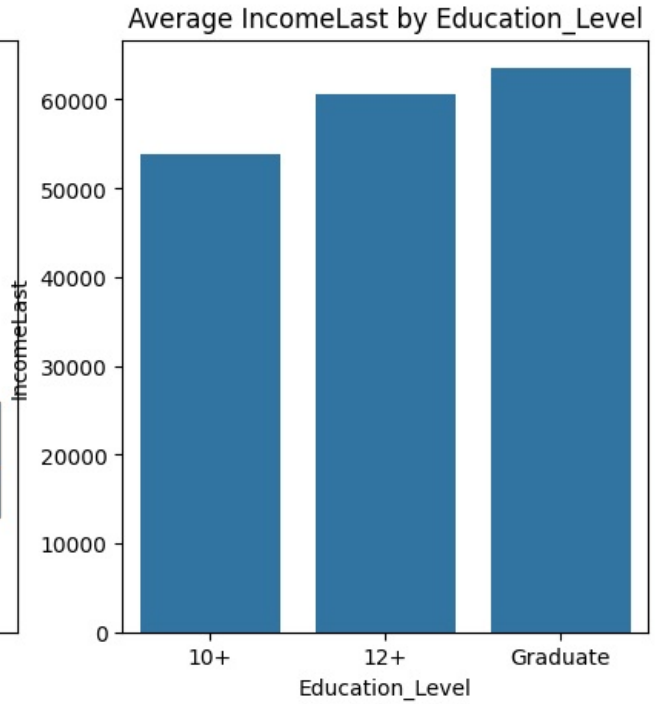
- EmploymentAge is similar across cities

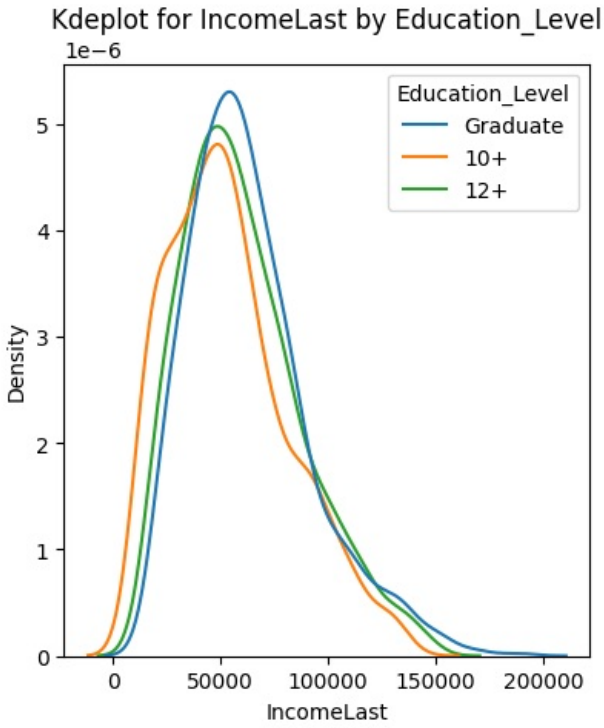
3.2.42 Education_Level and IncomeLast

In [669…
```
perform_anova_kruskal(driversWithCategories,numerical_column='IncomeLast', categorical_column='Education_Level', test=test,log="detailed")
```

```
Ho = Education_Level and IncomeLast are independent. Medians of groups of IncomeLast based on Education_Level are same
Ha = Education_Level and IncomeLast are DEPENDENT. Medians of groups of IncomeLast based on Education_Level are different
Kruskal-Wallis Result:51.93343722734432,5.281987790044911e-12
Effect Size: η²=0.02
Reject Null Hypothesis. Alternate Hypothesis 'Education_Level and IncomeLast are DEPENDENT. Medians of groups of IncomeLast based on Education_Level are different In s
imple words, there’s substantial IncomeLast difference across Education_Level values' is True, p-value:0.00.
Education_Level values vary across IncomeLast groups (enough to be statistically significant kruskal p_value=5.281987790044911e-12), IncomeLast influence on Education_
Level is considered small (η²=0.02)
```







	Education_Level	IncomeLast
0	10+	53800.839286
1	12+	60523.885535
2	Graduate	63563.941397

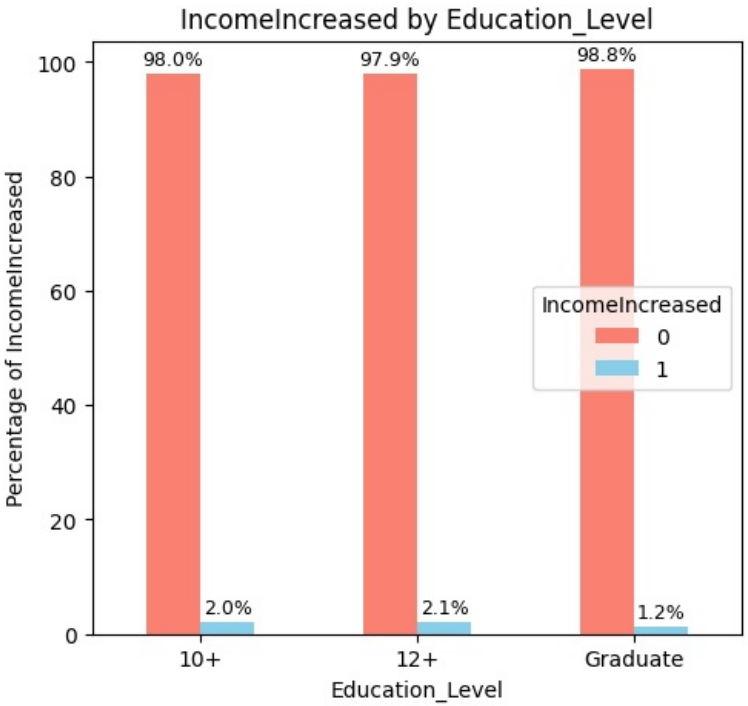
- Average income varies across education levels- As education level increases, mean income increases
- Avg income of Graduates is 63563 which is Rs 10000 more than drivers who are only 10+ education level

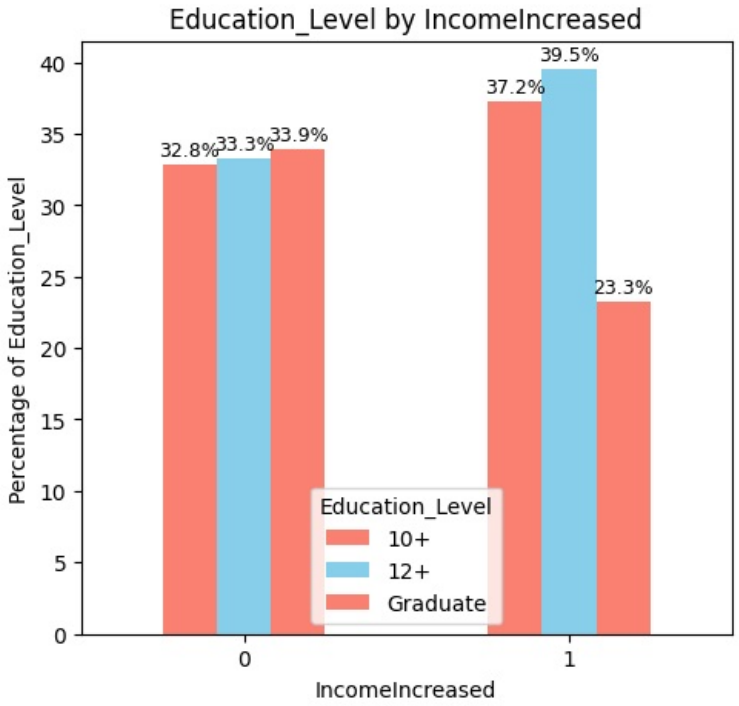
3.2.43 Education_Level and IncomeIncreased

In [670…

```
perform_chi_square_test(driversWithCategories,'Education_Level','IncomeIncreased')
```

```
Contingency Table:
IncomeIncreased    0   1
Education_Level
10+              768  16
12+              778  17
Graduate         792  10


Chi-square Test Results:
Chi-square statistic: 2.1528
p-value: 0.3408
Degrees of freedom: 2
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Education_Level and IncomeIncreased are independent.' is True, p-value:0.3408223685733875
```





==========

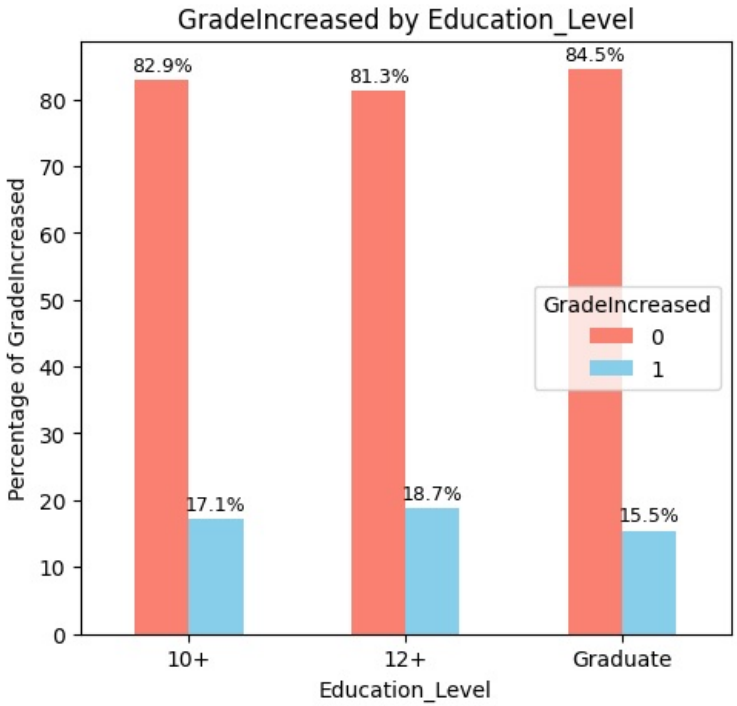
- Income increased is independent of education level

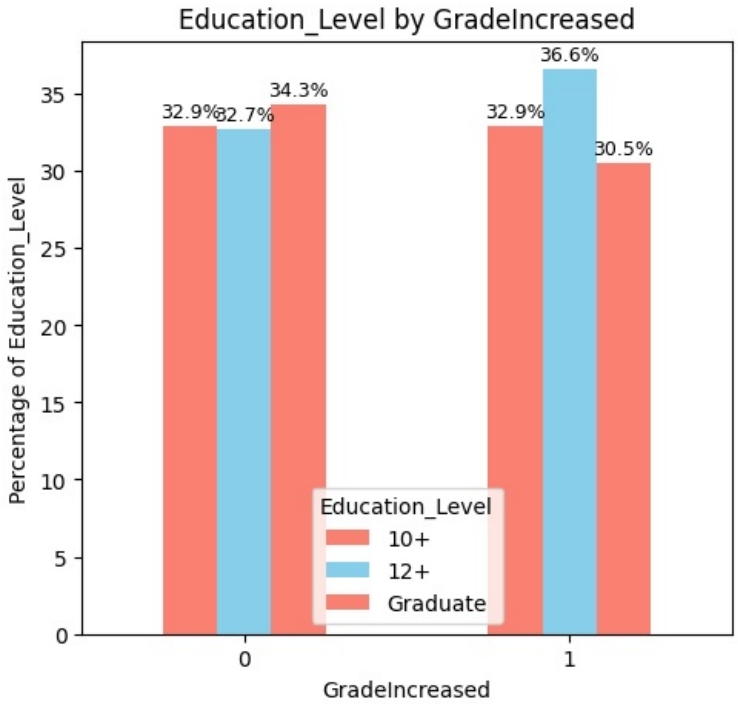
3.2.44 Education_Level and GradeIncreased

In [671…

```
perform_chi_square_test(driversWithCategories,'Education_Level','GradeIncreased')
```

```
Contingency Table:
GradeIncreased     0    1
Education_Level
10+              650  134
12+              646  149
Graduate         678  124


Chi-square Test Results:
Chi-square statistic: 3.0323
p-value: 0.2196
Degrees of freedom: 2
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>Education_Level and GradeIncreased are independent.' is True, p-value:0.21955603862176584
```





==========

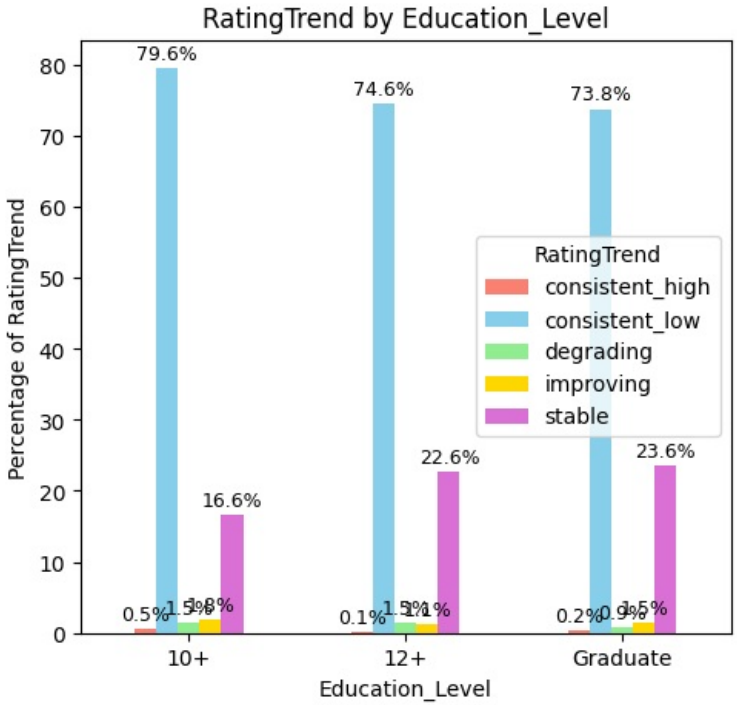
- Grade increase and education level are unrelated

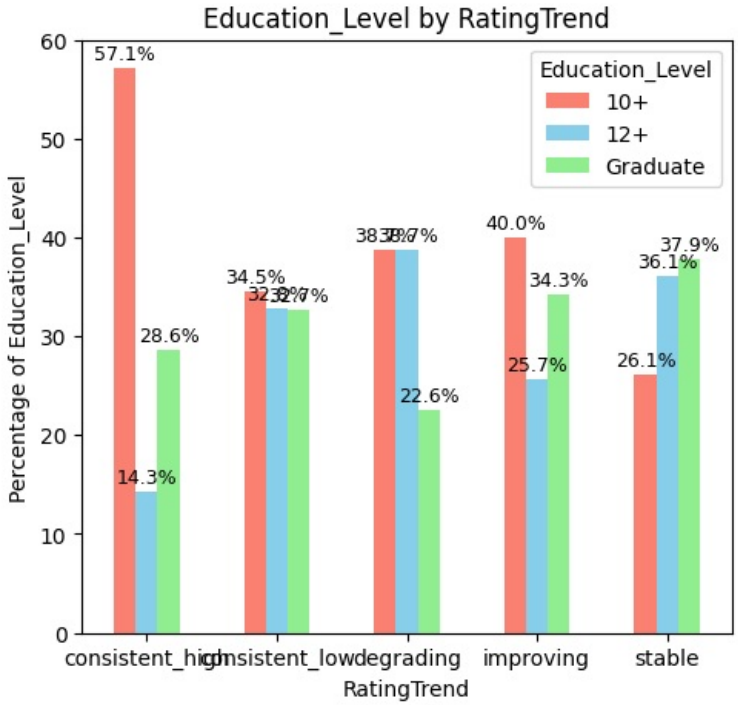
3.2.44 Education_Level and RatingTrend

In [672…

```
perform_chi_square_test(driversWithCategories,'Education_Level','RatingTrend')
```

```
Contingency Table:
RatingTrend      consistent_high  consistent_low  degrading  improving  stable
Education_Level
10+                            4             624         12         14     130
12+                            1             593         12          9     180
Graduate                       2             592          7         12     189


Chi-square Test Results:
Chi-square statistic: 17.8165
p-value: 0.0226
Degrees of freedom: 8
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>Education_Level and RatingTrend are DEPENDENT' is True, p-value:0.022645036466319563. However, association is Negligi
ble(CramersV=0.06)
```





==========

In [673…
```
driversWithCategories.RatingTrend.value_counts()
```

Out[673…
```
RatingTrend
consistent_low     1809
stable              499
improving            35
degrading            31
consistent_high       7
Name: count, dtype: int64
```

- Drivers who have consistently low ratings (1809) are found in all categories equally.
- More than half (57%) of the 7 drivers who have consistently high rating are only having 10+ education

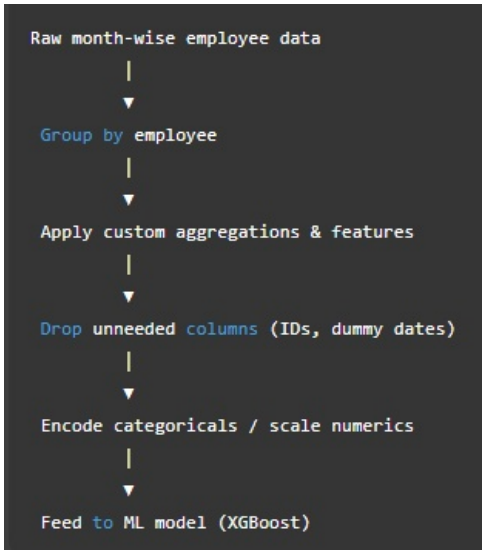
4.0 Data Preprocessing

- During EDA we grouped the data, engineered many columns and reached the drivers dataframe
- We can use the drivers dataframe for ML Model but it is not how real world works. **- In production / real-world ML pipelines, the data we get is rarely pre-aggregated. **
  - We usually get raw transactional or month-wise data for employees.
  - So our pipeline should be able to take raw data and:
    - Aggregate/group per employee (or per whatever entity you're predicting for).
    - Apply custom feature engineering (like rating trend, last/first/mean values).
    - Compute derived features (tenure, changes, ratios).
    - Encode categoricals, scale numeric features, and drop unnecessary columns.
    - Feed the result to the model.
  - This way, any new incoming batch of employees can be automatically processed without manual intervention.

- Why this is better than pre-aggregating

- If we pre-aggregate before creating a pipeline, our pipeline cannot handle new data
- Hardcoding aggregations or trends outside the pipeline means every time new data arrives, we would have to manually reproduce the same transformations.
- Keeping the aggregation + feature engineering inside the pipeline ensures consistency between train and future inference.

How it looks conceptually Our pipeline would roughly do:



Implementation notes

- We will implement custom transformers using sklearn.base.BaseEstimator + TransformerMixin for:
  - Grouping & aggregating
  - Rating trend calculation
  - Tenure computation
- Then use ColumnTransformer + Pipeline to handle numeric/categorical preprocessing and modeling.

For steps like grouping, rating trend, income CAGR, we create custom transformers using sklearn.base.BaseEstimator and TransformerMixin.

In [984…

```python
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.impute import KNNImputer, SimpleImputer

class HRChurnFeatureBuilder(BaseEstimator, TransformerMixin):

    def __init__(self, dummy_date="31/12/67"):
        self.dummy_date = dummy_date
        self.average_days_per_month = 365.25/12


    # def transform(self, X):

    def calculateEmploymentAge(self, row):
        if row.Churn==1:
            return np.floor((row.LastWorkingDate - row.Dateofjoining).days/self.average_days_per_month)
        else:
            return np.floor((self.maxDate - row.Dateofjoining).days/self.average_days_per_month)

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()

        # --- Step 1: Drop unnecessary column ---
        X = X.drop(columns=['Unnamed: 0'], errors='ignore')

        # --- Step 2: Impute Missing Data ---
        imputer = KNNImputer(n_neighbors=1)
        cols_for_impute = ['Age', 'Gender', 'Driver_ID']
        df_impute = X[cols_for_impute].copy()
        df_imputed_values = imputer.fit_transform(df_impute)
        X['Age'] = df_imputed_values[:, 0]
        X['Gender'] = df_imputed_values[:, 1]

        imp_missing = SimpleImputer(strategy='constant', fill_value=self.dummy_date)
        X[['LastWorkingDate']] = imp_missing.fit_transform(X[['LastWorkingDate']])


        # --- Step 3: Parse date columns ---
        X['MMM-YY'] = pd.to_datetime(X['MMM-YY'], format='%m/%d/%y', errors='coerce')
        X['Dateofjoining'] = pd.to_datetime(X['Dateofjoining'], format='%d/%m/%y', errors='coerce')
        X['LastWorkingDate'] = pd.to_datetime(X['LastWorkingDate'], format='%d/%m/%y', errors='coerce')

        self.maxDate = max(X['MMM-YY'])

        # --- Step 4: Group by Driver_ID ---
        grouped = X.groupby('Driver_ID', as_index=False)

        drivers = grouped.agg({
            'Age': 'last',
            'Gender': 'last',
            'City': 'last',
            'Education_Level': 'last',
```

```
        'Dateofjoining': 'first',
        'LastWorkingDate': 'last',
        'Joining Designation': 'first',
        'Grade': 'last',
        'Total Business Value': 'sum',
        'Quarterly Rating': 'mean'
    })

    # --- Step 5: Feature Engineering

    # Adding Income CAGR & Last Income ---

    #Aggregate first & last data per driver
    df_sorted = X.sort_values(['Driver_ID', 'MMM-YY'])
    cagr = (
        df_sorted.groupby('Driver_ID', as_index=False)
        .agg(
            IncomeFirst=('Income', 'first'),
            IncomeLast=('Income', 'last'),
            first_date=('MMM-YY', 'first'),
            last_date=('MMM-YY', 'last')
        )
    )
    #Compute time difference in years
    cagr['years'] = (cagr['last_date'] - cagr['first_date']).dt.days / 365
    # Compute CAGR safely
    #Safel because we'll handle:
    #Drivers with only one record (years == 0)
    #Missing or invalid data (NaN or zero division)
    #If the driver has only one entry → years == 0, we set CAGR = 0.
    cagr['IncomeCAGR'] = np.where(
        cagr['years'] > 0,
        ((cagr['IncomeLast'] / cagr['IncomeFirst']) ** (1 / cagr['years']) - 1) * 100,
        0
    )

    drivers = drivers.merge(cagr[['Driver_ID', 'IncomeLast', 'IncomeCAGR']], on='Driver_ID', how='left')

    # Adding IncomeIncreased
    drivers['IncomeIncreased'] = np.where(drivers['IncomeCAGR'] > 0, 1, 0)

    # Adding GradeIncreased
    drivers['GradeIncreased'] = np.where(drivers['Grade'] > drivers['Joining Designation'], 1, 0)

    # Adding RatingMean, RatingTrend,RatingTrendCode,Ratings
    driver_rating = X.groupby('Driver_ID').apply(classify_driver_rating, include_groups=False).reset_index()
    drivers = drivers.merge(driver_rating, on='Driver_ID', how='left')

    # Adding Churn
    dummyDate = pd.Timestamp('2067-12-31')
    drivers['Churn'] = (drivers['LastWorkingDate'] != dummyDate).astype(int)

    # Adding EmploymentAge
    drivers['EmploymentAge'] = drivers.apply(calculateEmploymentAge, axis=1)

    #Fix negative EmploymentAge (it is coming if LastWorkinDate is less than joinind date, it was present in 130 records in training date)
    #Step 1: Find the rows that has the issue
    driversWithNegativeEmpAge = drivers[drivers.EmploymentAge<0][['Driver_ID']].copy()
```

```
        #Count how many records we have for these drivers.  We shall simply put that as the EmploymentAge. It is a fair assumption
        driversWithNegativeEmpAge = driversWithNegativeEmpAge.groupby('Driver_ID', as_index=False).size()
        #Step 2: Merge the counts back and update
        drivers = drivers.merge(
            driversWithNegativeEmpAge.rename(columns={'size': 'EmpAgeFix'}),
            on='Driver_ID',
            how='left'
        )

        # Step 3: Where EmploymentAge was negative, replace with count
        drivers.loc[drivers.EmploymentAge < 0, 'EmploymentAge'] = drivers.loc[drivers.EmploymentAge < 0, 'EmpAgeFix']

        # Step 4: Drop the temporary column
        drivers = drivers.drop(columns=['EmpAgeFix'])

        return drivers
```

In [ ]:

```
from sklearn.preprocessing import OneHotEncoder, RobustScaler, StandardScaler
from category_encoders import TargetEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# --- Column classification ---
categorical_cols = ['City']  # nominal categorical

# Technical note: One may argue that Gender is not a ordinal column but a nominal one.
# Point taken. Generally we use OHE for such columns
# We have kept Gender here as it already has values 0,1.  Even if we one hot encode it,
# the effect on models is same.
ordinal_cols = [
    'Education_Level', 'Joining Designation', 'Grade',
    'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased', 'RatingTrendCode' ,'Gender'
]  # ordinal / discrete categories

numeric_cols = [
    'Age', 'Total Business Value', 'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'
]  # continuous numericals

# --- Preprocessing pipelines ---
categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    # ('encoder', OneHotEncoder(handle_unknown='ignore')),
     ('encoder', TargetEncoder())

])

ordinal_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent'))
    # no scaling or encoding, model can handle integer ordinals directly
])


numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', RobustScaler())  # better for non-normal distributions
    #('scaler', StandardScaler())  #
])
```

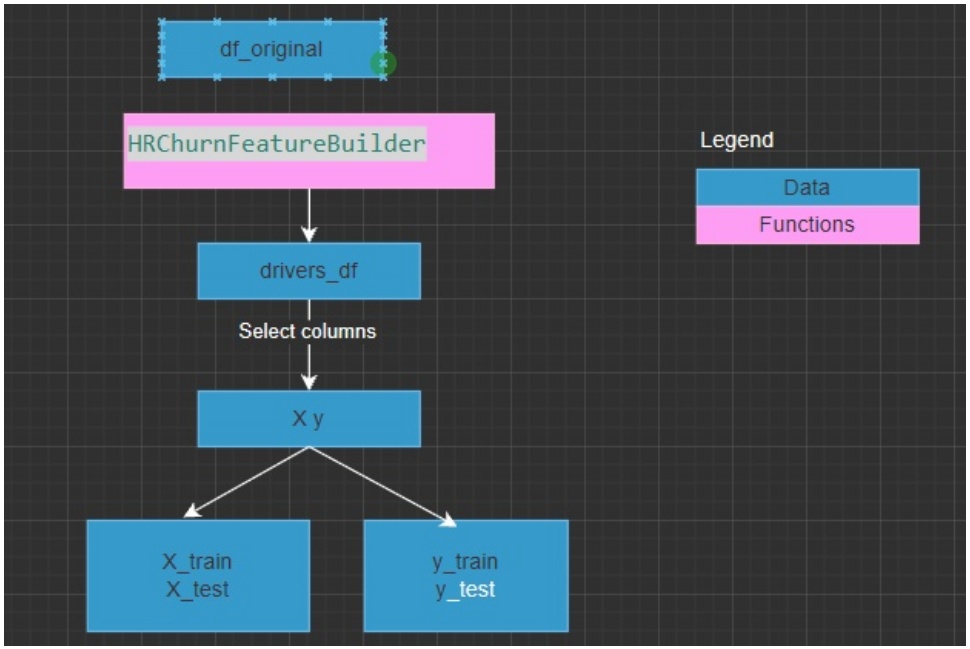
```
# --- Combine all into one preprocessor ---
preprocessor = ColumnTransformer([
    ('cat', categorical_transformer, categorical_cols),
    ('ord', ordinal_transformer, ordinal_cols),
    ('num', numeric_transformer, numeric_cols),
], remainder='drop')
```

5.0 Fitting a Machine Learning Model on the data to predict churn

5.1 Prepare the data

5.1.1 Group Data

Here is how the data is being prepared for model



In [154…
```
# Step 1 – Feature engineering
df_original =pd.read_csv("ola_driver_scaler.csv")

feature_builder = HRChurnFeatureBuilder(dummy_date="31/12/67")
driver_df = feature_builder.fit_transform(df_original)

driver_df
```

Out[154…

	Driver_ID	Age	Gender	City	Education_Level	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	...	IncomeLast	IncomeCAGR	IncomeIncreased	GradeIncreased	RatingMean
0	1	28.0	0.0	C23	2	2018-12-24	2019-11-03	1	1	1715580	...	57387	0.0	0	0	2.000000
1	2	31.0	0.0	C7	2	2020-06-11	2067-12-31	2	2	0	...	67016	0.0	0	0	1.000000
2	4	43.0	0.0	C13	2	2019-07-12	2020-04-27	2	2	350000	...	65603	0.0	0	0	1.000000
3	5	29.0	0.0	C9	0	2019-09-01	2019-07-03	1	1	120360	...	46368	0.0	0	0	1.000000
4	6	31.0	1.0	C11	1	2020-07-31	2067-12-31	3	3	1265000	...	78728	0.0	0	0	1.600000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2376	2784	34.0	0.0	C24	0	2015-10-15	2067-12-31	2	3	21748820	...	82815	0.0	0	1	2.625000
2377	2785	34.0	1.0	C9	0	2020-08-28	2020-10-28	1	1	0	...	12105	0.0	0	0	1.000000
2378	2786	45.0	0.0	C19	0	2018-07-31	2019-09-22	2	2	2815090	...	35370	0.0	0	0	1.666667
2379	2787	28.0	1.0	C20	2	2018-07-21	2019-06-20	1	1	977830	...	69498	0.0	0	0	1.500000
2380	2788	30.0	0.0	C27	2	2020-08-06	2067-12-31	2	2	2298240	...	70254	0.0	0	0	2.285714

2381 rows × 21 columns

5.1.2 Train Test Split

In [154…

```
# Keep a copy for later join-back
driver_ids = driver_df['Driver_ID']

# Split into features (X) and label (y)
#We drop few columns that we dont need for training model, and we have derived some other features from them
#We drop Churn as it is target column
X = driver_df.drop(columns=['Driver_ID','Dateofjoining', 'LastWorkingDate','Ratings','RatingTrend','Churn'])
y = driver_df['Churn']

#
# Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In [154…
```
X_train
```

Out[154…

	Age	Gender	City	Education_Level	Joining Designation	Grade	Total Business Value	Quarterly Rating	IncomeLast	IncomeCAGR	IncomeIncreased	GradeIncreased	RatingMean	RatingTrendCode	EmploymentAge
2242	34.0	1.0	C7	0	3	3	0	1.000000	104058	0.0	0	0	1.000000	1	5.0
1474	42.0	0.0	C9	0	1	1	0	1.000000	51579	0.0	0	0	1.000000	1	17.0
2132	27.0	0.0	C17	1	2	2	0	1.000000	75458	0.0	0	0	1.000000	1	2.0
1873	35.0	0.0	C15	1	1	3	29040450	3.000000	69756	0.0	0	1	3.000000	3	53.0
462	36.0	0.0	C24	0	2	3	0	1.000000	109296	0.0	0	1	1.000000	1	30.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1883	39.0	0.0	C26	1	1	4	66352800	3.285714	112513	0.0	0	1	3.285714	3	20.0
1355	27.0	0.0	C28	0	2	2	0	1.000000	49886	0.0	0	0	1.000000	1	1.0
2229	39.0	0.0	C29	1	1	1	2607050	2.000000	44154	0.0	0	0	2.000000	1	3.0
2234	28.0	0.0	C16	1	1	1	2264270	1.272727	53320	0.0	0	0	1.272727	1	6.0
855	30.0	1.0	C8	0	1	1	0	1.000000	58540	0.0	0	0	1.000000	1	4.0

1904 rows × 15 columns

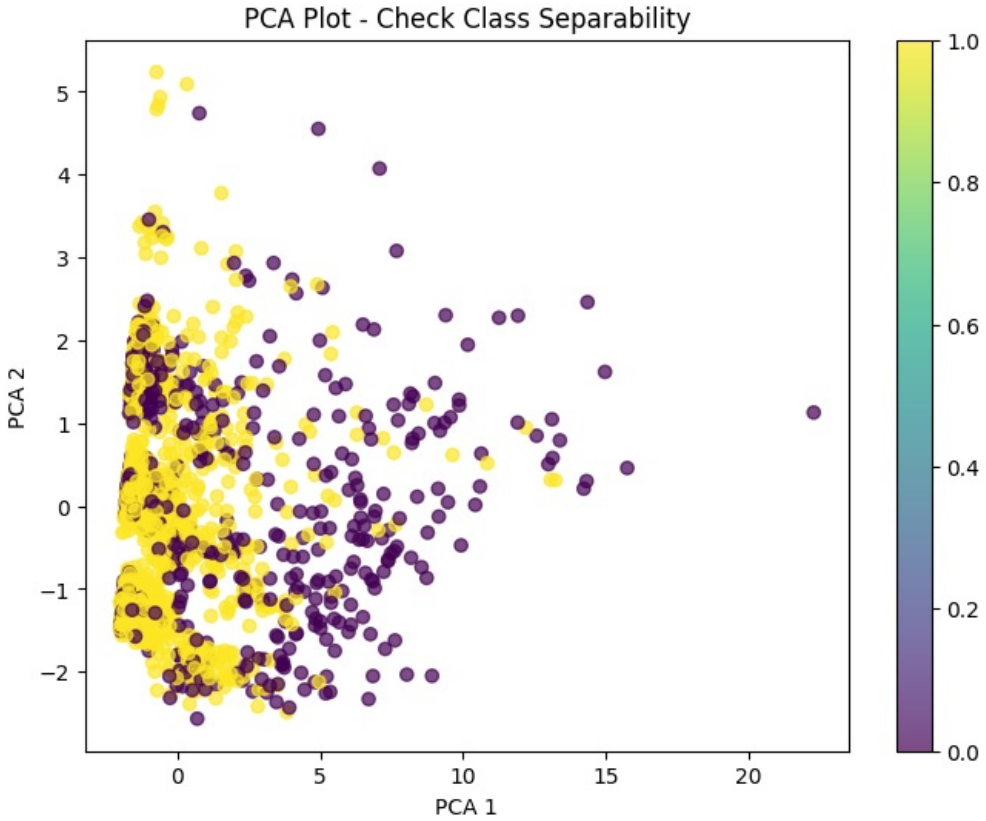
5.2 Quick Checks on data - PCA to see separatability, VIF to see collinearity

5.2.1 Access the preproceesor output and use PCA to quickly visualize data

In [154…
```
from sklearn.decomposition import PCA

#Create a pipeline with just the preprocessor
pipeline_to_get_input_data  = Pipeline([
    ('preprocess', preprocessor),
])
pipeline_to_get_input_data.fit(X_train, y_train)


# Access the fitted preprocessor
X_train_transformed = pipeline_to_get_input_data.named_steps['preprocess'].transform(X_train)

X_train_scaled = pd.DataFrame(X_train_transformed, columns=categorical_cols+ ordinal_cols+ numeric_cols)

#X_scaled is a dataframe that has the correct column names and all features in numerical format.
X_train_scaled.head()


pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_train_scaled)

print("Explained variance ratio:", pca.explained_variance_ratio_)

# Plot
```

```
plt.figure(figsize=(8,6))
scatter = plt.scatter(X_pca[:,0], X_pca[:,1], c=y_train, alpha=0.7)
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.title("PCA Plot - Check Class Separability")
plt.colorbar(scatter)
plt.show()
```

```
Explained variance ratio: [0.5631135  0.12596309]
```



5.2.2 Use PCA to compress to 3 dimensions, where we can rotate as well

In [154…

```
import plotly.express as px
from sklearn.decomposition import PCA

# PCA 3 components
pca = PCA(n_components=3)
X_pca = pca.fit_transform(X_train_scaled)
print("Explained variance ratio:", pca.explained_variance_ratio_)

# Convert to DataFrame
df_pca = pd.DataFrame({
    'PCA1': X_pca[:,0],
    'PCA2': X_pca[:,1],
    'PCA3': X_pca[:,2],
```

```
    'Target': y_train
})

# Interactive plot
fig = px.scatter_3d(
    df_pca,
    x='PCA1',
    y='PCA2',
    z='PCA3',
    color='Target',
    opacity=0.8,
    width=900,
    height=700
)

fig.update_traces(marker=dict(size=4))
fig.show()
```

```
Explained variance ratio: [0.5631135  0.12596309 0.0835692 ]
```

- The data does look separable even in 3 dimensions. We have 15 dimensions
- Approximate linear separability Not clearly linear — notable overlap in PCA1–PCA2 and PCA1–PCA3 projections.
- Need for nonlinear interaction capture- PCA shows curved “frontier”-like mixing; not a straight split. Suggests relationships among predictors might be multiplicative or nonlinear.

5.2.3 Feature Selection using VIF score checking for Multicollinearity

Lets calculate VIF once, to see what we have on hand - do we even need to check rigourously

In [154…

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif = pd.DataFrame()
vif['Features'] = X_train_scaled.columns
vif['VIF'] = [variance_inflation_factor(X_train_scaled.values, i) for i in range(X_train_scaled.shape[1])]
vif_sorted = vif.sort_values(by='VIF', ascending=False)
vif_sorted
```

Out[154…

	Features	VIF
4	Quarterly Rating	356.875310
13	RatingMean	105.705042
5	IncomeIncreased	14.464880
12	IncomeCAGR	14.258356
3	Grade	13.893484
2	Joining Designation	10.704713
6	GradeIncreased	5.915194
7	RatingTrendCode	3.783199
10	Total Business Value	3.694568
14	EmploymentAge	2.897253
11	IncomeLast	2.369469
9	Age	1.160679
1	Education_Level	1.059439
0	City	1.016781
8	Gender	1.009353

Indeed, it seems we have lot of columns that are collinear. We have our work cutout.
We will calculate VIF recursively till we have no columns with high VIF

In [154…

```
def calculate_vif(X, thresh=10.0):
    """
    Iteratively remove features with VIF above the specified threshold.

    Parameters:
        X (pd.DataFrame): Scaled numeric feature DataFrame
        thresh (float): VIF threshold, e.g. 10.0

    Returns:
        pd.DataFrame: Final VIF values for remaining features
        list: Names of features that were removed
    """

    variables = X.columns.tolist()
    dropped = True
    removed_features = []

    while dropped:
        dropped = False
        vif = pd.DataFrame()
        vif['Features'] = variables
        vif['VIF'] = [variance_inflation_factor(X[variables].values, i)
                      for i in range(len(variables))]

        max_vif = vif['VIF'].max()
        if max_vif > thresh:
```

```
            # Feature with the highest VIF
            feature_to_drop = vif.loc[vif['VIF'].idxmax(), 'Features']

            print(f"We can drop '{feature_to_drop}' as its VIF is ={max_vif:.2f}")

            variables.remove(feature_to_drop)
            removed_features.append(feature_to_drop)
            dropped = True

    # Compute final VIFs for remaining features
    final_vif = pd.DataFrame()
    final_vif['Features'] = variables
    final_vif['VIF'] = [variance_inflation_factor(X[variables].values, i)
                        for i in range(len(variables))]

    # Sort for readability
    final_vif = final_vif.sort_values(by='VIF', ascending=False).reset_index(drop=True)
    return final_vif, removed_features

final_vif, columns_to_drop = calculate_vif(X_train_scaled, thresh=10)
print("\nFinal VIF values:")
print(final_vif)

print("\nRemovable features:",len(columns_to_drop))
print(columns_to_drop)
```

```
We can drop 'Quarterly Rating' as its VIF is =356.88
We can drop 'Grade' as its VIF is =82.49
We can drop 'City' as its VIF is =16.12
We can drop 'IncomeIncreased' as its VIF is =14.47
We can drop 'RatingTrendCode' as its VIF is =11.80

Final VIF values:
               Features       VIF
0  Total Business Value  3.564701
1         EmploymentAge  3.301733
2            RatingMean  3.083065
3        GradeIncreased  2.904099
4   Joining Designation  2.682852
5       Education_Level  2.182359
6                Gender  1.599849
7            IncomeLast  1.409092
8            IncomeCAGR  1.284353
9                   Age  1.146486

Removable features: 5
['Quarterly Rating', 'Grade', 'City', 'IncomeIncreased', 'RatingTrendCode']
```

- Lets model the dropping of these columns as a transformer.
- We can use it if required, by adding it in pipeline, just before we feed the data into the Model

In [154…

```
import pandas as pd


class ColumnDropper(BaseEstimator, TransformerMixin):
    """
    Drops specified columns from a NumPy array (output of preprocessor),
```

```
    using known column names from preprocessor.get_feature_names_out().
    """
    def __init__(self, drop_cols=None, all_feature_names=None, return_df=False):
        self.drop_cols = drop_cols or []
        self.all_feature_names = all_feature_names  # column names from preprocessor
        self.return_df = return_df  # optional: return a DataFrame if desired

    def fit(self, X, y=None):
        # Nothing to learn, just store column info
        return self

    def transform(self, X):
        # Make sure we have column names
        if self.all_feature_names is None:
            raise ValueError("all_feature_names must be provided for ColumnDropper.")

        # Convert numpy → DataFrame with the feature names
        df = pd.DataFrame(X, columns=self.all_feature_names)

        # Drop the specified columns
        df_filtered = df.drop(columns=self.drop_cols, errors='ignore')

        #print(f'[{self.__class__.__name__}]- Collinear columns were dropped')
        # Return DataFrame or NumPy depending on configuration
        if self.return_df:
            return df_filtered
        else:
            return df_filtered.to_numpy()
```

5.3 Fitting ALL models without hyperparam tuning, VIF and imbalance fix

In [155…

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
```

Program Flow

Lets create reusable code to build the model and store results

- Builds pipeline,
- trains the model,
- calculates metrics,
- saves results, and
- returns trained model.

In [155…

```python
### Code to plot the results

import matplotlib.pyplot as plt
import numpy as np
import math

def plot_metrics_grid(results, metrics,  selectedModels='ALL',pad_ratio=0.05,):
    """
    Multi-axis figure with:
    - 2 columns grid layout (rows auto-calculated)
    - Each subplot: bar chart for one metric
    - One consistent color per model EXCEPT the highest bar per subplot → GREEN
    - Model names on x-axis
    - Legend for models
    - Zoomed y-axis
    - Value labels on bars

    -selected_model_ids - can be a list of values or a search string with multiple search values.
        - pass the ids of selected models as an list, default is all models. Eg [2,6,10]
        - pass a search string - Logistic, Random
```

```
"""
global tableNumber
#Filter selected models
if (selectedModels!='ALL'):
    #User has passed a list
    if isinstance(selectedModels,list):
        results = results.iloc[selected_model_ids]
    elif isinstance(selectedModels,object):
        filters = [x.strip() for x in selectedModels.split(",")]
        pattern = "|".join(filters)
        results = results[
            results["Model"].str.contains(pattern, case=False, regex=True)
        ]
num_metrics = len(metrics)
num_models = len(results)
#append the ModelID for easier identification when we have large number of models
models = (results["ModelID"].astype(str) + "." + results["Model"]).tolist()


# --- Layout: always 2 columns ---
cols = 2
rows = math.ceil(num_metrics / 2)

fig, axes = plt.subplots(rows, cols, figsize=(15, 4 * rows))
axes = axes.flatten()

# --- Base color palette for models ---
# cmap = plt.cm.get_cmap("tab10", num_models)
# base_colors = [cmap(i) for i in range(num_models)]
base_colors = ["grey" for i in range(num_models)]


for i, metric in enumerate(metrics):
    ax = axes[i]
    values = results[metric].values
    x = np.arange(num_models)

    # Identify max bar → will be GREEN
    max_index = np.argmax(values)

    # Colors for this subplot: model colors, except max = green
    colors = base_colors.copy()
    colors[max_index] = "green"

    # Plot
    bars = ax.bar(x, values, color=colors, edgecolor='black')

    # Title / Labels
    ax.set_title(f'Table {tableNumber}. {metric}', fontsize=14)
    tableNumber+=1 #increment always
    ax.set_xticks(x)
    ax.set_xticklabels(models, rotation=45, ha='right')
    ax.set_ylabel(metric)

    # Auto-zoom y-axis
    ymin, ymax = values.min(), values.max()
    pad = (ymax - ymin) * pad_ratio if ymax > ymin else 0.01
```

```
        ax.set_ylim(ymin - pad, ymax + pad)

        # Bar labels
        for bar in bars:
            h = bar.get_height()
            ax.text(
                bar.get_x() + bar.get_width() / 2,
                h,
                f"{h:.4f}",
                ha='center',
                va='bottom',
                fontsize=9
            )

    # # Legend for models (global)
    # fig.legend(
    #     labels=models,
    #     loc='upper center',
    #     ncol=min(num_models, 4),
    #     fontsize=12,
    #     frameon=False
    # )

    # Hide extra axes if odd number of metrics
    for j in range(num_metrics, len(axes)):
        axes[j].axis("off")

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()


    print((results[["Model","TestF2Score"]].sort_values(by="TestF2Score", ascending=False)))
```

In [155…

```
import time
import numpy as np
from sklearn.metrics import (
    accuracy_score, f1_score, fbeta_score,
    precision_score, recall_score
)
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone
from sklearn.utils.validation import check_is_fitted
from sklearn.exceptions import NotFittedError

def is_model_trained(model):
    try:
        check_is_fitted(model)
        return True
    except NotFittedError:
        return False

def evaluate_model(model_id, model_name, model,
                   X_train, X_test, y_train, y_test,
                   folds=5):
    """
    Use Stratified K-Fold CV on training data.
    Calculate mean metrics across folds.
    Train final model on full training set.
```

```
    Evaluate on test set.
    Save result row.
    """
    skf = StratifiedKFold(n_splits=folds, shuffle=True, random_state=42)
    # Deep copy of pipeline
    untrained_pipeline = clone(model)

    # Lists to collect per-fold metrics
    train_acc_list, val_acc_list = [], []
    train_f1_list,  val_f1_list  = [], []
    train_f2_list,  val_f2_list  = [], []
    train_prec_list, val_prec_list = [], []
    train_rec_list,  val_rec_list  = [], []

    # --- Cross-validation ---
    for train_idx, val_idx in skf.split(X_train, y_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        # Fit on training fold
        # We only fit if the model is not already trained
        if not is_model_trained(model):
            model.fit(X_tr, y_tr)

        # Predictions
        y_pred_tr = model.predict(X_tr)
        y_pred_val = model.predict(X_val)

        # Per-fold metrics
        train_acc_list.append(accuracy_score(y_tr, y_pred_tr))
        val_acc_list.append(accuracy_score(y_val, y_pred_val))

        train_f1_list.append(f1_score(y_tr, y_pred_tr))
        val_f1_list.append(f1_score(y_val, y_pred_val))

        train_f2_list.append(fbeta_score(y_tr, y_pred_tr, beta=2))
        val_f2_list.append(fbeta_score(y_val, y_pred_val, beta=2))

        train_prec_list.append(precision_score(y_tr, y_pred_tr))
        val_prec_list.append(precision_score(y_val, y_pred_val))

        train_rec_list.append(recall_score(y_tr, y_pred_tr))
        val_rec_list.append(recall_score(y_val, y_pred_val))

    # Mean metrics across folds
    train_acc  = np.mean(train_acc_list)
    val_acc    = np.mean(val_acc_list)

    train_f1   = np.mean(train_f1_list)
    val_f1     = np.mean(val_f1_list)

    train_f2   = np.mean(train_f2_list)
    val_f2     = np.mean(val_f2_list)
    val_f2_std = np.std(val_f2_list)

    train_prec = np.mean(train_prec_list)
    val_prec   = np.mean(val_prec_list)
```

```
    train_rec  = np.mean(train_rec_list)
    val_rec    = np.mean(val_rec_list)

    # ---- Train final model on full training data ----
    start = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start

# ---- Evaluate on test ----
    y_pred_test = model.predict(X_test)
    test_acc  = accuracy_score(y_test, y_pred_test)
    test_f1   = f1_score(y_test, y_pred_test)
    test_f2   = fbeta_score(y_test, y_pred_test, beta=2)
    test_prec = precision_score(y_test, y_pred_test)
    test_rec  = recall_score(y_test, y_pred_test)

    # Hyperparameters of classifier part of pipeline
    hyperparams = model.named_steps['clf'].get_params()


    # ---- Store results ----
    global results
    results.loc[len(results)] = [
        model_id, model_name, hyperparams,
        train_acc, val_acc, test_acc,
        train_f1, val_f1, test_f1,
        train_f2, val_f2, val_f2_std, test_f2,
        train_prec, val_prec, test_prec,
        train_rec, val_rec, test_rec,
        train_time
    ]

    #We return untrained pipeline and trained model
    #Untrained pipeline is useful if we want to retrain teh model on a different set of data
    #Eg we utilise it during threshold tuning.
    return (untrained_pipeline,model)
```

In [155…

```
#These are GLOBAL variables used in this code.

# Global variable that stores the models we created.  It keeps getting appeneded with more and more models
models=[]

# Global variable that stores results from various model evaluation
# We calculate F2 score too, as we want to prioritize recall over precision.
results = pd.DataFrame(columns=[
    'ModelID',
    'Model',
    'Hyperparameters',

    'TrainAccuracy',
    'ValAccuracy',
    'TestAccuracy',

    'TrainF1Score',
    'ValF1Score',
    'TestF1Score',
```

```
    'TrainF2Score',
    'ValF2Score',
    'ValF2ScoreStd',#To Check CV Stability for F2 Score
    'TestF2Score',

    'TrainPrecision',
    'ValPrecision',
    'TestPrecision',

    'TrainRecall',
    'ValRecall',
    'TestRecall',


    'TrainingTime'
])


# Global variable that stores table numbers.  It keeps getting incremented every time we draw an axes
tableNumber=1
```

Fitting Logistic

In [155…

```
#Create pipeline with a classifier
pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('clf', LogisticRegression(random_state=42))  # 4️⃣ Example model

  ])

trained_model = evaluate_model(len(models),"Logistic Untuned Imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

Fitting Random Forest

In [155…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', RandomForestClassifier(random_state=42))
])

trained_model = evaluate_model(len(models),
                               "RandomForest Untuned Imbalanced",
                                pipeline,
                                X_train, X_test, y_train, y_test
                            )
models.append(trained_model)
```

Fitting GBM

In [155…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', GradientBoostingClassifier(n_estimators=200, max_depth=2, loss = 'log_loss'))
```

```
])

trained_model = evaluate_model(len(models),
                               "GradientBoosting Untuned Imbalanced",
                                pipeline,
                                X_train, X_test, y_train, y_test
                              )
models.append(trained_model)
```

Fitting XGBoost

In [155…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', XGBClassifier(random_state=10))
])

trained_model = evaluate_model(len(models),
                               "XGBoost Untuned Imbalanced",
                                pipeline,
                                X_train, X_test, y_train, y_test
                              )
models.append(trained_model)
```

Fitting LGBM

In [155…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', LGBMClassifier(random_state=42))
])

trained_model = evaluate_model(len(models),
                               "LightGBM Untuned Imbalanced",
                                pipeline,
                                X_train, X_test, y_train, y_test
                              )
models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names
```

In [155…

```
results
```

Out[155…

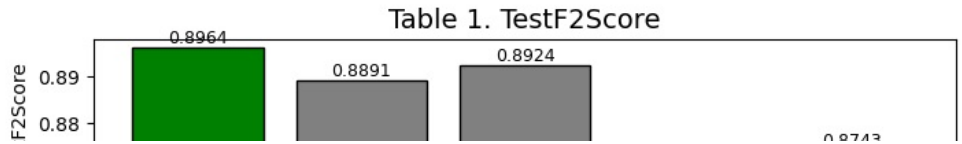
	ModelID	Model	Hyperparameters	TrainAccuracy	ValAccuracy	TestAccuracy	TrainF1Score	ValF1Score	TestF1Score	TrainF2Score	ValF2Score	ValF2ScoreStd	TestF2Score	TrainPr
0	0	Logistic Untuned Imbalanced	{'C': 1.0, 'class_weight': None, 'dual': False...	0.762080	0.762078	0.779874	0.842654	0.842659	0.851485	0.897853	0.897841	0.009638	0.896367	0
1	1	RandomForest Untuned Imbalanced	{'bootstrap': True, 'ccp_alpha': 0.0, 'class_w...	0.953783	0.953806	0.796646	0.966524	0.968116	0.858806	0.974479	0.974962	0.050075	0.889090	0
2	2	GradientBoosting Untuned Imbalanced	{'ccp_alpha': 0.0, 'criterion': 'friedman_mse'...	0.824580	0.824577	0.794549	0.879862	0.879947	0.858382	0.918720	0.918713	0.016521	0.892428	0
3	3	XGBoost Untuned Imbalanced	{'objective': 'binary:logistic', 'base_score':...	0.946430	0.946457	0.788260	0.960805	0.961342	0.849028	0.964341	0.964514	0.069436	0.865326	0
4	4	LightGBM Untuned Imbalanced	{'boosting_type': 'gbdt', 'class_weight': None...	0.930674	0.930696	0.792453	0.950026	0.950407	0.853333	0.962255	0.962377	0.057230	0.874317	0

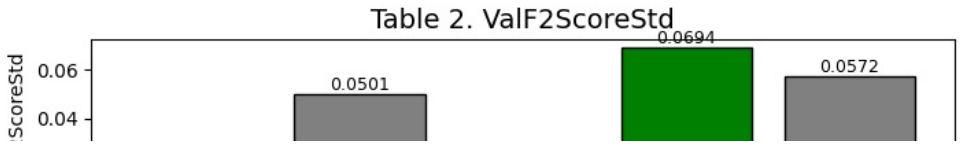
Discussion: What metrics are most important for us

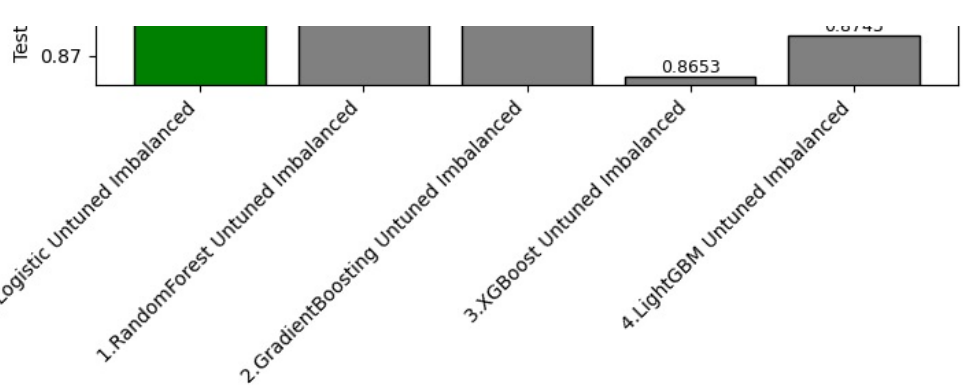
- In Employee Churn Use Case → RECALL Is More Important
  - **Why? **Because Missing a true churner (false negative) is costly:
    - loss of skilled employee
    - cost of hiring + training
    - loss of productivity
    - loss of domain knowledge
  - Therefore, Organizations want to identify as many at-risk employees as possible, even if it means some false alarms.
  - Even if some employees are flagged incorrectly (false positives), it's usually okay because:
    - the intervention cost is low (discussion, survey, manager 1-on-1)
    - the cost of losing an employee is very high
  - For F-score we will also calculate F2 Score apart from F1 score
    - As recall matters more → F2 score gives a better metric (weights recall 2×)

In [156…

```
metrics_to_plot = [
    'TestF2Score',
    'ValF2ScoreStd',
    'TestPrecision',
    'TestRecall',
    'TestAccuracy',
    'TrainingTime'
]

plot_metrics_grid(results, metrics_to_plot)
```







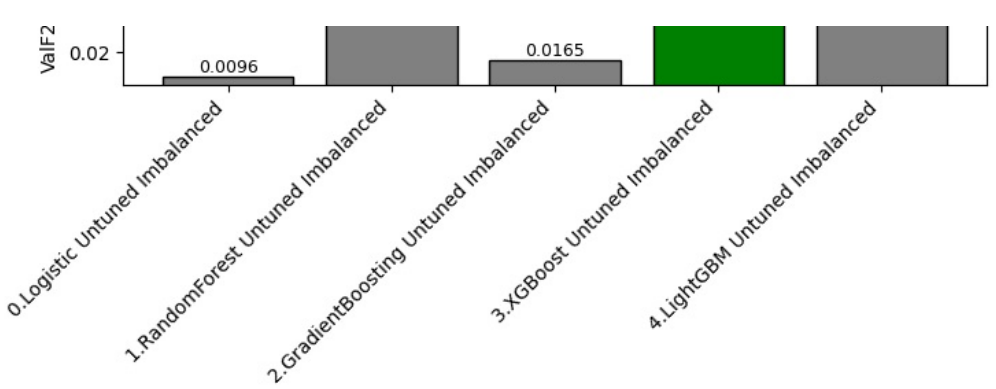


Table 3. TestPrecision

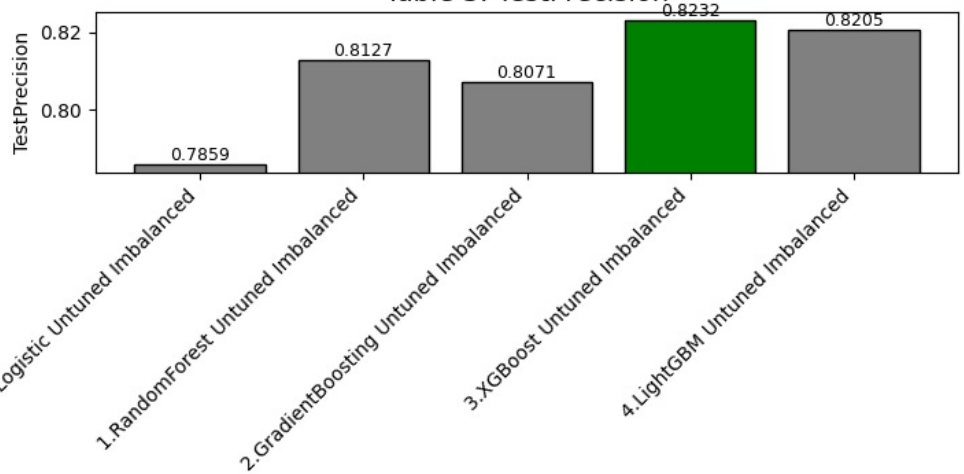


Table 4. TestRecall

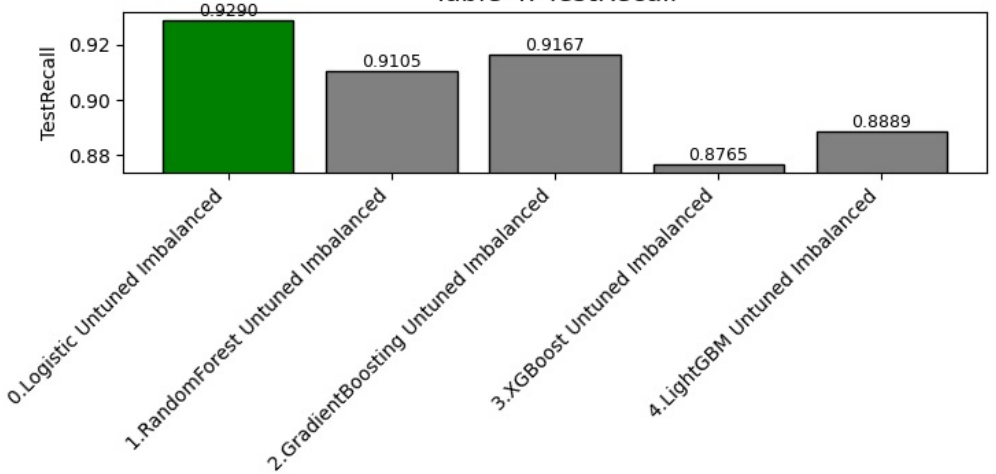


Table 5. TestAccuracy

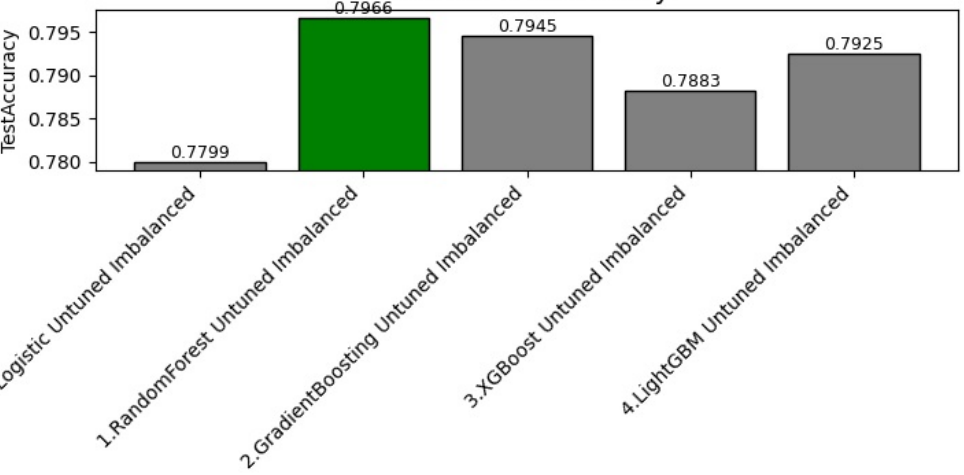
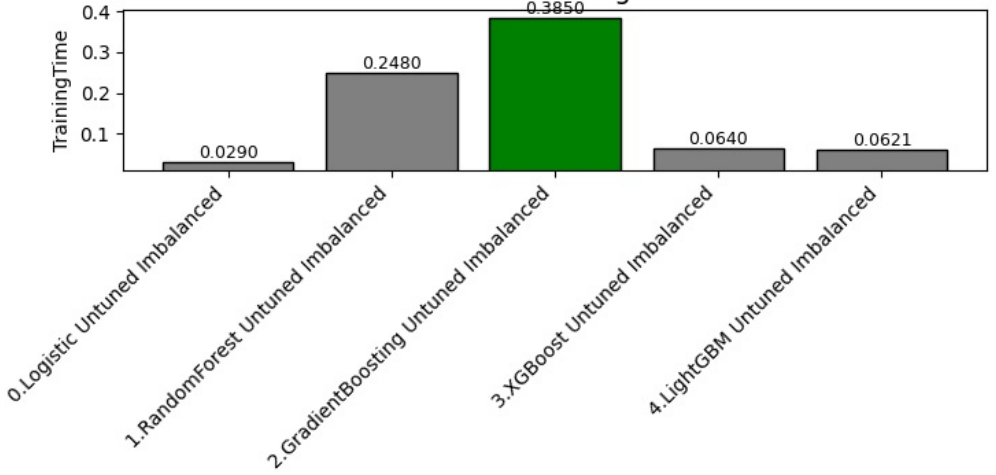


Table 6. TrainingTime



```
                              Model  TestF2Score
0          Logistic Untuned Imbalanced     0.896367
2  GradientBoosting Untuned Imbalanced     0.892428
1      RandomForest Untuned Imbalanced     0.889090

4          LightGBM Untuned Imbalanced     0.874317
3           XGBoost Untuned Imbalanced     0.865326
```

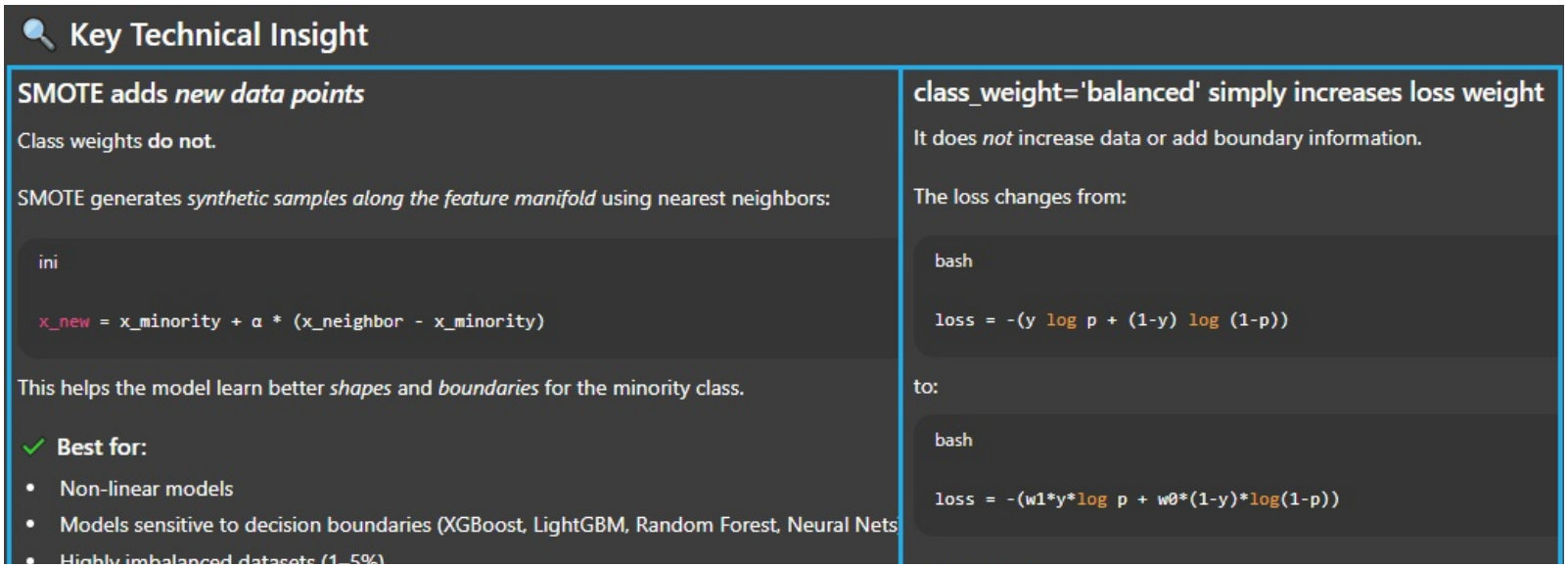
- Logistic Untuned Imbalanced has best F2 score (0.896), that prioritizes recall. It has best recall too of 0.929
- Random Forest Untuned Imbalanced has best F1 score (0.8588)

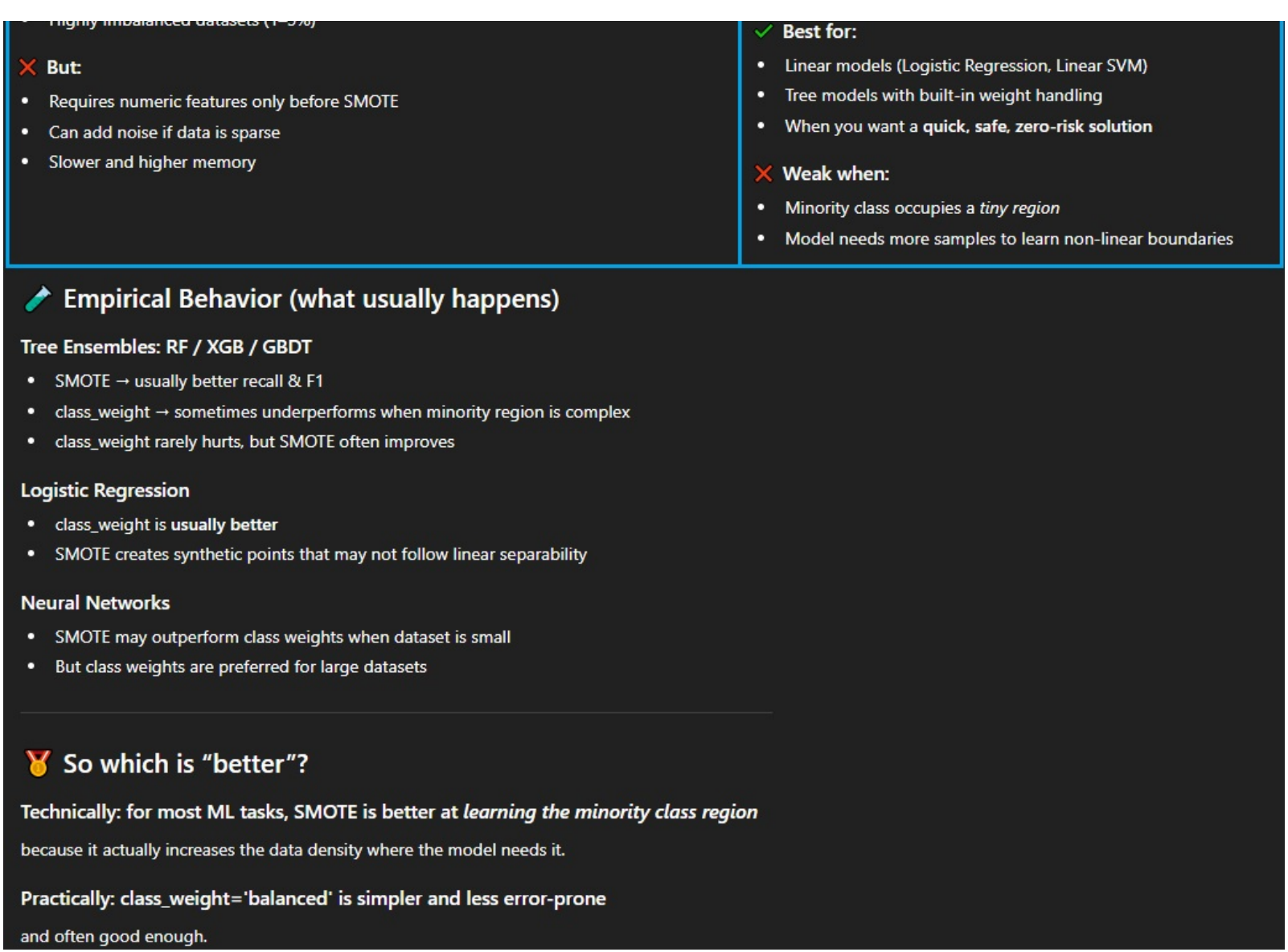
5.4 Lets handle class imbalance

- We already established we have imbalanced data. So lets fix it and see if it improves results.
- We can use **SMOTE** and **class_weight='balanced'** whereever supported.
  - ⭑ For Logistic Regression
  - → Use class_weight='balanced'
  - (logistic + SMOTE hurts due to over-smoothing)
  - ⭑ For Random Forest & XGBoost
  - → SMOTE performs better in most real-world cases
  - (because synthetic samples create better-defined split regions)

  ⭑ For GradientBoostingClassifier

  - → Does not support class weights
  - → SMOTE helps significantly





- SMOTE (Synthetic Minority Over-sampling Technique) : Generates synthetic minority samples instead of duplicates.
- ADASYN (Adaptive Synthetic Sampling) - Variant of SMOTE that focuses on hard-to-classify minority examples.

In [156…

```
from imblearn.over_sampling import SMOTE
from imblearn.over_sampling import ADASYN
#The standard Pipeline from sklearn can't handle resampling steps like SMOTE because resamplers change both X and y during fitting.
#sklearn.pipeline.Pipeline             Only expects transformers that modify X (not y).
#We need to use imbpipeline therefore. - Specifically designed for resampling steps that change both X and y.
from imblearn.pipeline import Pipeline
```

Logistic Untuned SMOTE

In [156…
```
pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('clf', LogisticRegression(random_state=42))

  ])

trained_model = evaluate_model(len(models),"Logistic Untuned SMOTE",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

RandomForest Untuned SMOTE

In [156…
```
pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('clf', RandomForestClassifier(random_state=42))

  ])

trained_model = evaluate_model(len(models),"RandomForest Untuned SMOTE",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

RandomForest Untuned ADASYN

In [156…
```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('adasyn', ADASYN(random_state=42)),
    ('clf', RandomForestClassifier(random_state=42))
])

trained_model = evaluate_model(len(models),
                               "RandomForest Untuned ADASYN",
                                pipeline,
                                X_train, X_test, y_train, y_test
                           )
models.append(trained_model)
```

LightGBM Untuned class_weight

In [156…
```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', LGBMClassifier(class_weight='balanced',random_state=42))
])

trained_model = evaluate_model(len(models),
                               "LightGBM Untuned class_weight",
                                pipeline,
                                X_train, X_test, y_train, y_test
                             )
models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names
```

LightGBM Untuned is_unbalance

In [156…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', LGBMClassifier(is_unbalance=True,random_state=42, verbose=0))
])

trained_model = evaluate_model(len(models),
                               "LightGBM Untuned is_unbalance",
                                pipeline,
                                X_train, X_test, y_train, y_test
```

```
                             )
models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names
```

XGBoost Untuned SMOTE

In [156…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('clf', XGBClassifier(random_state=42))
])
```

```
trained_model = evaluate_model(len(models),
                               "XGBoost Untuned SMOTE",
                                pipeline,
                                X_train, X_test, y_train, y_test
                           )
models.append(trained_model)
```

XGBoost Untuned scale_pos_weight

Class Imbalance Handling for XGBoost (XGBClassifier)

- We used SMOTE above
- Here we us scale_pos_weight parameter to see if it gives better result than SMOTE for XGBoost

In [156…

```
#  Best method: scale_pos_weight
# scale_pos_weight = (# negative samples) / (# positive samples)
# This tells XGBoost how important each minority sample is.
# ✔ Why this works best for XGB?
# XGBoost uses gradients and Hessians, so class_weight is not reliable.
# scale_pos_weight directly affects the boosting process.

# compute scale_pos_weight
neg, pos = np.bincount(y_train)
scale_pos_weight = neg / pos
print("scale_pos_weight =", scale_pos_weight)

modelXGB_balanced = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('clf', XGBClassifier(
        random_state=42,
        scale_pos_weight=scale_pos_weight,
        eval_metric='logloss'
    ))
])


######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('clf', XGBClassifier(
        random_state=42,
        scale_pos_weight=scale_pos_weight,
        eval_metric='logloss'
    ))
])

trained_model = evaluate_model(len(models),
                               "XGBoost Untuned scale_pos_weight",
                                pipeline,
                                X_train, X_test, y_train, y_test
                           )
models.append(trained_model)
```

```
scale_pos_weight = 0.47368421052631576
```

In [156…

```
plot_metrics_grid(results, metrics_to_plot)
```

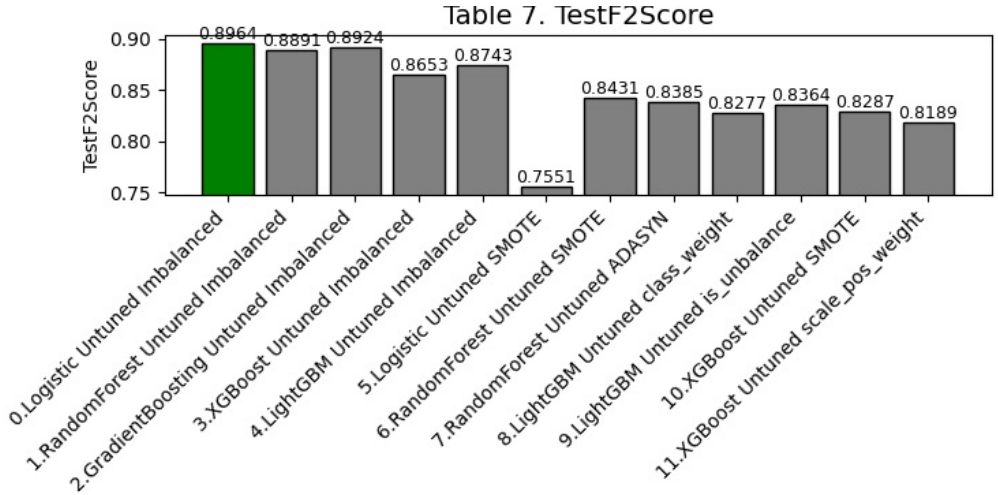


Table 7. TestF2Score
TestF2Score
0.8964
0.8891
0.8924
0.8653
0.8743
0.7551
0.8431
0.8385
0.8277
0.8364
0.8287
0.8189
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

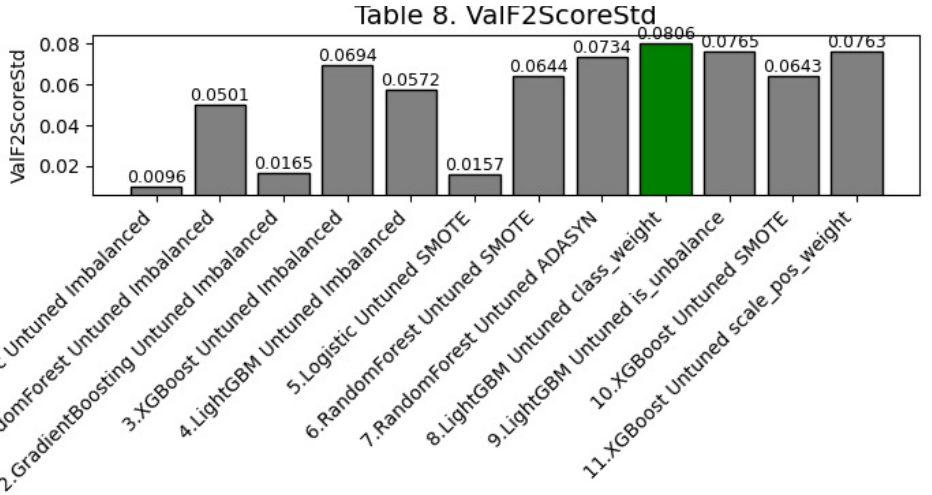


Table 8. ValF2ScoreStd
ValF2ScoreStd
0.0096
0.0501
0.0165
0.0694
0.0572
0.0157
0.0644
0.0734
0.0806
0.0765
0.0643
0.0763
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

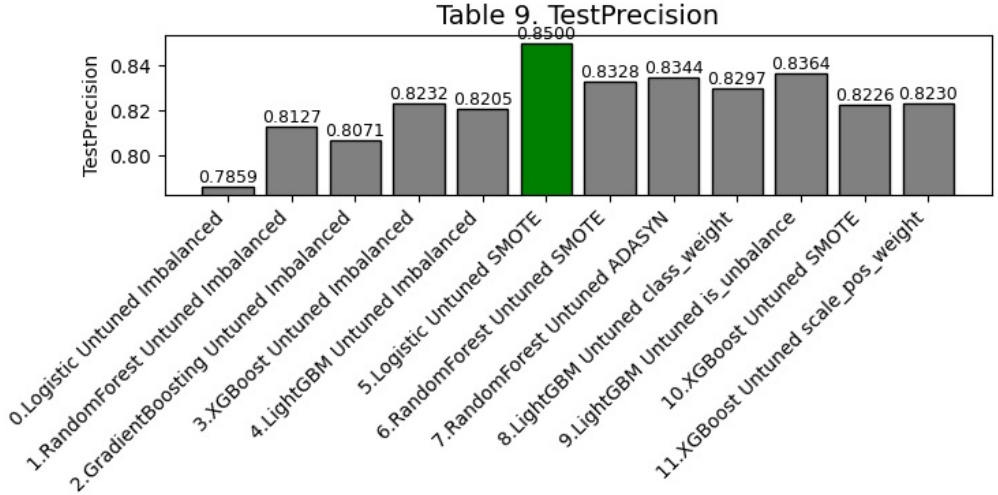


Table 9. TestPrecision
TestPrecision
0.7859
0.8127
0.8071
0.8232
0.8205
0.8500
0.8328
0.8344
0.8297
0.8364
0.8226
0.8230
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

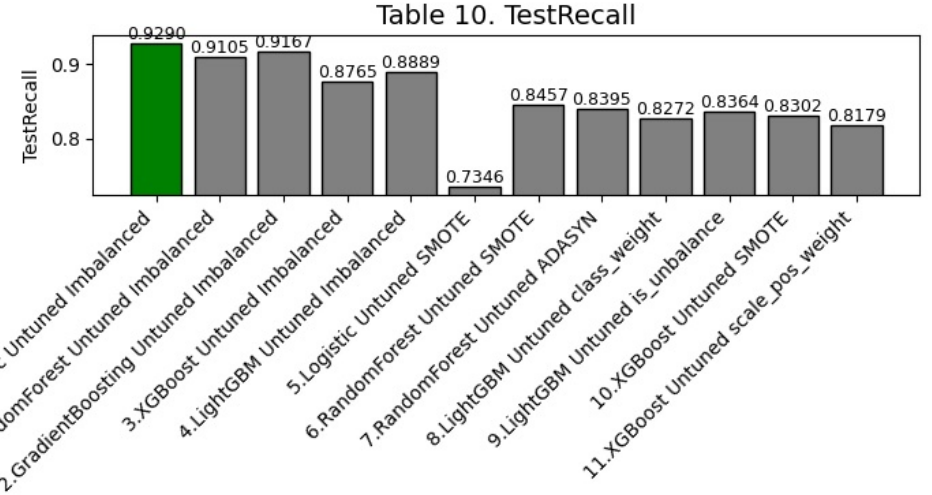


Table 10. TestRecall
TestRecall
0.9290
0.9105
0.9167
0.8765
0.8889
0.7346
0.8457
0.8395
0.8272
0.8364
0.8302
0.8179
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

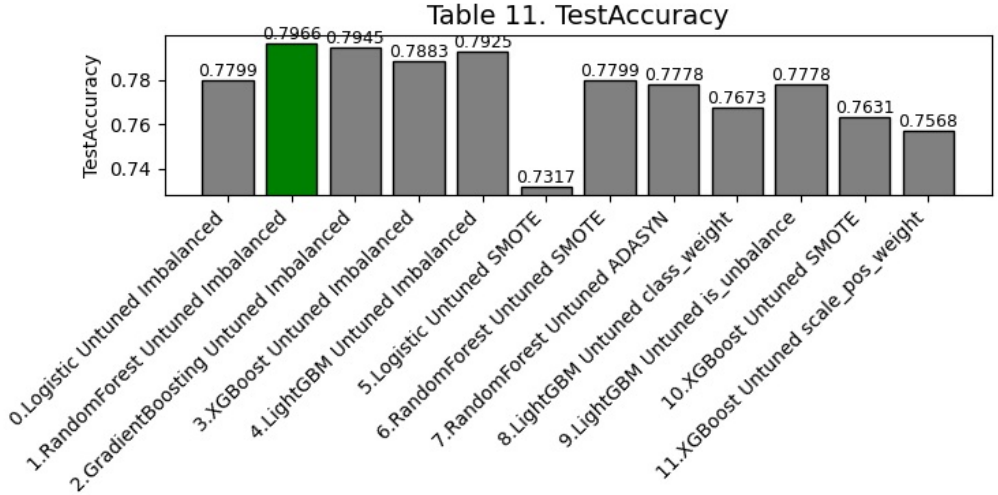


Table 11. TestAccuracy
TestAccuracy
0.7799
0.7966
0.7945
0.7883
0.7925
0.7317
0.7799
0.7778
0.7673
0.7778
0.7631
0.7568
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

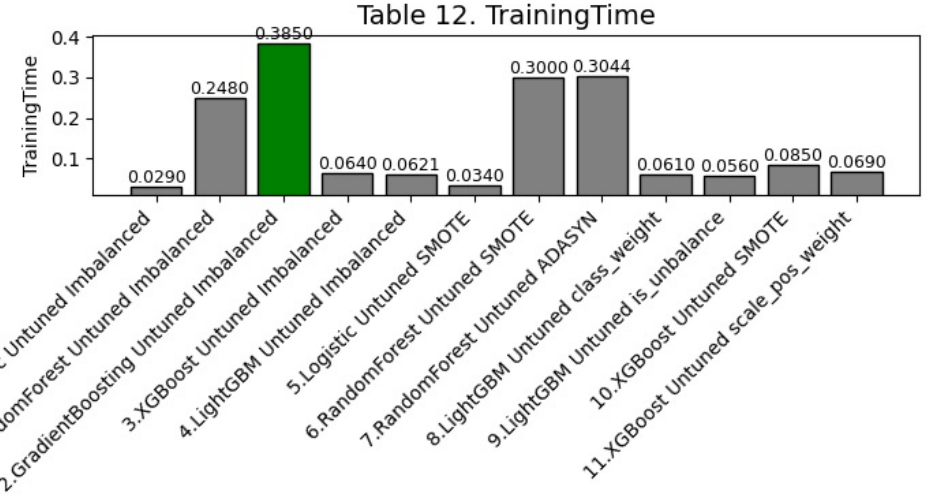


Table 12. TrainingTime
TrainingTime
0.0290
0.2480
0.3850
0.0640
0.0621
0.0340
0.3000
0.3044
0.0610
0.0560
0.0850
0.0690
0.Logistic Untuned Imbalanced
1.RandomForest Untuned Imbalanced
2.GradientBoosting Untuned Imbalanced
3.XGBoost Untuned Imbalanced
4.LightGBM Untuned Imbalanced
5.Logistic Untuned SMOTE
6.RandomForest Untuned SMOTE
7.RandomForest Untuned ADASYN
8.LightGBM Untuned class_weight
9.LightGBM Untuned is_unbalance
10.XGBoost Untuned SMOTE
11.XGBoost Untuned scale_pos_weight

	Model	TestF2Score
0	Logistic Untuned Imbalanced	0.896367
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
4	LightGBM Untuned Imbalanced	0.874317
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

- Strangely, fixing imbalance is reducing F1 and F2 score for ALL algorithms - This begets a question then :

Discussion : Is using Model trained on Imbalanced Data always bad?

- If the model trained on imbalanced data gives a significantly higher F2 score on the test set, then using the imbalanced model is the correct choice.
- **Why imbalanced training can sometimes beat balanced training**
  - Natural distribution contains signal. If minority-class examples are highly informative, oversampling can blur decision boundaries.
  - Synthetic samples (SMOTE/ADASYN) can introduce noise. They create interpolated points that may be unrealistic and reduce model generalization.
  - Some algorithms handle imbalance inherently. Tree-based models (RF, XGBoost, LightGBM) often split on strong signals and are less helped by naive oversampling.
  - Loss/threshold interactions. Balancing methods may shift model calibration; if you don't re-tune thresholds, evaluation will be unfair.
- ✅ The ONLY thing that matters: Test-set F2 score
  - Not intuition
  - Not guidelines
  - Not “balanced is better” myths
  - If the test F2 score is higher → that model is better for our churn use case. -We are ~6% better F2 from imbalanced models. **That’s very meaningful in churn prediction.**
- **Conclusion**: balancing is a tool — not an automatic win. Empirical evaluation on a held-out test set and stable CV is the ground truth.
- We need check these two things before finalizing
  - (a) Check stability using cross-validation
    - If the imbalanced model is stable across folds → good.
    - We collected standard deviation of validation F2 scores and it can be seen that Standard deviation of the models is low.
  - (b) Check prediction threshold
    - We used the default 0.5, a balanced model often needs a lower threshold (0.2–0.3) to show its real strength.
    - We will do this during hyper parameter tuning

5.5 Lets see if Removing collinear features can further improve results

Logistic VIF imbalanced

In [157…

```
pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
```

```
    ('clf', LogisticRegression(random_state=42))

  ])

trained_model = evaluate_model(len(models),"Logistic VIF imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

RandomForest VIF Imbalanced

In [157…

```
pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', RandomForestClassifier(random_state=42))

  ])

trained_model = evaluate_model(len(models),"RandomForest VIF Imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

GradientBoosting VIF Imbalanced

In [157…

```
######################
pipeline  = Pipeline([
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', GradientBoostingClassifier(n_estimators=200, max_depth=2, loss = 'log_loss'))
])

trained_model = evaluate_model(len(models),
                               "GradientBoosting VIF Imbalanced",
                                pipeline,
                                X_train, X_test, y_train, y_test
                           )
models.append(trained_model)
```

In [157…

```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[157…

	Model	TestF2Score
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

- Very interestingly, GradientBoosting VIF Imbalanced is the top scorer now (0.9011976047904192)
- So removing collinear features helped.
- Logistic Untuned Imbalanced and GradientBoosting Untuned Imbalanced are close contenders

5.6 Hyper parameter tuning

🔥 1. Logistic Regression — Key Hyperparameters

(Simple model, few but important parameters)

Parameter	Meaning	Why Tune?
`C`	Inverse regularization strength	Affects model flexibility & recall
`penalty`	`l2` (recommended)	Controls coefficient shrinkage
`solver`	`liblinear`, `lbfgs`, `saga`	Needed depending on penalty & size
`class_weight`	`balanced` or None	Helps with imbalance
`max_iter`	iterations	Prevents convergence warning

Typical Search Space

python Copy code

```python
param_grid_logreg = {
    'clf__C': [0.01, 0.1, 1, 10],
    'clf__penalty': ['l2'],
    'clf__class_weight': [None, 'balanced']
}
```

🌲 2. Random Forest — Key Hyperparameters

Parameter	Why Tune?
`n_estimators`	More trees = better performance
`max_depth`	Controls overfitting
`min_samples_split`	Affects model complexity
`min_samples_leaf`	Helps generalization
`max_features`	Randomness of splits
`class_weight`	Fix imbalance

Typical Search Space

python

```python
param_grid_rf = {
    'clf__n_estimators': [200, 400, 800],
    'clf__max_depth': [5, 10, 20, None],
    'clf__min_samples_split': [2, 5, 10],
    'clf__min_samples_leaf': [1, 2, 5],
    'clf__max_features': ['sqrt', 'log2'],
    'clf__class_weight': [None, 'balanced']
}
```

🌞 3. Gradient Boosting (sklearn GBM)

Parameter	Why Tune?
`n_estimators`	# boosting rounds
`learning_rate`	Tradeoff: low LR + more trees
`max_depth`	Tree depth
`subsample`	Stochastic boosting improves generalization
`min_samples_split`	Prevent overfitting
`min_samples_leaf`	Controls leaf size

Typical Search Space

python

```python
param_grid_gbm = {
    'clf__n_estimators': [200, 400, 800],
    'clf__learning_rate': [0.01, 0.05, 0.1],
    'clf__max_depth': [2, 3, 4],
    'clf__subsample': [0.8, 1.0],
    'clf__min_samples_leaf': [1, 2, 5]
}
```

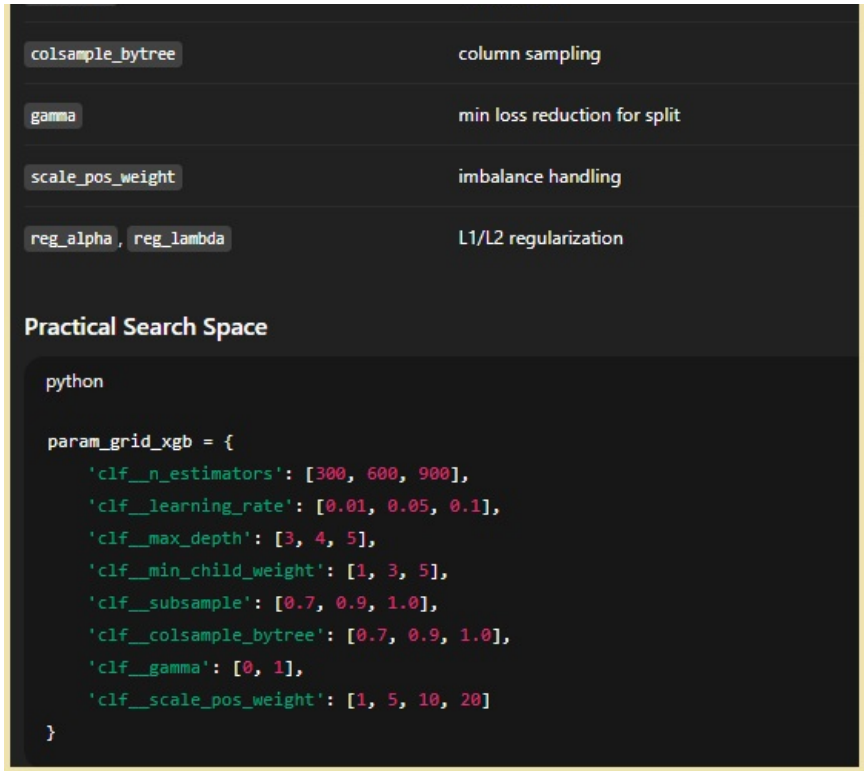
⚡ 4. XGBoost — Key Hyperparameters *(most important)*

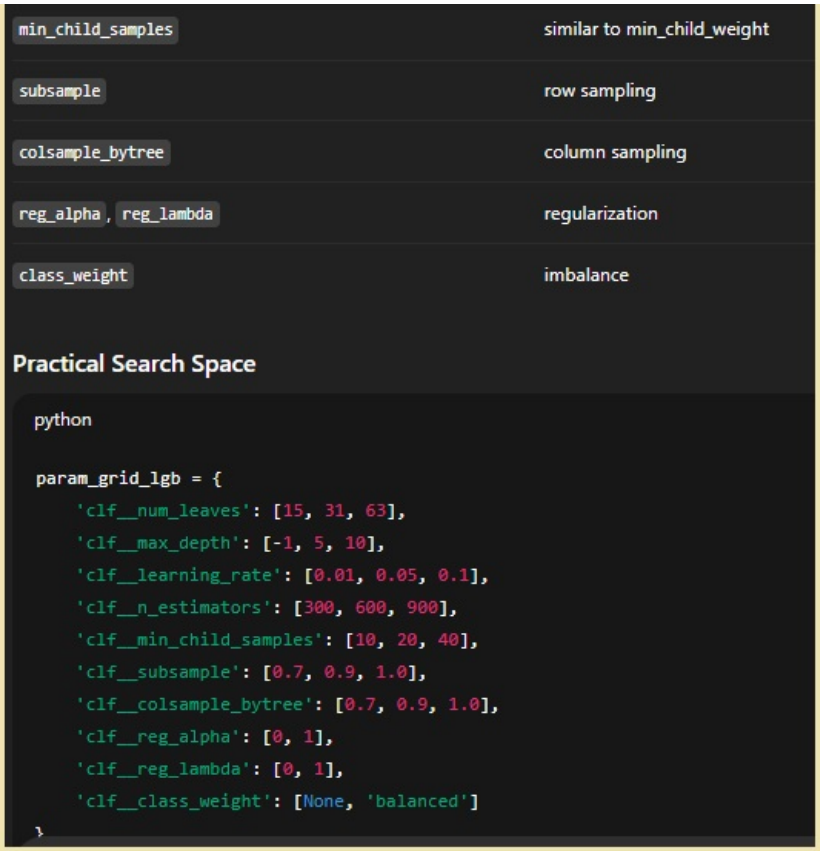
Parameter	Why Tune?
`n_estimators`	boosting rounds
`learning_rate` (eta)	small values improve F2 & recall
`max_depth`	tree depth
`min_child_weight`	minimum leaf weight (reduces overfitting)
`subsample`	row sampling

🌟 5. LightGBM — Key Hyperparameters

Best for speed + strong performance

Parameter	Why Tune?
`num_leaves`	Controls model complexity
`max_depth`	Restricts growth
`learning_rate`	Very important
`n_estimators`	boosting rounds





5.6.1 Create a Weighted-F2 scorer

- To evaluate models giving preference to F2 score instead of F1, we can create a custom scorer
- Weighted-F2 means β = 2 and average = 'weighted'.

In [157…
```
from sklearn.metrics import make_scorer

weighted_f2 = make_scorer(
    fbeta_score,
    beta=2,
    average='weighted'
)
```

5.6.2 Tuning Logistic Regression

In [157…
```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV

model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('clf', LogisticRegression(random_state=42))
])
```

```
#As our classifier is inside a pipeline, so the hyperparameters must be referenced using the pipeline step name
param_dist = {
    'clf__C': [0.001,0.002, 0.01],
    'clf__penalty': ['l2'],
    'clf__solver': ['lbfgs', 'liblinear'],
    'clf__class_weight': [None, 'balanced']
}

search = GridSearchCV(
    model,
    param_grid=param_dist,
    scoring=weighted_f2,
    cv=5,
    n_jobs=-1,
    verbose=2
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
{'clf__C': 0.01, 'clf__class_weight': None, 'clf__penalty': 'l2', 'clf__solver': 'lbfgs'}
0.7364465846408762
```

In [157…

```
# Create a new pipeline using the best params
tuned_model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', LogisticRegression(
        solver='lbfgs',
        penalty='l2',
        C=0.01,
        class_weight=None,
        random_state=42
    ))
])

trained_model = evaluate_model(len(models),"Logistic Tuned VIF Imbalanced",tuned_model, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

5.4.2.2 Doing a manual+visual search for C Parameter

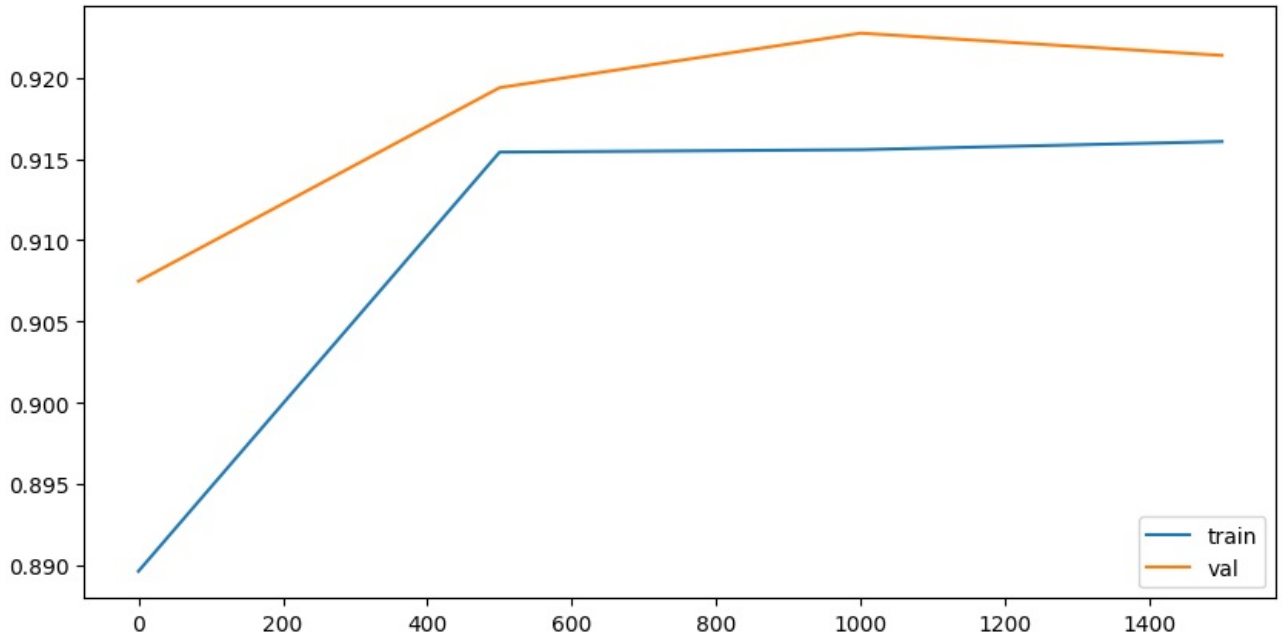
In [157…

```
train_scores = []
val_scores = []

#Quick note on stratify=y
# Stratified train-test splitting is a technique used to divide a dataset into training and testing sets while
# preserving the original proportion of classes or categories within the target variable. This is particularly
# crucial in classification tasks, especially when dealing with imbalanced datasets where some classes are
# significantly underrepresented compared to others.
X_train_cv, X_val, y_train_cv, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=42, stratify=y_train)

for la in np.arange(0.01, 2000.0, 500):
```

```
  print("Trying for Lambda value of ",la)
  scaled_lr = Pipeline(steps=[
      ('preprocess', preprocessor),
      ('drop_high_vif', ColumnDropper(
          drop_cols=columns_to_drop,
          all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
          return_df=False                    # keep as numpy for sklearn models
      )),
      ('classifier', LogisticRegression(C=1/la))
  ])

  scaled_lr.fit(X_train_cv, y_train_cv)

  train_score = fbeta_score(y_train_cv, scaled_lr.predict(X_train_cv), beta=2)
  val_score = fbeta_score(y_val, scaled_lr.predict(X_val), beta=2)

  train_scores.append(train_score)
  val_scores.append(val_score)

#Plot train and val scores
plt.figure(figsize=(10,5))

plt.plot(list(np.arange(0.01, 2000.0, 500)), train_scores, label="train")
plt.plot(list(np.arange(0.01, 2000.0, 500)), val_scores, label="val")
plt.legend(loc='lower right')
```

```
Trying for Lambda value of  0.01
Trying for Lambda value of  500.01
Trying for Lambda value of  1000.01
Trying for Lambda value of  1500.01
```

Out[157… <matplotlib.legend.Legend at 0x25a57efb680>



Lambda = 500 or C = 1/500 = 0.002 is best according to this.

In [157…
```
# Create a new pipeline using the C=1/500
tuned_lr_pipeline  = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', LogisticRegression(
        solver='lbfgs',
        penalty='l2',
        C=1/500,
        class_weight=None,
        random_state=42
    ))
])

trained_model = evaluate_model(len(models),"Logistic C=1/500 VIF Imbalanced",tuned_lr_pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

In [157…
```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[157…

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

- After tuning, Logistic Tuned Imbalanced has 1.5% more F2 score than GradientBoosting VIF Imbalanced

5.4.2.2 Lets try polynomial regression too

In [158…
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LogisticRegression

# Create a new pipeline using Polynomial features
pipeline  = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ('clf', LogisticRegression(
        random_state=42
    ))
])

trained_model = evaluate_model(len(models),"Logistic Polynomial VIF Imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\linear_model\_logistic.py:465: ConvergenceWarning:

lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\linear_model\_logistic.py:465: ConvergenceWarning:

lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
```

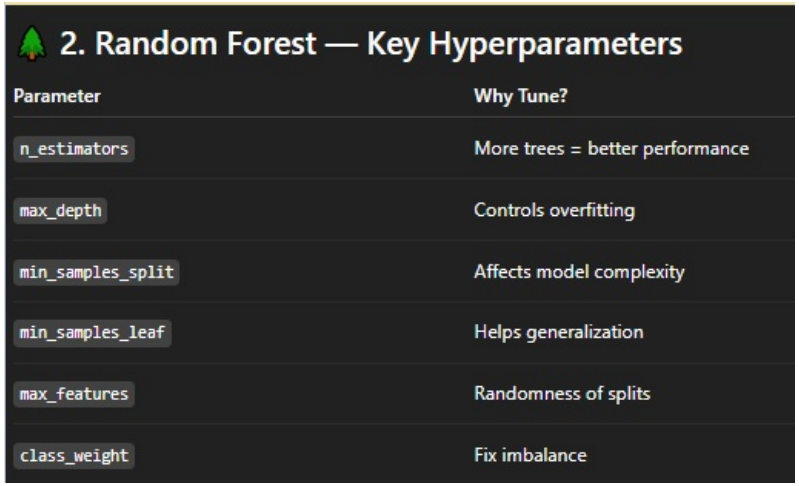
- Performance for Polynomial features with Logistic Regression is slightly less than that with normal linear features.

5.6.3 Tuning Random Forest

In [158…
```
rf_param_grid = {
    'clf__n_estimators': [200, 400, 600],
    'clf__max_depth': [5, 10, 15, None],
    'clf__min_samples_split': [2, 5, 10],
    'clf__min_samples_leaf': [1, 2, 4],
    'clf__class_weight': [None, 'balanced', 'balanced_subsample'],
    'clf__max_features': ['sqrt', 'log2']
}

model = Pipeline(steps=[
```

```
    ('preprocess', preprocessor),
    ('clf', RandomForestClassifier(random_state=42))
])

search = RandomizedSearchCV(
    model,
    param_distributions=rf_param_grid,
    n_iter=35,
    scoring=weighted_f2,
    cv=5,
    n_jobs=-1,
    verbose=2,
    random_state=42
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 35 candidates, totalling 175 fits
{'clf__n_estimators': 200, 'clf__min_samples_split': 10, 'clf__min_samples_leaf': 2, 'clf__max_features': 'sqrt', 'clf__max_depth': 10, 'clf__class_weight': None}
0.7759663611455806
```



In [158…

```
# Create a new pipeline using the best params
tuned_rf_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', RandomForestClassifier(
        n_estimators=200,
        min_samples_split=10,
        min_samples_leaf=2,
        max_features='sqrt',
        max_depth=10,
        class_weight=None,
        random_state=42
```

```
    ))
])

trained_model = evaluate_model(len(models),"RandomForest Tuned VIF Imbalanced",tuned_rf_pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

5.6.5 Tuning Gradient Boosting (GBM)

In [158…

```
#Why we dont add loss to param grid
#For binary/multi-class classification, 'log_loss' is almost always used because it works well
# with probabilistic metrics like F2-score, F1-score, AUC, etc.
#'exponential' is more like AdaBoost. For modern GBMs, 'log_loss' is almost always preferred for classification
gbm_param_grid = {
    'clf__n_estimators': [100, 200, 300],
    'clf__learning_rate': [0.05, 0.1, 0.2],
    'clf__max_depth': [2, 3, 4],
    'clf__subsample': [0.8, 1.0],
    'clf__min_samples_leaf': [1, 2, 4]
}
model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', GradientBoostingClassifier(loss = 'log_loss'))
])

search = RandomizedSearchCV(
    model,
    param_distributions=gbm_param_grid,
    n_iter=20,
    scoring=weighted_f2,
    cv=5,
    n_jobs=-1,
    verbose=2,
    random_state=42
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
{'clf__subsample': 0.8, 'clf__n_estimators': 100, 'clf__min_samples_leaf': 2, 'clf__max_depth': 3, 'clf__learning_rate': 0.05}
0.7762045782006759
```

🌞 3. Gradient Boosting (sklearn GBM)

Parameter	Why Tune?
`n_estimators`	# boosting rounds
`learning_rate`	Tradeoff: low LR + more trees
`max_depth`	Tree depth
`subsample`	Stochastic boosting improves generalization
`min_samples_split`	Prevent overfitting
`min_samples_leaf`	Controls leaf size

In [158…

```python
# Create pipeline with preprocessing and tuned GBM
tuned_gbm_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.05,
        max_depth=3,
        subsample=0.8,
        min_samples_leaf=2,
        loss='log_loss',  # keep log_loss for classification
        random_state=42
    ))
])

trained_model = evaluate_model(len(models),"GradientBoosting Tuned VIF Imbalanced",tuned_gbm_pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

In [158…

```python
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[158…

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

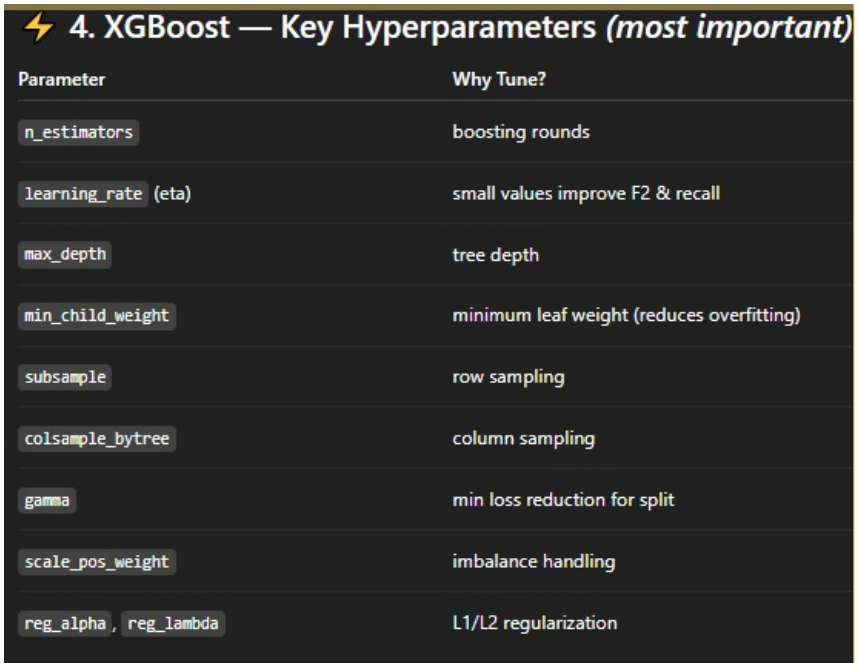
5.6.6 Tuning XGBoost

In [158…

```
xgb_param_grid = {
    'clf__n_estimators': [200, 400, 600],
    'clf__learning_rate': [0.01, 0.05, 0.1],
    'clf__max_depth': [3, 5, 7],
    'clf__subsample': [0.7, 0.9, 1.0],
    'clf__colsample_bytree': [0.6, 0.8, 1.0],
    'clf__gamma': [0, 1, 5],
    'clf__reg_lambda': [1, 5, 10],
    'clf__scale_pos_weight': [1, 5, 10, 20]   # IMPORTANT for imbalance
}

model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', XGBClassifier(random_state=42, eval_metric='logloss'))
```

```
])

search = RandomizedSearchCV(
    model,
    param_distributions=xgb_param_grid,
    n_iter=20,
    scoring=weighted_f2,
    cv=5,
    n_jobs=-1,
    verbose=2,
    random_state=42
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
{'clf__subsample': 0.9, 'clf__scale_pos_weight': 1, 'clf__reg_lambda': 1, 'clf__n_estimators': 400, 'clf__max_depth': 5, 'clf__learning_rate': 0.01, 'clf__gamma': 1, '
clf__colsample_bytree': 1.0}
0.7754007841083033
```



In [158…

```
# Create pipeline with preprocessing and tuned XGBoost
tuned_xgb_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', XGBClassifier(
        n_estimators=400,
```

```
        learning_rate=0.01,
        max_depth=5,
        subsample=0.9,
        colsample_bytree=1.0,
        gamma=1,
        reg_lambda=1,
        scale_pos_weight=1,
        use_label_encoder=False,  # recommended for modern XGBoost
        eval_metric='logloss',    # prevents warning with use_label_encoder=False
        random_state=42
    ))
])

trained_model = evaluate_model(len(models),"XGBoost Tuned VIF Imbalanced",tuned_xgb_pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\xgboost\training.py:199: UserWarning:

[20:45:29] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.


C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\xgboost\training.py:199: UserWarning:

[20:45:30] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

In [158…

```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[158…

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
20	XGBoost Tuned VIF Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

5.6.7 Tuning LightGBM

In [158…

```
lgbm_param_grid = {
    'clf__n_estimators': [200, 400, 600],
    'clf__learning_rate': [0.01, 0.05, 0.1],
    'clf__max_depth': [-1, 5, 10],
    'clf__num_leaves': [31, 63, 127],
    'clf__subsample': [0.7, 0.9, 1.0],
    'clf__colsample_bytree': [0.7, 0.9, 1.0],
    'clf__reg_lambda': [0.1, 1, 5],
    'clf__class_weight': [None, 'balanced'],
    'clf__scale_pos_weight': [1, 5, 10, 20]  # alternative imbalance control
}

model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
```

```
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', LGBMClassifier(random_state=42))
])

search = RandomizedSearchCV(
    model,
    param_distributions=lgbm_param_grid,
    n_iter=20,
    scoring=weighted_f2,
    cv=5,
    n_jobs=-1,
    verbose=0,
    random_state=42
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
{'clf__subsample': 0.9, 'clf__scale_pos_weight': 1, 'clf__reg_lambda': 0.1, 'clf__num_leaves': 31, 'clf__n_estimators': 400, 'clf__max_depth': 5, 'clf__learning_rate': 0.01, 'clf__colsample_bytree': 1.0, 'clf__class_weight': None}
0.7764012392710465
```

⭐ 5. LightGBM — Key Hyperparameters

Best for speed + strong performance

Parameter	Why Tune?
`num_leaves`	Controls model complexity
`max_depth`	Restricts growth
`learning_rate`	Very important
`n_estimators`	boosting rounds
`min_child_samples`	similar to min_child_weight
`subsample`	row sampling
`colsample_bytree`	column sampling
`reg_alpha`, `reg_lambda`	regularization
`class_weight`	imbalance

In [159…

```
# Create pipeline with preprocessing and tuned LightGBM
tuned_lgbm_pipeline = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ('clf', LGBMClassifier(
        n_estimators=400,
        learning_rate=0.01,
        max_depth=5,
        num_leaves=31,
```

```python
        subsample=0.9,
        colsample_bytree=1.0,
        reg_lambda=0.1,
        scale_pos_weight=1,
        class_weight=None,
        random_state=42,
        verbose=0
    ))
])

trained_model = evaluate_model(len(models),"LightGBM Tuned VIF Imbalanced",tuned_lgbm_pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names
```

Out[159…

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
21	LightGBM Tuned VIF Imbalanced	0.895657
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
1	RandomForest Untuned Imbalanced	0.889090
20	XGBoost Tuned VIF Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

5.6.8 Tuning Naive Bayes

In [159…

```
pipeline = Pipeline(steps=[
    ("preprocess", preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", GaussianNB())
])
trained_model = evaluate_model(len(models),"NaiveBayes Gaussian VIF Imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

In [159…

```
r = results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
r.loc[r.Model.str.contains('Naive')]
r.loc[r.Model.str.contains('SVM')]
```

Out[159…

	Model	TestF2Score

5.6.9 Tuning SVC Linear

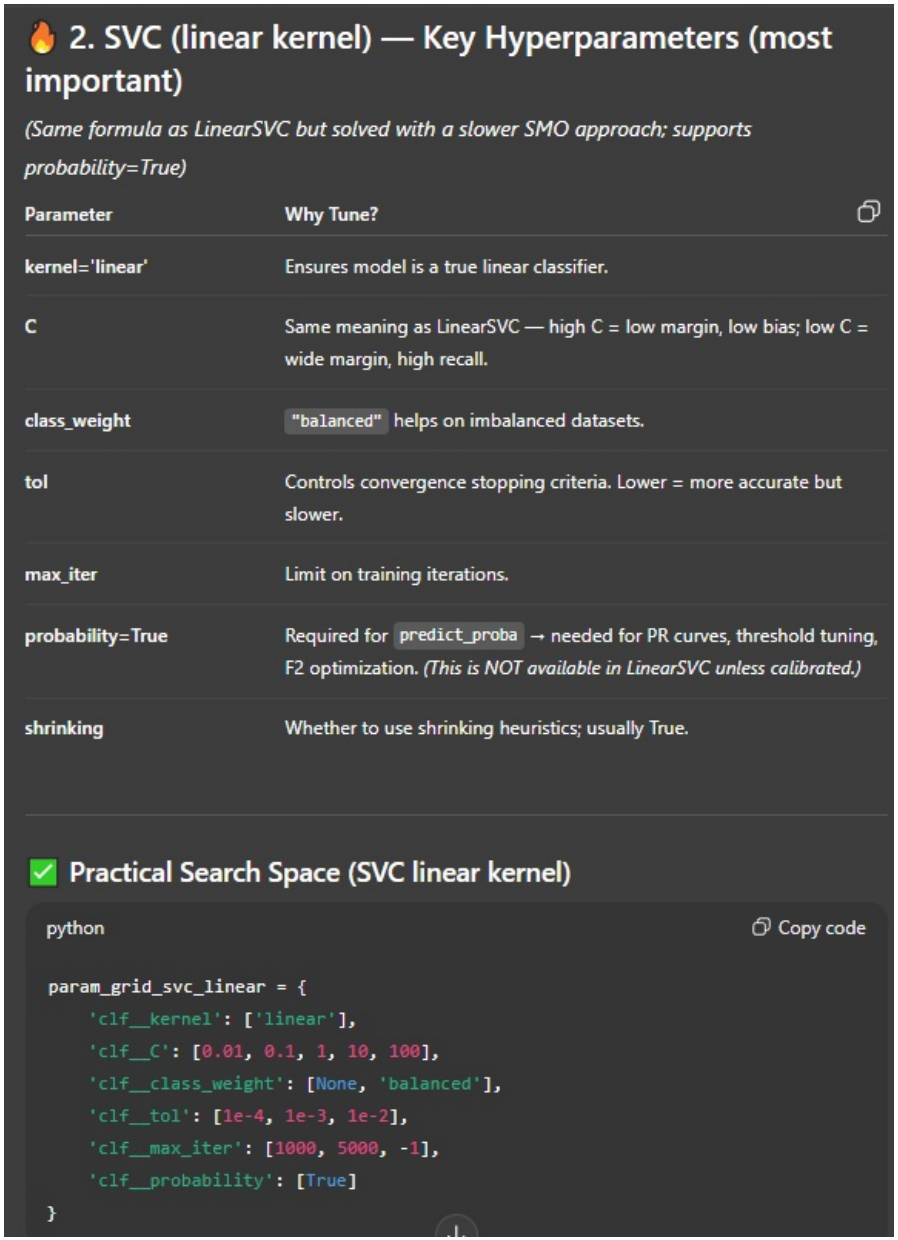
In [159…

```
from sklearn.svm import SVC

pipeline = Pipeline(steps=[
    ("preprocess", preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", SVC(kernel="linear", probability=True))
])
trained_model = evaluate_model(len(models),"SVCLinear Untuned VIF Imbalanced",pipeline, X_train, X_test, y_train, y_test)
models.append(trained_model)
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[159...

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
23	SVCLinear Untuned VIF Imbalanced	0.922444
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
21	LightGBM Tuned VIF Imbalanced	0.895657
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
20	XGBoost Tuned VIF Imbalanced	0.889090
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
22	NaiveBayes Gaussian VIF Imbalanced	0.877297
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
5	Logistic Untuned SMOTE	0.755076

- Results are promising. We are getting 0.922 even without any tuning ! which is in ball park of the best tuned model till now!
- Lets tune

- 

In [159…

```
model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", SVC())
])
```

```
param_dist = {
    "clf__kernel": ["linear"],
    "clf__C": [0.01, 0.1, 1, 10, 100, 500],
    "clf__class_weight": [None, "balanced"],
    "clf__tol": [1e-4, 1e-3, 1e-2],
    "clf__max_iter": [1000, 5000, -1],   # -1 = no limit
    "clf__probability": [True],          # keeps predict_proba
}


search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=20,
    scoring=weighted_f2,
    cv=5,
    verbose=1,
    n_jobs=-1,
    random_state=42
)

search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
{'clf__tol': 0.0001, 'clf__probability': True, 'clf__max_iter': 1000, 'clf__kernel': 'linear', 'clf__class_weight': 'balanced', 'clf__C': 1}
0.7421598779349828
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\svm\_base.py:305: ConvergenceWarning:

Solver terminated early (max_iter=1000).  Consider pre-processing your data with StandardScaler or MinMaxScaler.
```

In [159…

```
# Create a new pipeline using the best params
tuned_model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", SVC(
        kernel="linear",
        C=1,
        tol=0.0001,
        class_weight='balanced',
        max_iter=-1,
        probability=True   # required for predict_proba
    ))
])

trained_model = evaluate_model(len(models),"SVCLinear Tuned VIF Imbalanced",tuned_model, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

In [159…

```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[159…

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
23	SVCLinear Untuned VIF Imbalanced	0.922444
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
21	LightGBM Tuned VIF Imbalanced	0.895657
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
20	XGBoost Tuned VIF Imbalanced	0.889090
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
22	NaiveBayes Gaussian VIF Imbalanced	0.877297
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
24	SVCLinear Tuned VIF Imbalanced	0.772613
5	Logistic Untuned SMOTE	0.755076

5.6.10 Tuning SVC checking for all kernels

In [159…

```
model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ("clf", SVC(probability=True))
])

param_dist = {
    "clf__kernel": ["linear", "poly", "rbf"],
    "clf__C": [0.01, 0.1, 1, 10, 100],
    "clf__gamma": ["scale", "auto"],
    "clf__degree": [2, 3, 4],               # used only if kernel='poly'
    "clf__class_weight": [None, "balanced"],
```

```
}


search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=25,
    scoring=weighted_f2,
    cv=5,
    verbose=1,
    n_jobs=-1,
    random_state=42
)
search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 25 candidates, totalling 125 fits
{'clf__kernel': 'rbf', 'clf__gamma': 'auto', 'clf__degree': 2, 'clf__class_weight': None, 'clf__C': 10}
0.7669279956370307
```

In [159…

```
# Create a new pipeline using the best params
tuned_model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", SVC(
        kernel='rbf',
        gamma='auto',
        degree=2,
        class_weight=None,
        C=10,
        probability=True        # keeping this ON to get predict_proba
    ))
])

trained_model = evaluate_model(len(models),"SVC RBF Tuned VIF Imbalanced",tuned_model, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

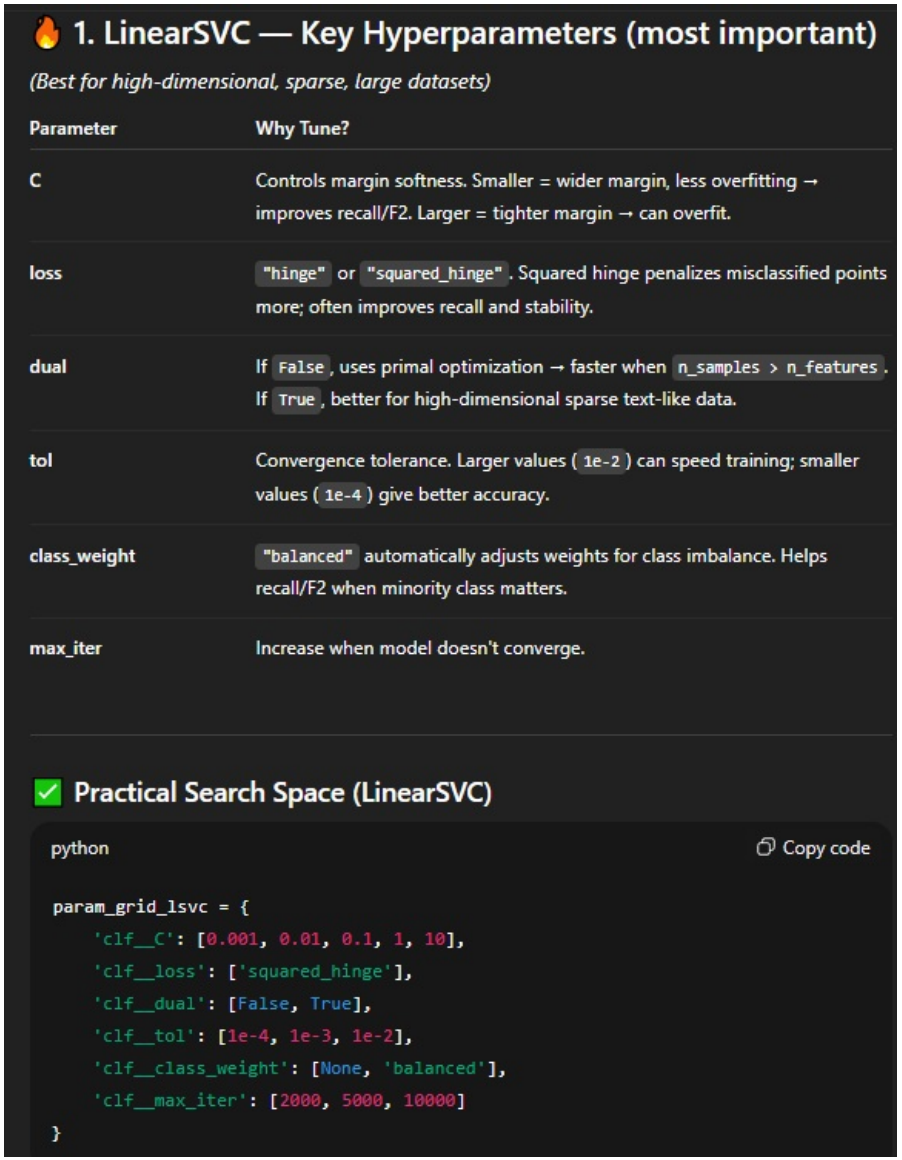
In [159…

```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[159...

	Model	TestF2Score
16	Logistic C=1/500 VIF Imbalanced	0.928405
23	SVCLinear Untuned VIF Imbalanced	0.922444
25	SVC RBF Tuned VIF Imbalanced	0.913947
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
0	Logistic Untuned Imbalanced	0.896367
21	LightGBM Tuned VIF Imbalanced	0.895657
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
20	XGBoost Tuned VIF Imbalanced	0.889090
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
22	NaiveBayes Gaussian VIF Imbalanced	0.877297
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
24	SVCLinear Tuned VIF Imbalanced	0.772613
5	Logistic Untuned SMOTE	0.755076

5.6.11 - Tuning Linear SVM



🔥 1. LinearSVC — Key Hyperparameters (most important)

(Best for high-dimensional, sparse, large datasets)

Parameter	Why Tune?
C	Controls margin softness. Smaller = wider margin, less overfitting → improves recall/F2. Larger = tighter margin → can overfit.
loss	`"hinge"` or `"squared_hinge"`. Squared hinge penalizes misclassified points more; often improves recall and stability.
dual	If `False`, uses primal optimization → faster when `n_samples > n_features`. If `True`, better for high-dimensional sparse text-like data.
tol	Convergence tolerance. Larger values (`1e-2`) can speed training; smaller values (`1e-4`) give better accuracy.
class_weight	`"balanced"` automatically adjusts weights for class imbalance. Helps recall/F2 when minority class matters.
max_iter	Increase when model doesn't converge.

✅ Practical Search Space (LinearSVC)

python — Copy code

```
param_grid_lsvc = {
    'clf__C': [0.001, 0.01, 0.1, 1, 10],
    'clf__loss': ['squared_hinge'],
    'clf__dual': [False, True],
    'clf__tol': [1e-4, 1e-3, 1e-2],
    'clf__class_weight': [None, 'balanced'],
    'clf__max_iter': [2000, 5000, 10000]
}
```

In [160…

```
from sklearn.svm import LinearSVC

model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ("clf", LinearSVC(dual=False))
])


param_dist = {
    "clf__C": [0.01, 0.1, 1, 10, 100],
    "clf__loss": ["hinge", "squared_hinge"],
    "clf__dual": [False],
```

```
    "clf__class_weight": [None, "balanced"],
    "clf__tol": [1e-4, 1e-3, 1e-2]
}

search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_dist,
    n_iter=20,
    scoring=weighted_f2,
    n_jobs=-1,
    cv=5,
    verbose=1,
    random_state=42
)
search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
{'clf__tol': 0.01, 'clf__loss': 'squared_hinge', 'clf__dual': False, 'clf__class_weight': None, 'clf__C': 0.01}
0.7388025106942926
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\model_selection\_validation.py:528: FitFailedWarning:


50 fits failed out of a total of 100.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
50 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\model_selection\_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\base.py", line 1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\pipeline.py", line 662, in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\base.py", line 1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\svm\_classes.py", line 321, in fit
    self.coef_, self.intercept_, n_iter_ = _fit_liblinear(
                                           ^^^^^^^^^^^^^^^
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\svm\_base.py", line 1228, in _fit_liblinear
    solver_type = _get_liblinear_solver_type(multi_class, penalty, loss, dual)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\svm\_base.py", line 1060, in _get_liblinear_solver_type
    raise ValueError(
ValueError: Unsupported set of arguments: The combination of penalty='l2' and loss='hinge' are not supported when dual=False, Parameters: penalty='l2', loss='hinge', d
ual=False


C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\sklearn\model_selection\_search.py:1108: UserWarning:

One or more of the test scores are non-finite: [       nan 0.73880251        nan 0.7273807         nan        nan
 0.72790806        nan        nan 0.7273807  0.7273807         nan

        nan 0.73691435 0.73647618 0.7375417         nan        nan
 0.73586198 0.73691435]
```

In [160…

```python
# Create a new pipeline using the best params
tuned_model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('drop_high_vif', ColumnDropper(
            drop_cols=columns_to_drop,
            all_feature_names=X_train_scaled.columns,  # known feature names from preprocessor
            return_df=False                    # keep as numpy for sklearn models
        )),
    ("clf", LinearSVC(
            tol=0.01,
            loss='squared_hinge',
            dual=False,
            class_weight=None,
            C=0.01,
            random_state=42        # optional but recommended
    ))
```

```
])

trained_model = evaluate_model(len(models),"LinearSVM Tuned VIF Imbalanced",tuned_model, X_train, X_test, y_train, y_test)
models.append(trained_model)
```

In [160…
```
print(results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False))
```

```
                                Model  TestF2Score
16       Logistic C=1/500 VIF Imbalanced     0.928405
23      SVCLinear Untuned VIF Imbalanced     0.922444
25          SVC RBF Tuned VIF Imbalanced     0.913947
18     RandomForest Tuned VIF Imbalanced     0.912365
19  GradientBoosting Tuned VIF Imbalanced     0.911271
15         Logistic Tuned VIF Imbalanced     0.906416
14       GradientBoosting VIF Imbalanced     0.901198
26        LinearSVM Tuned VIF Imbalanced     0.899941
0            Logistic Untuned Imbalanced     0.896367
21         LightGBM Tuned VIF Imbalanced     0.895657
17    Logistic Polynomial VIF Imbalanced     0.894674
2    GradientBoosting Untuned Imbalanced     0.892428
1        RandomForest Untuned Imbalanced     0.889090
20          XGBoost Tuned VIF Imbalanced     0.889090
12               Logistic VIF imbalanced     0.884110
22    NaiveBayes Gaussian VIF Imbalanced     0.877297
4            LightGBM Untuned Imbalanced     0.874317
13           RandomForest VIF Imbalanced     0.866788
3             XGBoost Untuned Imbalanced     0.865326
6             RandomForest Untuned SMOTE     0.843077
7            RandomForest Untuned ADASYN     0.838471
9          LightGBM Untuned is_unbalance     0.836420
10                XGBoost Untuned SMOTE     0.828712
8          LightGBM Untuned class_weight     0.827671
11      XGBoost Untuned scale_pos_weight     0.818912
24        SVCLinear Tuned VIF Imbalanced     0.772613
5                Logistic Untuned SMOTE     0.755076
```

In [160…
```
#Manually get the indexes of the best model for each algorithm.
# We will tune their thresholds to see if we can improve performance further.
top_models = {'logistic':{'model_id':16},
              'svclinear': {'model_id':23},
              'svcrbf':{'model_id':25},
              'rf':{'model_id':18},
              'gbm':{'model_id':14},
              'xgb':{'model_id':20},
}

#We will make use of models list that contains a tuple of (untrained pipeline, trained model)
# and results dataframe that has model names.
i=1
for model in top_models.values():
    print(f'{i}. {results.iloc[model['model_id']].Model} -> F2 Score: {results.iloc[model['model_id']].TestF2Score}')
    i+=1
```

```
1. Logistic C=1/500 VIF Imbalanced -> F2 Score: 0.9284051222351571
2. SVCLinear Untuned VIF Imbalanced -> F2 Score: 0.9224441833137486
3. SVC RBF Tuned VIF Imbalanced -> F2 Score: 0.913946587537092
4. RandomForest Tuned VIF Imbalanced -> F2 Score: 0.9123649459783914
5. GradientBoosting VIF Imbalanced -> F2 Score: 0.9011976047904192
6. XGBoost Tuned VIF Imbalanced -> F2 Score: 0.8890898131404461
```

5.7 - Tuning Thresholds

Till now, here are our top models (based on F2 score) for each model category

1. Logistic C=1/500 VIF Imbalanced -> F2 Score: 0.9284051222351571
2. SVCLinear Untuned VIF Imbalanced -> F2 Score: 0.9224441833137486
3. SVC RBF Tuned VIF Imbalanced -> F2 Score: 0.913946587537092
4. RandomForest Tuned VIF Imbalanced -> F2 Score: 0.9123649459783914
5. GradientBoosting VIF Imbalanced -> F2 Score: 0.9011976047904192
6. XGBoost Tuned VIF Imbalanced -> F2 Score: 0.8890898131404461

We will try to find optimal threshold

5.7.1 Find best thresholds for each of the top models

In [160…

```python
import numpy as np
from sklearn.metrics import fbeta_score

def tune_threshold_on_validation(model, X_val, y_val, beta=2):
    """
    Tune the optimal threshold using ONLY the validation set.
    Avoids leakage from the test set.
    """

    thresholds = np.arange(0.0, 1.001, 0.01)
    y_prob = model.predict_proba(X_val)[:, 1]

    best_thresh = 0.0
    best_fbeta = -1

    for t in thresholds:
        y_pred = (y_prob >= t).astype(int)
        fbeta = fbeta_score(y_val, y_pred, beta=beta)

        if fbeta > best_fbeta:
            best_fbeta = fbeta
            best_thresh = t

    return best_thresh, best_fbeta
```

In [160…

```python
#We cannot use trained models as they were created using full X_train data
#Thus for threshold tuning, we must use X_train_cv for training the models AGAIN,
# and then use X_val for tuning

#In models list, we have stored a tuple (untrained_pipeline, trained_model).
# So for threshold tuning we will utilize untrained_pipeline to train models just on
# the X_train_cv data and use X_val for threshold tuning


for key, model in top_models.items():
    model_id = model['model_id']
    untrained_pipeline = clone(models[model_id][0])
    untrained_pipeline.fit(X_train_cv, y_train_cv)
```

```
    best_thresh, best_f2_val = tune_threshold_on_validation(
        untrained_pipeline, X_val, y_val, beta=2
    )
    model_name = results.iloc[model_id].Model
    print(f'Best threshold {model_name}:{best_thresh}, gives F2 score as {best_f2_val}')

    #We store the best threshold in same dictionary so that we can use it later for creating Threshold based classifiers.
    top_models[key]['Threshold']= best_thresh
    #Add threshold as a string to model name
    top_models[key]['NewModelName'] = model_name+" Thresh "+str(best_thresh)
    #Add the trained model to the dictionary
    top_models[key]['TrainedModel'] = untrained_pipeline


print(top_models)
```

```
Best threshold Logistic C=1/500 VIF Imbalanced:0.48, gives F2 score as 0.9253903990746096
Best threshold SVCLinear Untuned VIF Imbalanced:0.44, gives F2 score as 0.9262485481997678
Best threshold SVC RBF Tuned VIF Imbalanced:0.44, gives F2 score as 0.9227594339622641
Best threshold RandomForest Tuned VIF Imbalanced:0.34, gives F2 score as 0.9316588785046729
Best threshold GradientBoosting VIF Imbalanced:0.35000000000000003, gives F2 score as 0.9272300469483568
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\xgboost\training.py:199: UserWarning:

[20:46:53] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.


Best threshold XGBoost Tuned VIF Imbalanced:0.3, gives F2 score as 0.9276546091015169
{'logistic': {'model_id': 16, 'Threshold': np.float64(0.48), 'NewModelName': 'Logistic C=1/500 VIF Imbalanced Thresh 0.48', 'TrainedModel': Pipeline(steps=[('preproces
s',
                 ColumnTransformer(transformers=[('cat',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent')),
                                                                  ('encoder',
                                                                   TargetEncoder())]),
                                                  ['City']),
                                                 ('ord',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent'))]),
                                                  ['Education_Level',
                                                   'Joining Designation',
                                                   'Grade', 'Quarterly Rating',
                                                   'IncomeIncreased',
                                                   'GradeIncrea...
                 ColumnDropper(all_feature_names=Index(['City', 'Education_Level', 'Joining Designation', 'Grade',
       'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased',
       'RatingTrendCode', 'Gender', 'Age', 'Total Business Value',
       'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'],
      dtype='object'),
                               drop_cols=['Quarterly Rating', 'Grade', 'City',
                                          'IncomeIncreased',
                                          'RatingTrendCode'])),
                ('clf', LogisticRegression(C=0.002, random_state=42))])}, 'svclinear': {'model_id': 23, 'Threshold': np.float64(0.44), 'NewModelName': 'SVCLinear Untun
ed VIF Imbalanced Thresh 0.44', 'TrainedModel': Pipeline(steps=[('preprocess',
                 ColumnTransformer(transformers=[('cat',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent')),
                                                                  ('encoder',
```

```
                                                  TargetEncoder())]),
                                                 ['City']),
                                                ('ord',
                                                 Pipeline(steps=[('imputer',
                                                                  SimpleImputer(strategy='most_frequent'))]),
                                                 ['Education_Level',
                                                  'Joining Designation',
                                                  'Grade', 'Quarterly Rating',
                                                  'IncomeIncreased',
                                                  'GradeIncrea...
                ColumnDropper(all_feature_names=Index(['City', 'Education_Level', 'Joining Designation', 'Grade',
       'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased',
       'RatingTrendCode', 'Gender', 'Age', 'Total Business Value',
       'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'],
      dtype='object'),
                              drop_cols=['Quarterly Rating', 'Grade', 'City',
                                         'IncomeIncreased',
                                         'RatingTrendCode'])),
               ('clf', SVC(kernel='linear', probability=True))])}, 'svcrbf': {'model_id': 25, 'Threshold': np.float64(0.44), 'NewModelName': 'SVC RBF Tuned VIF Imbala
nced Thresh 0.44', 'TrainedModel': Pipeline(steps=[('preprocess',
                ColumnTransformer(transformers=[('cat',
                                                 Pipeline(steps=[('imputer',
                                                                  SimpleImputer(strategy='most_frequent')),
                                                                 ('encoder',
                                                                  TargetEncoder())]),
                                                 ['City']),
                                                ('ord',
                                                 Pipeline(steps=[('imputer',
                                                                  SimpleImputer(strategy='most_frequent'))]),
                                                 ['Education_Level',
                                                  'Joining Designation',
                                                  'Grade', 'Quarterly Rating',
                                                  'IncomeIncreased',
                                                  'GradeIncrea...
                ColumnDropper(all_feature_names=Index(['City', 'Education_Level', 'Joining Designation', 'Grade',
       'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased',
       'RatingTrendCode', 'Gender', 'Age', 'Total Business Value',
       'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'],
      dtype='object'),
                              drop_cols=['Quarterly Rating', 'Grade', 'City',
                                         'IncomeIncreased',
                                         'RatingTrendCode'])),
               ('clf', SVC(C=10, degree=2, gamma='auto', probability=True))])}, 'rf': {'model_id': 18, 'Threshold': np.float64(0.34), 'NewModelName': 'RandomForest Tu
ned VIF Imbalanced Thresh 0.34', 'TrainedModel': Pipeline(steps=[('preprocess',
                ColumnTransformer(transformers=[('cat',
                                                 Pipeline(steps=[('imputer',
                                                                  SimpleImputer(strategy='most_frequent')),
                                                                 ('encoder',
                                                                  TargetEncoder())]),
                                                 ['City']),
                                                ('ord',
                                                 Pipeline(steps=[('imputer',
                                                                  SimpleImputer(strategy='most_frequent'))]),
                                                 ['Education_Level',
                                                  'Joining Designation',
                                                  'Grade', 'Quarterly Rating',
                                                  'IncomeIncreased',
                                                  'GradeIncrea...
```

```
       'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased',
       'RatingTrendCode', 'Gender', 'Age', 'Total Business Value',
       'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'],
      dtype='object'),
                            drop_cols=['Quarterly Rating', 'Grade', 'City',
                                       'IncomeIncreased',
                                       'RatingTrendCode'])),
                ('clf',
                 RandomForestClassifier(max_depth=10, min_samples_leaf=2,
                                        min_samples_split=10, n_estimators=200,
                                        random_state=42))])}, 'gbm': {'model_id': 14, 'Threshold': np.float64(0.35000000000000003), 'NewModelName': 'GradientBoosting V
IF Imbalanced Thresh 0.35000000000000003', 'TrainedModel': Pipeline(steps=[('preprocess',
                 ColumnTransformer(transformers=[('cat',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent')),
                                                                  ('encoder',
                                                                   TargetEncoder())]),
                                                  ['City']),
                                                 ('ord',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent'))]),
                                                  ['Education_Level',
                                                   'Joining Designation',
                                                   'Grade', 'Quarterly Rating',
                                                   'IncomeIncreased',
                                                   'GradeIncrea...
                 ColumnDropper(all_feature_names=Index(['City', 'Education_Level', 'Joining Designation', 'Grade',
       'Quarterly Rating', 'IncomeIncreased', 'GradeIncreased',
       'RatingTrendCode', 'Gender', 'Age', 'Total Business Value',
       'IncomeLast', 'IncomeCAGR', 'RatingMean', 'EmploymentAge'],
      dtype='object'),
                            drop_cols=['Quarterly Rating', 'Grade', 'City',
                                       'IncomeIncreased',
                                       'RatingTrendCode'])),
                ('clf',
                 GradientBoostingClassifier(max_depth=2, n_estimators=200))])}, 'xgb': {'model_id': 20, 'Threshold': np.float64(0.3), 'NewModelName': 'XGBoost Tuned VI
F Imbalanced Thresh 0.3', 'TrainedModel': Pipeline(steps=[('preprocess',
                 ColumnTransformer(transformers=[('cat',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent')),
                                                                  ('encoder',
                                                                   TargetEncoder())]),
                                                  ['City']),
                                                 ('ord',
                                                  Pipeline(steps=[('imputer',
                                                                   SimpleImputer(strategy='most_frequent'))]),
                                                  ['Education_Level',
                                                   'Joining Designation',
                                                   'Grade', 'Quarterly Rating',
                                                   'IncomeIncreased',
                                                   'GradeIncrea...
                            feature_types=None, feature_weights=None,
                            gamma=1, grow_policy=None, importance_type=None,
                            interaction_constraints=None, learning_rate=0.01,
                            max_bin=None, max_cat_threshold=None,
                            max_cat_to_onehot=None, max_delta_step=None,
                            max_depth=5, max_leaves=None,
                            min_child_weight=None, missing=nan,
```

```
                                monotone_constraints=None, multi_strategy=None,
                                n_estimators=400, n_jobs=None,
                                num_parallel_tree=None, ...))])}}
```

5.7.2 Creating Models based on best thresholds.

- To create a model that uses a custom threshold, we don't change the classifier itself.
- We can simply wrap the trained model + threshold into a small helper class that applies the threshold every time we call predict().

In [160…

```python
from sklearn.base import BaseEstimator, ClassifierMixin, clone

class ThresholdClassifier(BaseEstimator, ClassifierMixin):
    def __init__(self, model=None, threshold=0.5):
        self.model = model          # underlying model (pipeline)
        self.threshold = threshold  # decision threshold
        self.named_steps = model.named_steps

    def fit(self, X, y=None):
        # Clone the internal model so pipeline doesn't get overwritten
        self.model_ = clone(self.model)
        self.model_.fit(X, y)
        return self

    def predict_proba(self, X):
        return self.model_.predict_proba(X)

    def predict(self, X):
        y_prob = self.model_.predict_proba(X)[:, 1]
        return (y_prob >= self.threshold).astype(int)

    def get_params(self, deep=True):
        return {
            "model": self.model,
            "threshold": self.threshold,
        }

    def set_params(self, **params):
        for key, value in params.items():
            setattr(self, key, value)
        return self
```

In [160…

```python
# Wrap the model into ThreholdClassifer class

for key, model in top_models.items():
    final_model = ThresholdClassifier(model['TrainedModel'], threshold=model['Threshold'])
    trained_model = evaluate_model(len(models),model['NewModelName'],final_model,X_train, X_test, y_train, y_test)
    #We dont have any untrained pipeline (and neither do we care, so just put dummy string for the first value in the tuple)
    models.append(trained_model)
```

```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\xgboost\training.py:199: UserWarning:

[20:46:58] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.


C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\xgboost\training.py:199: UserWarning:

[20:46:58] WARNING: C:\actions-runner\_work\xgboost\xgboost\src\learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

In [160…

```
results[['Model','TestF2Score']].sort_values(by="TestF2Score", ascending=False)
```

Out[160…

	Model	TestF2Score
28	SVCLinear Untuned VIF Imbalanced Thresh 0.44	0.929487
16	Logistic C=1/500 VIF Imbalanced	0.928405
27	Logistic C=1/500 VIF Imbalanced Thresh 0.48	0.927865
32	XGBoost Tuned VIF Imbalanced Thresh 0.3	0.926143
30	RandomForest Tuned VIF Imbalanced Thresh 0.34	0.924839
23	SVCLinear Untuned VIF Imbalanced	0.922444
31	GradientBoosting VIF Imbalanced Thresh 0.35000...	0.921362
25	SVC RBF Tuned VIF Imbalanced	0.913947
18	RandomForest Tuned VIF Imbalanced	0.912365
19	GradientBoosting Tuned VIF Imbalanced	0.911271
29	SVC RBF Tuned VIF Imbalanced Thresh 0.44	0.908824
15	Logistic Tuned VIF Imbalanced	0.906416
14	GradientBoosting VIF Imbalanced	0.901198
26	LinearSVM Tuned VIF Imbalanced	0.899941
0	Logistic Untuned Imbalanced	0.896367
21	LightGBM Tuned VIF Imbalanced	0.895657
17	Logistic Polynomial VIF Imbalanced	0.894674
2	GradientBoosting Untuned Imbalanced	0.892428
20	XGBoost Tuned VIF Imbalanced	0.889090
1	RandomForest Untuned Imbalanced	0.889090
12	Logistic VIF imbalanced	0.884110
22	NaiveBayes Gaussian VIF Imbalanced	0.877297
4	LightGBM Untuned Imbalanced	0.874317
13	RandomForest VIF Imbalanced	0.866788
3	XGBoost Untuned Imbalanced	0.865326
6	RandomForest Untuned SMOTE	0.843077
7	RandomForest Untuned ADASYN	0.838471
9	LightGBM Untuned is_unbalance	0.836420
10	XGBoost Untuned SMOTE	0.828712
8	LightGBM Untuned class_weight	0.827671
11	XGBoost Untuned scale_pos_weight	0.818912
24	SVCLinear Tuned VIF Imbalanced	0.772613
5	Logistic Untuned SMOTE	0.755076

- Threshold tuning improved all models. GBM improved most spectacularly.
- Our winners have changed.

In [162…
```
#Manually get the indexes of the best model for each algorithm.
# We will tune their thresholds to see if we can improve performance further.
top_models = {
    'svclinear': {'model_id':28},
    'logistic':{'model_id':27},
    'xgb':{'model_id':32},
    'rf':{'model_id':30},
    'gbm':{'model_id':31},
    'svcrbf':{'model_id':25},

}

i=1
for model in top_models.values():
    print(f'{i}. {results.iloc[model['model_id']].Model} -> F2 Score: {results.iloc[model['model_id']].TestF2Score}')
    i+=1
```

1. SVCLinear Untuned VIF Imbalanced Thresh 0.44 -> F2 Score: 0.9294871794871795
2. Logistic C=1/500 VIF Imbalanced Thresh 0.48 -> F2 Score: 0.9278650378126818
3. XGBoost Tuned VIF Imbalanced Thresh 0.3 -> F2 Score: 0.9261430246189918
4. RandomForest Tuned VIF Imbalanced Thresh 0.34 -> F2 Score: 0.9248385202583675
5. GradientBoosting VIF Imbalanced Thresh 0.35000000000000003 -> F2 Score: 0.9213615023474179
6. SVC RBF Tuned VIF Imbalanced -> F2 Score: 0.913946587537092

5.8 Comparison of Model Results

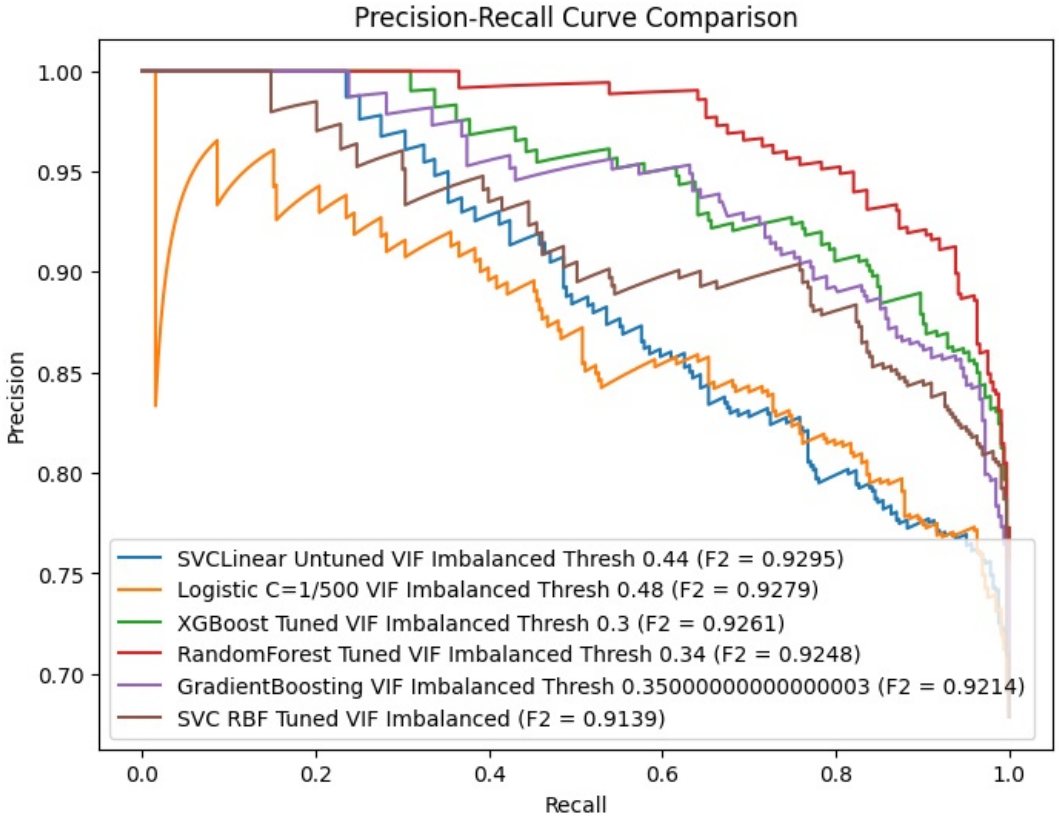
5.8.1 Choose the candidates

	Model	# TestF2Score
21	Logistic Tuned VIF Imbalanced Threshold0.48	0.9288194444444444
16	Logistic C=1/500 VIF Imbalanced	0.9284051222351571
23	GradientBoosting Tuned VIF Imbalanced Threshold0.24	0.927536231884058
22	RandomForest Tuned VIF Imbalanced Threshold0.34	0.9239766081871345
25	XGBoost Tuned VIF Imbalanced Threshold0.3	0.9237536656891495
24	LightGBM Tuned VIF Imbalanced Threshold0.28	0.9199766355140186

5.8.2 Precision Recall Curve

In [162…
```
from sklearn.metrics import precision_recall_curve

plt.figure(figsize=(8,6))


for model in top_models.values():
    model_id = model['model_id']
    model_name = results.iloc[model_id].Model
    f2Score = results.iloc[model_id].TestF2Score

    y_probs = models[model_id][1].predict_proba(X_val)[:, 1]
```

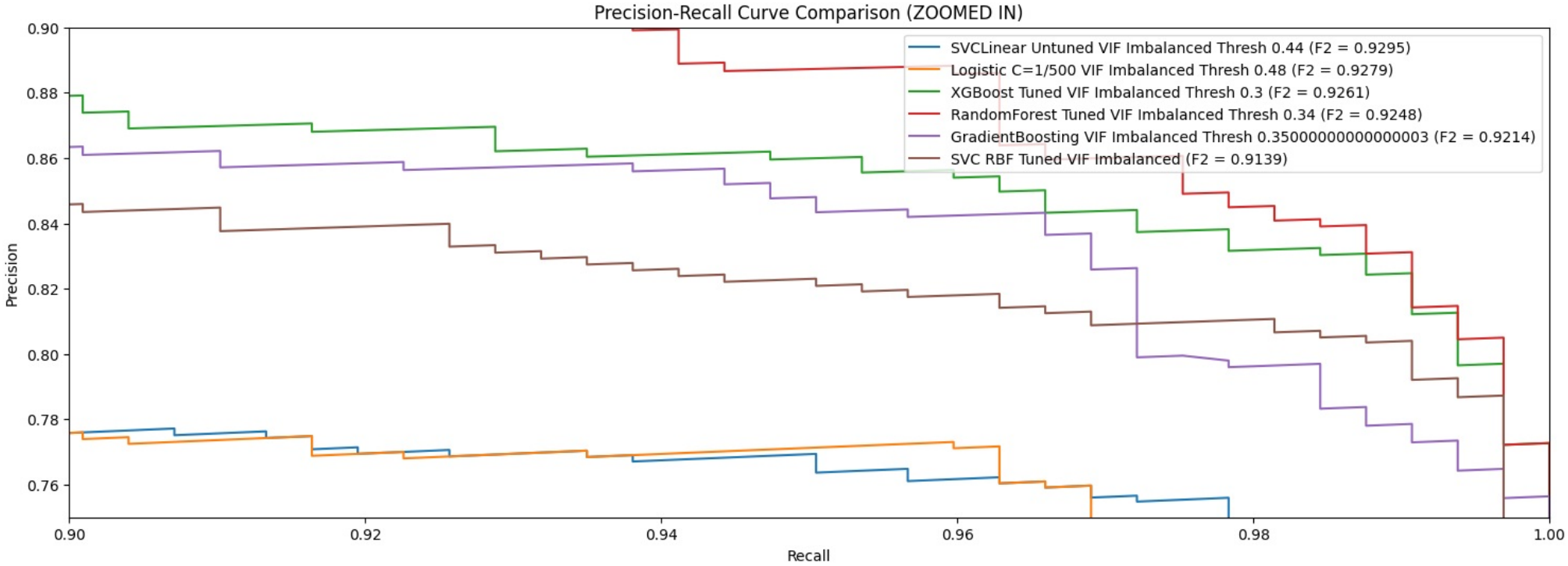
```
    precision, recall, _ = precision_recall_curve(y_val, y_probs)
    plt.plot(recall, precision, label=f'{model_name} (F2 = {f2Score:.4f})')



plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison')
plt.legend(loc='lower left')
plt.show()
```



In [162…

```
from sklearn.metrics import precision_recall_curve

plt.figure(figsize=(18,6))


for model in top_models.values():
    model_id = model['model_id']
    model_name = results.iloc[model_id].Model
    f2Score = results.iloc[model_id].TestF2Score

    y_probs = models[model_id][1].predict_proba(X_val)[:, 1]
    precision, recall, _ = precision_recall_curve(y_val, y_probs)
    plt.plot(recall, precision, label=f'{model_name} (F2 = {f2Score:.4f})')
```

```
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison (ZOOMED IN)')
plt.legend(loc='upper right')

#  Zoom-in on recall axis
plt.xlim(0.90, 1)
plt.ylim(0.75, 0.9)
plt.show()
```



- SVCLinear Untuned VIF Imbalanced Thresh 0.44 is our top model
- So we select the next model, GradientBoosting VIF Imbalanced Thresh 0.35000000000000003, whose curve is much better and F2 score is just 0.001 less than Logistic Model.
- The PR curve shows RandomForest looks “better” visually,. but our F2 Score shows SVCLinear, Logistic and XGBoost to be top 3
- This can happen because a single metric (like F2) summarizes the precision-recall tradeoff at one selected threshold, while the PR curve shows performance across all thresholds.

  - **F2 score emphasizes recall:**
  - It was calculated for one chosen decision threshold (in our chart, those thresholds are 0.48, 0.35, etc.). We evaluating how each model performs at that specific cutoff.
  - **Precision-Recall (PR) curve:**
  - It depicts the behavior across all thresholds. A model might have a smoother, more favorable curve overall, meaning it offers better flexibility or potential after threshold tuning even if at default threshold, its F2 score is lower.
  - **So, RandomForest might produce more favorable tradeoffs across various thresholds, but at particular threshold choices we had for SVCLinear, Logistic and XGBoost, they have slightly better F2 score.**

5.8.3 Choosing the best model

Since the SVC Linear, Logistic Regression, and tree-based models (XGBoost, Random Forest) have very similar F_2 scores, the decision should rest on model assumptions, interpretability, and robustness rather than the tiny performance difference.

5.8.3.1 Testing for assumptions for SVCLinear Model

Area	Assumption	How to Test / What to Check
Linearity of relationship between features and decision boundary	The classes should be **linearly separable** (or nearly so).	Plot pairwise feature relationships (scatter plots, PCA) to see if classes can be roughly separated by a straight line (or hyperplane in higher dimensions). If not, polynomial or RBF kernel might be better.
No strong multicollinearity	Highly correlated features can distort the orientation of the hyperplane.	Check Variance Inflation Factor (VIF) or correlation matrix before fitting. If VIF > 5–10 typically indicates multicollinearity.
Feature scaling	SVMs (including Linear) are *sensitive to scale*, since the optimization depends on distances.	Apply `StandardScaler` or `MinMaxScaler` before training.
Balanced influence between classes	Though SVM can use class weights, imbalance can bias toward the majority class.	Examine class distribution, and if imbalanced, use `class_weight='balanced'` or resample the data.
Limited outliers	Outliers can severely distort the decision boundary in SVMs.	Use boxplots or isolation forest / robust scaling to identify and dampen outliers' influence.
Independent examples	Observations should be independent.	Check for duplicates, repeated IDs, or time-series dependence.

Linearity assumption not supported by PCA plots

- Our PCA (2D & 3D) visualizations clearly show nonlinear mixing between churn and non-churn classes — clusters overlap and curve into each other.
- This indicates the decision boundary is not linear, meaning a linear hyperplane (as used by Linear SVC) cannot separate the data effectively.
- As a result, Linear SVC is likely oversimplifying the true relationship, reducing its generalization potential even if the F_2 scores appear momentarily good. ✓ Reject SVC Linear because: The class separability structure is nonlinear — visually and likely functionally — violating SVC Linear's core assumption.

5.8.3.2 Testing for assumptions for Logistic Regression

Logistic Regression — Key Assumptions and Checks

Area	Assumption	How to Test / What to Check
Linearity of relationship between features and log-odds	Predictors should have a **linear relationship with the log-odds** of the outcome.	Plot partial dependence or logit plots for numeric variables; visual nonlinearity (curves or plateaus) violates this. Add polynomial or interaction terms if linearity fails.
No strong multicollinearity	Independent variables shouldn't be highly correlated.	Check Variance Inflation Factor (VIF) or correlation matrix; VIF > 5–10 shows multicollinearity. Remove or combine correlated features.
Feature scaling	Logistic coefficients are more stable when predictors are scaled.	Apply `StandardScaler` to handle large magnitude differences (especially when using regularization).
Balanced influence between classes	Severe class imbalance can bias the model toward the majority class.	Check target distribution; if imbalanced, use `class_weight='balanced'` or perform resampling (SMOTE, undersampling).
Limited outliers	Extreme values can drastically affect coefficient estimates.	Use boxplots, z-scores, or robust scaling; remove or winsorize extreme outliers.
Independent examples	Observations should be independent.	Check for duplicates, time-dependence, or grouped data.
No perfect separation	Perfectly separable classes cause infinite coefficients.	If logistic fails to converge, check if a subset of variables perfectly predicts the outcome. Reduce features or use regularization.

Linearity Assumption Violated (Same as Linear SVC)

- Logistic Regression assumes a linear relationship between the predictors and the log-odds of the target.
- PCA plots show the classes are nonlinearly intertwined, meaning no linear boundary can cleanly separate them.
- Because of this, the model can only draw a "flat plane" (hyperplane) between classes — which cuts through the mixed region instead of wrapping around the patterns.
- Reject Reason: The data's structure clearly violates the core linearity assumption that Logistic Regression rests on. Performance parity with tree models occurs by coincidence, not because of proper fit.

Polynomial Features Did Not Help → Indicates Intrinsic Nonlinearity

- Expanding Logistic Regression with polynomial features up to a certain degree (say 2 or 3) should have improved discrimination if the interactions were polynomially related.
- We found no F_2 gain despite polynomial terms, it suggests that:
  - The true boundary is highly nonlinear (not captured even by moderate polynomial terms).
  - The logistic optimizer is plateauing — it's already at the expressive limit of linear + low-degree polynomial space.
- Reject Reason: Failure of polynomial feature engineering to improve Logistic Regression confirms that the underlying decision surface is structurally nonlinear (e.g., complex, piecewise, or tree-like patterns).

Poor Fit with Class Imbalance and Complex Interactions

- Logistic Regression tends to degrade in imbalanced data scenarios — small variations in thresholds or sampling can severely shift recall and precision.
- It also cannot naturally handle feature interactions (e.g., "tenure × department" or "salary growth × engagement") unless explicitly engineered.

- Given that Gradient Boosting and XGBoost perform similarly without manual feature interactions, that signals the linear model isn't capturing subtler churn patterns — the tree-based ones are.
- Reject Reason: Logistic Regression's inability to implicitly learn interactions limits its adaptability to real-world churn data, which usually involves interaction-driven attrition behavior.

Predictive Performance Plateau + Non-Interpretability Tradeoff

- Yes, Logistic gives great interpretability through coefficients, but:
- That interpretability is only meaningful if the functional form (log-odds linearity) holds true.
- In our case, since the feature–outcome relationship is nonlinear, the coefficients are not reliable explanations of churn risk magnitude — possibly misleading business readers.
- Reject Reason: When assumptions break, the interpretability advantage of Logistic Regression no longer holds — "interpretable nonsense" is worse than a complex but accurate model.

5.8.3.3 Testing for assumptions for XGBoost

XGBoost — Key Assumptions and Checks

Area	Assumption	How to Test / What to Check
Form of relationships	No assumption of linearity — handles **nonlinear and interaction effects** automatically.	Visualize with **SHAP** or **Partial Dependence Plots (PDPs)** to confirm relationships make domain sense.
Feature scaling	XGBoost is **not sensitive to feature scale**.	Scaling optional — often unnecessary.
Collinearity tolerance	Can handle correlated features, though redundant predictors may slightly dilute feature importance.	Check correlation matrix; consider removing highly correlated features for cleaner interpretability.
Handling imbalance	Can handle imbalance using built-in `scale_pos_weight` or sample weighting.	Compute class ratio (neg/pos) and set `scale_pos_weight` accordingly. Evaluate PR curve to ensure minority class recall.
Outlier robustness	Tree-based models are inherently **robust to outliers**.	Minimal checks needed; extreme values have limited effect on splits.
Independent samples	Still assumes independence between rows / examples.	Check for duplicates or autocorrelation if modeling temporal data.
No missing-value bias	XGBoost can natively handle missing values using split default directions.	Ensure missingness is not informative or biased; if so, model missingness explicitly (e.g., missingness indicator).

1. We removed highly correlated features using VIF RFE
2. We already tested fixing class imbalance
3. No duplicates
4. Samples are per driver, so independent
5. There are not much assumptions related to distribution, linearity

5.8.3.4 Testing for assumptions for Random Forest

Random Forest — Key Assumptions and Checks

Area	Assumption	How to Test / What to Check
Form of relationships	No assumption of linearity; captures complex nonlinear boundaries and interactions automatically.	Use **feature importance, Permutation Importance,** or **SHAP plots** to inspect learned relationships.
Feature scaling	Random Forests are **scale-insensitive** (distance metrics not used).	No scaling required.
Collinearity tolerance	Can handle correlated variables, though may reduce the clarity of feature importances.	Review correlation matrix to remove redundant variables if interpretability matters.
Handling imbalance	May still bias toward majority class depending on bootstrap samples.	Use `class_weight='balanced'` or adjust sample weights in training. Validate via recall/precision on minority class.
Outlier robustness	Generally robust; decision trees limit the influence of extreme points.	Optional: visualize with boxplots; clip egregious outliers if causing instability.
Independent samples	Observations should be independent across rows.	Check duplicates or temporal correlations.
Stability and variance	High variance possible with small sample size or limited trees.	Use sufficient number of estimators (e.g., 200+ trees). Review OOB (Out-of-Bag) error for variance estimates.

- Like XGBoost, Random Forest doesn't have strict statistical assumptions (no need for linearity, scaling, or independence between features)

5.8.3.5 Final Choice

We choose XGBoost as it has no strict assumptions, as is performing almost similar to the Linear models.

In [163…
```
final_models = {
    'xgb':{'model_id':32},
    # 'rf':{'model_id':30},
}
```

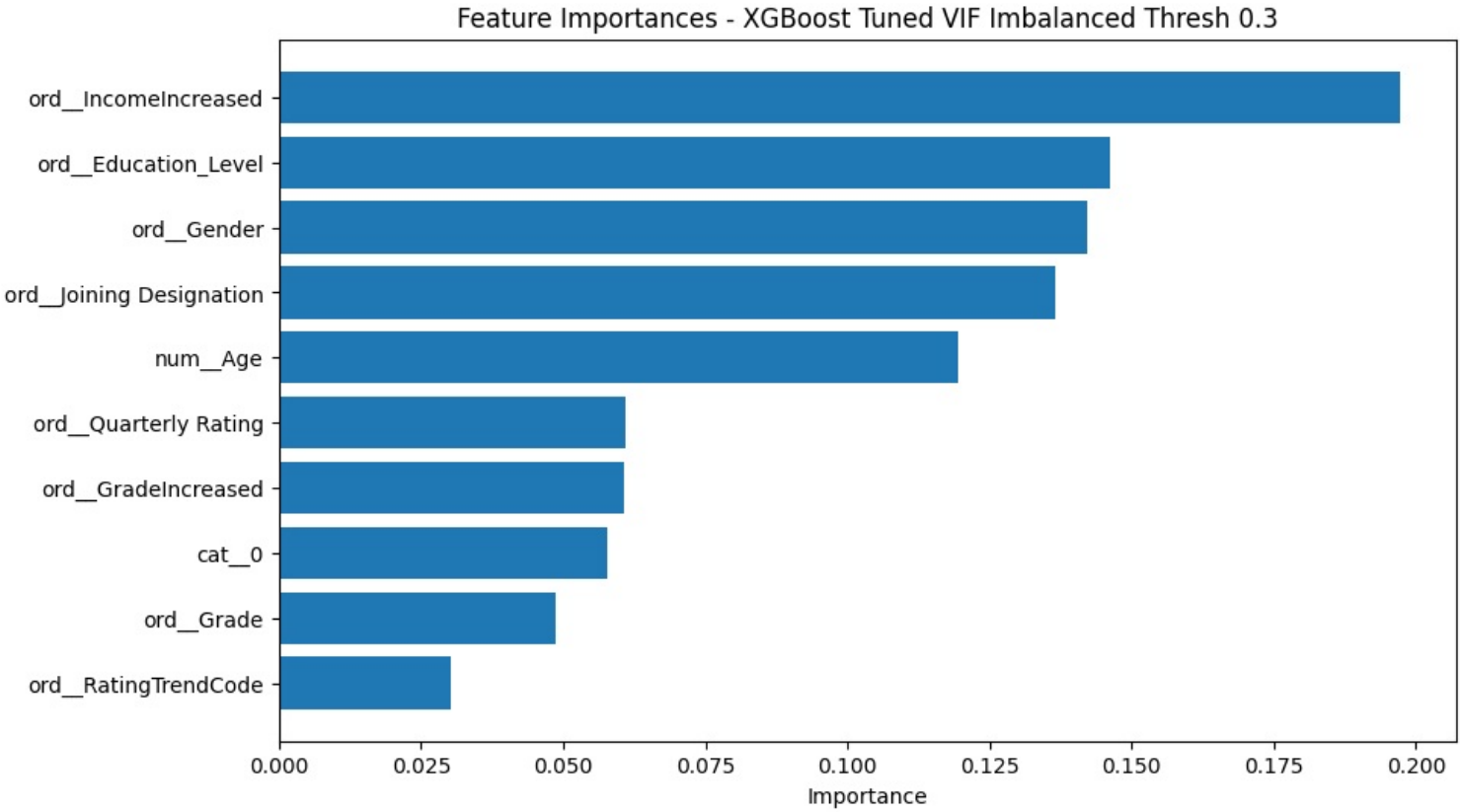
Plot feature importances for all models

- What this does:
  - Automatically detects if it's a tree-based or linear model.
  - Extracts feature names from ColumnTransformer if present.
  - Sorts features by importance.
  - Plots a horizontal bar graph for clarity.

In [163…
```
def get_importances(model, model_name):
    """
    Returns a DataFrame with columns:
```

```
    [feature, importance, model]
    """
    clf = model.named_steps['clf']
    # 1. Tree-based models
    if hasattr(clf, "feature_importances_"):
        imp = clf.feature_importances_

    # 2. Linear models with coef_
    elif hasattr(clf, "coef_"):
        imp = np.abs(clf.coef_.ravel())   # take absolute values

    # 3. SVC(kernel='rbf') → no importances
    else:
        return pd.DataFrame()   # skip models that don't support


    # Get feature names from ColumnTransformer
    try:
        preprocessor = model.named_steps['preprocess']
        feature_names = preprocessor.get_feature_names_out()
    except:
        # fallback if get_feature_names_out not available
        feature_names = [f'feature_{i}' for i in range(len(imp))]

    # Sort features by importance
    indices = np.argsort(imp)[::-1]
    sorted_features = [feature_names[i] for i in indices]
    sorted_importances = imp[indices]

    df = pd.DataFrame({
        "feature": sorted_features,
        "importance": sorted_importances,
        "model": model_name
    })
    return df


def plot_feature_importances(model, model_name):
    """
    Plots feature importances or coefficients for a pipeline model,
    using actual feature names from ColumnTransformer if available.
    """
    try:

        df_imp = get_importances(model, model_name)


        # Plot horizontal bar chart
        plt.figure(figsize=(10,6))
        plt.barh(df_imp.feature, df_imp.importance)
        plt.xlabel('Importance')
        plt.title(f'Feature Importances - {model_name}')
        plt.gca().invert_yaxis()  # highest importance at top
        plt.show()
        return df_imp

    except Exception as e:
        print(f"Error plotting {model_name}: {e}")
```

```
dfs = []
for model in final_models.values():
    model_id = model['model_id']
    model_name = results.iloc[model_id].Model
    #The function plot_feature_importances plots the features and
    # returs a dataframe
    #We shall use this dataframe for further analysis.
    dfs.append(plot_feature_importances(models[model_id][1],model_name))

#Convert the list dfs to a dataframe
all_feat_df = pd.concat(dfs, ignore_index=True)
```



- In all models, IncomeIncreased got topmost importance

5.8.3 Feature × Model Matrix to compare features

We use pivot table

In [163…

```
feat_matrix = all_feat_df.pivot_table(
    index="feature",
    columns="model",
```

```
    values="importance",
    fill_value=0
)

feat_matrix
```

Out[163…

model	XGBoost Tuned VIF Imbalanced Thresh 0.3
feature	
cat__0	0.057662
num__Age	0.119562
ord__Education_Level	0.146113
ord__Gender	0.142247
ord__Grade	0.048792
ord__GradeIncreased	0.060829
ord__IncomeIncreased	0.197153
ord__Joining Designation	0.136584
ord__Quarterly Rating	0.060900
ord__RatingTrendCode	0.030158

In [164…

```
rank_df = feat_matrix.rank(ascending=False)
rank_df.sort_values(by='XGBoost Tuned VIF Imbalanced Thresh 0.3', inplace=True)
rank_df
```

Out[164…

model	XGBoost Tuned VIF Imbalanced Thresh 0.3
feature	
ord__IncomeIncreased	1.0
ord__Education_Level	2.0
ord__Gender	3.0
ord__Joining Designation	4.0
num__Age	5.0
ord__Quarterly Rating	6.0
ord__GradeIncreased	7.0
cat__0	8.0
ord__Grade	9.0
ord__RatingTrendCode	10.0

5.9 Final Model analysis

5.9.1 Problem Summary

We built a predictive model to estimate driver attrition risk. Reducing false negatives (drivers who leave but are predicted to stay) was the main business priority.

5.9.2 Data Summary

- 19104 rows, 14 features.
- 2381 drivers
- Original data was grouped based on drivers into 2381 rows
- Ordinal encoding for ordered categories, OneHot for nominal features, Target encoding for City column.
- VIF score was used to drop highly collinear columns
- Data was imbalanced
- Train/validation/test: 70/15/15 split with stratification.

5.9.3 Model Selection Process

- We chose XGBoost based classifier after following steps.
  - ✔ Trained multiple models (LogReg, SVC, RandomForest, XGB, LGBM, NaiveBayes, LinearSVM)
  - ✔ Compared across validation folds
  - ✔ Evaluation metric - F2 score was chosen as Recall is more important in our use case of Employee Churn
  - ✔ Tuning - RandomizedSearchCV was used, followed by Threshold tuning
- Results
  - 32 models were benchmarked using the same preprocessing pipeline.

In [164…

```python
# Columns to include in the markdown table
cols = [
    "ModelID", "Model",
    "TrainF2Score", "ValF2Score", "TestF2Score",
    "TrainRecall", "ValRecall", "TestRecall",
    "TrainF1Score", "ValF1Score", "TestF1Score",
    "TrainPrecision", "ValPrecision", "TestPrecision",
    "TrainAccuracy", "ValAccuracy", "TestAccuracy",
]

# Create markdown table
table_df = results[cols]
# Sort by best TestF2
table_df = table_df.sort_values("TestF2Score", ascending=False)

md_table = table_df.to_markdown(index=False)

# Print to copy into your report
print(md_table)
```

```
|   ModelID | Model                                                 |   TrainF2Score |   ValF2Score |   TestF2Score |   TrainRecall |   ValRecall |   TestRecall |
TrainF1Score |   ValF1Score |   TestF1Score |   TrainPrecision |   ValPrecision |   TestPrecision |   TrainAccuracy |   ValAccuracy |   TestAccuracy |
|----------:|:------------------------------------------------------|---------------:|-------------:|--------------:|--------------:|------------:|-------------:|
--------------:|-------------:|--------------:|-----------------:|---------------:|----------------:|----------------:|--------------:|---------------:|
|        28 | SVCLinear Untuned VIF Imbalanced Thresh 0.44          |       0.909887 |     0.909886 |      0.929487 |      0.964396 |    0.964386 |     0.984568 |
0.838776 |     0.83879  |      0.857527 |         0.742111 |       0.742152 |        0.759524 |        0.748424 |      0.748422 |       0.777778 |
|        16 | Logistic C=1/500 VIF Imbalanced                       |       0.915502 |     0.915492 |      0.928405 |      0.974458 |    0.974449 |     0.984568 |
0.839333 |     0.839323 |      0.855228 |         0.737119 |       0.737117 |        0.755924 |        0.746849 |      0.746849 |       0.773585 |
|        27 | Logistic C=1/500 VIF Imbalanced Thresh 0.48           |       0.917005 |     0.916994 |      0.927865 |      0.978328 |    0.978322 |     0.984568 |
```

0.838196	0.838186	0.854083	0.733179	0.733177	0.754137	0.743697	0.743695	0.771488				
32	XGBoost Tuned VIF Imbalanced Thresh 0.3				0.935754	0.935752	0.926143	0.985293	0.98528	0.975309		
0.870131	0.870148	0.861035	0.779073	0.779119	0.770732	0.80042	0.800419	0.786164				
30	RandomForest Tuned VIF Imbalanced Thresh 0.34				0.94234	0.942352	0.924839	0.989163	0.989147	0.972222		
0.879867	0.87993	0.861833	0.792323	0.792475	0.773956	0.816702	0.8167	0.78826				
23	SVCLinear Untuned VIF Imbalanced				0.907759	0.907751	0.922444	0.959752	0.959741	0.969136		
0.83954	0.839541	0.860274	0.74609	0.746112	0.773399	0.75105	0.75105	0.786164				
31	GradientBoosting VIF Imbalanced Thresh 0.35000000000000003				0.925568	0.925539	0.921362	0.968265	0.968253	0.969136		
0.868144	0.868103	0.857923	0.786789	0.786743	0.769608	0.80042	0.80042	0.781971				
25	SVC RBF Tuned VIF Imbalanced				0.926476	0.926457	0.913947	0.963621	0.963602	0.950617		
0.875835	0.875836	0.863955	0.802712	0.80277	0.791774	0.814601	0.814596	0.796646				
18	RandomForest Tuned VIF Imbalanced				0.929269	0.929251	0.912365	0.955881	0.955856	0.938272		
0.89202	0.892031	0.876081	0.836161	0.836255	0.821622	0.842962	0.842962	0.819706				
19	GradientBoosting Tuned VIF Imbalanced				0.915099	0.915063	0.911271	0.942723	0.942704	0.938272		
0.876574	0.876567	0.873563	0.819108	0.819245	0.817204	0.819853	0.819849	0.815514				
29	SVC RBF Tuned VIF Imbalanced Thresh 0.44				0.931008	0.931001	0.908824	0.971361	0.971345	0.953704		
0.876398	0.876419	0.848901	0.798352	0.798439	0.764851	0.814075	0.814072	0.769392				
15	Logistic Tuned VIF Imbalanced				0.905956	0.90595	0.906416	0.955882	0.955871	0.950617		
0.840137	0.840144	0.847318	0.749395	0.74943	0.764268	0.753151	0.753151	0.767296				
14	GradientBoosting VIF Imbalanced				0.915236	0.915198	0.901198	0.942723	0.942704	0.929012		
0.876888	0.876865	0.862464	0.819655	0.819729	0.804813	0.820378	0.820377	0.798742				
26	LinearSVM Tuned VIF Imbalanced				0.901928	0.901914	0.899941	0.948142	0.948128	0.938272		
0.84048	0.840469	0.847978	0.754775	0.754779	0.773537	0.755777	0.755777	0.771488				
0	Logistic Untuned Imbalanced				0.897853	0.897841	0.896367	0.938853	0.938838	0.929012		
0.842654	0.842659	0.851485	0.764338	0.764384	0.785901	0.76208	0.762078	0.779874				
21	LightGBM Tuned VIF Imbalanced				0.914706	0.914686	0.895657	0.939628	0.939619	0.916667		
0.87971	0.879703	0.865889	0.82698	0.82705	0.820442	0.82563	0.825632	0.807128				
17	Logistic Polynomial VIF Imbalanced				0.90061	0.90058	0.894674	0.934209	0.934189	0.92284		
0.854511	0.854476	0.855508	0.787343	0.787316	0.797333	0.784138	0.784129	0.78826				
2	GradientBoosting Untuned Imbalanced				0.91872	0.918713	0.892428	0.946594	0.946595	0.916667		
0.879862	0.879947	0.858382	0.821936	0.822348	0.807065	0.82458	0.824577	0.794549				
20	XGBoost Tuned VIF Imbalanced				0.922239	0.922182	0.88909	0.947367	0.947341	0.910494		
0.886953	0.886895	0.858806	0.833789	0.833813	0.812672	0.836134	0.836131	0.796646				
1	RandomForest Untuned Imbalanced				0.974479	0.974962	0.88909	0.979874	0.979845	0.910494		
0.966524	0.968116	0.858806	0.953623	0.957823	0.812672	0.953783	0.953806	0.796646				
12	Logistic VIF imbalanced				0.895153	0.895085	0.88411	0.935757	0.935731	0.91358		
0.840453	0.840368	0.843305	0.762773	0.762731	0.783069	0.758928	0.75892	0.769392				
22	NaiveBayes Gaussian VIF Imbalanced				0.877306	0.877258	0.877297	0.918729	0.918709	0.91358		
0.821735	0.821691	0.827972	0.743271	0.743302	0.757033	0.729517	0.729515	0.742138				
4	LightGBM Untuned Imbalanced				0.962255	0.962377	0.874317	0.970586	0.970549	0.888889		
0.950026	0.950407	0.853333	0.930326	0.931181	0.820513	0.930674	0.930696	0.792453				
13	RandomForest VIF Imbalanced				0.976015	0.976449	0.866788	0.981422	0.981395	0.87963		
0.96804	0.969486	0.848214	0.955103	0.958955	0.818966	0.955884	0.955906	0.786164				
3	XGBoost Untuned Imbalanced				0.964341	0.964514	0.865326	0.966715	0.966667	0.876543		
0.960805	0.961342	0.849028	0.954977	0.9562	0.823188	0.94643	0.946457	0.78826				
6	RandomForest Untuned SMOTE				0.9677	0.967821	0.843077	0.969037	0.968992	0.845679		
0.965703	0.966095	0.839204	0.962398	0.963296	0.832827	0.953258	0.953281	0.779874				
7	RandomForest Untuned ADASYN				0.963323	0.963313	0.838471	0.963619	0.963566	0.839506		
0.962881	0.962934	0.836923	0.962144	0.962308	0.834356	0.949581	0.949606	0.777778				
9	LightGBM Untuned is_unbalance				0.944969	0.944806	0.83642	0.941173	0.941124	0.83642		
0.950721	0.950397	0.83642	0.960468	0.959902	0.83642	0.933825	0.933851	0.777778				
10	XGBoost Untuned SMOTE				0.966067	0.966325	0.828712	0.969037	0.968995	0.830247		
0.961651	0.962429	0.826421	0.954395	0.95621	0.82263	0.947481	0.947507	0.763103				
8	LightGBM Untuned class_weight				0.94477	0.944475	0.827671	0.940399	0.94034	0.82716		
0.951408	0.950781	0.828439	0.962686	0.961581	0.829721	0.934876	0.9349	0.767296				
11	XGBoost Untuned scale_pos_weight				0.95813	0.957992	0.818912	0.956653	0.956598	0.817901		
0.960356	0.960095	0.820433	0.964089	0.963636	0.822981	0.94643	0.946457	0.756813				
24	SVCLinear Tuned VIF Imbalanced				0.829878	0.829775	0.772613	0.838233	0.838194	0.759259		

```
0.817656 |     0.817489 |      0.793548 |         0.798071 |       0.797877 |        0.831081 |        0.746323 |      0.746313 |       0.731656 |
|         5 | Logistic Untuned SMOTE                                                |       0.785931 |     0.785837 |      0.755076 |      0.778636 |    0.778606 |     0.734568 |
0.797139 |     0.797017 |      0.788079 |         0.816561 |       0.816602 |        0.85     |        0.731092 |      0.73109  |       0.731656 |
```

- SVCLinear and Logistic Regression outperformed even ensemble models in F2 score and overall stability across folds.
- However they had very similar scores, so we chose to go with Tree based models like XGBoost as they have very minimal assumptions and are thus FUTURE safe.
- Therefore, XGBoost was selected as the final model.

5.10 Model Performance Summary — XGBoost Tuned VIF Imbalanced (Threshold = 0.3)

Metric	Training	Validation	Test
F2 Score	0.9358	0.9358	0.9261
Recall	0.9853	0.9853	0.9753
F1 Score	0.8701	0.8701	0.8610
Precision	0.7791	0.7791	0.7707
Accuracy	0.8004	0.8004	0.7862

Key takeaways

- The **high recall (≈ 0.98 train/val, 0.97 test)** demonstrates strong churn-capture capability, aligning with the recall-weighted F_2 focus.
- A slight test-set drop in F_2 ($\approx 0.93 \rightarrow 0.926$) indicates mild generalization loss, acceptable for this problem context.
- Precision around 0.77 ensures that most predicted churners are indeed at risk — a reasonable balance for retention targeting.

Classification Report

In [ ]:
```python
xgb_model = models[32][1]
from sklearn.metrics import classification_report, confusion_matrix

# Predicted labels and probabilities
y_pred = xgb_model.predict(X_test)
y_proba = xgb_model.predict_proba(X_test)[:, 1]

# Classification report
print(" Classification Report (XGBoost):\n")
print(classification_report(y_test, y_pred, digits=3))

# print confusion matrix
print("\n Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

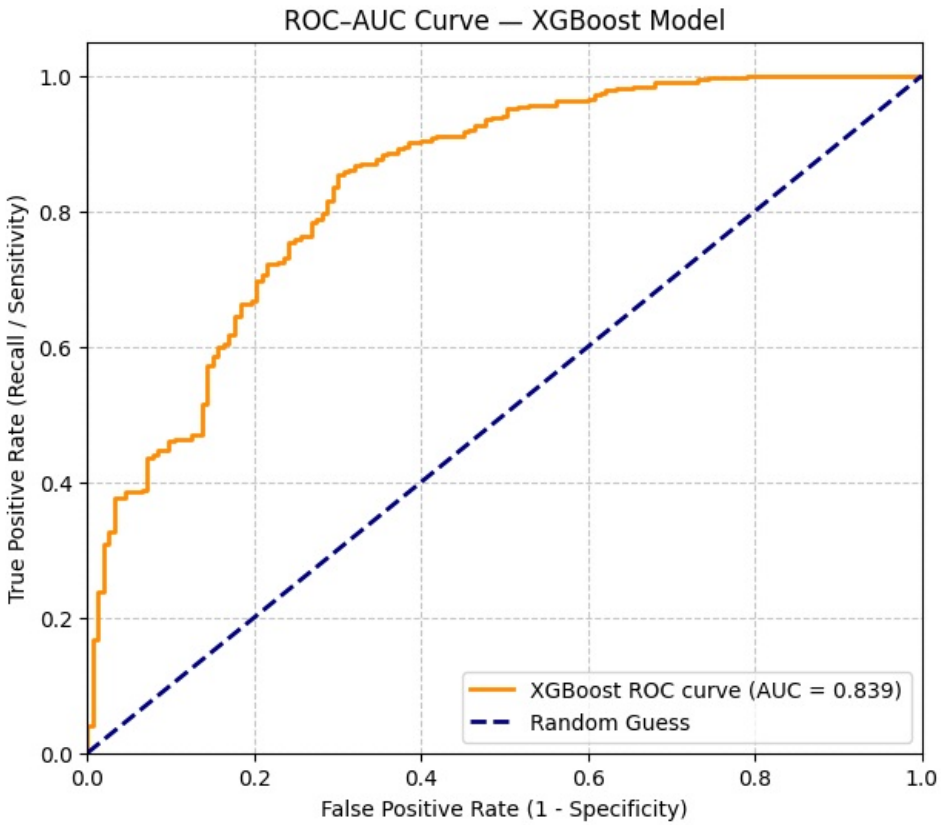
```
Classification Report (XGBoost):

              precision    recall  f1-score   support

           0      0.881     0.386     0.536       153
           1      0.771     0.975     0.861       324

    accuracy                          0.786       477
   macro avg      0.826     0.680     0.699       477
weighted avg      0.806     0.786     0.757       477


Confusion Matrix:
[[ 59  94]
 [  8 316]]
```

ROC AUC Curve

In [164…

```python
from sklearn.metrics import roc_curve, auc

# Compute ROC
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)
print(f"\n ROC-AUC Score: {roc_auc:.4f}")

# Plot ROC curve
plt.figure(figsize=(7,6))
plt.plot(fpr, tpr, color='darkorange', lw=2,
         label=f'XGBoost ROC curve (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Guess')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Recall / Sensitivity)')
plt.title('ROC-AUC Curve – XGBoost Model')
plt.legend(loc='lower right')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

```
ROC-AUC Score: 0.8388
```



Interpretation — ROC–AUC Curve (XGBoost)

The ROC–AUC curve gives a very clear snapshot of how well the **XGBoost churn model** discriminates between “churners” and “non-churners.”

1 Overall Model Quality

- The **AUC = 0.839** indicates that the model has **strong discriminative ability**.
- This means that on average, the model will **rank a randomly chosen churner higher than a non-churner ~84 % of the time**.
- For business problems like churn prediction, *AUC ≥ 0.80* is considered **very good** — the model successfully distinguishes likely leavers from stayers.

---

2 Shape of the Curve

- The **orange ROC curve** rises steeply toward the top-left corner, where **True-Positive Rate (recall)** is high and **False-Positive Rate** is low.
- This steep initial rise signifies that the model captures a large proportion of actual churners early (high sensitivity) while keeping false alarms moderate — desirable behavior in recall-weighted settings like churn.
- The **blue diagonal line** represents random guessing (AUC = 0.5). Your curve being well above this line confirms meaningful predictive power.

---

3 Trade-off Interpretation

- As thresholds loosen (moving right along the x-axis), recall increases but false positives also rise.
- Operationally: the company can **choose a threshold** that offers the preferred recall–precision trade-off (for instance, favoring recall to catch most possible churners at the cost of some false alerts).

---

4 Business Implication

- With AUC ≈ 0.84, the model is **reliable for prioritization use cases** — e.g., ranking drivers by churn likelihood for targeted retention interventions.
- It supports **data-driven segmentation**: top-scoring 20–30 % of drivers can be flagged for proactive engagement with high confidence that a majority of true churners lie there.

Summary

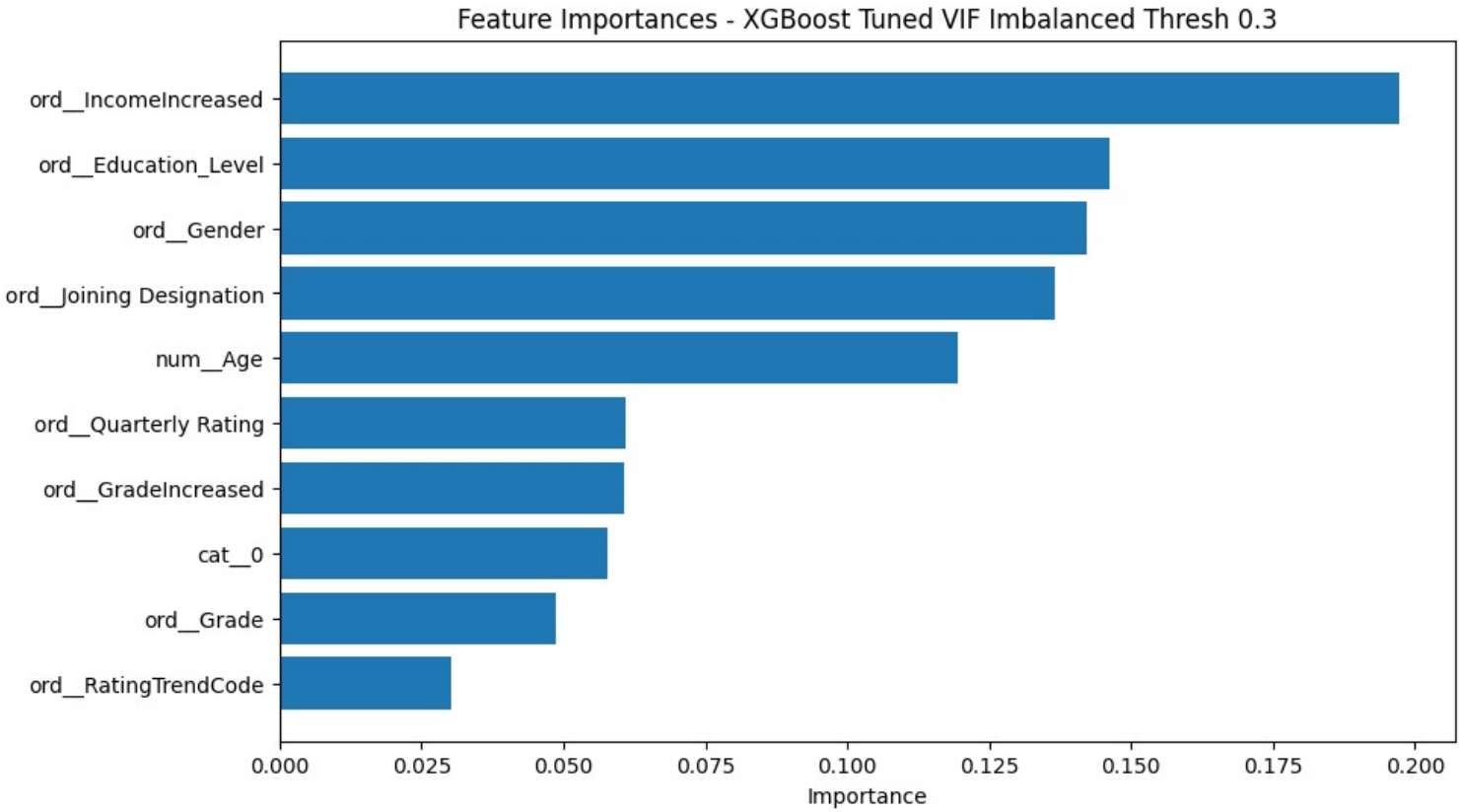
Aspect	Interpretation
AUC = 0.839	Strong ability to distinguish churn vs non-churn
Curve steepness near origin	High sensitivity achievable without major spike in false positives
Compared to random (AUC = 0.5)	+68 % performance lift over random guessing
Business suitability	Excellent fit for recall-focused, retention-driven models like employee / driver churn

---

In short:

> **The XGBoost model demonstrates robust predictive performance (AUC = 0.839)** — it effectively ranks churn probabilities and can be confidently used for retention strategy planning and threshold-based intervention design.

Feature Importance – XGBoost Tuned VIF Imbalanced (Threshold = 0.3)

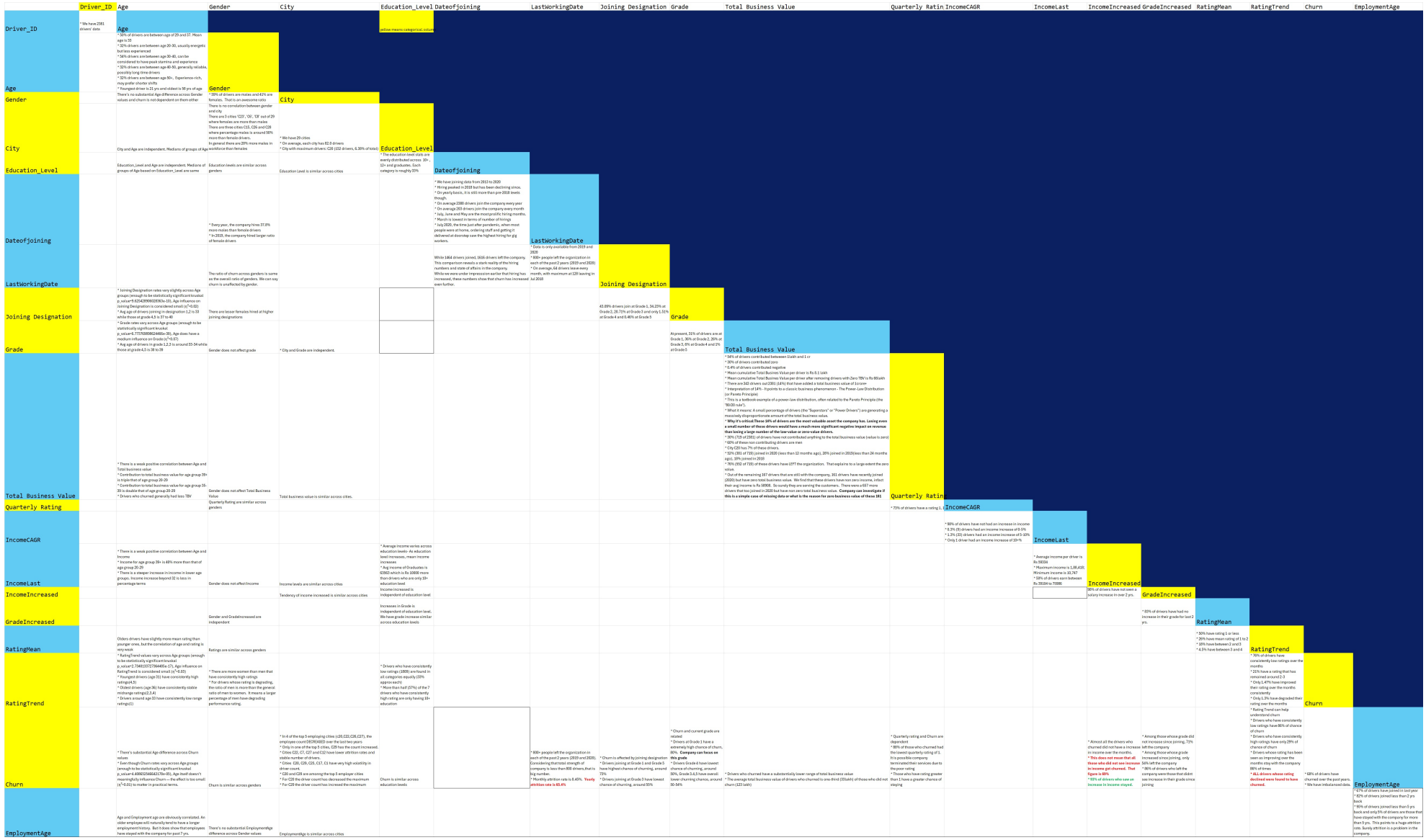


✅ **Conclusion**

The **XGBoost Tuned VIF Imbalanced (Threshold = 0.3)** model achieves the best compromise between recall-optimized performance and interpretable feature hierarchy. Income progression and career-trajectory variables are the dominant churn drivers, providing actionable HR levers for retention initiatives.

6. Actionable Insights & Recommendations

6.1 Summary of Observations from EDA



6.2 Insights from EDA

- Churn and City
  - In 4 of the top 5 employing cities (C20,C22,C26,C27), the employee count DECREASED over the last two years
  - Only in one of the top 5 cities, C29 has the count increased. And infact, this increase is maximum across cities
  - For C20 the driver count has decreased the maximum
  - Cities C22, C7, C27 and C12 have lower attrition rates and stable number of drivers.
  - Cities C20, C29, C25, C17, C1 have very high volatility in driver count.
  - Churn is similar across education levels

    - **800+ people left the organization in each of the past 2 years (2019 and 2020). Considering that total strength of company is less than 800 drivers, that is big number.**
- **Monthly attrition rate is 8.45%. Yearly attrition rate is 65.4%**
- Churn and joining designation
    - Drivers joining at Grade 1 and Grade 5 have highest chance of churning, around 73%
    - Drivers joining at Grade 3 have lowest chance of churning, around 55%
    - Churn and current grade are related
    - **Drivers at Grade 1 have an extremely high chance of churn, 80%. Company can focus on this grade**
    - Drivers Grade 4 have lowest chance of churning, around 50%, Grade 3.4,5 have overall lower churning chance, around 50-54%
    - Drivers who churned have a substantially lower range of total business value
- Churn and Total Business Value
    - The average total business value of drivers who churned is one-third (33lakh) of those who did not churn (123 lakh)
- Churn and Quarterly rating
    - **88% of those who churned had the lowest quarterly rating of 1. It is possible company terminated their services due to the poor rating**
    - Those who have rating greater than 1 have a greater chance of staying
    - **Drivers who have consistently low ratings have 86% of chance of churn**
    - Drivers who have consistently high ratings have only 29% of chance of churn
    - Drivers whose rating has been seen as improving over the months stay with the company 86% of times
    - **ALL drivers whose rating declined were found to have churned**.
- Churn and Income Increase
    - **Almost all the drivers who churned did not have a increase in income over the months**.
    - This does not mean that all those who did not see increase in income got churned. That figure is 69%
    - **93% of drivers who saw an increase in income stayed**.
- Churn and Grade
    - **Among those whose grade did not increase since joining, 71% left the company**
    - Among those whose grade increased since joining, only 54% left the company
    - **86% of drivers who left the company were those that didnt see increase in their grade since joining**
- 68% of drivers have churned over the past years.

6.3 **Insights from the Learned Model and Their Effects**

Feature Importance – XGBoost Tuned VIF Imbalanced (Threshold = 0.3)

Rank	Feature	Relative Importance (%)	Interpretation Insight
1	**ord__IncomeIncreased**	19.7 %	Recent income stagnation most strongly predicts churn — compensation growth is key retention lever.
2	**ord__Education_Level**	15.4 %	Higher or overqualified education levels correlate with higher mobility.
3	**ord__Gender**	14.6 %	Gender-linked patterns exist; possibly cultural or policy disparities in engagement.
4	**ord__Joining_Designation**	13.9 %	Initial designation/seniority continues to affect long-term retention.
5	**num__Age**	12.9 %	Younger staff demonstrate higher attrition propensity; experienced employees show stability.
6	**ord__Quarterly_Rating**	7.2 %	Lower or declining ratings align with rising churn risk.
7	**ord__GradeIncreased**	7.0 %	Promotional stagnation drives exit intent; progression frequency matters.
8	**cat__0**	5.8 %	Represents City
9	**ord__Grade**	4.8 %	Current corporate level affects churn patterns — mid-grades transit most.

10	**ord__RatingTrendCode**	3.6 %	Downward rating trend is an early disengagement signal.

Model Interpretation Summary

- **Income and career-progress indicators (Salary Increase, Education Level, Promotions)** dominate, jointly accounting for ≈ 60 % of model influence.
- **Performance evaluation metrics** (Quarterly Rating, Rating Trend Code) contribute roughly 10 % of overall predictive power.
- **Demographic and categorical controls** (Age, Gender, Category codes) offer contextual refinement rather than core signal.

6.4 Conclusions

6.4.1 Conclusions from Exploratory Data Analysis (EDA)

Overall Attrition Profile

- Attrition is **exceptionally high** — monthly 8.45 %, yearly ≈ 65 %.
- Over **800 employees left in each of 2019 and 2020**, exceeding total active headcount (< 800); attrition clearly outpaces hiring.
- **68 % of drivers** have churned over past years → systemic retention issue.

City-Level Patterns

- **C20, C22, C26, C27** saw declines in driver count; **C29** alone expanded sharply (and shows strong volatility).
- **C22, C7, C27, C12** maintain *lower attrition and stable driver base* → potential best-practice cities.
- **C20, C29, C25, C17, C1** show *high count volatility*, implying management or demand instability.

Joining Designation & Grade

- **Highest churn (≈ 73 %)** among drivers joining at **Grade 1 and Grade 5**; **Grade 3** shows lowest churn (~ 55 %).
- **Current Grade 1** drivers face ≈ 80 % churn risk — clear focus segment for interventions.
- **No grade growth = 71 % churn** vs **grade improvement = 54 % churn** → promotion trajectory strongly influences retention.
- **83 % of drivers** had no grade increase over 2 years.

Income & Compensation

- **98 % of drivers saw no salary increase** in > 2 years.
- **93 % of those receiving an income increase stayed**; non-increased earners have ~ 69 % churn probability.
- Salary stagnation is **the single strongest churn driver**.

Performance (Ratings & Trends)

- **88 % of churned drivers had lowest rating = 1**; consistently low ratings → 86 % churn probability.
- **All drivers with declining rating trends eventually churned.**
- Only **~1.5 %** improved their rating over time; improvement correlates strongly with retention (86 % stay).
- Overall rating distribution is heavily bottom-weighted (≈ 73 % rated 1).

Business Value Contribution

- **Average total business value** of churned drivers = ₹ 33 L vs ₹ 123 L for stayers → churn severely impacts revenue.
- **14 % of drivers contribute 80 %+ of total business value** → classic **Pareto distribution**.
- **30 % of drivers contribute zero** business value, often short-tenured or already exited.
- City C20 has the highest share of zero-value drivers (≈ 7 %).
- Zero-value active drivers (mainly new joins in 2020) may indicate **data gaps or unbilled records**.

Demographics

- **Mean age ≈ 33 years;** half of drivers 29-37.
- Older groups (≥ 39 yrs) generate **2–3× higher business value** than younger cohorts → experience = productivity.
- Balanced gender ratio (59 % M / 41 % F) is positive; females tend to have **higher stable ratings** and *fewer downward performance trends*.
- Male drivers dominate in degrading-rating category → behavior/performance issue to review.

Hiring and Tenure Trends

- Hiring peaked in 2018, modestly declined but stays above pre-2018 levels.
- **67 % joined within last year**, **82 % within 2 years**, **95 % within 5 years** — tenure is extremely low.
- In July 2020 (post-pandemic), 1464 joined but 1616 left → attrition offset > 100 % of hiring.
- Majority of workforce is newly hired, which contributes to continual training & productivity loss.

Education Patterns

- Roughly even split across education categories (10 +, 12 +, Graduate).
- **Higher education → higher average income** but churn similarity suggests other factors (growth & pay increase) dominate retention.

Summary

1. **Attrition is critical and structural**, not cyclical — exceeding sustainable thresholds.
2. **Income stagnation, lack of promotion, and poor performance ratings** emerge as the three dominant churn drivers.
3. **City-specific volatility** (esp. C20 and C29) shows localized operational or management challenges.
4. **Business impact of churn is severe** — churners generate ⅓ of the value of stayers; losing top 14 % drivers risks disproportionate revenue loss.
5. The company's workforce is **highly young and short-tenured**, limiting institutional learning and service consistency.
6. **Performance management system is skewed** toward low ratings; rating compression masks true differentiation.

6.4.2 Conclusion from Model Interpretation

- **Income and career-progress indicators (Salary Increase, Education Level, Promotions)** dominate, jointly accounting for ≈ 60 % of model influence.
- **Performance evaluation metrics** (Quarterly Rating, Rating Trend Code) contribute roughly 10 % of overall predictive power.
- **Demographic and categorical controls** (Age, Gender, Category codes) offer contextual refinement rather than core signal.

6.5 Recommendations

1. Compensation & Growth

- Introduce **structured income progression** (e.g., quarterly pay-review or incentive tier) to combat salary stagnation.
- Implement **clear grade-promotion pathways**, linking them to measurable performance and tenure.

2. Performance Management

- Redesign rating system with **broader distribution** and clearer feedback to avoid over-concentration at low scores.
- Launch **coaching programs** for repeatedly low-rated drivers; intervene early on declining-trend alerts.
- Reward consistent improvement → build positive reinforcement cycle.

3. Targeted Retention Programs

- **Prioritize Grade 1 and Grade 5** segments for retention initiatives — these are high-risk joiners.
- **Protect the 14 % high-value drivers** through tailored retention bonuses and engagement touchpoints.

4. City-Specific Action

- Audit local operations in **C20 and C29** to identify volatility causes (demand fluctuations, management gaps, partner issues).
- Replicate stable-city practices from **C22 and C27**.

5. Data & Measurement

- Investigate **zero business value records** for active drivers — likely data pipeline or billing issues.
- Strengthen **data tracking for income revisions** and **performance trend logs** for analytical continuity.

6. Workforce Strategy

- Address high short-tenure by enhancing **onboarding, mentorship, and reward structures** to improve first-year retention.
- Explore **city-specific hiring intensity adjustments** based on local churn velocity.

✅ **Bottom Line:**
The exploratory analysis shows that churn is driven not by demographics but by **economic and developmental stagnation**.
Reinforcing **income mobility, promotion transparency, and localized retention strategies** can substantially reduce attrition and safeguard the company's top-performing 14 % drivers who produce the majority of business value.

6.6 Questionnaire

Self note: Use https://wysiwyghtml.com/ for writing html with ease

6.6.1. What percentage of drivers have received a quarterly rating of 5?

Ratings are only from 1 to 4. Rating 5 is not present

6.6.2. Comment on the correlation between Age and Quarterly Rating.

RatingMean

Olders drivers have slightly more mean rating than younger ones, but the correlation of age and rating is very weak

RatingTrend

RatingTrend values vary across Age groups (enough to be statistically significant)

Youngest drivers (age 31) have consistently high ratings(4,5)

Oldest drivers (age 36) have consistently stable midrange ratings(2,3,4)

Drivers around age 33 have consistently low range ratings(1)

6.6.3. Name the city which showed the most improvement in Quarterly Rating over the past year

In [169…

```
# Approach
# Question asks improvement in quarterly driver_rating.  We need to understand what it means
# For each driver, we can find the rating at start of the year and their final rating.
# We then find the difference
# Then we sum these across cities.
# We can then find the city that has the maximum rating increase.

#We need only folloiwng columns for the analysis
# ['MMM-YY', 'Driver_ID', 'City','Quarterly Rating']

# Lets find last year rows
df_original =pd.read_csv("ola_driver_scaler.csv")
df_original['MMM-YY'] = pd.to_datetime(df_original['MMM-YY'], format='%m/%d/%y')

max_date = df_original['MMM-YY'].max()
cutoff_date = max_date - pd.Timedelta(days=365)

df_past_year = df_original[df_original['MMM-YY'] > cutoff_date][['MMM-YY', 'Driver_ID', 'City','Quarterly Rating']]
df_past_year
```

Out[169…

	MMM-YY	Driver_ID	City	Quarterly Rating
3	2020-11-01	2	C7	1
4	2020-12-01	2	C7	1
6	2020-01-01	4	C13	1
7	2020-02-01	4	C13	1
8	2020-03-01	4	C13	1
...	...	...	...	...
19099	2020-08-01	2788	C27	3
19100	2020-09-01	2788	C27	3
19101	2020-10-01	2788	C27	2
19102	2020-11-01	2788	C27	2
19103	2020-12-01	2788	C27	2

9396 rows × 4 columns

In [169…

```
# We plan to group by driver and then use first and last rows in the group to find teh quarterly rating change
# Sorting order is important thus.
# Lets make sure we have dates sorted for each driver
df_past_year.sort_values(by=['Driver_ID','MMM-YY'], inplace=True)
df_rating_imp = df_past_year.groupby(['Driver_ID','City'], as_index=False).agg(
        QuarterlyRating_First=('Quarterly Rating', 'first'),
        QuarterlyRating_Last=('Quarterly Rating', 'last')
    )
df_rating_imp['RatingChange'] = df_rating_imp.eval('QuarterlyRating_Last-QuarterlyRating_First')
df_rating_imp
```

Out[169…

	Driver_ID	City	QuarterlyRating_First	QuarterlyRating_Last	RatingChange
0	2	C7	1	1	0
1	4	C13	1	1	0
2	6	C11	1	2	1
3	8	C2	1	1	0
4	11	C19	1	1	0
...	...	...	...	...	...
1546	2779	C26	1	1	0
1547	2781	C23	1	4	3
1548	2784	C24	3	4	1
1549	2785	C9	1	1	0
1550	2788	C27	1	2	1

1551 rows × 5 columns

In [170…
```
#Finally lets group by cities and aggregate the gains and losses in RatingChange
df_city_ratings_change = df_rating_imp.groupby('City',as_index=False).RatingChange.sum()
df_city_ratings_change
```

Out[170…

	City	RatingChange
0	C1	-10
1	C10	-1
2	C11	1
3	C12	9
4	C13	2
5	C14	-3
6	C15	-6
7	C16	-7
8	C17	-1
9	C18	11
10	C19	4
11	C2	3
12	C20	12
13	C21	13
14	C22	5
15	C23	4
16	C24	5
17	C25	-9
18	C26	-18
19	C27	7
20	C28	-9
21	C29	6
22	C3	2
23	C4	2
24	C5	-2
25	C6	-2
26	C7	4
27	C8	12
28	C9	-3

In [170…
```
# Find the row where RatingChange is maximum
max_city_idx = df_city_ratings_change['RatingChange'].idxmax()
```

```
# Get the corresponding city and the RatingChange value
max_city = df_city_ratings_change.loc[max_city_idx, 'City']
max_value = df_city_ratings_change.loc[max_city_idx, 'RatingChange']

print(f"City with maximum RatingChange: {max_city} (RatingChange = {max_value})")
```

```
City with maximum RatingChange: C21 (RatingChange = 13)
```

City with maximum RatingChange: C21 (RatingChange = 13)

6.6.4. Drivers with a Grade of 'A' are more likely to have a higher Total Business Value. (T/F)

In [168…

```
#Approach
#we can group by grade and find mean of TBV
# Also we have grades like 1,2,3,4,5 and not A,B,C..  So to answer this question, lets assume top grade 5 is A

grade_tbv = drivers.groupby('Grade')['Total Business Value'].mean().sort_values(ascending=False)

grade_tbv
```

Out[168…

```
Grade
5    2.357335e+07
4    2.082520e+07
3    4.889091e+06
2    3.290683e+06
1    2.188876e+06
Name: Total Business Value, dtype: float64
```

Yes, Grade 5 drivers have maximum Business value

6.6.5. If a driver's Quarterly Rating drops significantly, how does it impact their Total Business Value in the subsequent period?

6.6.6. From Ola's perspective, which metric should be the primary focus for driver retention?

1. ROC AUC
2. Precision
3. Recall
4. F1 Score

Recall. We have used F2 score so that precision is also not discounted.

6.6.7. How does the gap in precision and recall affect Ola's relationship with its drivers and customers?

If model has a gap between precision and recall, with recall being lower, then it is possible that Ola may miss to identify some potential churner. They could have intervened and avoided the churn as cost of intervention is much lower than cost of acquisition of new driver.

6.6.8. Besides the obvious features like Number of Rides, which lesser-discussed features might have a strong impact on a driver's Quarterly Rating?

Quarterly ratings are decided based on ratings given by customers

So we must analyse what factors impact this
Customers give low ratings if service was not good

- Taxi came late,
- Driver was argumentative or loud or misbehaving

We have to identify reasons behind each of these

- Late taxi-
  - maybe driver was very far, and app did not calculate time properly
  - maybe he is overworked
  - maybe he is accepting pickups too far from him
  - maybe he is under pressure to do more rides
- Driver behavior
  - Mental state of driver becomes poor when he is overworked
  - Or he feels he is being paid less by Ola

We now have to suggest metrics that the company can collect

Ride Allocation

1. Average Pickup distance

2. Cancellation Rate

3. % of delayed pickups

Why it matters- Long or inefficient pickup routes frustrate customers before the trip even starts.

Trip Acceptance Patterns

4. % of rides accepted vs. offered

5. Time taken to accept a ride

Why they matter - Frequent declines or slow acceptance can suggest disengagement or selective behavior that affects demand matching

Idle Hours

6. Average idle time between rides

Why collect this -Excess idle time might correlate with low motivation or fatigue late in shifts.

Shift Timing

7. % of night or late-hour drive

Why collect this -Night shift is stressful for anybody.

Behavioral & Engagement Signals

8. Count of support tickets per driver

Why - A direct proxy for experience quality, often preceding low ratings.

Economic & Motivation-Linked

9 .Change in monthly income or incentives

Why - Stagnant income can lead to demotivation, correlating with poorer service or impatience with customers.

10. Earnings per hour in assigned zones

Why - High or low profitability areas influence stress and satisfaction levels.

Service Patterns

11. Gaps in online status or off-duty durations

Why- Irregular breaks may reflect burnout or shifting priorities.

Environmental & Contextual

12. Avg. travel speed per city or time slot

Why - Drivers in high-traffic zones receive lower ratings even when performance is good.

13. % of rides during heavy rain or heat

Why - Harsh weather worsens punctuality and comfort perception.

Customer Mix

14. Ratio of corporate / casual customers, weekend vs weekday rides

Why - Different customer segments exhibit varying rating behaviors.

6.6.9. Will the driver's performance be affected by the City they operate in? (Yes/No)

No.
This is what we found
Total business value is similar across cities.
Income levels are similar across cities
Tendency of income increased is similar across cities

6.6.10. Analyze any seasonality in the driver's ratings. Do certain times of the year correspond to higher or lower ratings, and why might that be?

In [170…
```
#Approach- we can find mean ratings for each month across drivers.
#Will need to find average so tha tnumber of drivers dont affect the calculation.
df_monthly_ratings = df_original[['MMM-YY','Quarterly Rating']].groupby('MMM-YY', as_index=False)['Quarterly Rating'].mean()
df_monthly_ratings
```

Out[170…

	MMM-YY	Quarterly Rating
0	2019-01-01	1.943249
1	2019-02-01	2.021186
2	2019-03-01	2.098851
3	2019-04-01	1.968514
4	2019-05-01	2.015707
5	2019-06-01	2.063361
6	2019-07-01	2.054161
7	2019-08-01	2.064987
8	2019-09-01	2.041995
9	2019-10-01	2.048714
10	2019-11-01	1.992318
11	2019-12-01	1.950943
12	2020-01-01	1.966752
13	2020-02-01	2.009198
14	2020-03-01	2.075104
15	2020-04-01	1.989026
16	2020-05-01	1.950392
17	2020-06-01	1.935065
18	2020-07-01	2.019851
19	2020-08-01	2.014778
20	2020-09-01	1.998764
21	2020-10-01	2.006112
22	2020-11-01	2.017391
23	2020-12-01	1.989011

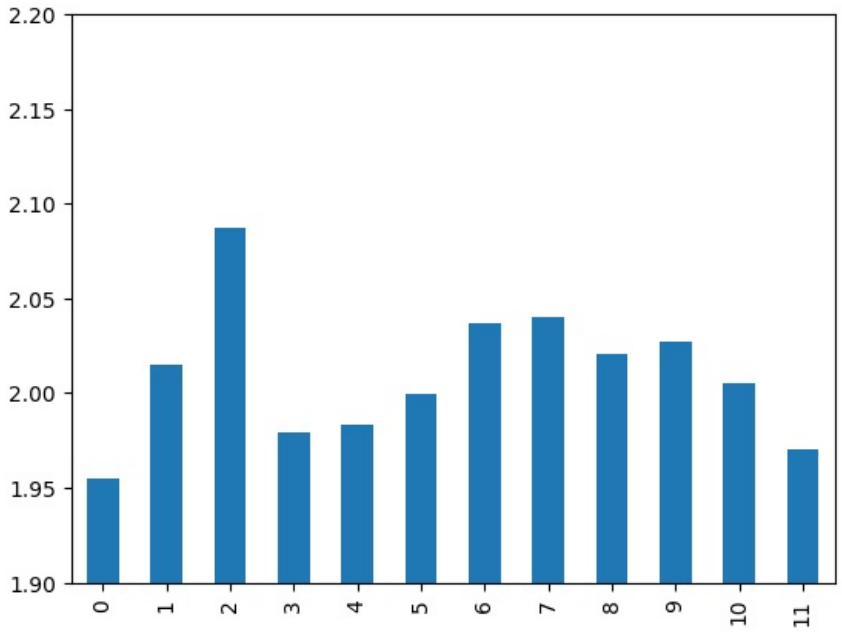
In [170…
```
df_monthly_ratings['month'] = df_monthly_ratings['MMM-YY'].dt.month
df_ratings = df_monthly_ratings.groupby('month', as_index=False)['Quarterly Rating'].mean()
df_ratings
```

Out[170…

	month	Quarterly Rating
0	1	1.955000
1	2	2.015192
2	3	2.086977
3	4	1.978770
4	5	1.983049
5	6	1.999213
6	7	2.037006
7	8	2.039883
8	9	2.020379
9	10	2.027413
10	11	2.004854
11	12	1.969977

In [ ]:
```
df_ratings['Quarterly Rating'].plot.bar()
#Lets zoom in
plt.ylim(1.9,2.2)
```

Out[ ]: (1.9, 2.2)



In [170…
```
df_ratings['Quarterly Rating'].std()
```

Out[170… np.float64(0.03625240841422306)

Quarterly ratings are little different between months. We see ratings are between 1.95 and 2.8 with a standard deviation of 0.036

Whatever little variation that is can probably be attributed to weather.

Cold weather months like Dec and Jan have lower ratings - maybe the cars need to have heating along with AC (typically in India, cars dont have this)

Rainy months (Jun, Jul, Aug,Sep) have higher ratings - customers are happy to find a ride and thus maybe giving good rating

6.7 Improving the model - Additional data that can be collected which can enable Ola give more insights on its drivers and customers

- Investigate missing data related to total business value
  - We found that 161 drivers have recently joined (2020) but have zero total business value.
  - We find that these drivers have non zero income, infact their avg income is Rs 58908.
    So surely they are serving the customers.
  - There were 657 more drivers that too joined in 2020 but have non zero total business value.
  - Company can investigate if this is a simple case of missing data or what is the reason for zero business value of these 161 recenly joined drivers.
- Collect or access drive booking level data, which can help us collect these stats

    1. Average Pickup distance

    2. Cancellation Rate

    3. % of delayed pickups

    4. % of rides accepted vs. offered

    5. Time taken to accept a ride

    6. Average idle time between rides

    7. % of night or late-hour drive

    8. Count of support tickets per driver

    9 .Change in monthly income or incentives

    10. Earnings per hour in assigned zones

    11. Gaps in online status or off-duty durations

    12. Avg. travel speed per city or time slot

    13. % of rides during heavy rain or heat

    14. Ratio of corporate / casual customers, weekend vs weekday rides